Robot Reinforcement Learning

From First Principles to Real Robots

Bruce Lu

Contents

How to Use This Book

Edition 1.0.2 — 2026-09-04

This book is designed for active self-study. Read with a Python prompt, a notebook, or a robot simulator nearby. When an equation appears, first name every symbol and predict how changing it would affect an experiment. Then work the numerical example before looking at the answer. When code appears, run it, change one input, and explain the output before continuing.

The material has three connected layers:

1. **Foundations** explain the reinforcement-learning problem, the mathematics, and the major algorithm families.
2. **The Microduck laboratory** shows where those abstractions live in a real GPU-parallel robot-learning project, from environment configuration through ONNX deployment.
3. **Modern robot intelligence** connects locomotion to demonstrations, offline data, world models, vision-language-action policies, hierarchical planning, and reproducible research.

Microduck is the running hands-on case study, not the boundary of the subject. Its main walking policy is a command-conditioned local controller: it does not see a camera or obstacle map and it does not choose a destination. That clear boundary makes it a useful base for learning how perception and planning can be added without placing cloud latency inside a real-time balance loop.

On GitHub, chapter-end answers are collapsed so that you can attempt each exercise first. In this PDF edition they are expanded and placed at the end of their chapter. The primary-source index at the end records the papers and official open-source projects behind the research chapters. Treat every state-of-the-art claim as scoped to its dated task, data, embodiment, and evaluation protocol.

Visual language in this PDF

The PDF uses a restrained semantic color system to reduce search effort while studying. Color is never the only carrier of meaning: heading size, labels, frames, alignment, and whitespace repeat every cue, so the structure remains usable in grayscale and for readers with color-vision differences.

Visual cue	Meaning and study action
navy headings	a major idea or conceptual landmark; pause and restate it in your own words
teal navigation	the current step in an explanation or workflow
amber answer banner	revealed material; attempt the associated problem before reading further
blue-gray panel	executable code, a command, data, or a diagram; trace it line by line
navy display math	a mathematical model; name every symbol before manipulating it

Suggested routes

- **Theory first:** Chapters 1–7, the 13 runnable programs in `examples/`, and the worked problems in Chapter 20.
- **Build and deploy:** Chapters 1–13, alongside a pinned Microduck checkout.
- **Research literacy:** Chapters 14–20 after the foundations, recording the evidence boundary of every paper before accepting its headline result.

Do not measure progress by pages read. Advance when you can produce the deliverable at the end of a chapter and explain what observation, action, reward, data, and safety contract a robot-learning result actually used.

1. Reinforcement Learning Foundations

1.1 The central idea

Reinforcement learning trains a decision-making rule by letting it interact with an environment. The learner is not given the correct motor command for every possible situation. Instead, it tries actions, observes consequences, and adjusts its policy so that useful outcomes become more likely.

For Microduck, one interaction looks like this:

```
simulated robot and command
        |
        | observation: orientation, joint state, previous action, command
        v
neural-network policy
        |
        | action: 14 normalized joint-position targets
        v
Better Actuator Models (BAM) motor model + MuJoCo physics
        |
        | next robot state, reward, termination flag
        +-----------------------------------------------> repeat
```

An **actuator** is the component that turns a command into physical force or motion; here it is a Dynamixel servo represented by Better Actuator Models (BAM). During training, thousands of simulated robots execute this loop in parallel. During deployment, one real robot executes only the learned policy; Proximal Policy Optimization (PPO) and the critic are no longer updating weights.

1.2 Reinforcement learning (RL) compared with nearby ideas

Supervised learning starts with labeled examples such as "for this image, the answer is cat." RL usually has no label saying "the correct 14 motor targets are these." It has an objective, such as tracking velocity while staying upright, and must discover a sequence of actions.

Classical control starts from an explicit model or error law, such as a proportional–integral–derivative (PID) controller. RL can learn a nonlinear controller where writing the full law by hand is difficult. Classical controllers remain valuable as baselines, safety layers, and inner loops.

Planning chooses goals or sequences over a longer horizon. A walking policy is normally below planning: a planner asks for a local velocity, and the policy produces stable joint targets. Asking a cloud model for raw motor torques would collapse these layers and remove the timing and safety boundary.

Why reinforcement learning became a separate field

The distinctive difficulty is **credit assignment**: deciding which earlier actions deserve credit or blame for a later result. A supervised label can say that one image is a cat. It usually cannot say which ankle target 1.3 seconds ago prevented a fall after three later contacts. The action also changes what data becomes available next, so examples are neither independent nor drawn from a fixed distribution.

The modern formal thread has several roots rather than one birthday:

- Richard Bellman's 1950s dynamic programming made a long decision problem recursive: solve the future subproblem, then choose the present action.
- Trial-and-error psychology and optimal-control research contributed ideas about reward, feedback, and sequential behavior.
- Temporal-difference learning joined sampled experience with Bellman-style bootstrapping; the 1983 actor-critic work of Barto, Sutton, and Anderson is an early recognizable learning controller.
- Watkins's Q-learning, Williams's REINFORCE estimator, neural function approximation, and later deep-learning scale produced today's major value-learning and policy-gradient branches.

The annotated primary-source index links the original or stable records. This history matters because current methods recombine old answers to four questions: what should be predicted, how should delayed credit be assigned, which data may be reused, and how far may the policy change at once?

1.3 A small guide to the notation

Math notation is compact language. Read the symbols rather than trying to memorize the visual shape:

Notation	Read it as	Example meaning here
x_t	"x at time step t"	robot state now
x_{t+1}	"x at the next step"	robot state 20 ms later
$a \in \mathcal{A}$	"a is an element of set A"	this action is allowed
$p(x \mid y)$	"probability of x given y"	next-state probability given state/action
$\sum_i x_i$	"sum x over index i"	add reward terms or future rewards
$\mathbb{E}[X]$	"expected or long-run average X"	average return across sampled rollouts
$x \sim p$	"sample x from distribution p"	sample a randomized mass
θ	"theta," a parameter collection	all actor network weights
$\approx$	approximately equal	critic estimate versus unknown true value

A **vector** is an ordered list of numbers. A **tensor** is the general multidimensional form used by PyTorch: a scalar is 0D, a vector is 1D, a table or batch is 2D, and images/batched histories use more dimensions. In a batch of 4,096 actor observations, the observation tensor has shape `(4096, 61)`.

1.4 The Markov Decision Process

The standard mathematical model is a Markov Decision Process (MDP):

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$$

Symbol	General meaning	Microduck example
$s_t \in \mathcal{S}$	complete state at time t	all simulated positions, velocities, contacts, and randomized physics
$a_t \in \mathcal{A}$	action selected at time t	14 joint-position commands
$P(s_{t+1} \mid s_t, a_t)$	transition dynamics	MuJoCo plus the BAM actuator, contacts, delay, and noise
$R(s_t, a_t, s_{t+1})$	reward	velocity tracking, uprightness, foot behavior, and regularization
γ	discount factor	`0.99` in the main PPO configuration

The Markov property says the current state contains enough information to model the distribution of the next state. The policy does not receive the complete simulator state, however. It receives an observation o_t.

More precisely, Markov means that after conditioning on the present state and action, the earlier history adds no predictive information:

$$P(S_{t+1} \mid S_0, A_0, \ldots, S_t, A_t) = P(S_{t+1} \mid S_t, A_t).$$

This does **not** mean the next state is deterministic. A randomized push, noisy sensor, or uncertain contact can still produce a distribution of next states. It means that a correctly defined state carries the information needed to specify that distribution.

The main actor observation has 61 values:

```
base angular velocity          3
projected gravity              3
joint position                14
joint velocity                14
previous action               14
twist command                  3
head-pose command              4
body-pose command              6
                              --
total                         61
```

This makes the real problem partially observable. For example, the actor does not directly receive true base linear velocity. Previous actions and physical dynamics provide some short-term context without making the policy recurrent.

The formal extension is a Partially Observable Markov Decision Process (POMDP). It adds an observation model $O(o_t \mid s_t)$: the hidden state emits what the agent can measure. An ideal decision maker could maintain a **belief state**—a probability distribution over hidden states given the history:

$$b_t(s) = P(S_t = s \mid o_0, a_0, \dots, o_t).$$

A filter, recurrent network, or history encoder approximates this idea. Merely renaming o_t as "state" does not restore missing velocity, friction, delay, or terrain information.

1.5 Policy, trajectory, and return

A policy is a conditional distribution over actions:

$$\pi_\theta(a_t \mid o_t)$$

θ represents all neural-network weights. The vertical bar means "conditioned on," so the formula reads: "the probability of action a_t when the observation is o_t." During training, the actor defines a Gaussian distribution so it can explore nearby actions. A rollout is a trajectory:

$$\tau = (o_0, a_0, r_0, o_1, a_1, r_1, \dots)$$

The discounted return from time t is:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots$$

The capital sigma means "add all the following terms." With $\gamma = 0.99$, a reward one step later counts almost as much as a reward now, while very distant rewards gradually matter less. Discounting helps make long sums finite and expresses a preference for reliable, sooner outcomes.

The same definition gives a recurrence used in real rollout code. Peel off the first reward:

$$\begin{aligned} G_t &= r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots \\ &= r_t + \gamma(r_{t+1} + \gamma r_{t+2} + \cdots) \\ &= r_t + \gamma G_{t+1}. \end{aligned}$$

This is not a new assumption; it is the same sum regrouped. Implementations therefore compute all returns in one backward pass:

```
future = bootstrap_value
for t in reversed(range(len(rewards))):
    future = rewards[t] + gamma * future
    returns[t] = future
```

Run `examples/returns_and_occupancy.py` to see this recurrence produce [3.5, 3.0, 4.0] from rewards [2, 1, 4] at $\gamma = 0.5$.

Discounting, horizon, and physical time

If reward is a constant 1 forever and $0 \le \gamma < 1$, the return is a geometric series:

$$1 + \gamma + \gamma^2 + \cdots = \frac{1}{1-\gamma}.$$

At $\gamma = 0.99$, the total is 100. This motivates the rough **effective horizon** $1/(1-\gamma)$: about 100 policy steps. At 50 Hz that is about two seconds. It is only a rule of thumb—the weight after k steps is exactly γ^k—but it exposes a common error: copying the same γ between 20 Hz and 200 Hz changes the amount of physical future being valued.

To preserve an approximate continuous-time discount time constant τ, use

$$\gamma = e^{-\Delta t/\tau},$$

where Δt is the policy period. If $\tau = 2$ s and $\Delta t = 0.02$ s, $\gamma \approx 0.99005$. This derivation explains why `0.99` is plausible for a 50 Hz controller; it does not prove it is optimal for every reward design.

The learning objective is to find weights with high expected return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T-1} \gamma^t r_t\right]$$

Read this as: "adjust network weights θ to maximize the average discounted reward over trajectories sampled from the policy." "Expected" is important. Initial poses, commands, contact events, actuator parameters, and sampled actions vary. A policy should work across that distribution, not memorize one rollout.

The policy changes its own training distribution

Let $d_t^\pi(s) = P(S_t = s \mid \pi)$ be the probability of visiting state s at time t under policy π. A discounted state-occupancy distribution is

$$d_\gamma^\pi(s) = (1-\gamma)\sum_{t=0}^{\infty} \gamma^t d_t^\pi(s).$$

The leading factor normalizes the weights to sum to one. With this notation, the continuing objective can be viewed as reward averaged over the states and actions the policy itself visits:

$$J(\pi) = \frac{1}{1-\gamma}\mathbb{E}_{s \sim d_\gamma^\pi,\, a \sim \pi(\cdot|s)}[r(s,a)].$$

This equation explains several practical facts at once:

- changing the policy changes the data distribution;
- a rare recovery state supplies almost no gradient if resets and failures almost never reach it;
- behavior cloning can fail after one mistake because it leaves the expert's state distribution; and

- reverse-curriculum starts work by deliberately adding occupancy near the missing part of a maneuver.

The occupancy demo in the runnable example estimates this distribution in a two-state balance process. It is a small programmatic bridge between the probability definition and the state histograms used in robot evaluation.

1.6 Reward is a specification, not a feeling

The robot has no concept of "walk naturally" unless the measurable reward and termination conditions make natural walking the best available strategy. If a policy can earn more reward by shuffling, leaning, falling into a stable pose, or repeating a violent motion, PPO may discover exactly that.

A typical per-step reward is a weighted sum:

$$r_t = \sum_i w_i f_i(s_t, a_t, s_{t+1})$$

For example, a velocity-tracking term may be positive while action-rate and foot-slip terms are costs. The sign convention is a software contract, not a mathematical universal: some functions return a nonnegative cost and therefore need a negative weight; some Microduck penalty helpers already return a negative value and therefore need a positive weight. The observable invariant is that every logged penalty contribution should be at most zero.

The most common beginner mistake is to optimize a proxy and then judge the policy by the intended goal. Judge both:

- Does the logged reward term improve?
- Does the rollout visibly perform the desired behavior?
- Does it work across commands, resets, and randomized conditions?

Return-equivalent reward shaping—and its boundary

A shaping signal can make learning easier without changing which policy is optimal, but only under specific conditions. Potential-based shaping adds the difference of a scalar potential Φ between consecutive states:

$$F(s_t, s_{t+1}) = \gamma\Phi(s_{t+1}) - \Phi(s_t),$$

$$r'_t = r_t + F(s_t, s_{t+1}).$$

Summing its discounted contribution causes the intermediate potential terms to cancel:

$$\sum_{t=0}^{T-1} \gamma^t F_t = -\Phi(s_0) + \gamma^T \Phi(s_T).$$

For an infinite discounted task with bounded Φ, the last term vanishes. For a fixed start state, every policy's return shifts by the same constant, so the optimal policy is preserved. In finite episodes, terminal potentials and truncation handling must be defined carefully.

This gives a mathematical reason that "progress since last step" is often less farmable than paying for merely *being* in an intermediate pose. Arbitrary weighted bonuses do not receive this guarantee. The policy-invariance result originated in Ng, Harada, and Russell's 1999 reward-shaping analysis, linked in the source index.

Reward, constraint, and acceptance test are different

Keep three mechanisms separate:

Mechanism	Role	Example
reward	ranks behavior during optimization	track commanded velocity
constraint/supervisor	restricts allowed behavior	motor-current and tilt cutoff
acceptance metric	decides whether an artifact may advance	zero falls in a fixed test battery

A large negative reward for overcurrent is still something the optimizer can trade against positive reward. It cannot replace a current limiter. Conversely, a hard limiter alone gives no learning gradient until it activates.

1.7 Commands make one policy represent many behaviors

The velocity policy does not learn one fixed walk. A command c_t is part of the observation, so the policy is more accurately written as:

$$\pi_\theta(a_t \mid o_t, c_t)$$

Training samples many forward, lateral, turn, stand, and head-pose commands. Playback also resamples commands, which can look like a "random walk." The actions are not random at deployment: an application can deliberately set the command.

This distinction separates skill from intention:

```
planner or operator: where/why to move
          |
          | local velocity and pose command
          v
RL locomotion policy: how to move the joints stably
```

The current velocity task has no global waypoint, obstacle observation, or navigation memory. Those belong in an upper layer unless a new observation contract and task are intentionally designed.

1.8 Episodes and termination

Training divides experience into episodes. An episode resets when it reaches a time limit or a terminal failure condition. Resetting gives the policy fresh initial states and bounds the return.

Termination design changes the problem. Ending immediately on a fall teaches the locomotion policy to avoid falling. A stand-up policy must not terminate merely because it begins on the ground; the ground state is its task. This is why task templates differ even when they use the same robot.

Time limits need special care. A **termination** says the modeled task reached a true absorbing end such as failure or success. A **truncation** says data collection stopped for an external reason such as a fixed horizon. For a true terminal transition, future value is zero. For a time-limit truncation in a continuing task, a critic should usually bootstrap the value that would have followed. Treating every time limit as death systematically undervalues states near the horizon.

1.9 On-policy learning

PPO is on-policy: each update uses recent data sampled by the current or very recent policy. After several optimization epochs, that rollout is discarded and new behavior is collected.

Consequences:

- simulation throughput matters;
- a rare state may receive too little training data;
- curricula and reset distributions control what data the policy sees; and
- old demonstrations or checkpoints are not automatically replayed as an off-policy dataset.

Reverse-curriculum resets are useful when a maneuver learns its beginning but never reaches its final state: starting some episodes near completion creates on-policy data at the missing frontier.

1.10 A complete two-state derivation

Consider a toy balance controller with states `upright` (U) and `fallen` (F). `fallen` is terminal. In U, action `careful` gives reward 1 and stays upright with probability 0.9; action `fast` gives reward 2 but stays upright with probability 0.5. Let $\gamma = 0.9$.

If the policy always chooses `careful`, Bellman self-consistency gives

$$V^{careful}(U) = 1 + 0.9[0.9V^{careful}(U) + 0.1V(F)].$$

Since $V(F) = 0$:

$$V^{careful}(U) = 1 + 0.81V^{careful}(U),$$

$$(1 - 0.81)V^{careful}(U) = 1,$$

$$V^{careful}(U) = \frac{1}{0.19} \approx 5.263.$$

For always `fast`:

$$V^{fast}(U) = 2 + 0.9[0.5V^{fast}(U)],$$

$$V^{fast}(U) = \frac{2}{1 - 0.45} \approx 3.636.$$

The larger immediate reward loses because it causes earlier termination. This tiny example contains the full sequential problem: transition dynamics, discounted future, policy-dependent occupancy, and a reward that cannot be judged one step at a time.

An implementation does not solve these two equations symbolically for a continuous biped. It samples transitions and trains a critic to approximate the same self-consistency relationship. Chapter 3 turns that statement into Bellman and temporal-difference updates; Chapter 5 turns it into PPO.

1.11 From definition to environment code

Every mathematical object should have an identifiable software owner:

Mathematical object	Typical environment implementation	Evidence to inspect
$\mathcal{S}$	simulator state and hidden randomized parameters	model, scene, event configuration
o_t	observation-term functions and concatenation order	resolved shape, units, ranges
$\mathcal{A}$	action transform and limits	normalized-to-physical mapping
P	physics, actuator, delay, contact, randomization	timestep and identified parameters
R	reward-term functions and weights	per-term logs and sign tests

Mathematical object	Typical environment implementation	Evidence to inspect
initial distribution	reset events	spawn histograms
terminal/truncated flags	termination manager and time limit	bootstrapping behavior
π_θ	actor network and action distribution	checkpoint plus resolved config

Minimal environment pseudocode makes the mapping explicit:

```
def step(normalized_action):
    physical_target = home_pose + action_scale * normalized_action
    for _ in range(decimation):
        torque = actuator_model(physical_target, measured_joint_state)
        simulator.integrate(torque, physics_dt)

    observation = build_actor_observation(simulator, command)
    reward_terms = compute_reward_terms(simulator, command, normalized_action)
    terminated = unsafe_tilt_or_forbidden_contact(simulator)
    truncated = episode_steps >= time_limit_steps
    return observation, sum(reward_terms), terminated, truncated
```

This is not Microduck's exact application programming interface (API). It is a reading guide: if one line has no concrete counterpart in a project, the task definition is incomplete or hidden elsewhere.

1.12 Check your understanding

1. Why is the 3D velocity command part of the observation rather than part of the action?
2. If an obstacle is visible in the renderer but absent from the actor observation, can the policy deliberately avoid it?
3. What is the difference between a high instantaneous reward and a high expected return?
4. Why can a reward function produce behavior its author did not intend?
5. Which state should terminate a walking episode but be a valid initial state for stand-up training?
6. At 50 Hz, roughly how much physical time does the heuristic horizon $1/(1-\gamma)$ represent for $\gamma = 0.99$?
7. Derive $G_t = r_t + \gamma G_{t+1}$ from the discounted sum in one line of algebra. Why is this useful in code?
8. A continuing task is cut into 10-second log files. Is the file boundary a termination or a truncation? Should a value target normally bootstrap?
9. Why does a policy change the distribution of observations used to train its next update?
10. Design a potential $\Phi(s)$ for standing upright. State one way an incorrectly handled terminal potential could still change behavior.
11. In the two-state example, find the immediate reward r_{fast} at which the always-fast and always-careful policies have equal value.
12. Map "joint command arrives one cycle late" to the MDP/POMDP vocabulary.

Continue with the math and neural-network toolkit.

1.13 Folded solutions

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show answers to Section 1.12

1. The velocity command describes the behavior requested *of* the policy, so it must be an input. The action is the policy's response: fourteen immediate joint targets. If forward velocity were an action, the network would merely repeat the request instead of deciding how to move the mechanism.
2. No—not deliberately before contact. Pixels in a viewer are not numbers in the actor observation. The policy could learn a generic cautious gait or a post-contact reaction, but pre-contact avoidance needs an obstacle feature, map, range sensor, or an upper planner that changes the local command.
3. Instantaneous reward evaluates one transition. Expected return averages a discounted sequence of rewards over trajectories and uncertainty. A policy can accept one small immediate cost when it produces a much better future, or exploit a repeated small reward whose long-run sum is large.

4. A reward is a proxy written in code. Any unspecified orientation, contact, timing, or terminal condition may provide a cheaper way to score. The policy follows the mathematical signal, not the author's unstated visual intent.
5. A fallen body should normally terminate walking because continued fallen transitions are outside the skill. The same fallen pose is a legitimate reset state for stand-up, where recovery from it is the task.
6. The heuristic horizon is $1/(1 - 0.99) = 100$ steps. At 50 steps per second, that is roughly two seconds. The exact weight two seconds ahead is $0.99^{100} \approx 0.366$, so "horizon" is not a hard cutoff.
7. Factor one γ from every term after r_t:

$$G_t = r_t + \gamma(r_{t+1} + \gamma r_{t+2} + \cdots) = r_t + \gamma G_{t+1}.$$

   A backward loop then obtains every return in linear time without repeatedly summing the same suffix of rewards.
8. It is a truncation: the underlying control task could continue. The critic should normally bootstrap from the final observation. A true fall or task completion would instead set future value to zero.
9. Actions alter future states. A stable policy spends more samples upright; an unstable one creates more fall/recovery observations. Updating the policy therefore changes the occupancy distribution that generates the next training batch.
10. One potential is upright progress, such as $\Phi(s) = k\,\hat{z}_{body} \cdot \hat{z}_{world}$, possibly combined with a bounded height term. If a failure terminal is assigned a nonzero potential but the final $\gamma^T \Phi(s_T)$ term is silently dropped or treated inconsistently, terminating can receive an unintended bonus or penalty and policy invariance no longer follows from the telescoping argument.
11. Always-fast has value $r_{fast}/(1 - 0.9 \times 0.5)$. Set it equal to $1/(1 - 0.9 \times 0.9)$:

$$\frac{r_{fast}}{0.55} = \frac{1}{0.19} \quad \Rightarrow \quad r_{fast} = \frac{0.55}{0.19} \approx 2.895.$$

   Below about 2.895, careful has greater return; above it, the immediate reward compensates for the greater fall probability under this model.
12. The delay belongs to transition dynamics and observation history. If the current state includes the queued prior action, the augmented process can be Markov. If the actor cannot observe the queue/delay, it faces partial observability. Adding previous action and history can expose evidence, while randomizing delay trains robustness rather than revealing its exact value.

2. Math and Neural-Network Toolkit

Deep reinforcement learning combines probability, calculus, linear algebra, and optimization. You do not need to master each subject before beginning. You do need to know what the symbols mean, what shape each quantity has, and what an equation asks the computer to do.

This chapter builds that minimum toolkit. Keep a pencil nearby: predicting a number before running code is much more educational than only reading it.

Where this toolkit came from

Deep reinforcement learning (RL) did not invent its mathematics. Bellman's dynamic programming uses probability and recursive value equations; Robbins and Monro's 1951 stochastic approximation explains learning from noisy samples; likelihood-ratio estimators make policy gradients possible; and reverse-mode automatic differentiation turns the chain rule into practical neural-network training. Modern libraries compose these ideas at tensor scale.

The goal of this chapter is therefore not to survey all of calculus or linear algebra. It is to build a trace from every symbol used later to an operation a program performs, including its shape, units, estimate error, and failure mode.

2.1 Scalars, vectors, matrices, and tensors

A **scalar** is one number, such as the reward at one instant:

$$r_t = 1.7.$$

A **vector** is an ordered list. A simple mobile-robot command might be

$$c_t = [v_x, v_y, \omega_z] = [0.2, 0.0, -0.4].$$

The order is part of the interface. Swapping v_x and v_y does not merely rename two values; it asks for a different physical motion.

A **matrix** is a rectangular array. A neural-network layer maps an input vector x to an output vector z using a weight matrix W and bias b:

$$z = Wx + b.$$

If x has 61 values and the layer has 512 neurons, then

$$x \in \mathbb{R}^{61}, \quad W \in \mathbb{R}^{512\times 61}, \quad b \in \mathbb{R}^{512}, \quad z \in \mathbb{R}^{512}.$$

Read $\mathbb{R}^{61}$ as "a vector of 61 real numbers." The matrix shape says 512 rows, one per output, and 61 columns, one per input. Shape reasoning catches many bugs before training.

A **tensor** is the programming term for a multidimensional numeric array. A Microduck rollout tensor might have shape

```
(time steps, parallel environments, observation values)
             (24, 4096, 61)
```

The word tensor is often used casually in deep-learning code; here it does not require advanced tensor calculus.

Units are an invisible part of every vector

Shape alone is not enough. A value might be radians, radians per second, meters, normalized units, or a Boolean encoded as 0/1. Write interfaces as both shape and meaning:

```
base angular velocity: shape (3,), unit rad/s, expressed in body frame
projected gravity:      shape (3,), dimensionless, body frame
joint position delta:  shape (14,), unit rad, relative to HOME
```

This discipline matters because a numerically valid 61-vector with the wrong units or frame still produces a physically wrong action.

Dot products, norms, and batches

The **dot product** combines two equal-length vectors into one scalar:

$$x^T y = \sum_{i=1}^{n} x_i y_i.$$

It appears in every neural layer and many robot costs. If $x = [1, 2, -1]$ and $y = [3, 0, 4]$, then $x^T y = 3 + 0 - 4 = -1$.

The Euclidean norm measures vector magnitude:

$$\|x\|_2 = \sqrt{x^T x} = \sqrt{\sum_i x_i^2}.$$

A squared tracking error $\|v - v^*\|_2^2$ therefore means "square each component error and add." It grows quadratically and makes one large miss more expensive than several small ones with the same absolute sum. The L_1 norm, $\|x\|_1 = \sum_i |x_i|$, grows linearly and has a corner at zero. Which one is used changes both optimization gradients and outlier sensitivity.

For a batch matrix $X \in \mathbb{R}^{B \times 61}$ and a layer stored as $W \in \mathbb{R}^{512 \times 61}$, frameworks commonly evaluate

$$Z = XW^T + \mathbf{1}b^T,$$

so $Z \in \mathbb{R}^{B \times 512}$. The bias is **broadcast**—logically copied across the B rows. A layer with 61 inputs and 512 outputs has $61 \times 512 + 512 = 31{,}744$ learned scalar parameters.

2.2 Functions and parameters

A **function** maps an input to an output. A policy is a function:

$$a_t = \pi_\theta(o_t).$$

- o_t is the observation at time t;
- a_t is the action;
- π is the policy; and
- θ is the collection of trainable weights and biases.

The subscript θ means changing those parameters changes the function. Training searches for parameters that make useful trajectories more likely.

A **hyperparameter** is chosen by the experimenter rather than learned by the optimizer: learning rate, discount factor, network width, Proximal Policy Optimization (PPO) clip value, or rollout length are examples.

2.3 Probability is how reinforcement learning (RL) represents uncertainty

Robots face uncertainty from sensor noise, contact variation, delayed state, random commands, randomized simulation, and exploration.

A **random variable** is a numeric outcome of an uncertain process. We write

$$X \sim p(x)$$

as "X is sampled from probability distribution p." A Gaussian (normal) distribution is written

$$X \sim \mathcal{N}(\mu, \sigma^2),$$

where μ is the mean and σ is the standard deviation. Larger σ means more spread.

During training, a continuous policy commonly produces a mean action and a standard deviation, then samples:

$$a_t \sim \pi_\theta(\cdot \mid o_t) = \mathcal{N}(\mu_\theta(o_t), \mathrm{diag}(\sigma^2)).$$

Read this as: "given observation o_t, the network proposes the center of an action distribution, and an action is sampled from it." During deployment, the deterministic mean is often used instead.

Conditional probability

$p(a \mid o)$ means "the probability of action a given observation o." The vertical bar means "conditioned on." A policy must respond differently to a tilting-left observation and a tilting-right observation, so it represents a conditional distribution rather than one fixed action distribution.

Joint and conditional probability are related by the product rule:

$$p(x, y) = p(x \mid y)p(y) = p(y \mid x)p(x).$$

Rearranging gives Bayes' rule:

$$p(x \mid y) = \frac{p(y \mid x)p(x)}{p(y)}.$$

In state estimation, $p(x)$ is a prior belief about robot state, $p(y \mid x)$ is a sensor likelihood, and $p(x \mid y)$ is the posterior after observing the measurement. A learned recurrent actor need not explicitly calculate this fraction, but its history-processing role is related: infer hidden conditions from their observable effects.

Expectation

The **expectation** $\mathbb{E}[X]$ is a probability-weighted average. For a six-sided die,

$$\mathbb{E}[X] = \frac{1 + 2 + 3 + 4 + 5 + 6}{6} = 3.5.$$

No roll produces 3.5. Expectation describes the long-run average, not a guaranteed single result. Likewise, maximizing expected return does not guarantee that every robot episode succeeds. Tail failures need explicit measurement and safety handling.

Variance

Variance measures spread around the mean:

$$\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2].$$

Two policies can have the same mean return while one fails violently in 10% of trials. Reporting only the mean hides that distinction.

For a finite distribution with outcomes x_i and probabilities p_i:

$$\mu = \sum_i p_i x_i, \qquad \mathrm{Var}(X) = \sum_i p_i (x_i - \mu)^2.$$

Standard deviation is $\sigma = \sqrt{\mathrm{Var}(X)}$ and has the original unit. Variance of an angle measured in radians has units radian squared; standard deviation is again in radians.

Two variables can vary together. Their covariance is

$$\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])].$$

A diagonal Gaussian policy assumes action noise has zero cross-joint covariance after conditioning on the observation. The neural mean can still coordinate joints; the simplifying assumption concerns sampled residual noise. Full-covariance policies can represent correlated exploration but require more parameters and stable matrix operations.

From population expectation to a sample estimate

The objective contains expectations over distributions we cannot enumerate. With N sampled values $x_1, \ldots, x_N$, the Monte Carlo estimate is

$$\hat{\mu} = \frac{1}{N} \sum_{i=1}^{N} x_i.$$

If samples are independent with variance σ^2, then $\mathrm{Var}(\hat{\mu}) = \sigma^2/N$ and the standard error scales as $1/\sqrt{N}$. Four times as many independent samples roughly halves this sampling uncertainty, not quarters it. Robot trajectory samples are correlated within episodes, so treating every time step as independent can greatly overstate confidence; evaluation usually aggregates by seed, episode, or hardware trial.

2.4 Why logarithms appear in policy gradients

For independent events, probabilities multiply. Products of many numbers smaller than one become numerically tiny. Logarithms turn products into sums:

$$\log(xy) = \log x + \log y.$$

The log-probability of a sampled action is convenient to differentiate. A key identity is

$$\nabla_\theta \log \pi_\theta(a \mid o) = \frac{\nabla_\theta \pi_\theta(a \mid o)}{\pi_\theta(a \mid o)}.$$

You do not need to memorize the fraction yet. The practical idea is that the gradient can adjust the probability of actions that were actually sampled. Positive advantage pushes their log-probability up; negative advantage pushes it down.

For one scalar Gaussian action, the density is

$$\pi(a \mid o) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left[-\frac{(a-\mu)^2}{2\sigma^2}\right].$$

Taking its logarithm turns multiplication into addition:

$$\log \pi(a \mid o) = -\frac{1}{2}\left[\frac{(a-\mu)^2}{\sigma^2} + 2\log\sigma + \log(2\pi)\right].$$

Differentiate with respect to the mean:

$$\frac{\partial}{\partial \mu} \log \pi(a \mid o) = \frac{a-\mu}{\sigma^2}.$$

If the sampled action is above the mean, increasing the mean makes it more likely; if it is below, the gradient points downward. Smaller σ makes the same deviation more surprising and produces a larger score magnitude. This local derivative is the concrete mechanism behind "increase the probability of an advantageous action."

For an independent d-dimensional diagonal Gaussian, implementations sum one such log-probability per action dimension. Run `examples/gaussian_policy_math.py` to compare the analytic derivative with a finite difference and to calculate Gaussian entropy.

2.5 Derivatives: sensitivity to change

The derivative of a scalar function says how its output changes for a small input change. If

$$y = x^2,$$

then

$$\frac{dy}{dx} = 2x.$$

At $x = 3$, the derivative is 6. Increasing x by approximately 0.01 increases y by approximately $6 \times 0.01 = 0.06$.

A **partial derivative** changes one input while holding others fixed. For

$$f(x, y) = x^2 + 3xy,$$

$$\frac{\partial f}{\partial x} = 2x + 3y, \qquad \frac{\partial f}{\partial y} = 3x.$$

A **gradient** collects all partial derivatives:

$$\nabla f = \begin{bmatrix} \partial f/\partial x \\ \partial f/\partial y \end{bmatrix}.$$

The gradient points toward the locally steepest increase. To minimize a loss $L(\theta)$, gradient descent takes a small step in the opposite direction:

$$\theta_{new} = \theta_{old} - \alpha \nabla_\theta L,$$

where α is the **learning rate**, the step-size hyperparameter.

When both input and output are vectors, derivatives form a **Jacobian**. For $f : \mathbb{R}^n \to \mathbb{R}^m$:

$$J_{ij} = \frac{\partial f_i}{\partial x_j}, \qquad J \in \mathbb{R}^{m \times n}.$$

Robot kinematics uses a Jacobian to map joint velocity to end-effector velocity. Neural-network backpropagation also multiplies Jacobian-vector products, but automatic differentiation avoids materializing every huge matrix.

Numerical example

Let $L(w) = (w - 4)^2$ and start at $w = 1$.

$$\frac{dL}{dw} = 2(w - 4) = -6.$$

With $\alpha = 0.1$:

$$w_{new} = 1 - 0.1(-6) = 1.6.$$

The update moves w toward 4. A huge learning rate could overshoot; a tiny one could make progress impractically slow.

2.6 The chain rule and backpropagation

Neural networks compose functions. Suppose

$$u = wx, \qquad y = \tanh(u), \qquad L = (y - y^*)^2.$$

The **chain rule** multiplies local sensitivities along the path:

$$\frac{dL}{dw} = \frac{dL}{dy}\frac{dy}{du}\frac{du}{dw}.$$

Substitute each derivative:

$$\frac{dL}{dw} = 2(y - y^*)\left(1 - \tanh^2(u)\right)x.$$

Backpropagation is an efficient application of this chain rule through a computation graph. Automatic differentiation libraries such as PyTorch compute it, but understanding the path helps diagnose detached tensors, exploding gradients, saturation, and unintended objectives.

2.7 A neural network is a learned nonlinear function

A feed-forward layer is

$$h = \phi(Wx + b),$$

where ϕ is an **activation function** applied element by element. Without nonlinear activations, many stacked linear layers collapse to one linear map.

Common activations include:

Activation	Formula or behavior	Practical note
rectified linear unit (ReLU)	$\max(0, x)$	cheap; negative side has zero gradient
exponential linear unit (ELU)	x if positive, smooth negative saturation otherwise	used by the Microduck actor/critic
tanh	maps to (–1, 1)	useful for bounded outputs; can saturate
sigmoid	maps to (0, 1)	common for probabilities or gates

The Microduck actor is conceptually

```
61 observations -> 512 ELU -> 256 ELU -> 128 ELU -> 14 action means
```

The numbers 512, 256, and 128 are hidden-layer widths. This network is not a database of motions. Its weights define a smooth high-dimensional mapping from observations and commands to actions.

2.8 Loss, objective, reward, and return are different

These words are easy to mix up:

- **reward**: one scalar emitted by the environment at one step;
- **return**: a discounted sum of future rewards along a trajectory;
- **objective**: the quantity the algorithm conceptually wants to maximize;
- **loss**: a quantity the optimizer minimizes in code.

An implementation may minimize the negative policy objective, add a value loss, and subtract an entropy bonus. Seeing a negative sign in code does not by itself mean the robot is punished.

2.9 Stochastic gradient descent, minibatches, and epochs

Computing a gradient over all possible trajectories is impossible. RL estimates it from sampled experience.

A **batch** is the collected set of samples. A **minibatch** is one subset used for an optimizer step. An **epoch** is one pass through the batch. PPO can reuse one on-policy rollout for several epochs, but too much reuse makes the updated policy drift away from the policy that generated the data.

Optimizers such as Adam maintain moving estimates of gradient scale. Adam can make training easier, but it does not repair a wrong reward, missing observation, impossible target, or simulator error.

Why a sampled gradient can still be useful

Let g_i be a gradient contribution from sample i. A minibatch estimator is

$$\hat{g} = \frac{1}{B} \sum_{i=1}^{B} g_i.$$

If the sampling procedure matches the objective, $\mathbb{E}[\hat{g}]$ can equal the desired gradient even though any one minibatch is noisy. Stochastic gradient descent follows these noisy directions repeatedly. In RL, the match is delicate: trajectories come from a policy, adjacent steps are correlated, and reusing old data changes the distribution. Algorithm design is partly the art of constructing a gradient estimator whose bias and variance remain manageable.

Gradient clipping and parameter update are not action clipping

Global gradient-norm clipping rescales a parameter gradient when $\|g\|_2$ exceeds threshold c:

$$g_{used} = g \min\left(1, \frac{c}{\|g\|_2}\right).$$

This limits an optimizer step caused by an unusually large batch. It does not bound the robot action, motor current, or next network output. Action bounds, target rate limits, and hardware supervisors are separate layers.

2.10 Normalization and numerical scale

Suppose one observation ranges around 0.01 and another around 1000. The same weight scale treats them very differently. Observation normalization maintains running mean μ and variance σ^2, then computes approximately

$$\hat{o} = \frac{o - \mu}{\sqrt{\sigma^2 + \epsilon}}.$$

ϵ is a small constant that avoids division by zero. Normalization can improve optimization, but it becomes part of the learned input contract. If training uses $\hat{o}$ and deployment feeds raw o, the actor receives a different problem. That is why Microduck's export path bakes the normalizer into Open Neural Network Exchange (ONNX).

Reward scaling also matters. Multiplying every reward by 100 leaves the mathematical optimal policy unchanged in an ideal tabular setting, but it changes gradient magnitude, value targets, clipping interactions, and numerical behavior in a practical deep-RL implementation.

Numerical stability is part of the algorithm

Mathematically equivalent expressions need not behave equally in finite precision. For example, directly computing $\log(e^{x_1} + e^{x_2})$ can overflow for large x_i. The stable log-sum-exp identity subtracts the maximum m:

$$\log \sum_i e^{x_i} = m + \log \sum_i e^{x_i - m}, \qquad m = \max_i x_i.$$

Likewise, policy code stores `log_std`, adds epsilon before square roots, and often computes probability ratios from differences of log-probabilities:

$$\frac{\pi_{new}(a \mid s)}{\pi_{old}(a \mid s)} = \exp[\log \pi_{new}(a \mid s) - \log \pi_{old}(a \mid s)].$$

These are not cosmetic implementation tricks. A not-a-number (NaN) value in one parallel environment can contaminate normalization statistics and then an entire policy update.

2.11 Bias and variance: a recurring tradeoff

An estimator has **bias** when its average estimate is systematically shifted. It has **variance** when estimates fluctuate strongly across samples.

- Monte Carlo return uses a complete sampled future: low modeling bias, often high variance.
- A one-step temporal-difference target bootstraps from a learned estimate: potentially more bias, usually lower variance.
- Generalized Advantage Estimation introduces λ to move along this tradeoff.

"Unbiased" does not automatically mean "better." A very noisy gradient may need so much data that a slightly biased, lower-variance estimator learns more reliably.

Approximation, estimation, and optimization error

When a learned policy fails, separate three sources:

1. **approximation error**: the chosen network/function family cannot represent the needed mapping;
2. **estimation error**: finite, noisy, or poorly covered data cannot identify the best representable mapping; and
3. **optimization error**: the optimizer did not find good parameters even for the available data and model class.

Adding a larger network targets approximation error but can worsen estimation or optimization. Collecting diverse resets targets coverage, not network capacity. Lowering a learning rate targets optimization dynamics, not a missing camera input. This decomposition prevents "tune the neural network" from becoming a universal but untestable explanation.

2.12 A tiny gradient check

For a differentiable scalar function, compare the analytic derivative with a finite difference:

```
def f(w: float) -> float:
    return (w - 4.0) ** 2

w = 1.0
eps = 1e-5
finite_difference = (f(w + eps) - f(w - eps)) / (2.0 * eps)
analytic = 2.0 * (w - 4.0)
print(finite_difference, analytic)
```

They should be close to –6. Finite differences are slow in large networks, but the idea is valuable for checking a new reward derivative or small custom operation.

2.13 Exercises

1. A batch has shape `(24, 4096, 61)`. How many observation scalars does it contain?
2. A Gaussian policy has $\mu = 0.3$ and $\sigma = 0.1$. Is action 0.31 more or less typical than action 0.8? Why?
3. Compute one gradient-descent step for $L(w) = (w + 2)^2$, starting at $w = 1$ with learning rate 0.25.
4. Why can two policies with equal expected return have different hardware safety risk?
5. What breaks if an exported policy receives joint angles in degrees while training used radians?
6. Explain backpropagation without using the phrase "the computer learns."
7. Why does an observation normalizer belong in the deployment contract?
8. A linear layer maps 61 inputs to 512 outputs. Derive its parameter count, including bias. What is the output batch shape for 4,096 observations?
9. Compute the dot product and Euclidean norm of $x = [3, 4]$. Give one robot interpretation of each.
10. A return distribution is 0 with probability 0.1 and 10 with probability 0.9. Compute its expectation, variance, and standard deviation. Why does the expectation alone hide an important fact?
11. For a scalar Gaussian with $\mu = 0$, $\sigma = 0.5$, and sampled action $a = 0.25$, compute $\partial \log \pi / \partial \mu$. If advantage is positive, which way will a policy-gradient update tend to move μ?
12. Assuming independent samples, by what factor must the sample count grow to reduce standard error by a factor of three? Why may robot time steps not satisfy the independence assumption?
13. A gradient is $g = [6, 8]$ and global norm threshold is 5. Compute the clipped gradient. Does this bound the motor command?
14. Classify each proposed fix by its primary target: add history for hidden backlash; collect more rough-terrain resets; widen a network that cannot represent a discontinuous gate; lower a diverging optimizer step.

The small bandit example in `examples/bandit_incremental_mean.py` uses expectation, sampling, and an incremental mean without any neural network. Run it before continuing with Bellman methods from tables to a Deep Q-Network (DQN).

2.14 Folded solutions

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show answers to Section 2.13

1. Multiply every axis:

$$24 \times 4096 \times 61 = 5{,}996{,}544$$

   These are scalar entries, not independent trajectories; the batch still has 24 correlated time positions per environment.

2. Action 0.31 is only 0.01/0.1 = 0.1 standard deviations from the mean. Action 0.8 is (0.8 – 0.3)/0.1 = 5 standard deviations away, so 0.31 is vastly more typical under that Gaussian.

3. The gradient at $w = 1$ is $2(w + 2) = 6$. Gradient descent gives $w_{new} = 1 - 0.25(6) = -0.5$. A direct check is:

```
w = 1.0
learning_rate = 0.25
gradient = 2.0 * (w + 2.0)
w -= learning_rate * gradient
assert w == -0.5
```

4. Equal means do not imply equal distributions. One policy may score consistently while another mixes excellent trials with rare destructive failures. Report quantiles, failure categories, interventions, and raw trials in addition to expected return.

5. One radian is about 57.3 degrees. Feeding degree-valued numbers into a model trained on radians changes its input scale and distribution, often driving normalized features far outside training experience and producing unsafe actions.

6. Backpropagation applies the chain rule from the final loss toward earlier operations. It multiplies each downstream sensitivity by each local derivative, accumulating how a small change in every weight would change the loss. The optimizer then uses those gradients to update the weights.

7. The actor learned a function of normalized observations, not raw sensor numbers. Mean, variance, clipping, epsilon, order, and units therefore form part of the deployed function and must travel with or be embedded in the exported model.

8. There are $61 \times 512 = 31{,}232$ weights and 512 biases, for 31,744 parameters. A batch $X \in \mathbb{R}^{4096\times61}$ produces $Z \in \mathbb{R}^{4096\times512}$.

9. The dot product with itself is $x^T x = 3^2 + 4^2 = 25$; the norm is $\sqrt{25} = 5$. A dot product between unit body-up and world-up measures orientation alignment. A norm can measure the magnitude of a velocity or tracking-error vector.

10. The mean is $0.1(0) + 0.9(10) = 9$. The variance is

$$0.1(0 - 9)^2 + 0.9(10 - 9)^2 = 8.1 + 0.9 = 9,$$

    so the standard deviation is 3. The mean of 9 conceals a 10% complete- failure rate, which could be unacceptable on hardware.

11. The score is $(a - \mu)/\sigma^2 = 0.25/0.25 = 1$. With positive advantage, gradient ascent tends to increase the action's log-probability, moving the mean upward toward the sampled value (subject to other samples/loss terms).

12. Standard error scales as $1/\sqrt{N}$. Reducing it by three requires $3^2 = 9$ times as many independent samples. Consecutive robot steps share state, command, terrain, and episode conditions, so they are correlated; the effective independent sample count is smaller than the transition count.

13. The original norm is $\sqrt{6^2 + 8^2} = 10$. Scale by 5/10 to obtain $g_{used} = [3, 4]$. This bounds the parameter-update gradient norm only. Motor commands require their own action transform, range/rate limits, and safety checks.

14. History primarily targets partial observability/representation; rough-terrain resets target estimation coverage; widening the network targets approximation capacity; lowering the step targets optimization stability. Each diagnosis should still be tested because the categories can interact.

3. Bellman Methods: From Tables to Deep Q-Learning

Before Proximal Policy Optimization (PPO), learn the simpler idea that organizes much of reinforcement learning (RL): the value of a decision is its immediate reward plus the value of what follows. This is the **Bellman principle**.

The tabular setting in this chapter is deliberately small. When every state and action can be listed, the algorithms expose their logic without a neural network hiding mistakes.

Historical thread: one recursive idea, several data regimes

Bellman's 1957 dynamic programming assumed a known model and decomposed a long optimization into one-step subproblems. Monte Carlo methods estimated outcomes from complete samples. Temporal-difference methods learned a Bellman fixed point from incomplete experience. Watkins's 1989 thesis and the 1992 Watkins–Dayan paper established Q-learning's off-policy control rule in the tabular setting. The 2013 Deep Q-Network (DQN) work combined Q-learning with a convolutional network, replay, and a target network at influential scale.

The changes are not just "use a bigger network." Each step relaxes an assumption—known model, enumerable state, on-policy data—while introducing a new source of approximation or instability. The primary records are indexed in SOURCES.md.

3.1 Start with a bandit: action without state transition

A **multi-armed bandit** has several actions ("arms"). Each arm produces reward from an unknown distribution. There is no changing state and no delayed consequence. The learner must balance:

- **exploration**: try uncertain actions to gain information; and
- **exploitation**: choose the action currently estimated to be best.

If action a has been selected $N(a)$ times, an incremental sample-average estimate is

$$Q_{new}(a) = Q_{old}(a) + \frac{1}{N(a)}\left(r - Q_{old}(a)\right).$$

Read the term in parentheses as **prediction error**: observed reward minus the old prediction. The $1/N(a)$ step size shrinks as evidence accumulates.

An ϵ-greedy policy chooses a random action with probability ϵ and the estimated best action otherwise. If $\epsilon = 0.1$, about 10% of decisions explore.

Robotics connection: choosing among calibration routines or high-level skills can resemble a contextual bandit, but balancing a moving robot is sequential. An action changes the next state and therefore future choices.

3.2 State value and action value

For a policy π, the **state-value function** is the expected discounted return starting from state s:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s].$$

The **action-value function**, usually called the Q-function, also fixes the first action:

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a].$$

Plain language:

- $V^{\pi}(s)$ asks, "How good is this situation if I continue with policy π?"
- $Q^{\pi}(s, a)$ asks, "How good is taking this action here, then continuing with π?"

These are expectations over future transition randomness and future policy sampling. They are not promises for one rollout.

3.3 The Bellman expectation equation

Split the return into the next reward and the remaining future:

$$G_t = R_{t+1} + \gamma G_{t+1}.$$

Taking expectations gives

$$V^{\pi}(s) = \sum_{a} \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V^{\pi}(s') \right].$$

Do not let the sums obscure the idea:

1. consider each action the policy might take;
2. consider each next state and reward the environment might produce;
3. add immediate reward to discounted next-state value; and
4. average using their probabilities.

This is a self-consistency equation: a correct value estimate agrees with a one-step lookahead using itself.

Matrix form: policy evaluation is a linear system

For a finite Markov Decision Process (MDP) and a fixed policy, collect all state values into vector v, expected one-step rewards into r_{π}, and policy-induced transition probabilities into matrix P_{π}. Then

$$v = r_{\pi} + \gamma P_{\pi} v.$$

Move the value term to the left:

$$(I - \gamma P_{\pi}) v = r_{\pi},$$

and, when the inverse exists,

$$v = (I - \gamma P_{\pi})^{-1} r_{\pi}.$$

I is the identity matrix. This exact solve is practical only for small problems, but it reveals what iterative methods approximate: a fixed point of a linear operator. In a million-dimensional or continuous robot state, storing P_{π} is impossible, while sampling one transition is easy.

Numerical example

Suppose a state has one action. It gives reward 2 and moves deterministically to a state worth 5. With $\gamma = 0.9$:

$$V(s) = 2 + 0.9 \times 5 = 6.5.$$

If the next state were terminal, its future value would be zero and the answer would be 2.

3.4 Bellman optimality

The optimal action-value function chooses the best next action:

$$Q^*(s,a) = \sum_{s',r} p(s', r \mid s, a)\left[r + \gamma \max_{a'} Q^*(s', a')\right].$$

Once Q^* is known, a greedy optimal policy chooses

$$\pi^*(s) = \arg\max_a Q^*(s,a).$$

arg max returns the action producing the largest value, not the value itself.

3.5 Dynamic programming: when the model is known

Dynamic programming (DP) assumes the transition/reward model is known. Value iteration repeatedly applies an optimal Bellman backup:

$$V_{k+1}(s) = \max_a \sum_{s',r} p(s', r \mid s, a)\left[r + \gamma V_k(s')\right].$$

Repeated backups propagate information backward from goals and hazards.

Why discounted Bellman iteration converges in the tabular case

Define the Bellman optimality operator

$$(TV)(s) = \max_a \mathbb{E}[R_{t+1} + \gamma V(S_{t+1}) \mid s, a].$$

For two bounded value functions V and W, the maximum and expectation cannot amplify their largest difference beyond the discount:

$$\|TV - TW\|_\infty \le \gamma \|V - W\|_\infty.$$

This makes T a **contraction** when $\gamma < 1$. After k exact sweeps,

$$\|V_k - V^*\|_\infty \le \gamma^k \|V_0 - V^*\|_\infty.$$

The error shrinks geometrically toward a unique fixed point. This clean result depends on exact tabular backups. A neural network update changes many values at once and may use off-policy sampled targets, so the proof does not transfer unchanged to deep Q-learning.

Run:

```
python examples/gridworld_value_iteration.py
```

The program has the full grid transition model, so it can evaluate every one-step outcome without collecting experience. In most real robotics tasks, the exact stochastic model is unknown and continuous state makes a table impossible. DP still provides the conceptual reference.

3.6 Monte Carlo learning: wait for the outcome

Monte Carlo (MC) methods estimate value from complete sampled returns. If a state is visited and the later rewards are 1, 0, 3, with $\gamma = 0.9$:

$$G_t = 1 + 0.9(0) + 0.9^2(3) = 3.43.$$

The estimate can move toward 3.43 after the episode finishes. MC does not need a transition model and does not bootstrap from its own current value estimate. But it must wait for an outcome and can have high variance.

The sample return is unbiased for the value of the policy under the sampled start condition, but a single rollout mixes every future source of randomness. Long robot episodes can therefore give extremely noisy credit. Truncating an episode and incorrectly setting the remaining return to zero adds bias of a different kind.

3.7 Temporal-difference learning: learn before the episode ends

Temporal-difference (TD) learning updates from one transition:

$$V(S_t) \leftarrow V(S_t) + \alpha\delta_t,$$

where

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t).$$

δ_t is the **TD error**:

```
new one-step target - old prediction
```

The next state's current estimate supplies the unfinished future. Reusing an estimate inside its own target is called **bootstrapping**.

Numerical example

Let $V(S_t) = 4$, reward $R_{t+1} = 1$, $V(S_{t+1}) = 6$, $\gamma = 0.9$, and $\alpha = 0.1$.

$$\delta_t = 1 + 0.9(6) - 4 = 2.4,$$

$$V(S_t) \leftarrow 4 + 0.1(2.4) = 4.24.$$

The experience was better than predicted, so the estimate rises.

The spectrum between one-step TD and Monte Carlo

An n-step target uses n observed rewards and then bootstraps:

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n V(S_{t+n}).$$

- $n = 1$ is the one-step temporal-difference (TD) target.
- n reaching the true terminal state is a Monte Carlo target.
- Intermediate n trades a longer piece of real outcome against a later learned bootstrap.

The forward-view λ-return mixes all n-step returns:

$$G_t^\lambda = (1-\lambda)\sum_{n=1}^{\infty}\lambda^{n-1}G_t^{(n)}.$$

Eligibility traces compute an equivalent backward credit signal online in the tabular continuing case. Generalized Advantage Estimation in Chapter 5 uses the same geometric weighting idea on TD residuals. This is an example of an old mechanism surviving inside a newer deep algorithm.

3.8 State–Action–Reward–State–Action (SARSA) and Q-learning

SARSA is an on-policy TD control method. Its name lists the transition tuple: $S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}$. Its update target follows the action the behavior policy actually selects:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\right].$$

Q-learning is off-policy. Its target assumes the greedy next action even if the behavior policy explored:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\right].$$

On-policy means learning about the behavior policy generating the data. **Off-policy** means the policy being learned can differ from the behavior policy. Off-policy learning enables data reuse but introduces harder stability and distribution-shift problems.

Run:

```
python examples/tabular_q_learning.py
```

Try three values of ϵ. Record both final success and the number of episodes needed. Exploration is not free, and no single setting is universally best.

The implementation maps directly to the equation:

```
next_best = 0.0 if done else max(q[(next_state, a)] for a in ACTIONS)
td_error = reward + GAMMA * next_best - q[(state, action)]
q[(state, action)] += ALPHA * td_error
```

The first line implements the terminal-aware bootstrap. The second constructs the bracketed Q-learning error. The third applies the step size α. Read the complete runnable mapping in `examples/tabular_q_learning.py`, then alter only the target to use the actually selected next action to obtain SARSA.

In the classic cliff example, that difference matters. Q-learning learns the greedy target path even while exploratory behavior sometimes falls from it; SARSA evaluates its exploratory behavior and can prefer a safer path farther from the cliff. Neither label means “safe”: the result depends on reward, exploration, and environment.

3.9 From Q-table to Deep Q-Network

A robot camera produces too many possible observations for a table. A Deep Q-Network (DQN) approximates all discrete action values with a neural network:

$$Q_\theta(o, a).$$

The squared TD loss for one sample is

$$L(\theta) = \left(r + \gamma \max_{a'} Q_\theta(o', a') - Q_\theta(o, a)\right)^2.$$

$\bar{\theta}$ denotes a slowly updated **target network**. Two important DQN mechanisms are:

- **experience replay**: store transitions and sample shuffled minibatches, which reuses data and reduces adjacent-sample correlation;
- **target network**: hold the target parameters fixed temporarily so the prediction does not chase a target changing on every optimizer step.

The original DQN paper demonstrated value learning from pixels on Atari. That is historically important evidence for discrete-action domains; it is not evidence that DQN is the natural choice for 14 simultaneous continuous joint targets.

One DQN update, from tensor to optimizer

For a replay minibatch of size B, a practical update is:

```
with no_gradient():
    next_q = target_network(next_observation).max(axis=1)
    target = reward + gamma * (1.0 - terminated) * next_q

all_q = online_network(observation)           # shape (B, number_of_actions)
chosen_q = gather(all_q, action_index)        # shape (B,)
loss = mean(huber_loss(chosen_q, target))
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

`gather` is the code counterpart of selecting $Q(s, a)$ for the action stored in each transition. The maximum selects the target action. `no_gradient` and the target network prevent the target branch from being optimized in the same step. A Huber loss is quadratic near zero and linear for large errors, reducing the influence of rare huge temporal-difference targets relative to pure squared loss.

The official CleanRL DQN implementation is a compact executable reference; the original DeepMind DQN result should be read from the paper because later repositories contain many extensions.

What followed DQN, and which problem each change addresses

Several influential extensions are often bundled into “modern DQN”:

Extension	Problem targeted	Core change
Double DQN	maximization overestimation	select next action online, evaluate it with target network
prioritized replay	many stored transitions have little current learning signal	sample larger-error transitions more often and importance-correct
dueling network	action values share a large state-value component	estimate value and action advantage streams
multi-step return	reward propagates slowly through one-step backups	include several observed rewards before bootstrapping
distributional RL	an expectation hides outcome structure	predict a return distribution, not only its mean
noisy networks	hand-designed ϵ schedule	learn parameter-space exploration noise

Rainbow combined six such components and tested their interaction on Atari. These are alternatives within deep **discrete** value learning, not a universal frontier for continuous robotics. As of 2026, value distributions, representation learning, large-scale replay, and offline value learning remain active ingredients, while continuous motor control usually uses an actor, planning, or action-generation model to avoid enumerating actions.

3.10 Why continuous robot action changes the algorithm

For four discrete actions, computing $\max_a Q(s, a)$ is easy: evaluate all four. A 14D continuous action space contains infinitely many possible vectors. One cannot enumerate them.

Continuous-control families handle this differently:

- policy-gradient methods directly learn an action distribution;
- deterministic actor-critic methods learn an actor that approximately maximizes the critic;
- stochastic off-policy methods such as Soft Actor-Critic (SAC) optimize value plus entropy; and
- model-based methods plan through known or learned dynamics.

Microduck uses PPO because its simulator can generate large fresh on-policy batches and its action is continuous. This is an engineering fit, not a theorem that PPO is always the best robot algorithm.

3.11 The "deadly triad" warning

Learning can become unstable when three ingredients combine:

1. **function approximation**: a neural network generalizes across states;
2. **bootstrapping**: targets depend on current estimates; and
3. **off-policy learning**: training distribution differs from the target policy's distribution.

This combination is called the **deadly triad**. It does not mean every such algorithm fails. It explains why replay buffers, target networks, double critics, conservative objectives, and careful evaluation exist.

The danger can be understood as a feedback loop:

```
slightly wrong value at an unsupported next action
        -> enters a bootstrap target
        -> shared network generalizes the error to other states
        -> greedy/off-policy selection seeks the inflated region
        -> new targets amplify the same error
```

A lower training loss does not prove the values are correct: predictions and targets can move together. Held-out Bellman error is also incomplete because the target contains estimates. Policy return, calibration against realized returns, target statistics, Q-scale, and coverage all provide complementary evidence.

3.12 Partial observability and memory

Bellman equations are written in terms of Markov state: all information needed to predict the future distribution. A deployed robot usually receives an observation o_t, not full state s_t. Hidden terrain compliance, actuator temperature, backlash, payload, and delayed contacts can make the problem a Partially Observable Markov Decision Process (POMDP).

Common responses are:

- stack recent observations;
- add an explicit estimator;
- use a recurrent network with internal memory;
- train an adaptation module that infers latent environment properties; or
- add sensors.

No optimizer can reconstruct information that leaves no trace in the available history.

3.13 Exercises

1. With rewards $2, 1, 4$ and $\gamma = 0.5$, compute the three-step return.
2. Calculate the TD error when $V(s) = 3$, $r = -1$, $V(s') = 5$, and $\gamma = 0.9$. Does the value rise or fall for positive α?
3. Explain the difference between a model, a value function, and a policy.
4. Why can Q-learning learn a greedy target policy while collecting with an ϵ-greedy behavior policy?
5. Name the three parts of the deadly triad and one mitigation used by DQN.
6. Why is ordinary DQN inconvenient for a 14D continuous action vector?

7. Give one hidden physical property that can make a robot observation non-Markov and one way to expose or infer it.
8. For a two-state fixed policy with $P_\pi = \begin{bmatrix} 0.5 & 0.5 \\ 0 & 1 \end{bmatrix}$, $r_\pi = [1, 0]^T$, $\gamma = 0.8$, and the second state terminal with value zero, solve the first state's Bellman equation without a matrix inverse.
9. If $\gamma = 0.9$ and the initial maximum value error is 10, what upper bound does the contraction result give after 20 exact Bellman sweeps?
10. Compute the three-step target for rewards $[1, 2, 3]$, $\gamma = 0.9$, and bootstrap value $V(S_{t+3}) = 4$. Identify the observed and estimated parts.
11. In the DQN pseudocode, explain why `gather` and `max` act on different action choices.
12. Write the Double DQN target using an online network to select and a target network to evaluate the next action. Which bias is it intended to reduce?
13. Why can a critic's training loss fall while its induced robot policy gets worse?
14. Choose between a return expectation and a return distribution for a robot whose two outcomes are "normal landing" and "rare destructive impact." State what the richer prediction still cannot guarantee.

Continue with the reinforcement-learning algorithm map, where these dimensions become a practical selection framework.

3.14 Folded solutions

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show answers to Section 3.13

1. The three-step return is $2 + 0.5(1) + 0.5^2(4) = 2 + 0.5 + 1 = 3.5$.
2. The TD error is $-1 + 0.9(5) - 3 = 0.5$. Because it is positive, an update with positive learning rate raises $V(s)$.
3. A **model** predicts what the environment will do after an action. A **value function** predicts accumulated future reward. A **policy** selects or distributes actions. A system may learn any one, two, or all three.
4. Q-learning's target uses the greedy next value, $\max_a Q(s', a)$, regardless of which exploratory action the behavior policy actually selects next. The behavior policy supplies coverage; the backup defines the target policy.
5. The deadly triad is function approximation, bootstrapping, and off-policy learning. DQN mitigates instability with a replay buffer and a temporarily fixed target network; neither is a universal convergence guarantee.
6. A 14D continuous vector has infinitely many candidates, so ordinary DQN cannot enumerate every action to compute a maximum. Continuous actor-critic methods learn an actor that proposes the maximizing action instead.
7. Hidden ground friction is one example. Recent wheel/contact history or a learned adaptation latent can infer its effect; an additional force/slip sensor can expose it more directly. Motor temperature, payload, and backlash are other valid examples.
8. Let the first value be v. Its equation is $v = 1 + 0.8(0.5v + 0.5 \times 0) = 1 + 0.4v$. Thus $0.6v = 1$ and $v = 5/3 \approx 1.667$. The self-loop repeatedly earns reward 1; transition to the terminal state ends future reward.
9. The bound is $10(0.9)^{20} \approx 1.216$. It is an upper bound in the maximum norm, not a claim that every instance attains exactly that error.
10. The target is

$$1 + 0.9(2) + 0.9^2(3) + 0.9^3(4) = 1 + 1.8 + 2.43 + 2.916 = 8.146.$$

The first three terms are observed rewards; the final term estimates all value beyond the sampled three-step segment.

11. `gather` selects the value of the action that was actually stored in each replay transition, producing the prediction being trained. `max` chooses the greedy next action used by the Q-learning target. They refer to current behavior evidence and next-state target behavior respectively.
12. With online parameters θ and target parameters $\bar{\theta}$:

$$a^* = \arg\max_a Q_\theta(s', a), \qquad y = r + \gamma(1 - d)Q_{\bar{\theta}}(s', a^*).$$

Separating selection from evaluation reduces the positive bias caused by maximizing noisy estimates from the same estimator.

13. Bootstrapped targets contain critic predictions, so prediction and target can become mutually consistent at wrong values. Loss is measured on the replay distribution, while the actor may seek unsupported high-value actions elsewhere. Scale collapse, distribution shift, or exploitation of approximation error can therefore reduce loss and harm realized return.
14. A return distribution can expose probability mass on destructive impact that the same expected value would hide. It supports quantile or risk- sensitive decisions. It still depends on correct data coverage/modeling and cannot replace hard hardware protection or guarantee safety outside the evaluated distribution.

The arithmetic behind answers 1 and 2 can be checked with:

```
discount = 0.5
three_step_return = 2 + discount * 1 + discount**2 * 4
td_error = -1 + 0.9 * 5 - 3
assert three_step_return == 3.5
assert td_error == 0.5
```

4. The Reinforcement-Learning Algorithm Map

Algorithm names can make reinforcement learning (RL) feel like a collection of unrelated inventions. Most methods can be located along a few design axes. Learning those axes is more durable than memorizing a leaderboard.

A dated lineage, not a leaderboard

The major families answer recurring problems:

Period	Representative origin or milestone	Problem it made explicit
1950s	Bellman dynamic programming	recursive planning with a known model
1980s–1990s	actor-critic, Q-learning, REINFORCE	learn values or policies from sampled experience
2000s	natural policy gradient	account for how parameters alter a probability distribution
2013–2018	Deep Q-Network, Deep Deterministic Policy Gradient, Trust Region Policy Optimization, Proximal Policy Optimization, Twin Delayed DDPG, Soft Actor-Critic	stabilize scalable neural discrete/continuous control
2019–2023	conservative/implicit offline RL, Dreamer, TD-MPC	learn from fixed data or learned latent dynamics
2023–2026	action generators, generalist policies, massive off-policy robot training	reuse diverse data and scale representations/control

Here **Deep Deterministic Policy Gradient (DDPG)** is the actor-critic precursor from which Twin Delayed DDPG addresses value-error failures; **Trust Region Policy Optimization (TRPO)** is the constrained-update precursor that motivates Proximal Policy Optimization's simpler surrogate. The dates locate ideas, not declare old methods obsolete. Proximal Policy Optimization (PPO) remains a strong robot baseline because an algorithm's value depends on the data and system around it.

4.1 First ask what data interaction is allowed

Online RL

In **online RL**, the agent collects new experience while learning. The data distribution changes as the policy changes.

- Simulation makes online interaction cheap and safe enough for millions or billions of transitions.
- Physical online learning may consume robot time, wear actuators, or discover unsafe actions.

Offline RL

In **offline RL**, learning uses a fixed logged dataset and cannot request new transitions. It is attractive when robot data already exists or new exploration is dangerous. It is difficult because value estimates for actions absent from the dataset can be arbitrarily wrong.

The behavior distribution is part of the mathematics

Let $\mu(a \mid s)$ be the **behavior policy** that collected data and $\pi(a \mid s)$ the **target policy** being evaluated or improved. For an expectation over actions at a covered state, importance sampling gives

$$\mathbb{E}_{a\sim\pi}[f(a)] = \mathbb{E}_{a\sim\mu}\left[\frac{\pi(a \mid s)}{\mu(a \mid s)} f(a)\right].$$

Derivation for a discrete action set is substitution:

$$\sum_a \mu(a \mid s)\frac{\pi(a \mid s)}{\mu(a \mid s)} f(a) = \sum_a \pi(a \mid s) f(a).$$

The ratio is valid only where $\mu(a \mid s) > 0$ whenever $\pi(a \mid s) > 0$. Large ratios create high variance. Full trajectory ratios multiply across time and can become unusably large or small. Modern off-policy algorithms therefore combine bootstrapping, clipping, conservative objectives, learned critics, and coverage assumptions rather than naively reweighting long robot trajectories.

Imitation learning

Imitation learning learns from demonstrations, often by treating expert actions as supervised labels. It may not use a reward or Bellman backup at all. Modern robot-policy research mixes imitation, offline RL, online fine-tuning, and pretrained representations, so "robot learning" is broader than RL.

4.2 Model-free versus model-based

A **dynamics model** predicts how state changes:

$$\hat{s}_{t+1} = f_\psi(s_t, a_t).$$

- **Model-free RL** learns a policy and/or value without using a learned transition model for planning.
- **Model-based RL** uses a known or learned model to evaluate or optimize future action sequences.

The labels do not mean model-free robot design has no physics knowledge. A Proximal Policy Optimization (PPO) policy trained in MuJoCo relies heavily on a simulator model to generate data; the learning algorithm itself simply does not differentiate through or plan with a learned f_ψ at runtime.

A second distinction is **planning at decision time** versus **amortized control**. Model Predictive Control (MPC) optimizes an action sequence again at each state. A feed-forward actor spends compute during training so that a single network evaluation approximates a good decision later. Temporal Difference Learning for Model Predictive Control, second generation (TD-MPC2) uses both: a learned policy proposes actions and a learned model/value refines a local plan.

Model-based methods can be more sample efficient because one transition helps learn dynamics reusable by many plans. Their danger is **model bias**: planning can exploit small prediction errors and imagine impossible success.

4.3 Value-based, policy-based, and actor-critic

Value-based

Value-based methods learn V or Q and derive behavior by choosing a high-value action. A Deep Q-Network (DQN) is the canonical deep discrete-action example.

Policy-based

Policy-gradient methods directly adjust a parameterized policy $\pi_\theta(a \mid s)$ toward actions associated with higher return. Continuous actions are natural because the policy can output distribution parameters.

Actor-critic

An **actor-critic** has both:

- actor: produces actions;

- critic: evaluates states or state-action pairs to improve the actor.

PPO, Soft Actor-Critic (SAC), and Twin Delayed Deep Deterministic Policy Gradient (TD3) are actor-critic methods, but their data use and objectives differ substantially.

4.4 On-policy versus off-policy

An on-policy method updates from data collected by the current or very recent policy. PPO is approximately on-policy. Old data becomes stale after a policy change, so it is normally discarded.

An off-policy method can update a target policy using data from other behavior policies. SAC, TD3, DQN, Implicit Q-Learning (IQL), and Conservative Q-Learning (CQL) are off-policy in different settings. A replay buffer improves sample reuse but makes the learning distribution less like current deployment.

The tradeoff is not simply "off-policy is better because it reuses data":

Property	On-policy	Off-policy
data reuse	low	high
implementation/stability	often simpler	often more delicate
massively parallel simulation	excellent fit	possible, needs care
scarce real interaction	expensive	often preferable
distribution correction	limited by fresh data	central challenge

4.5 Stochastic versus deterministic policies

A **stochastic policy** describes a distribution over actions. Exploration can come from sampling that distribution. PPO and SAC train stochastic policies.

A **deterministic policy** maps a state to one action. TD3 learns a deterministic actor and adds noise during data collection. A deployed stochastic-policy actor may also use its mean deterministically.

Stochastic does not mean careless or random behavior. The distribution is conditioned on the observation, and its spread is learned or configured.

4.6 Core families at a glance

Temporal Difference Learning for Model Predictive Control, second generation (TD-MPC2) is the newer member of the TD-MPC method family. The long form describes its combination of temporal-difference value learning and model-predictive planning; the "2" identifies the later scalable method.

Method	Action	Data	Learned objects	Characteristic idea	Typical fit
tabular Q-learning	discrete	online, off-policy	Q-table	greedy Bellman target	small teaching/control problem
DQN	discrete	online replay, off-policy	Q-network	replay + target network	games or finite skill choices
PPO	discrete or continuous	online, on-policy	actor + value critic	clipped policy-ratio objective	high-throughput simulation
TD3	continuous	online replay, off-policy	deterministic actor + twin critics	clipped double-Q + delayed actor	sample-efficient state control
SAC	continuous; discrete variants exist	online replay, off-policy	stochastic actor + critics	maximize reward and entropy	continuous control with costly data
IQL/CQL	usually continuous	fixed offline data	values/critics + policy	avoid optimistic unseen actions	logged robot datasets

Method	Action	Data	Learned objects	Characteristic idea	Typical fit
DreamerV3	varied	online replay, model-based	latent world model + actor/critic	learn through imagined rollouts	pixels/general domains
TD-MPC2	continuous	online replay, model-based	latent model + values/policy	local latent trajectory optimization	sample-efficient continuous control
behavior cloning	any represented in data	demonstrations	policy	supervised action prediction	strong expert data
diffusion policy	continuous action chunks	demonstrations	conditional diffusion model	model multimodal action sequences	visual manipulation

This table is a map, not a ranking. Performance depends on implementation, task, data, compute, observation/action choices, and evaluation.

A unifying "what is fitted?" view

Many method differences become concrete by locating the regression or optimization target:

```
behavior cloning: observation -----------------> demonstrated action
value learning:   state/action ----------------> Bellman return target
policy gradient:  sampled log-probability -----> advantage-weighted objective
world model:      state/history + action ------> next latent/reward/continuation
offline RL:       fixed data ------------------> conservative value + extracted policy
```

This gives a code-reading strategy. Find the batch fields, construct the target, find the loss, then find which parameters receive gradients. Algorithm names matter less than these four concrete facts.

4.7 Four influential objectives in plain language

DQN: make Q agree with a bootstrapped target

$$\min_{\theta} \left(r + \gamma \max_{a'} Q_{\theta}(s', a') - Q_{\theta}(s, a) \right)^2 .$$

Move the current action value toward immediate reward plus the target network's best next value.

PPO: improve sampled actions without moving probability too far

$$\max_{\theta} \mathbb{E}\left[\min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t) \right].$$

Increase probability for better-than-expected sampled actions and decrease it for worse ones, while clipping the incentive for a large probability-ratio change. Chapter 5 derives every term.

From natural gradient to TRPO to PPO

An ordinary parameter step measures distance in weight coordinates. But the same weight change can barely alter one policy and radically alter another. The Kullback–Leibler (KL) divergence measures change in action distributions. Trust Region Policy Optimization poses a local problem of the form

$$\max_{\theta} \mathbb{E}_{(s,a)\sim\pi_{old}} \left[\frac{\pi_{\theta}(a \mid s)}{\pi_{old}(a \mid s)} A^{\pi_{old}}(s, a) \right]$$

subject to

$$\mathbb{E}_{s\sim\pi_{old}}[D_{KL}(\pi_{old}(\cdot \mid s) \,\|\, \pi_{\theta}(\cdot \mid s))] \le \delta.$$

Near the old parameters, the KL curvature is approximated by the Fisher information matrix F. The natural-gradient direction is

$$\Delta\theta \propto F^{-1}\nabla_\theta J.$$

TRPO approximately solves this constrained problem using conjugate gradients and a line search. PPO replaces that machinery with first-order clipped or KL- penalized surrogates that are easier to implement and batch. PPO is therefore best understood as a practical descendant of trust-region reasoning, not as a proof that clipping alone bounds every policy change.

TD3: trust the smaller of two learned critics

$$y = r + \gamma \min_{i=1,2} Q_{\phi_i}(s', \pi_\theta(s') + \text{clipped noise}).$$

Using the smaller critic reduces overestimation; delayed actor updates let the critics settle between policy changes. See the primary TD3 paper.

SAC: value both reward and action entropy

$$J(\pi) = \mathbb{E}\left[\sum_t \gamma^t \big(r_t + \alpha \mathscr{H}(\pi(\cdot \mid s_t))\big)\right].$$

Entropy $\mathscr{H}$ measures distributional spread. SAC rewards task success and, weighted by temperature α, retaining multiple plausible actions. This often improves exploration and robustness. See the primary SAC paper.

4.8 Exploration is part of the system design

Exploration mechanisms include:

- ϵ-greedy random discrete actions;
- action noise around a deterministic actor;
- sampling a stochastic actor;
- an entropy bonus;
- randomized initial states and goals;
- curricula that expose increasingly difficult regions;
- demonstrations or reverse-curriculum starts that place the agent near rare useful states; and
- intrinsic rewards for novelty or prediction error.

Robot exploration must respect hardware. “Let the agent discover it” is not a safety plan. Most aggressive motor-skill exploration belongs in simulation, with a separately enforced hardware envelope.

Exploration can also be reasoned about as uncertainty. For a simple bandit, an upper confidence bound (UCB) action score is

$$\mathrm{UCB}(a) = \hat{Q}(a) + c\sqrt{\frac{\log t}{N(a)}},$$

where the second term is large for rarely tried actions. Deep robotics rarely uses this exact formula at every motor dimension, but the principle survives in ensembles, uncertainty bonuses, active system identification, goal sampling, and model-predictive exploration. Random action noise is only one possible information-acquisition strategy.

4.9 The frontier as of 2026 is conditional

No defensible source supports one algorithm as "the state of the art for robot intelligence." The evaluated regimes are too different. A dated frontier map is more useful:

Regime	Strong current baselines or directions	Main unresolved pressure
massively parallel proprioceptive locomotion	PPO/Robotic Systems Lab reinforcement learning (RSL-RL) recipes; emerging high-update off-policy SAC/TD3 variants	robustness and fair wall-clock/compute comparison
scarce online physical interaction	SAC-style replay, model-based world models, residual/hybrid control	safety, resets, nonstationary hardware
fixed logged robot data	behavior cloning, IQL, CQL, diffusion/action-chunk policies	coverage and out-of-distribution action error
pixel control with online learning	Dreamer-family latent models, TD-MPC-family planning	contact/model error and runtime compute
broad language-conditioned manipulation	transformer/diffusion/flow imitation plus selective RL fine-tuning	data provenance, embodiment transfer, calibrated evaluation
safety-constrained control	constrained Markov Decision Process (MDP) objectives, shields, control-barrier filters, MPC	expectation constraints do not equal hard guarantees

The 2025 FastSAC/FastTD3 work is notable because it challenges the assumption that on-policy PPO is inherently the fastest choice under massive simulation; its training-time claim is tied to the reported simulator, update ratio, hardware, and humanoid tasks. The right response is a matched experiment, not automatic replacement. Chapters 6, 14–16, and 18 examine each regime using primary papers and official artifacts.

4.10 A practical selection procedure

Ask in this order:

1. **Is the action discrete or continuous?** DQN does not naturally solve a 14D continuous joint command.
2. **Can you collect fresh interaction cheaply?** If yes, on-policy simulation may be simple and effective. If no, consider replay, offline data, or a model.
3. **Is the observation low-dimensional state or high-dimensional vision?** Pixels increase representation and data demands.
4. **Do you have demonstrations?** A supervised initialization may remove the hardest exploration phase.
5. **Is a reliable model available?** Model-predictive control or model-based RL may exploit it.
6. **Does the task need memory/adaptation?** Select history, recurrence, or latent inference explicitly.
7. **What is the safety boundary?** Constraints and an independent runtime supervisor may matter more than algorithm choice.
8. **What baseline must learning beat?** Start with a scripted or classical controller when one exists.

4.11 Why PPO is sensible for Microduck

Microduck has:

- a 14D continuous action;
- thousands of graphics processing unit (GPU)-parallel simulated robots;
- inexpensive fresh rollouts;
- dense task/recovery signals; and
- a small actor needed for 50 Hz deployment.

These properties fit PPO. One iteration with 4,096 environments and 24 steps collects 98,304 fresh transitions. Low sample reuse is less painful when simulation generates this much data quickly.

PPO does not cause walking by itself. Robot model, observation, command distribution, reward, reset distribution, actuator dynamics, randomization, and curriculum define the experience from which PPO learns.

4.12 Why a different robot may need a different method

Consider three projects:

A simulated biped locomotion controller

Continuous action, cheap massively parallel experience, 50–100 Hz inference. PPO is a strong baseline.

A real arm with 200 successful demonstrations

Physical trial cost is high and demonstrations already specify behavior. Behavior cloning or a diffusion/action-chunk policy is a sensible first baseline; offline RL may improve it if reward labels and dataset coverage are credible.

A wheeled robot choosing among "follow," "dock," and "stop"

The high-level choice is discrete and slow, but each skill has its own continuous controller. A hierarchical architecture may use a planner or discrete policy above classical/RL motor skills. One monolithic algorithm is not required.

4.13 Common selection mistakes

- Choosing the newest paper before defining the observation and action.
- Comparing algorithms with different reward, model, environment count, or compute budgets.
- Calling behavior cloning "RL" because the policy is a neural network.
- Calling a simulator-trained model "model-based RL" merely because the simulator has physics.
- Assuming sample efficiency implies wall-clock efficiency.
- Reporting the best seed instead of a distribution.
- Using a generalist visual model in a hard realtime loop without measuring worst-case latency.

4.14 Exercises

For each scenario, select a starting algorithm and state what evidence would change your mind:

1. Cart-pole with left/right discrete actions and unlimited simulation.
2. Microduck velocity tracking in 4,096 parallel environments.
3. A robot arm with a fixed dataset and no permission for online exploration.
4. An image-based manipulation task with several valid grasp trajectories.
5. A rover with a known dynamics model, strict constraints, and a 20 Hz local planner.
6. A legged robot whose payload changes during an episode.

Then answer these mechanism questions:

7. A behavior policy selects an action with probability 0.2 and the target policy assigns it probability 0.5. Compute its importance ratio. What makes a product of 100 such ratios dangerous?
8. Explain why a small Euclidean change in neural-network weights does not necessarily imply a small policy change. What quantity do trust-region methods measure instead?
9. For the same task, PPO uses 100 million transitions while SAC uses 10 million but takes twice the wall-clock time and three times the GPU energy. Which is "more efficient"? Give a non-misleading report.
10. Locate each system on two axes—model-free/model-based and online/offline: PPO in simulation, behavior cloning from logs, Dreamer trained while interacting, and MPC with an identified fixed model.
11. A paper reports state-of-the-art mean reward on one simulator. List four facts needed before choosing it for a physical biped.

Then make a two-column list: what the learning algorithm decides versus what the system architecture must decide. Continue with PPO from equations to code.

4.15 Folded solutions

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show reference answers to Section 4.14

1. **Cart-pole:** begin with tabular Q-learning after discretization or DQN for a neural baseline. Unlimited simulation and two discrete actions fit value learning. A continuous-action variant or poor pixel sample efficiency would motivate an actor-critic or representation change.
2. **Microduck:** begin with PPO. Its continuous action and 4,096 cheap parallel simulators make fresh on-policy data practical. A matched-budget experiment showing better robustness or wall-clock cost from SAC/TD3 would change the choice.
3. **Fixed arm dataset:** start with behavior cloning, then compare IQL or CQL if rewards and coverage support offline improvement. Permission for safe online interaction would open a separate fine-tuning phase.
4. **Multimodal visual manipulation:** start with a diffusion or other multimodal imitation policy. Plain squared-error behavior cloning (BC) may average incompatible grasps. Additional reward-bearing data could justify offline RL.
5. **Known constrained rover:** start with Model Predictive Control (MPC), which can use the known model and express constraints. Learn a residual/model only if held-out traces reveal systematic error the baseline cannot handle.
6. **Changing payload:** start with a history-conditioned/adaptive locomotion actor, using privileged teacher information only during training. If the payload is measured directly, explicit estimation plus a robust controller may be simpler.

The learning algorithm decides how experience changes a policy, value, or model. The architecture still decides sensor/calibration ownership, control rate, action semantics, planner decomposition, hard limits, watchdog, E-stop, compute placement, data privacy, and fallback. Algorithm selection cannot make those system decisions disappear.

7. The ratio is $0.5/0.2 = 2.5$, so this sample receives greater target-policy weight. A trajectory product 2.5^{100} is enormous; other paths can yield products near zero. Such products create extreme variance and numerical instability, and support failure occurs if behavior probability is zero.
8. Neural policies are nonlinear and parameterized redundantly; local output sensitivity depends on state and current weights. The same weight-space norm can cause very different action-distribution changes. Trust-region methods use a distributional distance, commonly expected Kullback–Leibler divergence, and natural gradient incorporates its local curvature.
9. SAC is five times more transition-efficient; PPO is twice as wall-clock efficient and three times as energy/compute efficient under the stated measurements. Neither is simply “more efficient.” Report all three axes, final held-out performance, hardware, simulator throughput, update count, and uncertainty across seeds.
10. PPO in simulation is model-free at the learning/inference level and online in data collection. Behavior cloning is offline and does not learn/use a dynamics model. Interacting Dreamer is online and model-based because it learns a world model. MPC with a fixed identified model is model-based control but not necessarily RL; if it performs no learning from a dataset, online/offline RL is the wrong label.
11. At minimum: exact robot/task and action/observation contract; data and compute budget; baseline implementations and tuning; seeds/trials and uncertainty; simulator-to-real evidence; runtime latency/model size; safety protocol; and availability/license of code, configs, data, and checkpoints. Any four well-explained items expose why one benchmark mean is insufficient.

5. Proximal Policy Optimization (PPO) from Equations to Code

This chapter explains how Proximal Policy Optimization turns a batch of Microduck rollouts into improved actor and critic networks.

Origin and scope

Williams's 1992 REINFORCE algorithm supplied a general sampled policy-gradient estimator. Actor-critic methods learned a baseline to reduce its variance. Natural policy gradient and Trust Region Policy Optimization (TRPO) measured steps in policy-distribution space. The 2017 PPO paper proposed clipped and Kullback–Leibler-penalty surrogates that retain this "do not move too far on one batch" motivation with ordinary first-order optimization.

PPO is an algorithm for updating a policy from on-policy trajectories. It does not define robot observations, action semantics, reward, actuator physics, reset distribution, or safety. Those surrounding choices are why two projects that both say "PPO" can solve very different problems.

5.1 Actor and critic

The actor chooses actions. The critic estimates how much discounted reward is still available from the current situation.

$$\text{actor: } \pi_\theta(a_t \mid o_t)$$

$$\text{critic: } V_\phi(x_t) \approx \mathbb{E}[G_t \mid x_t]$$

The actor must use observations available on the real robot. The critic is needed only during training and may receive privileged simulator information. In the main velocity task:

Network	Input	Hidden layers	Output
Actor	61	512 → 256 → 128, exponential linear unit (ELU)	14 action means
Critic	76	512 → 256 → 128, ELU	1 value estimate

The critic's extra information includes true base linear velocity and contact features. This asymmetric actor-critic pattern can make training easier without creating a deployment dependency.

The actor uses a Gaussian distribution during training. Its network predicts the mean action, and an exploration standard deviation allows nearby actions to be sampled. Deployment uses the inference policy rather than deliberately sampling exploratory motor commands.

The layer widths, learning rate, discount, and similar choices are **hyperparameters**: settings chosen by the engineer rather than weights learned by gradient descent. A **gradient** describes how a small parameter change would change the loss; backpropagation efficiently computes those gradients through the network.

5.2 Why a critic helps

Suppose a robot receives reward 1 in a state. Is that good? If similar states normally produce reward 5, it was worse than expected. If they normally produce reward 0, it was better than expected.

The advantage measures this relative quality:

$$A_t = Q(o_t, a_t) - V(o_t)$$

A positive advantage means the sampled action led to a better outcome than the critic expected. A negative advantage means it was worse. Policy-gradient learning increases the probability of positive-advantage actions and decreases the probability of negative-advantage actions.

Deriving the likelihood-ratio policy gradient

Start with expected trajectory return:

$$J(\theta) = \sum_{\tau} p_\theta(\tau) R(\tau).$$

The trajectory probability factors into initial-state probability, policy probabilities, and environment transitions:

$$p_\theta(\tau) = p(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t).$$

Only the policy factors depend on θ. Differentiate J and multiply and divide by $p_\theta(\tau)$:

$$\begin{aligned} \nabla_\theta J &= \sum_{\tau} \nabla_\theta p_\theta(\tau) R(\tau) \\ &= \sum_{\tau} p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) R(\tau) \\ &= \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) R(\tau) \right]. \end{aligned}$$

The identity $\nabla p = p \nabla \log p$ is the likelihood-ratio or score- function trick. It lets the environment be nondifferentiable: contacts and resets are sampled, while gradients pass through the actor's log-probability.

Rewards before action a_t cannot be caused by that action, so they may be removed, replacing whole-trajectory $R(\tau)$ with reward-to-go G_t. Then subtract a state baseline $b(s_t)$:

$$\nabla_\theta J = \mathbb{E} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) (G_t - b(s_t)) \right].$$

Why does a baseline not bias the expectation? Conditional on s:

$$\begin{aligned} \mathbb{E}_{a \sim \pi}[\nabla_\theta \log \pi_\theta(a \mid s) b(s)] &= b(s) \sum_a \pi_\theta(a \mid s) \nabla_\theta \log \pi_\theta(a \mid s) \\ &= b(s) \sum_a \nabla_\theta \pi_\theta(a \mid s) \\ &= b(s) \nabla_\theta 1 = 0. \end{aligned}$$

Choosing $b = V(s)$ turns $G_t - b(s)$ into an advantage estimate. A better critic reduces variance; an inaccurate critic can add bias when bootstrapped targets are used, which is why critic learning and advantage diagnostics matter.

5.3 Temporal-difference error and Generalized Advantage Estimation (GAE)

The one-step temporal-difference residual is:

$$\delta_t = r_t + \gamma V_\phi(x_{t+1}) - V_\phi(x_t)$$

Generalized Advantage Estimation (GAE) combines a sequence of these residuals:

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \cdots$$

Microduck uses $\gamma = 0.99$ and $\lambda = 0.95$. Lower λ relies more on the critic and usually lowers variance; higher λ uses longer sampled returns and usually lowers bias. The chosen values are a common compromise, not a law of robotics.

The value target is commonly formed from the advantage and old value estimate:

$$\hat{G}_t = \hat{A}_t + V_{\text{old}}(x_t)$$

The critic learns by reducing error between $V_\phi(x_t)$ and this target.

Deriving the efficient backward recurrence

Let $\rho = \gamma\lambda$. Starting from the infinite-looking sum:

$$\hat{A}_t = \delta_t + \rho\delta_{t+1} + \rho^2\delta_{t+2} + \cdots,$$

factor one ρ from the tail:

$$\hat{A}_t = \delta_t + \rho(\delta_{t+1} + \rho\delta_{t+2} + \cdots) = \delta_t + \gamma\lambda\hat{A}_{t+1}.$$

That recurrence gives all advantages in one reverse loop. A true terminal mask $m_t \in \{0, 1\}$ prevents both value bootstrapping and propagation across an episode boundary:

$$\delta_t = r_t + \gamma m_t V(x_{t+1}) - V(x_t),$$

$$\hat{A}_t = \delta_t + \gamma\lambda m_t\hat{A}_{t+1}.$$

For a time-limit truncation of an underlying continuing task, $V(x_{t+1})$ should normally be retained even though the rollout buffer ends. Libraries may represent masks and timeout bootstrapping differently, so inspect code rather than infer semantics from a field named `done`.

Run `examples/gae_walkthrough.py`. Its `generalized_advantage_estimation` function is the equation translated line by line, including the terminal mask and value targets. Change the last flag from `True` to `False` and observe why the provided bootstrap value suddenly matters.

What λ is mixing

Expanding each δ shows that Generalized Advantage Estimation mixes one-step, two-step, and longer value targets. With a perfect critic, a shorter target can already assign useful credit with low variance. With a biased critic, longer sampled returns reduce dependence on its error. Therefore changing λ is

not just “more or less smoothing”; it changes how much the actor’s credit signal trusts the learned value function.

Practical PPO implementations commonly normalize advantages within the batch:

$$\tilde{A}_i = \frac{\hat{A}_i - \overline{A}}{\sqrt{\mathrm{Var}_{batch}(\hat{A}) + \varepsilon}}.$$

This stabilizes gradient scale. It also makes the update depend on batch composition: a sample’s normalized advantage is relative to other commands, terrains, and episodes in that batch. Preserve this detail when comparing implementations.

5.4 The policy ratio

After collecting a rollout with policy $\pi_{\theta_{old}}$, PPO asks how the new policy changes the probability of each sampled action:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid o_t)}{\pi_{\theta_{old}}(a_t \mid o_t)}$$

- $r_t = 1$ means no probability change.
- $r_t > 1$ means the sampled action became more likely.
- $r_t < 1$ means it became less likely.

For a 14-dimensional diagonal Gaussian, the joint log-probability is the sum of per-joint log densities:

$$\log \pi_\theta(a \mid o) = \sum_{j=1}^{14} \log \mathcal{N}(a_j; \mu_{\theta,j}(o), \sigma_j^2).$$

The ratio is calculated stably as

$$r_t(\theta) = \exp\left[\log \pi_\theta(a_t \mid o_t) - \log \pi_{old}(a_t \mid o_t)\right].$$

The old log-probability must be stored when collecting the action. Recomputing it after updating the actor would make numerator and denominator describe the same new policy and erase the intended comparison.

A basic policy-gradient objective would maximize $r_t\hat{A}_t$. Reusing the same rollout for aggressive updates can move the policy so far that the data no longer represents it. PPO limits the incentive for a large move.

5.5 The clipped PPO objective

The clipped surrogate objective is:

$$L^{CLIP}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)\hat{A}_t, \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right]$$

Microduck uses $\epsilon = 0.2$, so the clipped interval is $[0.8, 1.2]$.

Example: an action has advantage +2, its old probability was 0.10, and its new probability is 0.13. The ratio is 1.3. The unclipped objective is 2.6, but the clipped positive-advantage objective is $1.2 \times 2 = 2.4$. Increasing its probability further offers no clipped benefit for this sample.

The sign of advantage changes which side clips. With $\epsilon = 0.2$:

Advantage	Ratio direction that would help the surrogate	Clipped plateau
$A > 0$	increase probability, $r > 1$	no extra benefit above 1.2
$A < 0$	decrease probability, $r < 1$	no extra benefit below 0.8

For a negative advantage and $r = 1.5$, clipping does **not** protect the sample: the minimum selects $1.5A$, which is more negative than $1.2A$. PPO blocks updates only when the probability change would improve the surrogate beyond the clip boundary. It still penalizes moves in the wrong direction.

Run `examples/ppo_clip_demo.py` and inspect both advantage signs. This four-line scalar function is the direct mapping of the paper objective:

```
unclipped = ratio * advantage
clipped = clip(ratio, 1.0 - epsilon, 1.0 + epsilon) * advantage
objective = min(unclipped, clipped)
```

Clipping does not mathematically guarantee a small update. The Robotic Systems Lab reinforcement learning (RSL-RL) library also tracks the approximate Kullback–Leibler (KL) divergence, a measure of how different the old and new action distributions are, and uses an adaptive learning-rate schedule with desired KL `0.01`. Gradient norm is capped at `1.0`.

5.6 Value, entropy, and the combined update

The optimizer conceptually combines three goals:

```
maximize clipped policy improvement
minimize critic value error
preserve enough action-distribution entropy to explore
```

One common minimization form is:

$$L = -L^{CLIP} + c_v L^{value} - c_e H[\pi_\theta]$$

The main configuration uses value-loss coefficient `1.0` and entropy coefficient `0.01`. Entropy is not "randomness is always good." Early in training it prevents the policy from collapsing before discovering useful motion. Too much entropy can keep behavior noisy; too little can freeze a poor strategy.

For an unbounded scalar Gaussian, entropy is

$$H[\mathcal{N}(\mu, \sigma^2)] = \frac{1}{2}\log(2\pi e\sigma^2).$$

It depends on σ, not μ: translating the distribution does not change its spread. For a diagonal Gaussian, sum this term over action dimensions. If policy outputs are transformed or clipped, the actual action distribution may differ; entropy and log-probability must follow the same distribution semantics.

A simple critic loss is

$$L^{value} = \frac{1}{B}\sum_{i=1}^{B}\left(V_\phi(x_i) - \hat{G}_i\right)^2.$$

Some PPO implementations also clip the value prediction change. This is an implementation option, not part of the one universal PPO definition. Record it when reproducing results.

The signs in the combined loss follow the optimizer: gradient descent minimizes, so the actor objective and entropy bonus receive minus signs, while value error receives a plus sign. A useful code audit starts by writing the desired maximize/minimize direction beside every scalar before interpreting a logged loss.

5.7 What one Microduck iteration contains

The default runner uses:

```
parallel environments       4096 in a normal full run
steps per environment         24
rollout transitions       98,304 per iteration
minibatches                    4
learning epochs                5
initial learning rate       0.001
```

The same rollout is shuffled into four **minibatches** (smaller parts of the full collected batch) and visited for five optimization **epochs** (complete passes over that rollout). Then the updated policy collects a new rollout.

At a 50 Hz policy rate, 24 steps represent 0.48 seconds per environment. The large parallel batch contains many commands, initial conditions, contacts, and randomized robot instances even though each individual fragment is short.

The logical loop is:

```
for iteration in range(max_iterations):
    rollout = collect_current_policy(num_envs=4096, steps=24)
    advantages, returns = generalized_advantage_estimation(rollout)

    for epoch in range(5):
        for minibatch in split_and_shuffle(rollout, count=4):
            update_actor_with_clipped_ppo(minibatch)
            update_critic(minibatch)

    log_metrics_and_periodically_save_checkpoint()
```

This is explanatory pseudocode. RSL-RL owns the actual buffers, distribution, losses, optimizer, and checkpoint format.

The official RSL-RL PPO source is the implementation mapping to read after the pseudocode. Trace in this order:

1. rollout storage retains observations, actions, values, rewards, masks, and old action log-probabilities;
2. return computation performs the reverse GAE recurrence;
3. the minibatch generator yields stored old quantities and current inputs;
4. the actor evaluates the stored actions under its current distribution;
5. ratio, clipped surrogate, value loss, entropy, gradient norm, and optimizer step implement the equations above.

Pin the exact RSL-RL revision from the Microduck lockfile before comparing line numbers; `main` can evolve after a training run was produced.

5.8 Observation normalization

Joint angles, angular velocities, gravity components, and commands have different numerical scales. The actor and critic use empirical observation normalizers so one large-magnitude feature does not dominate merely because of its units.

Conceptually, each feature becomes:

$$\tilde{o}_i = \frac{o_i - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}}$$

The running mean and variance are learned from training observations and saved in the checkpoint. The Open Neural Network Exchange (ONNX) exporter includes the actor's normalizer in the graph. A hand-converted network that omits it receives a different numerical problem and may fail on hardware even if playback from the checkpoint looked correct.

5.9 The important hyperparameters in this repository

Setting	Main value	Practical effect
`gamma`	0.99	how strongly later rewards count
`lam`	0.95	GAE bias/variance tradeoff
`clip_param`	0.2	clips the policy-ratio incentive
`learning_rate`	0.001	initial optimizer step scale
`desired_kl`	0.01	target used by adaptive schedule
`entropy_coef`	0.01	exploration pressure
`num_learning_epochs`	5	reuse passes over each rollout
`num_mini_batches`	4	subdivisions per epoch
`max_grad_norm`	1.0	gradient clipping threshold
actor/critic normalization	on	normalizes each observation stream

Do not tune these merely because a reward curve looks noisy. First verify the environment, reward signs, resets, observation validity, and actual rollout. Most project failures have been specification or physics failures rather than a need for exotic PPO settings.

5.10 Reading a PPO update scientifically

Four diagnostics describe different mechanisms:

Diagnostic	What it measures	Suspicious pattern
approximate KL	old/new policy distribution change	persistent overshoot beyond target
clip fraction	fraction of samples whose improving direction is clipped	near 0 with no learning, or near 1 with violent updates
explained variance	how much return-target variation the critic predicts	negative: worse than predicting the batch mean
entropy/action standard deviation	exploration spread	early collapse or uncontrolled growth

Explained variance is commonly

$$1 - \frac{\mathrm{Var}(\hat{G} - V_\phi)}{\mathrm{Var}(\hat{G})}.$$

A value near 1 indicates predictions explain most target variation; 0 is no better than predicting a constant mean; a negative value is worse. It is not a policy-success metric: a critic can accurately predict returns for a bad policy.

For scalar Gaussians $p = \mathcal{N}(\mu_0, \sigma_0^2)$ and $q = \mathcal{N}(\mu_1, \sigma_1^2)$:

$$D_{KL}(p \,\|\, q) = \log \frac{\sigma_1}{\sigma_0} + \frac{\sigma_0^2 + (\mu_0 - \mu_1)^2}{2\sigma_1^2} - \frac{1}{2}.$$

This shows that changing either action means or exploration scales changes the policy. Summing across 14 independent dimensions means modest per-joint shifts can create a large joint policy change.

PPO alternatives do not remove the environment problem. TRPO enforces a more explicit approximate trust region but costs additional linear algebra. Soft Actor-Critic reuses replay and optimizes a maximum-entropy objective but relies on state-action critics. Model-based methods may use fewer real transitions but introduce model error and planning compute. As of 2026, PPO remains a reference for massively parallel locomotion, while high-update off-policy methods are credible matched-budget challengers rather than automatic drop-in replacements.

5.11 Why PPO does not understand intent

PPO only sees sampled data and scalar objectives. It does not know that a forward roll should be sagittal rather than a shoulder roll, or that "stand" should not mean balancing on the head. Hard state-based gates and correct task distributions translate that intent into the optimization problem.

This is also why total reward can improve while the task fails. The agent may learn cheaper regularizers without improving the main skill. Always inspect individual weighted reward terms and rollouts.

5.12 Check your understanding

1. Why can the critic use information that is unavailable on the real robot?
2. What does an advantage of zero mean for a sampled action?
3. What problem does PPO clipping reduce?
4. How many transitions are collected in one 4,096-environment iteration?
5. Why must observation normalization be included in the deployed ONNX graph?
6. In the policy-gradient derivation, why do simulator transition probabilities disappear from $\nabla_\theta \log p_\theta(\tau)$?
7. Prove in one line why a state-only baseline has zero expected score term. Why may it still reduce variance?
8. Given rewards [1, 2], values [0.5, 0.25], a terminal final transition, $\gamma = 0.9$, and $\lambda = 0.8$, compute both GAE advantages backward.
9. The old joint log-probability is −8.0 and the new one is −7.7. Compute the PPO ratio. With $A = 2$ and $\epsilon = 0.2$, compute the clipped surrogate.
10. With $A = -2$ and ratio 0.5, which branch of the minimum is selected? Explain why this is the correct clipping direction.
11. Why must `old_log_probability` be stored before optimization rather than recomputed from the updated actor?
12. A run has explained variance 0.95, increasing total reward, and a video of the robot farming reward while fallen. What has and has not succeeded?

Continue with off-policy and model-based reinforcement learning.

5.13 Folded solutions

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show answers to Section 5.12

1. The critic is used to estimate returns/advantages during training and is not part of the deployed actor. Simulator-only truth can therefore reduce critic noise while the actor remains restricted to reproducible sensors. Leakage into actor inputs would break this argument.
2. Zero advantage means the sampled result matched the critic's baseline for that observation. The policy-gradient term has no first-order reason to make that action more or less likely, though entropy and other loss terms may still update the policy.
3. Clipping reduces the incentive for one batch to move action probabilities too far from the behavior policy that collected it. It is a practical trust- region surrogate, not a proof that every update improves the real task.
4. One iteration collects $4096 \times 24 = 98{,}304$ transitions.
5. Normalization changes the function's actual input. Omitting the saved mean and variance makes the ONNX actor see a different numeric distribution than the network optimized in training.
6. The environment transition kernel and initial-state distribution do not depend on actor parameters θ in the standard derivation. Their log derivatives are therefore zero. If the environment itself is jointly parameterized by θ, that assumption must be revisited.
7. Conditional on state s:

$$\mathbb{E}_{a\sim\pi}[b(s)\nabla \log \pi(a \mid s)] = b(s)\nabla \sum_a \pi(a \mid s) = b(s)\nabla 1 = 0.$$

A baseline centered near expected return makes the remaining advantage smaller and less variable, so finite-batch gradient estimates fluctuate less even though their expectation is unchanged.

8. At the final step, continuation is zero:

$$\delta_1 = 2 - 0.25 = 1.75, \qquad \hat{A}_1 = 1.75.$$

At the first step:

$$\delta_0 = 1 + 0.9(0.25) - 0.5 = 0.725,$$

$$\hat{A}_0 = 0.725 + 0.9(0.8)(1.75) = 1.985.$$

9. The ratio is $\exp(-7.7 - (-8.0)) = e^{0.3} \approx 1.350$. The improving positive-advantage branch clips at 1.2, so the surrogate is $1.2 \times 2 = 2.4$ rather than about 2.700.
10. Unclipped is $0.5(-2) = -1$; clipped is $0.8(-2) = -1.6$. The minimum selects -1.6, preventing extra surrogate benefit from reducing the probability of a negative-advantage action below the lower boundary.
11. The denominator must describe the behavior policy that generated the stored action. Recomputing after updates would replace historical evidence with the current distribution, often making the ratio artificially near one and breaking the importance-ratio interpretation.
12. The critic has succeeded at explaining the return targets of the sampled (reward-hacking) behavior, and the optimizer has raised its specified objective. The environment specification and intended-task acceptance have failed. High explained variance says nothing about whether the reward represents the desired skill.

A minimal transition-count check is:

```
num_envs = 4096
steps_per_env = 24
assert num_envs * steps_per_env == 98_304
```

6. Off-Policy and Model-Based Reinforcement Learning

Proximal Policy Optimization (PPO) throws away old rollouts after a few update epochs. That can be entirely reasonable in fast parallel simulation, but it feels wasteful when one physical robot transition is expensive. Off-policy and model-based methods try to learn more from each interaction.

This chapter explains the motivation and failure modes before the algorithms.

Two historical routes to transition efficiency

Off-policy value learning descends from Q-learning: learn about one target policy while data may come from another. Deep Deterministic Policy Gradient (DDPG) extended this route to continuous actions using a learned actor; Twin Delayed Deep Deterministic Policy Gradient (TD3) directly addressed its overestimation and brittle actor updates. Soft Actor-Critic (SAC) added a stochastic maximum-entropy policy and became a widely used continuous-control baseline.

Model-based control follows a different route. System identification and Model Predictive Control (MPC) plan through known or fitted dynamics. Probabilistic Ensembles with Trajectory Sampling (PETS) represented uncertainty; model-based policy optimization mixed short model rollouts with real replay; Dreamer learned and acted in a recurrent latent world; TD-MPC learned a control-centered latent model and planned locally. The 2026 frontier still contains both routes and hybrids—no theorem makes replay or imagination universally cheaper than fresh parallel simulation.

6.1 Replay buffers: experience as a reusable dataset

An off-policy learner commonly stores transitions

$$(s_t, a_t, r_t, s_{t+1}, d_t)$$

in a **replay buffer**, where d_t indicates termination. Training samples minibatches from the buffer rather than only the newest trajectory.

Replay provides:

- reuse of expensive experience;
- shuffled samples instead of strongly correlated time neighbors; and
- a mixture of behavior from several policy versions.

That mixture is also the difficulty. The current actor may visit states and actions differently from the buffer. A critic can be asked about actions for which the data provides weak evidence.

Two quantities describe the engineering regime:

- **replay ratio** or update-to-data ratio: gradient samples consumed divided by new environment transitions;
- **buffer age and coverage**: how old and behaviorally diverse sampled data is relative to the current policy.

A ratio of 32 means 32 sampled transition uses per new transition, not necessarily 32 unique optimizer steps. Larger ratios can improve reuse until the learner overfits a narrow buffer, targets drift, or compute becomes the bottleneck. Report both environment steps and gradient updates.

6.2 State-action critics

PPO usually uses a state-value critic $V(s)$. Continuous off-policy actor-critic methods often learn

$$Q_\phi(s, a),$$

the expected return for taking action a in state s and following a policy afterward.

A one-step critic target has the form

$$y = r + \gamma(1 - d)\,\text{next-value}.$$

The factor $(1 - d)$ removes future value after a true terminal transition. A time-limit truncation may need different handling because the underlying task could have continued. Confusing termination with truncation biases targets.

The squared critic objective over replay distribution D is

$$L_Q(\phi) = \mathbb{E}_{(s,a,r,s',d)\sim D}[(Q_\phi(s, a) - y)^2].$$

This loss only constrains state-action pairs represented in D. An actor can query $Q_\phi(s, a)$ at a different action $a = \pi_\theta(s)$, so low replay loss does not imply accurate gradients at actor-proposed actions.

Target networks reduce rapid feedback. A **Polyak** or exponential moving average update is

$$\bar{\phi} \leftarrow \rho\bar{\phi} + (1 - \rho)\phi,$$

with ρ close to 1. The target follows slowly rather than being copied on every critic step.

6.3 Why critics become overoptimistic

Suppose a critic has small random errors across actions. Selecting the maximum tends to select not only a genuinely good action but also a positive estimation error. Repeating this in bootstrapped targets can inflate values.

Robot consequence: the actor may exploit critic errors by proposing an out-of-distribution action that the critic imagines is excellent, even though the dataset never demonstrated it safely.

Two widely used responses are:

- learn two critics and use the smaller estimate; and
- constrain or regularize actions toward regions covered by data.

6.4 Twin Delayed Deep Deterministic Policy Gradient (TD3): deterministic continuous control

Twin Delayed Deep Deterministic Policy Gradient (TD3) learns:

- deterministic actor $a = \pi_\theta(s)$;
- two critics Q_{ϕ_1} and Q_{ϕ_2}; and
- slowly moving target copies.

Its target is approximately

$$y = r + \gamma(1 - d)\min_{i=1,2} Q_{\bar{\phi}_i}\left(s', \pi_\theta(s') + \epsilon\right),$$

where target noise ϵ is clipped.

The three ideas encoded in the name/paper are:

1. **clipped double Q**: use the smaller of two critics to reduce optimistic error;

2. **delayed policy update**: update the actor less often than critics; and
3. **target policy smoothing**: train the value of a neighborhood, not a narrow action spike.

The actor is optimized to choose actions its critic values:

$$\max_{\theta} \mathbb{E}_{s\sim D}[Q_{\phi_1}(s, \pi_\theta(s))].$$

The chain rule gives the deterministic actor gradient:

$$\nabla_\theta J \approx \mathbb{E}_{s\sim D}\left[\nabla_a Q_\phi(s,a)\big|_{a=\pi_\theta(s)} \nabla_\theta \pi_\theta(s)\right].$$

Read it from left to right: the critic says which small action change would increase predicted value, then the actor Jacobian says how weights must change to produce that action change. This makes critic smoothness and correctness at actor actions crucial. A narrow erroneous Q peak can directly pull the actor toward it.

Target policy smoothing adds bounded noise to the next action inside the target. It approximates valuing a neighborhood:

$$\tilde{Q}(s', a') \approx \mathbb{E}_\epsilon[Q(s', a' + \epsilon)],$$

so an isolated critic spike is less attractive. Exploration noise on actions collected into replay is a separate mechanism from this target noise.

The TD3 paper isolates these mechanisms. Its benchmark findings support those mechanisms in the evaluated continuous control tasks; they do not erase the need for robot-specific evaluation.

6.5 Soft Actor-Critic (SAC): maximum-entropy actor-critic

Soft Actor-Critic (SAC) learns a stochastic policy and adds entropy to the return:

$$J(\pi) = \mathbb{E}\left[\sum_{t=0}^{T-1} \gamma^t \left(r_t + \alpha \mathcal{H}(\pi(\cdot \mid s_t))\right)\right].$$

$\mathcal{H}$ is entropy and α is a temperature. In plain language, the policy values reward while retaining action diversity when several actions are plausible.

For continuous actions, implementations commonly sample an unconstrained Gaussian variable using a differentiable reparameterization, then apply `tanh` to bound it:

$$u = \mu_\theta(s) + \sigma_\theta(s) \odot \xi, \quad \xi \sim \mathcal{N}(0, I), \quad a = \tanh(u).$$

$\odot$ means elementwise multiplication. Writing randomness as an input ξ allows gradients to flow through μ and σ.

A soft critic target includes the next action's entropy term:

$$y = r + \gamma(1 - d)\left[\min_i Q_{\bar{\phi}_i}(s', a') - \alpha \log \pi_\theta(a' \mid s')\right].$$

The actor minimizes approximately

$$J_\pi = \mathbb{E}\left[\alpha \log \pi_\theta(a \mid s) - \min_i Q_{\phi_i}(s, a)\right].$$

Interpretation: prefer actions the critics value, while paying a cost for collapsing the action distribution too sharply. Modern SAC often tunes α toward a target entropy instead of fixing it.

Soft value and temperature, step by step

The maximum-entropy state value is

$$V(s) = \mathbb{E}_{a\sim\pi}[Q(s, a) - \alpha \log \pi(a \mid s)].$$

Because entropy is $-\mathbb{E}[\log \pi]$, subtracting $\alpha \log \pi$ adds an entropy bonus. Insert this value into the Bellman target to obtain the earlier SAC target. The actor loss is the negative of the same soft value under reparameterized samples.

Automatic temperature tuning can minimize

$$J(\alpha) = \mathbb{E}_{a\sim\pi}[-\alpha(\log \pi(a \mid s) + \mathcal{H}_{target})].$$

If entropy is below the chosen target, the update tends to increase α and put more weight on diversity; if entropy is above target, it reduces that pressure. The target entropy is still a design choice, not discovered task intent.

The `tanh` action transform requires a probability correction. If $a = \tanh(u)$, change of variables gives, componentwise,

$$\log \pi_A(a \mid s) = \log \pi_U(u \mid s) - \sum_j \log(1 - \tanh^2(u_j)).$$

Omitting the Jacobian term makes the actor optimize the wrong bounded-action density, especially near -1 and 1. Mature implementations clamp or rearrange this calculation for numerical stability.

The primary SAC paper motivated stable, sample-efficient continuous control. "Sample efficient" still depends on what counts as a sample, update-to-data ratio, environment, and compute.

6.6 PPO versus SAC as an engineering decision

Question	PPO tendency	SAC tendency
Fresh simulation is cheap?	excellent	also possible
Each transition is expensive?	wastes more data	replay is attractive
Thousands of synchronous environments?	natural fit	replay/update design needs care
Simplicity of distributed rollout?	relatively simple	buffer and target networks add state
Old data from prior policies?	mostly discarded	reusable if relevant
Primary instability concern	policy update size	critic error/distribution shift

Do not compare only final reward. Match environment steps, wall time, compute, network size, evaluation protocol, and number of seeds.

6.7 What a model adds

A dynamics model predicts the next state distribution and possibly reward:

$$p_\psi(s_{t+1}, r_t \mid s_t, a_t).$$

If the model is known, Model Predictive Control (MPC) can repeatedly:

1. observe the current state;
2. simulate candidate action sequences;
3. score predicted futures;
4. execute only the first action; and
5. replan from the next observation.

Executing only the first action is **receding-horizon control**. Replanning limits damage from prediction error because reality corrects the plan at every step.

For horizon H, a deterministic planner can solve

$$\max_{a_{t:t+H-1}} \left[\sum_{k=0}^{H-1} \gamma^k \hat{r}(\hat{s}_{t+k}, a_{t+k}) + \gamma^H \hat{V}(\hat{s}_{t+H}) \right]$$

subject to

$$\hat{s}_{t+k+1} = \hat{f}(\hat{s}_{t+k}, a_{t+k}).$$

The terminal value $\hat{V}$ summarizes reward beyond the short planning horizon. Without it, a short planner can be myopic; with a bad learned value, it can inherit critic error.

The cross-entropy method (CEM) is a gradient-free sequence optimizer:

```
initialize a Gaussian over H-step action sequences
repeat:
    sample candidate sequences
    roll each through the model and score it
    keep the lowest-cost elite fraction
    refit each time step's mean and standard deviation to elites
execute only the first mean action; observe reality; replan
```

Run `examples/cem_mpc_point_mass.py` to see the equations control a one-dimensional point mass. It deliberately contains no learning: replace its exact `dynamics` with a fitted model and it becomes the planning core of a simple model-based learning experiment.

Model-based reinforcement learning (RL) learns some or all of the model and uses it to improve behavior.

6.8 Model error compounds through imagination

Let a learned model have a small one-step error. Rolling it forward for 100 steps feeds predictions back as inputs, so error can compound. The policy or planner may also seek regions where the model is wrong in a favorable way.

A simple bound makes the pressure visible. Suppose true dynamics f and learned dynamics $\hat{f}$ differ by at most ϵ per step, and both are L-Lipschitz in state. If $e_k = \|s_k - \hat{s}_k\|$, then roughly

$$e_{k+1} \le L e_k + \epsilon.$$

Starting from the same state, unrolling gives

$$e_H \le \epsilon \sum_{i=0}^{H-1} L^i.$$

For $L = 1$, the bound grows as $H\epsilon$; for $L > 1$, it can grow geometrically. This is a worst-case bound, not a forecast, but it explains why longer imagination is not free and why unstable/contact-rich dynamics are difficult.

Mitigations include:

- short planning horizons;

- model ensembles to estimate disagreement;
- uncertainty penalties;
- frequent replanning from real observations;
- training the model on the policy's current distribution;
- latent representations that focus on control-relevant information; and
- value functions to approximate outcomes beyond the short model horizon.

6.9 Latent world models

Images contain far more pixels than control-relevant state. A **latent model** compresses observations into a learned representation z_t, predicts latent dynamics, and learns reward/value in that space.

Conceptually:

```
camera/history -> encoder -> latent state z_t
                                  |
                            learned dynamics
                      (z_t, a_t) -> z_{t+1}
                                  |
                          imagined trajectories
                                  |
                            actor/value updates
```

The latent state is not guaranteed to correspond to human-named variables. It is optimized to support reconstruction, prediction, reward, value, or a combination.

A recurrent state-space model separates deterministic memory h_t and a stochastic latent z_t:

$$h_t = f_\psi(h_{t-1}, z_{t-1}, a_{t-1}),$$

$$z_t \sim q_\psi(z_t \mid h_t, o_t) \quad \text{during observation,}$$

$$z_t \sim p_\psi(z_t \mid h_t) \quad \text{during imagination.}$$

The posterior q may inspect the real observation; the prior p must predict without it. Training balances prediction terms and a Kullback–Leibler (KL) divergence that makes prior and posterior compatible:

$$\begin{aligned} L_{model} \approx\ & -\log p(o_t \mid h_t, z_t) - \log p(r_t \mid h_t, z_t) \\ & -\log p(c_t \mid h_t, z_t) + \beta D_{KL}(q(z_t \mid h_t, o_t) \,\|\, p(z_t \mid h_t)) \,. \end{aligned}$$

c_t represents continuation. Exact DreamerV3 losses include additional balancing, free-bit, distribution, and normalization details; the equation is a reading map, not replacement code.

6.10 DreamerV3

DreamerV3 learns a world model from replay, then trains actor and critic from trajectories imagined in its latent space. The paper evaluates a single configuration across more than 150 tasks in several domains and emphasizes normalization and scale-robust objectives.

The important conceptual contribution for a beginner is not "Dreamer replaces PPO." It is the separation:

```
real/sim experience -> learn predictive latent world
learned world       -> generate imagined futures
imagined futures    -> improve policy/value
```

During imagination, the actor samples actions from latent states and the critic estimates a λ-return. Gradients can then improve many imagined steps per real transition. This sample reuse is valuable only insofar as the learned latent dynamics and reward remain decision-correct where the actor goes.

Questions to ask before applying it to a robot:

- Does the latent model predict contacts and discontinuities well enough?
- What observation history resolves partial observability?
- Does imagined success correlate with real rollout success?
- What is the runtime actor size and latency?
- How is uncertainty outside the replay distribution handled?

6.11 Temporal Difference Learning for Model Predictive Control, second generation (TD-MPC2)

TD-MPC2 combines a learned latent model, temporal-difference values, a policy prior, and local trajectory optimization. Rather than decode future images, it learns task-relevant latent predictions for control. The paper reports results across 104 online RL tasks and explores multi-task scaling.

At each decision, planning conceptually samples/refines short action sequences in latent space, uses learned reward and terminal value to score them, and executes the first action. The policy helps propose actions and can also serve as a fast behavior prior.

TD-MPC2 trains a representation so that predicted next latent, reward, and value agree with replay-derived targets. A schematic joint loss is

$$L = c_z \|\hat{z}_{t+1} - \text{stopgrad}(z_{t+1})\|^2 + c_r \ell(\hat{r}_t, r_t) + c_q \ell(\hat{Q}_t, y_t) + c_\pi L_{policy}.$$

`stopgrad` means the target representation is treated as a constant on that loss branch. The actual method uses distributional/value-normalization and planning details documented by its paper and official implementation. The mapping to look for is: encoder, latent dynamics, reward/value heads, policy prior, replay update, and trajectory optimizer.

This offers a different runtime tradeoff from a small feed-forward PPO actor: planning may improve data efficiency or adaptation but consumes more per-step compute and introduces planner/model failure modes.

Model-based alternatives through 2026

- **PETS** uses probabilistic ensembles and trajectory sampling to represent epistemic uncertainty, then plans without learning a separate actor.
- **Model-Based Policy Optimization (MBPO)** adds short learned-model rollouts to real replay, deliberately limiting horizon to control model bias.
- **DreamerV3** learns a generative recurrent latent world and amortized actor; it is attractive when observations are rich and online samples are scarce.
- **TD-MPC2** learns control-centered latents and retains decision-time planning; it trades actor simplicity for planner compute.
- **Known-physics MPC plus learned residual** preserves explicit constraints and asks learning to model only what nominal dynamics miss.

These methods answer different questions. Compare realized task success per real transition, total compute, planning deadline misses, model calibration, reset burden, and failure severity—not only a benchmark mean.

6.12 Known physics, learned residuals, and hybrid control

Robot learning need not choose between “all classical” and “all neural.” A hybrid can use:

- known rigid-body dynamics for nominal prediction;
- a learned residual for unmodeled friction or compliance;

- MPC for explicit constraints;
- an RL policy for fast local correction; and
- a hard safety supervisor outside both.

A **residual policy** outputs a bounded correction:

$$u = u_{nominal} + \Delta u_\theta, \qquad |\Delta u_\theta| \le u_{residual,max}.$$

This can reduce exploration scope and preserve an interpretable baseline. However, a poorly defined residual can still destabilize the nominal loop.

6.13 A fair comparison experiment

To compare PPO and SAC on one simulated robot:

1. freeze robot model, observations, actions, reward, resets, randomization, control rate, and evaluator;
2. give both enough network capacity but report parameter counts;
3. report environment transitions, wall-clock time, and graphics processing unit (GPU) time;
4. evaluate deterministic policies on identical held-out seeds and physics;
5. show median and uncertainty across training seeds;
6. include failure categories, not only average return; and
7. preserve resolved configs and checkpoints.

Changing algorithm and reward together answers no clean question.

6.14 Exercises

1. Why can a replay buffer improve sample efficiency but worsen distribution mismatch?
2. Explain why taking the minimum of two noisy critics can reduce optimistic targets. What new pessimistic bias can it introduce?
3. In SAC, what happens qualitatively as α approaches zero?
4. A true terminal transition has $d = 1$. Evaluate $y = r + \gamma(1 - d)V(s')$.
5. Why does planning farther into a learned model sometimes reduce real-world performance?
6. Compare actor-only inference with MPC through a learned model for a 100 Hz loop. What must be measured?
7. Design a bounded residual action for a wheeled-leg robot with a classical balance controller.
8. A learner performs 64 gradient sample uses for each new environment transition. What is its update-to-data ratio? Name two reasons increasing it further can hurt even though no new robot wear is incurred.
9. With current critic parameter 10, target parameter 4, and Polyak $\rho = 0.995$, compute the new target parameter.
10. Use the deterministic policy-gradient equation to explain how a false local Q peak can move an actor even if that action never appears in replay.
11. In SAC, why must a `tanh`-squashed Gaussian subtract a log-Jacobian term?
12. For model one-step error $\epsilon = 0.01$, Lipschitz factor $L = 1.2$, and horizon 3, evaluate the derived upper bound on state error.
13. Explain why CEM executes only the first planned action. What information is thrown away and why can discarding it be beneficial?
14. Compare PETS, DreamerV3, TD-MPC2, and known-physics MPC along: learned representation, decision-time planning, actor, and uncertainty/model risk.

Continue with robot dynamics, control, and estimation.

6.15 Folded solutions

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show answers to Section 6.14

1. Replay reuses each expensive transition and decorrelates adjacent samples, improving transition efficiency. But old data came from older policies and possibly older task/robot distributions, so its state-action coverage can differ from the policy currently being optimized.
2. If critic noise is sometimes optimistically high, taking the smaller of two estimates makes that error less likely to enter the target. The minimum can instead be systematically low, creating conservative or pessimistic bias.
3. As SAC temperature α approaches zero, entropy contributes less and the objective approaches ordinary reward-maximizing actor-critic behavior. Exploration/diversity pressure weakens.
4. With $d = 1$, the bootstrap multiplier is zero, so $y = r$. Bootstrapping beyond a true terminal state would incorrectly assign value after the episode has ended.
5. Small one-step model errors compound as predicted states become inputs to later predictions. Farther plans can exploit model mistakes or cross contact events the model predicts poorly; shorter receding horizons refresh from reality more often.
6. Measure worst-case—not just average—inference/planning time, jitter, deadline misses, horizon/sample count, model error, memory, observation age, action continuity, and fallback behavior. A 100 Hz loop has only 10 ms for the entire sensor-to-command path.
7. Let a classical balance controller output wheel target u_c and constrain the learned correction:

   ```
   residual_limit_rad_s = 1.0
   residual = max(-residual_limit_rad_s,
                  min(residual_limit_rad_s, actor_output))
   wheel_target = classical_target + residual
   ```

   Expire the residual on stale input or deadline miss, rate-limit the combined target, preserve microcontroller unit (MCU) current/speed/tilt limits, and prove that residual zero returns to the accepted baseline.
8. The ratio is 64 sampled transition uses per new transition. More reuse can overfit recent/narrow coverage, amplify bootstrapped critic error, make targets chase a fast learner, bias training toward stale behavior, or make optimizer compute dominate wall time. Robot wear is only one resource.
9. Polyak averaging gives

   $$\bar{\phi}_{new} = 0.995(4) + 0.005(10) = 3.98 + 0.05 = 4.03.$$

   The target moves only 0.03 toward the current critic.
10. The actor queries $\nabla_a Q(s, a)$ specifically at $a = \pi_\theta(s)$, not only at replay actions. If approximation creates a false rising slope/peak there, the chain rule turns that slope into an actor parameter update toward the unsupported action. Low error on stored actions does not constrain every queried gradient.
11. Probability density changes under a nonlinear coordinate transform. `tanh` compresses an unbounded interval near action limits, so equal intervals in pre-squash u do not map to equal intervals in a. The Jacobian correction accounts for that volume change; without it, entropy and actor objectives describe the wrong action density.
12. The bound is

    $$e_3 \le 0.01(1 + 1.2 + 1.2^2) = 0.01(3.64) = 0.0364.$$

    It is a worst-case norm bound under the stated uniform/Lipschitz assumptions, not an expected empirical error.
13. After one action, the system obtains a new real observation. Replanning replaces predicted state with measured/estimated state and can adapt to disturbances/model error. The remaining $H - 1$

candidate actions are discarded (or sometimes warm-start the next distribution); blindly executing them open-loop would accumulate prediction error.

14. PETS learns a probabilistic ensemble and plans at every decision without a required actor; ensemble disagreement exposes one kind of model uncertainty. DreamerV3 learns a recurrent stochastic latent world plus actor/critic and normally deploys the amortized actor; imagination error affects training. TD-MPC2 learns a control-centered latent, policy/value, and dynamics and retains online trajectory optimization. Known-physics MPC may fit only parameters/residuals, plans online, and can express constraints, but wrong nominal physics and state estimates remain risks. Runtime and data costs differ, so the comparison is conditional.

7. Robot Dynamics, Control, and State Estimation

A reinforcement learning (RL) policy becomes a robot controller only through mechanics, electronics, sensors, timing, and lower-level control loops. This chapter supplies the robotics background needed to understand those interfaces.

Why learning does not replace robotics

Robot control grew from mechanics, feedback, estimation, numerical optimization, and realtime systems. A learned policy enters an existing causal chain: sensors estimate a delayed physical state; software computes a bounded target; actuator electronics create torque; contacts feed forces back into the mechanism. The policy can approximate a difficult control law, but it cannot make frames, energy, latency, observability, or actuator limits disappear.

Modern alternatives are therefore usually compositional: inverse dynamics, whole-body control, or Model Predictive Control (MPC) for explicit structure; reinforcement learning for hard-to-model behavior; imitation for motion style; learned residuals for missing physics; and control-barrier/supervisory logic for constraints. "Classical versus learning" is rarely the most useful system boundary.

7.1 Configuration and degrees of freedom

A robot's **configuration** describes its pose. Joint coordinates are commonly collected in a vector q; joint velocities are $\dot{q}$ and accelerations are $\ddot{q}$.

A **degree of freedom** (DoF) is an independent coordinate needed to describe motion. A single revolute hinge has one DoF: its angle. A free rigid body in 3D has six: three position coordinates and three orientation coordinates.

Microduck has 14 actuated joint coordinates, but its simulated state also includes the free base pose and velocity. "14-action policy" therefore does not mean the full physical state has dimension 14.

Forward kinematics and the geometric Jacobian

Forward kinematics maps joint configuration to a point or link pose:

$$x = f(q).$$

For a small joint change Δq, first-order Taylor expansion gives

$$f(q + \Delta q) \approx f(q) + J(q)\Delta q,$$

where

$$J(q) = \frac{\partial f}{\partial q}$$

is the Jacobian. Dividing by a small time interval yields

$$\dot{x} = J(q)\dot{q}.$$

The transpose maps an end-effector wrench F into generalized joint torque:

$$\tau_{ext} = J(q)^T F.$$

This follows from equal virtual work:

$$F^T \delta x = F^T J \delta q = (J^T F)^T \delta q.$$

The Jacobian can lose rank at a **singularity**, where some Cartesian direction requires unbounded or unavailable joint velocity. A task-space RL action still needs inverse kinematics or a controller that handles these geometric limits.

7.2 Position, velocity, acceleration, force, and torque

- position tells where something is;
- velocity is the rate of position change;
- acceleration is the rate of velocity change;
- force changes linear momentum; and
- torque is the rotational counterpart of force.

For a revolute joint,

$$\tau = Ia$$

is the simplest analogy: torque τ produces angular acceleration a according to rotational inertia I. A whole robot is coupled, so moving one joint can accelerate many links.

Mechanical power is

$$P = F^T v \quad \text{or} \quad P = \tau^T \dot{q}.$$

Energy is the integral of power over time. A torque may be within its static limit yet demand excessive power at high speed; sustained current can also overheat a motor. This is why action, torque, speed, voltage, and temperature limits describe different safety constraints.

7.3 The robot dynamics equation

A standard compact rigid-body equation is

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) + f(\dot{q}) = \tau + J(q)^T F_{ext}.$$

Read it term by term:

- $M(q)\ddot{q}$: inertia resisting acceleration;
- $C(q, \dot{q})\dot{q}$: velocity-dependent Coriolis/centrifugal effects;
- $g(q)$: gravity torque;
- $f(\dot{q})$: friction and other losses;
- τ: actuator torque;
- $J(q)^T F_{ext}$: external/contact force mapped into joint torques.

You do not need to symbolically solve this equation to train Proximal Policy Optimization (PPO). You do need to understand that mass, center of mass, inertia, friction, contact, voltage, and delay change how the same command moves the robot.

Where the dynamics equation comes from

Let kinetic energy be $T(q, \dot{q})$ and potential energy be $U(q)$. The Lagrangian is $\mathscr{L} = T - U$. For each generalized coordinate q_i, the Euler–Lagrange equation is

$$\frac{d}{dt}\frac{\partial \mathscr{L}}{\partial \dot{q}_i} - \frac{\partial \mathscr{L}}{\partial q_i} = Q_i,$$

where Q_i collects actuator and external generalized forces. Expanding these equations for linked rigid bodies produces the inertia matrix, velocity terms, and gravity vector. This derivation explains useful structure:

- $M(q)$ is symmetric and positive definite away from constrained/redundant coordinates;
- gravity comes from the gradient of potential energy;
- contact forces enter through $J^T F$ because of virtual work; and
- actuator/friction models add nonconservative generalized forces.

A simulator numerically integrates these accelerations. With semi-implicit Euler for a simple coordinate:

$$\dot{q}_{k+1} = \dot{q}_k + \Delta t\, \ddot{q}_k,$$

$$q_{k+1} = q_k + \Delta t\, \dot{q}_{k+1}.$$

Changing timestep changes integration error and contact behavior even when the continuous equation is unchanged.

7.4 Coordinate frames

A vector is incomplete without a frame. “Velocity [1, 0, 0]” could mean world east or robot-forward.

Common frames are:

- **world frame**: fixed to the environment;
- **base/body frame**: moves with the trunk;
- **sensor frame**: aligned with an inertial measurement unit (IMU) or camera mounting; and
- **joint frame**: defined by the robot model’s joint axis.

A rotation matrix R_{WB} can map a body-frame vector into world coordinates:

$$v_W = R_{WB} v_B.$$

A reversed transform, wrong axis sign, or degrees/radians mismatch can look like a failed policy even if the network is correct.

7.5 Orientation representations

Common orientation representations include:

- roll-pitch-yaw/Euler angles: intuitive but singular at some orientations;
- rotation matrices: 9 numbers with orthogonality constraints;
- quaternions: 4 numbers with unit-length constraint; and
- projected gravity: gravity direction expressed in the body frame.

Projected gravity is useful for locomotion because it tells the policy which way is “down” without exposing a fragile Euler-angle parameterization. A quaternion and its negation represent the same physical rotation, which must be considered in errors and interpolation.

A unit quaternion $q = [w, x, y, z]$ satisfies

$$w^2 + x^2 + y^2 + z^2 = 1.$$

For an axis-angle rotation with unit axis u and angle θ:

$$q = \left[\cos\frac{\theta}{2},\ u\sin\frac{\theta}{2}\right].$$

Because q and $-q$ encode the same rotation, a naive Euclidean error $\|q - q^*\|$ can call identical orientations far apart. A sign- invariant similarity uses $|q^T q^*|$, and a shortest-path interpolation flips one quaternion when the dot product is negative.

7.6 Contacts make robot dynamics hybrid

A walking robot alternates between continuous flight/swing motion and discrete contact events. Impact can abruptly change velocity. Contact includes normal force, friction, compliance, collision geometry, and numerical solver choices.

This is a **hybrid system**: continuous dynamics plus mode switches. Small differences in foot geometry or timing can produce large rollout differences. That is one reason long sim and real trajectories need not be bit-identical even when the control interface matches.

An ideal unilateral contact has gap $\phi(q) \geq 0$ and normal force $\lambda_n \geq 0$. They satisfy complementarity:

$$\phi(q)\lambda_n = 0.$$

Either bodies are separated and normal force is zero, or the gap is closed and a nonnegative contact force may exist. Impacts add velocity-level rules and friction adds tangential constraints. Engines approximate or regularize these nonsmooth conditions differently, so solver iteration count, compliance, and collision geometry are part of the learned environment.

Coulomb friction intuition

A simple contact model limits tangential friction force:

$$|F_t| \leq \mu F_n,$$

where F_n is normal force and μ is a friction coefficient. If requested tangential force exceeds the limit, the foot slips. Real tires/feet also show compliance, velocity dependence, surface variation, and wear.

7.7 Actuators are not ideal commands

An **actuator** converts electrical energy into mechanical motion or force. Common robot actuators include direct-current and brushless direct-current (BLDC) motors with transmissions and integrated servos.

An ideal simulator position actuator instantly producing any required torque is physically impossible. Real behavior depends on:

- voltage and battery sag;
- motor torque/current limits;
- speed-torque relationship;
- gear ratio and efficiency;
- friction, backlash, and compliance;
- embedded-controller gains and rate;
- command/communication latency; and
- thermal protection.

A minimal voltage-to-torque derivation

For a simple direct-current motor,

$$V = Ri + L\frac{di}{dt} + k_e\omega,$$

$$\tau_m = k_t i.$$

V is applied voltage, i current, R resistance, L inductance, $k_e\omega$ back electromotive force, and k_t torque constant. At steady current, ignoring inductance:

$$i \approx \frac{V - k_e\omega}{R}, \qquad \tau_m \approx \frac{k_t}{R}(V - k_e\omega).$$

Available torque falls as speed increases because back electromotive force consumes more of the voltage budget. Battery sag lowers V; heating changes R; gearing transforms torque/speed and adds friction/backlash. An ideal position actuator with unlimited torque misses this causal chain.

Better Actuator Models (BAM) in the Microduck project

BAM means **Better Actuator Models**, the actuator-modeling approach used by the project for Dynamixel XL330 servos. Here it refers to a voltage-aware servo model with its own friction behavior, not a generic RL algorithm.

That distinction has a concrete consequence: randomizing MuJoCo's ordinary joint `dof_frictionloss` does not randomize the effective friction authority when BAM computes it inside the actuator. The project scales BAM's `friction_scale` instead.

7.8 Feedback control and proportional–derivative (PD) control

A feedback controller compares a target to a measurement. A proportional- derivative (PD) joint controller is

$$\tau = K_p(q_{target} - q) - K_d\dot{q}.$$

- proportional term pushes against position error;
- derivative term damps motion;
- K_p and K_d are gains.

High gains can track tightly but amplify noise, excite unmodeled dynamics, or hit current limits. Low gains are compliant but may need target overshoot to produce enough torque. Thus a policy's position target can intentionally lie beyond the actual desired joint position while the physical joint remains within limits.

Gain meaning from a one-joint closed loop

For one ideal joint with inertia I, viscous damping b, fixed target q^*, and error $e = q - q^*$, substitute PD torque into $I\ddot{q} + b\dot{q} = \tau$:

$$I\ddot{e} + (b + K_d)\dot{e} + K_p e = 0.$$

Divide by I and compare with the standard second-order form

$$\ddot{e} + 2\zeta\omega_n\dot{e} + \omega_n^2 e = 0.$$

This gives

$$\omega_n = \sqrt{\frac{K_p}{I}}, \qquad \zeta = \frac{b + K_d}{2\sqrt{IK_p}}.$$

ω_n is a nominal natural frequency and ζ a damping ratio. Larger K_p raises response frequency; K_d raises damping. Torque saturation, sample delay, coupled links, contact, friction, and motor dynamics invalidate the ideal formula quantitatively, but it supplies a starting hypothesis.

Run `examples/pd_joint_and_kalman.py`. Its `simulate_joint` maps the equation to an integration loop and changes only feedback delay; it is a teaching model, not a servo-identification substitute.

7.9 Choosing the policy action

An RL action need not equal motor current. Common choices are:

Action	Advantage	Risk/requirement
torque/current	direct dynamic authority	hard transfer and safety; needs fast loop
joint velocity target	natural for wheels	requires matched velocity controller
absolute joint position	interpretable	can jump if not rate-limited
position delta from HOME	centered, normalized	target scale and HOME must match
residual on nominal controller	bounded learning scope	nominal/residual interaction
task-space target	meaningful geometry	needs inverse kinematics (IK) or a controller underneath

Microduck uses 14 normalized actions that become joint-position targets around the default pose. The lower-level actuator dynamics turn targets into motion.

7.10 Nested control loops

A safe robot usually has several rates and responsibilities:

```
high-level planner / behavior selection        ~10 Hz
            |
local navigation / learned skill command      ~50 Hz
            |
RL locomotion policy                          ~50–200 Hz
            |
joint position/velocity control             ~1000–1000 Hz
            |
current/field-oriented motor control          ~40 kHz
            |
mechanics and contacts                      continuous
```

These are illustrative ranges, not universal specifications. Each robot must measure timing and stability requirements.

The fastest loop should not depend on a cloud round trip. A network outage must not remove motor current limits, watchdogs, balance protection, or emergency stop.

7.11 Sampling rate, latency, and delay

A 50 Hz policy period is

$$T = \frac{1}{50} = 0.02 \text{ s} = 20 \text{ ms}.$$

If physics simulates at 5 ms and the action is held for four physics steps, the policy sees a new observation every 20 ms. This hold count is called **decimation** in many robotics simulators.

Latency is time between measurement/decision and effect. **Jitter** is variation in that latency. A constant 10 ms delay and a delay varying from 0–20 ms can affect stability differently.

A pure delay T_d contributes frequency-dependent phase lag

$$\phi(\omega) = -\omega T_d$$

in radians. At angular frequency $\omega = 20$ rad/s, a 20 ms delay adds –0.4 rad, about –23°. Feedback that arrives too late can reinforce motion instead of damping it.

Sampling at frequency f_s cannot uniquely represent a sinusoid at or above the Nyquist frequency $f_s/2$. Real control needs margin well below that bound because filters, zero-order action holds, inference, bus transport, and actuators all add phase lag. “The policy runs at 50 Hz” is therefore incomplete without the bandwidth of the behavior it must stabilize.

Deployment must define:

- when sensors are sampled;
- whether measurements have different timestamps;
- inference worst-case execution time (WCET), not only average time;
- when commands reach actuators; and
- what happens on deadline miss.

An explicit timestamped loop is preferable to assuming every field is “now”:

```
sample = read_sensors_with_timestamp()
state = estimator.propagate_and_update(sample)
observation = contract.encode(state, command, previous_action)
action = actor(observation)
if clock.now() - sample.timestamp > maximum_observation_age:
    enter_fallback("stale observation")
else:
    send_rate_limited_target(action)
```

Training delay randomization should reproduce the measured distribution and age semantics of this loop. Random delay can teach robustness; it does not repair missing timestamps or an unbounded queue.

7.12 State estimation

The policy rarely observes the true state. A **state estimator** combines sensor readings and a process model to estimate quantities such as orientation or velocity.

Typical inputs:

- encoders: joint/motor position and sometimes velocity;
- IMU gyroscope: angular velocity;
- IMU accelerometer: specific force, including gravity effects;
- contact/force sensors;
- cameras, depth sensors, or light detection and ranging (LiDAR); and
- motor current/voltage/temperature.

An estimator may use complementary filtering, a Kalman-filter family, optimization, or a learned model. Every estimate has bandwidth, bias, noise, delay, and frame conventions.

Scalar Kalman update before the matrix version

Suppose a prior state estimate is Gaussian with mean $\hat{x}^-$ and variance P^-, and a direct measurement $z = x+v$ has independent noise variance R. The Kalman gain is

$$K = \frac{P^-}{P^- + R}.$$

The posterior is

$$\hat{x}^+ = \hat{x}^- + K(z - \hat{x}^-),$$

$$P^+ = (1 - K)P^-.$$

The term $z - \hat{x}^-$ is the **innovation**: what the sensor says beyond the prediction. If prior uncertainty is large relative to sensor noise, K is near 1 and the estimate moves toward the measurement. If the sensor is noisy, K is small and the prior dominates.

The matrix Kalman filter repeats this idea with a dynamics prediction, covariances, and an observation matrix. An extended Kalman filter linearizes nonlinear motion/sensors with Jacobians. All require calibrated noise and model assumptions; a covariance number is not automatically honest uncertainty.

The runnable `scalar_kalman_update` function implements these three equations. Change prior and measurement variances before running it and predict the gain direction.

Observation is not state

An observation o_t is what the policy receives. Simulator state s_t may also contain exact ground-truth velocity, contact forces, terrain parameters, or future commands unavailable on hardware.

Using richer information for a training-only critic is called **asymmetric actor-critic** or **privileged critic** training. It is valid only if the actor input remains deployable.

7.13 System identification before randomization

System identification estimates model parameters from measured input-output data. For an actuator:

1. command a safe excitation;
2. log target, encoder position/velocity, current, voltage, and timestamps;
3. fit delay, gain, friction, damping, or other parameters;
4. validate on a held-out motion; and
5. randomize around the residual uncertainty.

Randomization is not a substitute for calibration. A wildly wrong nominal model plus a huge randomization range can teach unnecessarily conservative or unphysical behavior.

Least squares as the simplest identification map

Suppose a low-speed joint model is

$$\tau_t = I\ddot{q}_t + b\dot{q}_t + \tau_c \operatorname{sign}(\dot{q}_t).$$

For each measured time step, form a feature row

$$\varphi_t = [\ddot{q}_t,\ \dot{q}_t,\ \operatorname{sign}(\dot{q}_t)]$$

and parameters $\theta = [I, b, \tau_c]^T$. Stacking samples gives

$$y = \Phi\theta + \epsilon.$$

Ordinary least squares minimizes squared residuals:

$$\hat{\theta} = \arg\min_{\theta} \|\Phi\theta - y\|_2^2.$$

When $\Phi^T\Phi$ is invertible,

$$\hat{\theta} = (\Phi^T\Phi)^{-1}\Phi^T y.$$

The formula does not rescue uninformative data. If velocity barely changes, inertia and damping may be impossible to separate. **Persistent excitation** means the safe input explores enough frequencies/directions to identify the chosen parameters. Derivative estimates amplify noise, current is not always torque, and friction is nonlinear; validate on held-out trajectories before turning a fit into randomization ranges.

7.14 Safety is a separate objective and mechanism

A reward penalty is a preference in expectation. It is not a hard guarantee. Hardware safety needs independent mechanisms:

- mechanical stops and suitable structure;
- current, voltage, speed, position, and temperature limits;
- watchdog and command timeout;
- communication validity and sequence checks;
- bounded policy outputs and rate limits;
- fall/tilt detection;
- physical emergency stop;
- tether/test stand during early trials; and
- a known rollback controller/artifact.

Constrained RL can represent expected costs separately from reward. For example, Constrained Policy Optimization optimizes return under expected constraints. Its theoretical/empirical properties do not replace electrical or realtime interlocks.

Expected constraints, barrier filters, and hard interlocks

A constrained Markov Decision Process can state

$$\max_{\pi} J_R(\pi) \quad \text{subject to} \quad J_{C_i}(\pi) \le d_i.$$

This limits an expected discounted cost. A rare catastrophic violation can still be compatible with an acceptable expectation.

A control barrier function instead defines a safe set $\mathscr{C} = \{x : h(x) \ge 0\}$ and filters a proposed action by requiring, in a continuous-time model,

$$\dot{h}(x, u) + \alpha(h(x)) \ge 0.$$

A small quadratic program can choose the closest action satisfying this local condition when the model and constraint are valid. Barrier certificates have stronger structure than a reward penalty, but sensor error, model error, infeasible constraints, discretization, and actuator saturation still need handling. Electrical current cutoffs and physical emergency stop remain a separate final authority.

As of 2026, safe robot learning is best treated as layers: train with costs and randomization, filter commands where a verified model permits, enforce realtime limits independently, and evaluate tail failures. Calling one layer “safe RL” does not prove the whole robot safe.

7.15 Exercises

1. A policy uses 5 ms physics and decimation 4. Compute its rate.
2. Explain each term in the rigid-body equation in one sentence.
3. Why might a low-K_p servo need a command outside the desired physical angle, and why must the real joint still have a safety limit?
4. Choose an action representation for a wheel and for a servo-height joint.
5. Draw the frames for a body IMU mounted rotated 90° about yaw. Which transform must the observation pipeline apply?
6. Give one constant bias, one random noise source, and one latency source on a real robot.
7. Explain why a reward penalty cannot serve as an emergency stop.
8. Design a system-identification trial that is informative but safe.
9. A two-link endpoint has Jacobian $J = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$ and joint velocity $\dot{q} = [0.2, -0.1]^T$. Compute endpoint velocity. Then map force $F = [3, 4]^T$ to joint torque.
10. A motor has $V = 8$ V, $R = 4\ \Omega$, $k_e = 0.2$ V·s/rad, $k_t = 0.2$ N·m/A, and $\omega = 10$ rad/s. Ignoring inductance, compute current and torque. What happens if battery voltage falls?
11. For $I = 0.02$, $b = 0.05$, $K_p = 8$, and $K_d = 0.8$, compute the ideal natural frequency and damping ratio. Name two effects omitted by this model.
12. Compute phase lag from a 15 ms delay at $\omega = 30$ rad/s in radians and degrees. Why is average inference time insufficient?
13. A scalar prior has mean 0, variance 0.25; a measurement is 0.4 with variance 0.01. Compute Kalman gain, posterior mean, and posterior variance.
14. Why can quaternion q and $-q$ break a naive squared pose loss? Give a sign-invariant alternative.
15. Explain what makes a least-squares identification matrix singular or ill-conditioned. Propose a safe excitation improvement.
16. Distinguish an expected constrained-RL cost, a control-barrier filter, and a motor-driver overcurrent trip by the guarantee each can and cannot offer.

Continue with the Microduck software and control architecture, where these concepts become concrete interfaces.

7.16 Folded solutions

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show answers to Section 7.15

1. The policy period is 5 ms × 4 = 20 ms, so its rate is 1/0.020 = 50 Hz.
2. In $M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \tau + J^T f_{ext}$: M maps joint acceleration to inertial torque; $C\dot{q}$ contains velocity-dependent Coriolis/centrifugal effects; g is gravity torque; τ is commanded or actuator torque; and $J^T f_{ext}$ maps external/contact forces into joint torque.
3. With low proportional gain, static load may leave a position error; a target beyond the desired angle can create enough torque to hold the actual angle. The physical joint still needs independent limits because target overshoot can otherwise request collision, overload, or hard-stop impact.
4. A wheel naturally accepts bounded velocity or torque/current target under a lower wheel loop. A height servo naturally accepts a bounded position target with rate/limit enforcement. The exact choice follows measured actuator and controller semantics.
5. Define sensor frame S and body frame B. For a sensor mounted +90° about body yaw, apply the calibrated rotation R_{BS} to every sensor-frame vector before constructing body-frame observations. Test known gravity and angular- rate directions; do not repair a fixed mounting transform with randomization.
6. Constant bias: gyro zero offset. Random noise: encoder quantization or IMU measurement noise. Latency: filtering plus bus/queue/inference delay.

7. A reward penalty changes expected optimization preference. It can be traded against positive reward, has approximation error, and acts only when the policy runs. An E-stop must independently and deterministically remove or bound actuator authority.
8. Clamp the robot in a fixture, begin at low voltage/current, and command a small multisine or step sequence inside accepted travel. Log command, encoder, current, voltage, load, and synchronized timestamps. Fit delay, gain, damping/friction on one sequence; validate on a different sequence; stop on current, temperature, position, velocity, or watchdog limits.
9. Endpoint velocity is

$$\dot{x} = J\dot{q} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 0.2 \\ -0.1 \end{bmatrix} = \begin{bmatrix} 0.2 \\ 0.1 \end{bmatrix}.$$

   Joint torque is

$$\tau = J^T F = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 7 \\ 4 \end{bmatrix}.$$

10. Back electromotive force is $k_e\omega = 2$ V. Current is $(8 - 2)/4 = 1.5$ A and torque is $0.2(1.5) = 0.3$ N·m. Lower battery voltage reduces the current and torque available at the same speed; if voltage approaches back electromotive force, motoring torque approaches zero in this simplified model.
11. $\omega_n = \sqrt{8/0.02} = 20$ rad/s. The damping ratio is $(0.05 + 0.8)/(2\sqrt{0.02 \times 8}) = 0.85/0.8 = 1.0625$, slightly overdamped in the ideal unsaturated model. Omissions include torque/current saturation, discrete delay, link coupling, contact, friction nonlinearities, motor electrical dynamics, and sensor noise.
12. $\phi = -\omega T_d = -30(0.015) = -0.45$ rad, about -25.8°. Worst-case time and jitter determine whether individual cycles lose enough phase margin or miss deadlines; a safe mean can hide rare long delays.
13. $K = 0.25/(0.25 + 0.01) = 0.961538$. The posterior mean is $0 + K(0.4) = 0.384615$, and posterior variance is $(1 - K)0.25 \approx 0.009615$. The confident measurement dominates.
14. They represent the same orientation but $\|q-(-q)\|^2 = 4$ for a unit quaternion. Use $1-|q^Tq^*|$, $1-(q^Tq^*)^2$, or flip the target sign so the dot product is nonnegative before computing a shortest-path error.
15. If excitation does not vary acceleration, velocity, or direction independently, feature columns become dependent or nearly so and several parameter combinations explain the same trace. Add bounded multi-frequency or multisine input across safe amplitudes/directions, measure synchronized outputs, and validate the resulting condition number and held-out error.
16. Expected constrained RL shapes a policy so average discounted cost stays under a learned/estimated budget; it allows estimation error and possibly rare violations. A barrier filter modifies proposed actions to keep a modeled state inside a mathematically defined safe set when its assumptions and feasibility hold. A motor-driver trip directly removes/bounds electrical authority when measured current crosses a threshold, independent of policy reward; it does not by itself guarantee balance or geometric collision avoidance.

8. Software and Control Architecture

A reinforcement learning (RL) experiment is not just a neural network. It is a chain of models, interfaces, clocks, and artifacts. A policy can be mathematically correct and still fail if one link changes between training and deployment.

This chapter is a worked software-architecture case study. The goal is not to memorize filenames; it is to learn how a mathematical reinforcement-learning problem becomes versioned interfaces whose units, order, clocks, and owners can be tested.

8.1 The project stack

```
Onshape/CAD and measured robot facts
        |
        v
MuJoCo XML model (MJCF) + collision models + HOME pose
        |
        v
MuJoCo physics through MuJoCo Warp on the GPU
        |
        +---- BAM XL330 actuator model
        +---- sensors, contacts, delays, noise, domain randomization
        |
        v
mjlab manager-based RL environment
        |
        v
RSL-RL PPO actor/critic and rollout storage
        |
        +---- checkpoints and metrics
        v
ONNX export with observation normalizer
        |
        v
CPU runtime at 50 Hz on the real robot
```

The major dependencies have distinct jobs:

Component	Responsibility
MuJoCo	rigid-body dynamics, joints, contacts, sensors
MuJoCo Warp / Warp	NVIDIA Compute Unified Device Architecture (CUDA) simulation across many graphics processing units (GPUs)
`mjlab`	environment managers, task registry, vectorized wrapper, viewer, export support
Robotic Systems Lab reinforcement learning (RSL-RL)	Proximal Policy Optimization (PPO), neural networks, rollout buffers, checkpoints
Better Actuator Models (BAM)	voltage-aware Dynamixel XL330 actuator and friction behavior
this repository	Microduck robot models, tasks, rewards, randomization, tests, export and rehearsal
Microduck runtime repository	real sensors, command sources, Open Neural Network Exchange (ONNX) inference, motor communication

MuJoCo's Extensible Markup Language (XML) model format (MJCF) is its robot/model description format. CUDA is NVIDIA's GPU-computing platform. ONNX is a portable neural-network file format. **Inference** means running a frozen trained network to obtain an output; unlike training, it does not update weights.

Four graphs coexist

Beginners often see only the neural-network graph. A deployed learned controller actually has four coupled graphs:

1. **physical graph**: bodies, joints, contacts, actuators, sensors;
2. **dataflow graph**: timestamped measurements to observation to action;
3. **learning graph**: rollout, returns, actor/critic losses, optimizer;
4. **artifact graph**: source revision, resolved configuration, checkpoint, normalizer, export, runtime build.

An edge mismatch in any graph can cause failure. For example, swapping two joint observations preserves tensor shape and neural arithmetic but changes the dataflow/physical mapping. Loading an actor without its normalizer preserves weights but breaks the artifact graph.

This architecture descends from two modern scaling choices: vectorized physics places many environment copies on a graphics processor, and a manager-based task expresses observations/rewards/events as independently configured terms. Alternative stacks may use one monolithic `step()` function, a differentiable simulator, an off-policy replay service, or distributed actor processes. The invariant questions remain: who owns state, when is it sampled, how is an action transformed, and which artifact proves the answer?

8.2 Repository map as a learning map

```
src/mjlab_microduck/├──
 robot/│
   ├── microduck_constants.py       robot choice, HOME pose, actuator setup│
   └── microduck/                   MJCF, scenes, meshes, backlash generator├──
 actuator/│
   └── friction_dr_bam.py           actuator friction DR and backlash feedback├──
 tasks/│
   ├── __init__.py                  task IDs and runner registration│
   ├── mdp.py                       custom rewards, events, observations, curricula│
   ├── symmetry.py                  optional 61D mirror transform│
   ├── backlash.py                  base-task to backlash-task wrapper│
   └── microduck_*_env_cfg.py       task-family configuration factories└──
 train_cli.py                     local/Hugging Face training entry point

scripts/├──
 export.py                        checkpoint -> normalized ONNX├──
 infer_policy.py                  CPU MuJoCo deployment rehearsal└──
 testbench_sim2real.py            measured actuator comparison tools

tests/                               CPU regression and configuration tests
logs/rsl_rl/<experiment>/<run>/      checkpoints, parameters, metrics, videos
```

Read in dependency order. Start with the task registration, then its configuration factory, then follow only the custom functions it references into `mdp.py`. A Markov Decision Process (MDP) organizes state, action, transition, reward, and discount; reading all of the large `mdp.py` module from top to bottom is not a good beginner exercise.

The live upstream links make the theory-to-code mapping inspectable:

- task registry maps a task name to concrete factories and runner;
- velocity configuration specializes scene, commands, observations, reward, events, and PPO settings;
- custom MDP functions implement the project-specific mathematics; and
- robot constants bind models, HOME pose, collisions, and BAM actuators.

Pin the commit used by a run. A link to `main` is a navigation aid, not reproducibility evidence.

8.3 Task registration

Every user-facing task **identifier (ID)** binds four things:

```
# Abridged from tasks/__init__.py
```

```
register_mjlab_task(
    task_id="Mjlab-Velocity-Flat-MicroDuck",
    env_cfg=make_microduck_velocity_env_cfg(),
    play_env_cfg=make_microduck_velocity_env_cfg(play=True),
    rl_cfg=MicroduckRlCfg,
    runner_cls=MicroduckOnPolicyRunner,
)
```

- `env_cfg` is the training environment.
- `play_env_cfg` makes one policy easier to inspect, often with fewer or more visible disturbances.
- `rl_cfg` defines the networks and PPO hyperparameters.
- `runner_cls` connects the environment to RSL-RL.

`uv run list-envs` reads this live registry. Documentation can become stale; the registry is executable truth.

Registration happens at Python import time. The factory call creates a concrete configuration object; managers later copy/resolve its term configurations. This has two consequences:

- changing source after a run does not retroactively change its saved resolved configuration; and
- mutating `env.cfg` after manager initialization may not change the manager's own copied term. Runtime curricula must use manager accessors such as `get_term_cfg(...)`.

The object lifecycle is approximately:

```
import task package
  -> execute registrations
  -> select task ID
  -> construct/copy environment config
  -> compile MuJoCo scene and resolve entity selectors
  -> construct managers from resolved term configs
  -> construct RSL-RL runner and networks
  -> collect rollout/update/save
```

A regular expression that matches the wrong joint can therefore fail during resolution before a learning step. Central processing unit (CPU) configuration tests deliberately force that early, cheap failure.

8.4 Manager-based environment anatomy

`mjlab` divides environment behavior into managers:

Manager/config section	Question it answers
`scene`	What robot, terrain, sensors, and spacing exist?
`actions`	How do policy outputs become simulator controls?
`observations`	What numeric information reaches actor and critic?
`commands`	What behavior is requested this episode?
`events`	What changes at startup, reset, or intervals?
`rewards`	What outcomes produce learning signal?
`terminations`	When is an episode over?
`curriculum`	How does difficulty or a parameter change with training steps?
`metrics`	What diagnostic values are logged without changing learning?

The velocity factory starts from `mjlab`'s closest locomotion template and modifies it:

```
def make_microduck_velocity_env_cfg(play=False, rough=False):
    cfg = make_velocity_env_cfg()
    cfg.scene.entities = {"robot": MICRODUCK_WALK_ROBOT_CFG}
    cfg.scene.sensors = (
        feet_ground_cfg,
        self_collision_cfg,
        foot_height_scan_cfg,
    )
    # Then specialize actions, observations, commands, rewards, DR, and terrain.
```

```
    return cfg
```

Building on a maintained template retains important sensor and safety wiring. A standalone task must recreate that wiring deliberately.

Manager composition is analogous to factoring an equation. If reward terms f_i and weights w_i are registered independently, the manager evaluates

$$r_t = \sum_i w_i f_i(s_t, a_t, s_{t+1}; \eta_i),$$

where η_i denotes term parameters such as standard deviation, command name, sensor selector, or gate. The resolved configuration must record both w_i and η_i; a scalar total reward cannot reconstruct either.

The same principle applies to observations. Concatenation is an interface function

$$o_t = \mathrm{concat}(g_1(s_t), g_2(s_t), \ldots, g_k(s_t)).$$

Changing term order changes the mathematical function even when total dimension remains 61.

8.5 One 20 ms policy step

The physics timestep is 5 ms and action **decimation** is 4, meaning one policy decision is reused for four smaller physics steps. One policy action is therefore held across four physics steps:

```
t = 0 ms     assemble observation; run actor; set action target
t = 5 ms     physics + actuator substep
t = 10 ms    physics + actuator substep
t = 15 ms    physics + actuator substep
t = 20 ms    physics + actuator substep; compute next observation/reward
```

This gives a 50 Hz policy rate while resolving contacts and actuator dynamics at 200 Hz. Changing either clock changes the control problem and must be matched in deployment.

The rate calculation is

$$\Delta t_{policy} = d\,\Delta t_{physics} = 4(0.005) = 0.020 \text{ s},$$

$$f_{policy} = \frac{1}{\Delta t_{policy}} = 50 \text{ Hz}.$$

Action holding is part of the transition kernel. A policy trained at 50 Hz and executed at 100 Hz does not merely react faster: it applies twice as many targets over the same physical time and receives differently spaced state changes.

Simulation time, wall time, and data age

One default rollout contains

$$4096 \text{ environments} \times 24 \text{ steps} = 98{,}304 \text{ transitions}.$$

Each environment advances only 24/50 = 0.48 simulated seconds, while the aggregate batch represents about 4096(0.48) = 1966 simulated robot-seconds. Parallelism increases throughput; it does not make any one trajectory see a longer future.

Deployment adds an **age of information** budget. If an inertial measurement was sampled at t_s, assembled into an observation at t_o, inference ends at t_i, and a servo consumes the command at t_a, then its

effective age at actuation is $t_a - t_s$. Queueing every stale sensor packet can make this age grow even when no packet is lost. A low-level control path normally wants a bounded latest-value queue, timestamps, and explicit stale-data fallback.

Training delay/noise must mimic this causal order. Adding independent noise to an already current vector is not the same as sampling heterogeneous sensor timestamps, holding an old action, or delaying a bus command.

8.6 Observation contract

The actor layout is intentionally fixed across the policy family:

```
[0:3]    base angular velocity
[3:6]    projected gravity
[6:20]   14 relative joint positions
[20:34]  14 joint velocities
[34:48]  14 previous actions
[48:51]  twist: vx, vy, yaw rate
[51:55]  head pose: four joint deltas
[55:61]  body pose: x, y, z, roll, pitch, yaw deltas
```

Unused command slots remain present and are zero-padded or sampled over tiny ranges. Deleting a term would change the input size and prevent runtime policy hot-swapping.

The actor excludes true base linear velocity because the real robot does not have a direct perfect measurement. The critic includes it because privileged information can improve value estimates during simulation training.

Observation corruption models hardware imperfections. The broader practice of sampling physical parameters and disturbances is called **domain randomization**: training across a range of simulated domains so the real robot is less likely to fall outside the learned experience.

- small additive sensor noise;
- encoder bias;
- inertial measurement unit (IMU) mounting variation;
- delayed angular velocity, gravity, joint velocity, and actuation;
- backlash-specific encoder views.

If the actor sees a biased or backlash-adjusted quantity, a tracking reward on that quantity must use the same view. Otherwise the policy is punished for correcting the measurement it receives.

The actual network input is normalized featurewise:

$$\tilde{o}_{t,i} = \frac{o_{t,i} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}}.$$

Thus the actor contract is not only “61 floats.” It is the ordered tuple

```
(meaning, unit, frame, timestamp/age, corruption, mean, variance)
```

for every feature. A useful deployment test freezes several physically named states and compares the complete normalized 61-vector between training-side and runtime encoders before comparing actions.

Previous action occupies 14 dimensions because commanded target history helps the actor infer actuator/delay state. It is not a complete memory: hidden temperature, payload, friction, and older contact history may still make the observation partially Markov. Alternatives include stacking more history, a recurrent actor, or a dedicated adaptation estimator; each increases training and deployment state that must be synchronized.

8.7 Action contract

The actor emits 14 floating-point values. The joint-position action uses `scale=1.0` and the model’s default joint pose as its offset. Conceptually:

$$q^{target} = q^{HOME} + a$$

The exact action manager also applies its configured ordering and any limits. The target then enters the BAM actuator model; it is not an ideal instantaneous joint angle.

The units expose why `scale=1.0` is not "no transform." A normalized actor number of 0.1 becomes a 0.1 radian target offset, about 5.73°:

$$0.1 \text{ rad} \times \frac{180^\circ}{\pi} \approx 5.73^\circ.$$

Action order is a matrix permutation. If runtime servo order differs, the physical target is

$$q^{runtime} = q^{HOME} + Pa,$$

where permutation matrix P should be identity for the documented contract. A shape check cannot detect $P \neq I$; a one-hot action probe can.

The 14 servo order is:

```
0..4    left hip yaw, hip roll, hip pitch, knee, ankle
5..8    neck pitch, head pitch, head yaw, head roll
9..13   right hip yaw, hip roll, hip pitch, knee, ankle
```

Roller and backlash MJCF models contain interleaved passive joints. Custom MDP code must use `_servo_joint_ids` and `_servo_joint_pos` rather than assuming that MuJoCo joint index equals servo index. All unactuated joints begin with `passive_`, and actuator/observation selectors exclude that prefix.

Network size and rollout memory are calculable

For the actor mean path `61` → `512` → `256` → `128` → `14`, including a bias per output neuron, the learned scalar count is

$$(61 \times 512 + 512) + (512 \times 256 + 256) + (256 \times 128 + 128) + (128 \times 14 + 14) = 197{,}774.$$

A learned 14-value log-standard-deviation vector brings the training actor to about 197,788 scalars, depending on the exact RSL-RL distribution implementation. At 32-bit floating point, the mean network weights occupy about 0.75 mebibytes (MiB) before file-format metadata and normalizer state. This explains why a small feed-forward actor can be plausible on a robot central processing unit (CPU), but measured worst-case latency still decides.

Rollout storage is larger because it scales with environments and time. Actor observations alone require approximately

$$4096 \times 24 \times 61 \times 4 = 23{,}986{,}176 \text{ bytes} \approx 22.9 \text{ MiB}.$$

Critic observations, actions, rewards, values, log-probabilities, advantages, masks, optimizer activations, simulator state, and gradients add substantially more. This is why "model size" and "training memory" are different budgets.

8.8 The actuator is part of the environment

The real XL330 is not an ideal position source. The BAM configuration includes firmware position-loop behavior, motor electrical effects, torque limits, friction, battery voltage, load-dependent voltage sag, and command delay.

The main configuration randomizes values such as:

```
FrictionDRBamActuatorCfg(
    motor_name="xl330",
    model="m6",
    target_names_expr=(r"^(?!passive_).*",),
    kp_fw=200.0,
    vin_range=(6.5, 8.2),
    vin_drop_gain_range=(0.0, 0.2),
    vin_min=6.0,
    delay_min_lag=3,
    delay_max_lag=6,
)
```

This is a real project example of model-based sim-to-real engineering: the policy learns to control a family of plausible actuators rather than a perfect one. Under BAM, MuJoCo's ordinary `dof_frictionloss` is not the friction authority; friction randomization must scale the actuator's own field.

Formally, let physical parameters be ξ—voltage, delay, friction, mass, center of mass, encoder bias, and so on. Domain-randomized training optimizes

$$J(\theta) = \mathbb{E}_{\xi \sim p_{train}(\xi),\, \tau \sim \pi_\theta, P_\xi}[G(\tau)].$$

The distribution p_{train} is part of the task. If real parameters lie outside its support, the expectation provides no coverage argument. If it is unphysically broad, one actor may become unnecessarily conservative or unable to distinguish contradictory dynamics.

At a 5 ms actuator update, a lag of three to six update slots corresponds to a nominal 15–30 ms command delay **if** the active actuator implementation defines lag in physics-update slots. Verify that dependency version rather than infer units from the field name. A configuration range without its update clock is not a physical specification.

Randomization must restore nominal values before sampling each episode. An additive center-of-mass update accidentally applied to the already randomized value performs a random walk over resets; eventually its distribution bears no relationship to the stated range.

8.9 Three robot models, one policy interface

Model	Why it exists
`robot_walk.xml`	light walking model; trunk/head ground contacts are stripped
`robot_allcollisions.xml`	lying, stand-up, tricks, and body contacts
`robot_allcollisions_rollers.xml`	full body plus passive wheel joints

Backlash versions add an unactuated hinge in series with every servo while keeping actor and action dimensions unchanged. A base and backlash A/B test must use otherwise matching robot models or the comparison is confounded.

8.10 Training artifacts and provenance

A run directory normally contains:

```
params/env.yaml       resolved environment used for the run
params/agent.yaml     resolved PPO/network configuration
model_<n>.pt          actor, critic, normalizers, optimizer, counters
events.out...         local scalar logs
*.onnx                exported inference policy, when exported
videos/play/          finite recorded rollouts, when requested
```

The resolved YAML Ain't Markup Language (YAML) file is valuable. Source code may change after a run; the saved parameters show what the checkpoint actually trained against. Keep it with any policy you evaluate or deploy.

A minimal policy manifest should bind:

```
task_id: Mjlab-Velocity-Flat-MicroDuck
source_commit: <git commit>
lockfile_sha256: <digest>
env_config_sha256: <digest>
agent_config_sha256: <digest>
checkpoint_sha256: <digest>
onnx_sha256: <digest>
observation_schema: microduck-policy-family-61d-v1
policy_period_s: 0.020
evaluator_commit: <git commit>
acceptance_report: <path or immutable URL>
```

Secure Hash Algorithm 256-bit (SHA-256) digests detect accidental artifact substitution; they do not prove the artifact is good. The acceptance report supplies behavioral evidence, and a trusted release/signing process supplies authenticity.

The provenance chain should be traversable in both directions:

```
hardware policy -> ONNX digest -> checkpoint -> resolved config/source
training run     -> checkpoint -> export command -> ONNX -> acceptance report
```

Without this chain, "the trained model" is ambiguous after several 20-hour runs.

8.11 Lab: trace a task without training

1. List the registry:

   ```
   uv run list-envs
   ```

2. Find the registration of `Mjlab-Velocity-Flat-MicroDuck` in `src/mjlab_microduck/tasks/__init__.py`.
3. Open `make_microduck_velocity_env_cfg` and identify one scene sensor, one command, one reward, one termination, and one curriculum.
4. Find the actor and critic hidden layers in `MicroduckRlCfg`.
5. Explain which of those objects exists during real ONNX inference.
6. Calculate policy period/rate and aggregate simulated seconds for a 24-step, 4,096-environment rollout.
7. Compute the actor mean-network parameter count independently. Which extra training parameter and export state are not in that sum?
8. Design a one-hot action-order test that detects a servo permutation while the robot is safely unpowered or simulated.
9. For one observation feature, write its complete contract: meaning, unit, frame, source timestamp, corruption, and normalization.
10. Explain why modifying `env.cfg` after manager construction can be a silent no-op and identify the correct curriculum access path.
11. Draw the provenance path from a deployed ONNX file back to source and forward to its acceptance evidence.
12. Compare a manager-based and monolithic environment. Give one advantage and one failure mode of each without declaring one universally superior.

Continue with Microduck setup and the first experiment.

8.12 Folded lab solution

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show a reference trace for Section 8.11

1. `uv run list-envs` proves registration through the installed project rather than a filename guess. Preserve the exact task ID it prints.
2. `tasks/__init__.py` registers `Mjlab-Velocity-Flat-MicroDuck` with `make_microduck_velocity_env_cfg()`, a play configuration, `MicroduckRlCfg`, and `MicroduckOnPolicyRunner`.
3. Valid examples include the `foot_height_scan` terrain ray sensor; `twist` command; `track_linear_velocity` reward; `nan_state` termination; and `action_rate_weight` curriculum. The important result is the path from factory to a named manager term, not memorizing these examples.
4. Actor and critic both use hidden dimensions `(512, 256, 128)` with an exponential linear unit (ELU) activation and observation normalization. Their inputs differ because the critic may receive privileged observations.
5. Real ONNX inference deploys the actor and its actor-observation normalizer. The critic, reward/termination/event/curriculum managers, rollout storage, PPO losses, optimizer, and simulator randomization exist only to generate or improve the checkpoint.
6. 4(5 ms) = 20 ms and 1/0.020 = 50 Hz. Each environment advances 24/50 = 0.48 s; aggregate experience is 4096(0.48) = 1966.08 simulated robot-seconds, or about 32.77 robot-minutes.
7. Sum

$$(61 \times 512 + 512) + (512 \times 256 + 256) + (256 \times 128 + 128) + (128 \times 14 + 14) = 197{,}774.$$

   A typical diagonal Gaussian adds 14 learned log-standard-deviation values during training. The deployed graph also needs actor normalizer statistics/logic, though those are not dense-layer weights.
8. In simulation or with torque disabled, set action vector to zero, then set exactly component j to a small safe value. Inspect the post-transform physical target array and assert only documented servo j changes by the expected radians. Repeat for all 14. Do not infer order by moving powered hardware first.
9. Example: projected-gravity x component; dimensionless unit vector component; body frame; derived from the inertial estimate sampled at timestamp t_s; delayed/noised by the configured term; normalized by saved feature mean μ_i, variance σ_i^2, and epsilon. The same template should be filled with actual evidence for every feature.
10. Managers deepcopy/resolve their term configurations at initialization, so later writes to the outer environment config need not touch the live term. A curriculum should read/update the manager's live term through an accessor such as `env.event_manager.get_term_cfg(...)`, then set it through the supported manager path.
11. ONNX digest → export record → checkpoint digest → saved agent/environment YAML digests → lockfile/source commit → task/model sources. In the other direction, ONNX digest → frozen evaluator/config → per-condition metrics, rollouts, reviewer decision, and hardware stage authorization.
12. Managers make terms independently configurable/loggable/testable and reuse templates, but copied/resolved configuration can surprise runtime mutation and hidden term interactions remain possible. A monolithic `step()` makes causal order locally visible and may be easier for a tiny environment, but reward/observation reuse, per-term logging, and systematic configuration can become tangled. Evidence and project scale decide.

A repeatable source trace is:

```
rg -n 'Mjlab-Velocity-Flat-MicroDuck' src/mjlab_microduck/tasks/__init__.py
rg -n 'MicroduckRlCfg|hidden_dims|obs_normalization' \
  src/mjlab_microduck/tasks/microduck_velocity_env_cfg.py
```

9. Setup and First Experiment

This chapter builds confidence in layers. Do not begin with a 20-hour training run. First prove the checkout, dependency lock, task registry, central processing unit (CPU) tests, graphics processing unit (GPU) simulation, short Proximal Policy Optimization (PPO) loop, checkpoint load, and viewer.

The goal is not merely to make a command stop without an error. A first experiment is a chain of falsifiable claims:

```
source and lock are known
  -> dependencies import
  -> task registers
  -> model and selectors resolve
  -> simulation steps with finite numbers
  -> rollout and PPO update complete
  -> checkpoint reloads
  -> rendered behavior matches the evaluated artifact
```

Each stage should fail cheaply before the next stage consumes more time or money. A long run cannot repair a broken observation permutation, and an attractive video cannot prove that the checkpoint came from the configuration you intended.

9.1 What the first experiment can establish

Separate four kinds of claim:

Claim	Smallest useful evidence	What the evidence does not prove
software integrity	clean sync, import, CPU tests	GPU kernels work
integration integrity	five-iteration GPU smoke run	a useful skill was learned
learning behavior	multi-seed metric and rollout battery	hardware transfer
deployment behavior	timed runtime rehearsal and staged robot test	unrestricted safety

This hierarchy prevents a common reasoning error: treating success at a lower layer as success at every higher layer. For example, a smoke run establishes that gradients can be computed; it says almost nothing about the statistical quality of the learned gait.

Before typing the training command, write a one-sentence hypothesis and an acceptance test. A useful first hypothesis is:

> At the pinned commit and lockfile, the flat-walking task can complete five PPO updates on this GPU with 61 finite actor observations, 14 finite actions, a saved checkpoint, and no negative-valued term appearing as a positive penalty contribution.

The hypothesis is narrow enough to test. “Training works” is not.

9.2 Requirements and the compatibility chain

For local GPU training you need:

- Linux;
- an NVIDIA Compute Unified Device Architecture (CUDA)-capable GPU and working driver;
- enough disk space for Python/CUDA packages and checkpoints;
- Git; and
- `uv` for the locked Python 3.12 environment.

The project requires Python `>=3.12,<3.13`; let `uv` honor the repository configuration rather than installing packages into an unrelated system Python.

Training can also be submitted to Hugging Face Jobs. Playback and deployment rehearsal still need a local environment that can load the model.

A modern robot simulator is a stack, not one program:

```
NVIDIA driver
  -> CUDA-enabled PyTorch wheel
  -> Warp kernels and PyTorch/Warp interoperation
  -> MuJoCo and MuJoCo Warp
  -> mjlab environment managers
  -> RSL-RL learner
  -> Microduck task and model assets
```

An arrow means "depends on a compatible interface from." The installed driver need not have the same version number as the CUDA runtime bundled with a PyTorch wheel, but it must be new enough to support that runtime. Similarly, seeing a device in `nvidia-smi` proves that the operating-system driver sees the GPU; it does not prove that the selected Python wheel contains CUDA kernels.

The repository's `pyproject.toml` expresses direct requirements and source rules. Its `uv.lock` records the exact resolved package graph and wheel selection. This distinction matters:

- a version *constraint* describes allowed solutions;
- a lockfile records the solution that was tested; and
- the Git commit identifies the code and robot assets that consumed it.

Together, commit plus lockfile are the minimum software identity of a run.

9.3 Clone and synchronize

```
git clone https://github.com/pollen-robotics/microduck_rl
cd microduck_rl
uv sync
```

`uv sync` is the reproducibility test. A package that exists only because it was manually installed into a developer's environment will be absent on a clean machine or remote job.

On Linux running the **64-bit Arm architecture (AArch64)**, set a longer first-download timeout if needed:

```
export UV_HTTP_TIMEOUT=600
uv sync
```

The project pins Torch directly and selects the CUDA 12.9 wheel index on AArch64. Do not remove the apparently redundant direct pin; `uv` source routing depends on it.

For a strict reproduction, ask `uv` to refuse an implicit lock update:

```
uv sync --frozen
```

Use ordinary `uv sync` while intentionally changing dependencies; inspect and commit the resulting lockfile diff. Use `--frozen` when the experiment must consume the recorded resolution exactly.

Do not "fix" a clean-environment failure with an unrecorded `pip install`. Change the project dependency, regenerate the lock, and repeat from a clean environment. Otherwise the next machine will rediscover the same failure.

9.4 Capture a baseline manifest

Run identity should be recorded before training, not reconstructed after a surprising result:

```
git rev-parse HEAD
git status --short
sha256sum uv.lock
uname -a
nvidia-smi --query-gpu=name,driver_version,memory.total \
  --format=csv,noheader
```

```
uv run python -VV
```

If `git status --short` is non-empty, save the patch with the run. A commit hash alone does not identify uncommitted code. Never put access tokens or the full environment variable set in the manifest; provenance and secrets have different purposes.

Two random streams are exposed by the live task interface:

```
uv run train Mjlab-Velocity-Flat-MicroDuck \
    --env.seed 101 \
    --agent.seed 101 \
    --env.scene.num-envs 64 \
    --agent.max-iterations 5
```

The environment seed controls stochastic simulation setup; the agent seed controls learner initialization and sampling. Fixing both makes a debugging reproduction easier, but it does not make GPU execution mathematically bit-for-bit deterministic. For a scientific result, vary seeds deliberately and report the distribution rather than choosing the most attractive run.

9.5 Verify the GPU stack

```
nvidia-smi

uv run python - <<'PY'
import torch
import warp as wp

print("torch:", torch.__version__)
print("torch CUDA runtime:", torch.version.cuda)
print("CUDA available:", torch.cuda.is_available())
print("GPU count:", torch.cuda.device_count())
if torch.cuda.is_available():
    print("GPU 0:", torch.cuda.get_device_name(0))
wp.init()
PY
```

Do not proceed to a long run if `torch.cuda.is_available()` is false. A visible GPU in `nvidia-smi` is necessary but not sufficient; the Python wheel must also include compatible CUDA support.

Interpret failures from the bottom of the dependency chain upward:

1. if `nvidia-smi` fails, repair driver/device access;
2. if it passes but Torch reports no CUDA runtime, inspect the installed wheel;
3. if Torch works but Warp initialization fails, inspect Warp/runtime compatibility;
4. if both work but task import fails, inspect Python dependencies and entry points; and
5. only after those pass should you debug a task configuration.

Changing reward weights cannot fix any of the first four layers.

9.6 Discover the live tasks

```
uv run list-envs
```

Look for `Mjlab-Velocity-Flat-MicroDuck`. If it is missing, task package entry points were not installed or importing `mjlab_microduck.tasks` failed. Fix that before debugging PPO.

The task identifier (ID) is a contract key. Similar names may select different robots, contact models, terrain, or backlash variants. Copy it from the live registry and record it verbatim.

9.7 Run the CPU regression suite

```
uv run --with pytest pytest tests/
```

These tests are more than unit-test decoration. They protect properties such as:

- the 61D observation family contract;
- servo indices on roller/backlash models;
- reward sign conventions;
- not-a-number (NaN) handling;
- reset and curriculum wiring; and
- task registration.

A passing suite does not prove a good policy, but a failing invariant makes an expensive run untrustworthy.

9.8 Run the five-iteration smoke train

```
uv run train Mjlab-Velocity-Flat-MicroDuck \
    --env.scene.num-envs 64 \
    --agent.max_iterations 5
```

What this should prove:

1. the robot and sensors compile;
2. Warp can execute on the selected GPU;
3. actor observation shape is 61 and action shape is 14;
4. every reward and termination can run;
5. PPO can collect data, compute losses, and update weights;
6. a checkpoint and resolved parameters can be written; and
7. values remain finite for this short run.

What it does **not** prove: that the robot has learned to walk. Five iterations are a software smoke test.

The default logger uses Weights & Biases. Authenticate with `wandb login` when you want cloud tracking, or use W&B's offline mode for a local-only experiment:

```
WANDB_MODE=offline uv run train Mjlab-Velocity-Flat-MicroDuck \
    --env.scene.num-envs 64 \
    --agent.max_iterations 5
```

The smoke-test log should be read, not merely awaited. Check at least:

- observations, actions, rewards, advantages, losses, and gradients are finite;
- every penalty's weighted episode contribution has the intended sign;
- episode length is plausible for the termination logic;
- the requested environment and iteration counts were resolved; and
- the expected checkpoint and configuration snapshots exist.

If a self-negating penalty is accidentally assigned a negative weight, it becomes a positive reward for violating the constraint. A total return can rise while the robot learns exactly the wrong behavior, so inspect named terms.

9.9 Budget arithmetic: what did the smoke test sample?

Robotic Systems Lab reinforcement learning (RSL-RL) collects 24 policy steps from each environment before one PPO update. For N parallel environments and I iterations, the number of transitions is

$$N_{\text{transition}} = N \times 24 \times I.$$

The smoke run therefore collects

$$64 \times 24 \times 5 = 7{,}680$$

transitions. At the 50 hertz policy rate, each individual environment advances

$$\frac{24 \times 5}{50} = 2.4 \text{ s},$$

while all environments together represent

$$\frac{7{,}680}{50} = 153.6 \text{ s}$$

of aggregate simulated experience. The robot did **not** walk one continuous 153.6-second episode; 64 worlds produced short trajectories in parallel.

This arithmetic explains both the value and the limitation of the smoke test. It touches thousands of transitions and several update paths, but each world has seen only 2.4 seconds and only one learner seed has been sampled.

Measure two operational quantities:

$$\text{throughput} = \frac{N_{\text{transition}}}{t_{\text{wall}}}, \qquad \text{aggregate real-time factor} = \frac{N_{\text{transition}}/50}{t_{\text{wall}}}.$$

Here t_{wall} is elapsed wall-clock seconds. Throughput helps select an environment count; it is not a measure of policy quality.

9.10 Inspect the run artifacts

```
find logs/rsl_rl/velocity -maxdepth 3 -type f | sort | tail -40
```

Open the newest run directory and inspect:

```
sed -n '1,160p' logs/rsl_rl/velocity/<run>/params/agent.yaml
sed -n '1,220p' logs/rsl_rl/velocity/<run>/params/env.yaml
```

Questions to answer:

- How many environments actually ran?
- How many iterations were requested?
- Is actor normalization on?
- What is the simulator timestep and action decimation?
- Which robot spec function was resolved?

This habit prevents evaluating a checkpoint under a task you merely assumed it used.

Also preserve the exact source identity and content hashes:

```
sha256sum logs/rsl_rl/velocity/<run>/model_*.pt
git diff --binary > logs/rsl_rl/velocity/<run>/working-tree.patch
```

Only create `working-tree.patch` when the tree was dirty, and inspect it for secrets before uploading. The resolved `env.yaml` and `agent.yaml` answer what the program actually instantiated; the source file answers why those values were chosen. Keep both.

9.11 Play a local checkpoint

```
uv run play Mjlab-Velocity-Flat-MicroDuck \
    --checkpoint-file logs/rsl_rl/velocity/<run>/model_<iteration>.pt \
    --num-envs 1 \
    --viewer native
```

Or load a run from W&B:

```
uv run play Mjlab-Velocity-Flat-MicroDuck \
```

```
    --wandb-run-path <entity>/mjlab_microduck/<run_id> \
    --num-envs 1
```

The viewer resamples training-style commands, so the robot may change direction. The command arrow is the requested local velocity, not an obstacle sensor or global route.

Verify these startup facts in the terminal:

```
checkpoint name is the one you intended
environment device is cuda:0 (or the explicitly selected device)
actor observation shape is (61,)
action shape is 14
actor first layer has 61 inputs
```

An actuator-selector warning that says 14 joints were matched while similarly named sites also match is informational in the current stack; the resolved action dimension must still be 14.

Playback is a controlled evaluation, not decoration. Record the command, seed, checkpoint digest, task, and whether training noise/randomization is disabled or retained. If two videos differ in all of those variables, their visual difference cannot be attributed to the checkpoint alone.

9.12 Record a finite rollout

A video is durable evidence and exits after the requested number of policy steps:

```
uv run play Mjlab-Velocity-Flat-MicroDuck \
    --checkpoint-file logs/rsl_rl/velocity/<run>/model_<iteration>.pt \
    --num-envs 1 \
    --video True \
    --video-length 500
```

At 50 Hz, 500 policy steps represent 10 seconds of simulated behavior. The video is written under the checkpoint run's `videos/play/` directory.

When comparing checkpoints, keep the task identifier (ID), commands, random seed or evaluation battery, camera, and video length consistent.

9.13 Find a useful environment count

Parallel worlds improve device utilization until memory, kernel scheduling, or the learner becomes the bottleneck. More environments do not automatically mean faster learning in wall time. They also change the batch size per update:

$$B = N_{\text{env}} \times N_{\text{step}}.$$

With 24 steps per environment, 4,096 worlds produce a batch of 98,304 transitions per iteration. If four minibatches are used, the nominal minibatch contains 24,576 transitions before accounting for the implementation's epoch reshuffling.

Benchmark resource scaling with the same task, seed, and short iteration count:

Environments	Peak GPU memory	Transitions/s	Aggregate real-time factor	Result
64	measure	measure	calculate	baseline
256	measure	measure	calculate	
1,024	measure	measure	calculate	
4,096	measure	measure	calculate	

Select the largest stable count near the throughput plateau, leaving memory headroom for compilation and transient allocations. An out-of-memory error is an infrastructure limit, not evidence that the reward or algorithm is wrong.

9.14 Start a full run only after the smoke test

```
uv run train Mjlab-Velocity-Flat-MicroDuck \
    --env.scene.num-envs 4096
```

Environment count is a throughput/memory tradeoff. Do not assume 4,096 fits every GPU or task. Reduce it if compilation or memory fails; do not change the task merely to hide a resource problem.

Remote alternative:

```
uv run train Mjlab-Velocity-Flat-MicroDuck \
    --env.scene.num-envs 4096 \
    --hf-jobs
```

See the Microduck repository's `scripts/hf/README.md` for authentication, job flavor, namespace, and retrieval details.

Before committing 20 hours, fill this gate:

Question	Pass evidence
Is the exact code recoverable?	commit plus saved dirty patch
Is the environment reproducible?	lock hash and resolved configuration
Does the cheap suite pass?	command and test summary
Does the GPU smoke pass?	finite five-iteration log and artifact
Are reward signs credible?	named term audit
Is the skill measurable?	fixed evaluation battery and success definition
Can progress be recovered?	checkpoint upload/copy tested

A "no" does not always forbid exploratory training, but it must be an explicit risk rather than a surprise discovered after the budget is spent.

9.15 Resume rather than discard a useful run

```
uv run train Mjlab-Velocity-Flat-MicroDuck \
    --env.scene.num-envs 4096 \
    --agent.run-name resume \
    --agent.load-checkpoint model_29999.pt \
    --agent.resume True
```

Resume only when the environment and observation/action contract are compatible with the checkpoint. A checkpoint is not a generic starting point for an arbitrary task.

Resume semantics deserve care. Loading network weights is different from restoring optimizer state, observation-normalizer statistics, iteration count, and curriculum phase. `--agent.resume True` requests a training continuation; an ad-hoc weight load may instead be a warm start. Record which operation was used because they define different experiments.

9.16 Diagnose by layer, not by guess

When a run fails, first classify the failure:

Layer	Typical symptom	First evidence to inspect
host/device	GPU absent, allocation failure	`nvidia-smi`, memory use

Layer	Typical symptom	First evidence to inspect
package/runtime	import, symbol, kernel error	lock, wheel versions, minimal import
task construction	selector or shape failure	registry, resolved config, CPU tests
numerical learning	NaN, exploding loss	first bad iteration and named tensors
objective	stable but undesirable behavior	per-term reward mass and rollouts
evaluation	contradictory videos/metrics	checkpoint hash and fixed battery

This order is causal. If the task cannot resolve its joints, changing PPO's learning rate adds a second variable without addressing the first failure.

Use a minimal reproduction that preserves the failing layer. For a numerical failure, save the earliest offending checkpoint/configuration and reduce the environment count. For a behavior failure, do not reduce away the command or terrain slice on which it occurs.

Common first-run failures

GPU visible to the driver but not Torch

Symptom: `nvidia-smi` works, while Torch reports zero devices or `mjlab` fails while selecting a GPU. Inspect the installed Torch build and CUDA runtime. On AArch64, preserve the repository's direct Torch pin and CUDA index routing.

Viewer does not open

Check `DISPLAY` on the X Window System version 11 (X11), or use a finite video/remote viewer appropriate to the machine. Do not interpret a display failure as a policy failure; first confirm the simulation process and checkpoint load.

Observation-size mismatch

The current family expects 61 actor inputs. A 51D legacy Open Neural Network Exchange (ONNX) policy or a task that deleted command slots is incompatible. Use the correct checkpoint/task or intentionally version the runtime contract.

NaN during training

Record the task, iteration, offending term, and resolved configuration. Run the CPU NaN regression tests, reduce the case to a reset/step reproduction, and inspect contacts and sensors. Simply restarting an unchanged long run loses the most useful evidence.

A successful smoke policy falls immediately

Expected. Five iterations proved the pipeline, not locomotion.

Training is finite but no skill appears

Check whether the main task reward grows, not only whether total return grows. Then inspect command coverage, episode length, curriculum stage, observation normalization, and a rollout. Regularizers can become less negative while the robot still does nothing. This is an objective/data diagnosis, not a reason to reinstall CUDA.

9.17 Lab report template

Record this after the first experiment:

```
commit:
task ID:
GPU and Torch/CUDA versions:
command:
run directory:
checkpoint:
```

```
actor/action shapes:
tests result:
smoke result:
viewer or video result:
first anomaly observed:
```

This small report is the beginning of reproducible reinforcement learning (RL) work.

Add an experiment table before a multi-run study:

Run	Code/config difference	Env seed	Agent seed	Hypothesis	Acceptance metric
A	baseline	101	101	pipeline is finite	smoke checklist
B	baseline	202	202	result is not seed-only	same evaluation battery
C	one intended change	101	101	change improves named failure	predeclared metric

Run C is a paired comparison with A because its seeds match. B estimates seed sensitivity. Many more seeds are normally needed for a publishable conclusion, but this structure already prevents changing code, randomization, and seeds at once.

Continue with the anatomy of a Microduck environment.

9.18 Exercises

1. For 256 environments, 24 steps per environment, and 10 iterations, calculate transitions, per-environment simulated time, and aggregate simulated time at 50 hertz.
2. Explain why `nvidia-smi` succeeding while `torch.cuda.is_available()` is false points to a different layer than a missing task ID.
3. A 64-environment smoke run passes, but 4,096 environments run out of memory. What has been falsified? What remains supported?
4. Why are both a Git commit and a dirty patch needed when the worktree has uncommitted changes?
5. Design a four-row environment-count benchmark and state which variables must remain fixed.
6. A run's total return rises, the velocity-tracking term is flat, and the action-rate penalty approaches zero. Give the most likely interpretation and the next two measurements.
7. Explain why one fixed seed is useful for debugging but insufficient for a learning claim.
8. Distinguish a training resume from a warm start. Name at least three pieces of state that may differ.
9. Draft an acceptance test for "the exported checkpoint is the policy shown in this video."
10. Find and correct the stale flags in this deliberately wrong command:

```
uv run train Mjlab-Velocity-Flat-MicroDuck \
    --num-envs 64 --max-iterations 5
```

9.19 Folded solutions and example report

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show exercise solutions

1. The transition count is $256 \times 24 \times 10 = 61{,}440$. Each environment advances $24 \times 10/50 = 4.8$ seconds. Aggregate simulated time is $61{,}440/50 = 1{,}228.8$ seconds. The latter is summed parallel experience, not the duration of one trajectory.

2. `nvidia-smi` tests driver/device visibility, while Torch tests the Python wheel and CUDA-enabled runtime. A missing task ID occurs higher in the stack, usually at package entry-point registration or task import. The two failures therefore demand different minimal reproductions.
3. The claim "4,096 worlds fit this device/configuration" is falsified. The passing smoke run still supports task construction, finite stepping, PPO integration, and checkpoint writing at 64 worlds. It does not support skill quality at either count.
4. The commit identifies tracked repository content. The dirty patch records changes not represented by that commit. Without it, another learner cannot reconstruct the program that ran.
5. Use, for example, 64, 256, 1,024, and 4,096 worlds. Hold commit, lock, task, both seeds, rollout steps, iteration count, logging overhead, and GPU fixed. Measure peak memory, transitions per second, failures, and aggregate real-time factor. Select based on the stable throughput curve, not count alone.
6. The learner may be discovering inactivity: smoother actions reduce the regularization cost while the task is not improving. Inspect a fixed-command rollout and the weighted mass of every reward term. Also compare episode length/terminations to see whether survival or early termination explains the return.
7. Fixed seeds make an observed change reproducible and permit paired debugging. They reveal only one initialization and one stochastic path. Several independently seeded runs are needed to estimate variability and distinguish a robust effect from luck.
8. A resume normally restores weights, optimizer moments, normalization statistics, iteration/curriculum position, and sometimes random-number state. A warm start may load only weights into a newly initialized run. These histories produce different update dynamics even at the same first visible checkpoint.
9. Record the checkpoint's Secure Hash Algorithm 256-bit (SHA-256) digest, export command and output digest, task and resolved config, inference command, seed/command sequence, and video path. The runtime log must print or record the loaded artifact digest and expected 61-input/14-output contract. Replaying that manifest should reproduce the evaluated artifact.
10. The live hierarchical flags are:

```
uv run train Mjlab-Velocity-Flat-MicroDuck \
    --env.scene.num-envs 64 \
    --agent.max-iterations 5
```

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show an annotated first-experiment report

This is a format example, not evidence from your machine. Replace every value and retain the commands/logs that support it.

```
commit:                 FULL_GIT_SHA (plus dirty patch, if any)
task ID:                Mjlab-Velocity-Flat-MicroDuck
GPU and Torch/CUDA:     exact device / torch build / CUDA runtime
command:                uv run train ... --env.scene.num-envs 64
                        --agent.max-iterations 5
run directory:          logs/<experiment>/<timestamp-or-run>
checkpoint:             exact model_*.pt and Secure Hash Algorithm 256-bit
                        (SHA-256) digest
actor/action shapes:    61 / 14, confirmed from runtime output
tests result:            N passed, M skipped; paste command and date
smoke result:            completed 5 iterations; finite observations/rewards
viewer or video result:  pipeline rendered; locomotion quality not claimed
first anomaly observed:  exact warning, metric, frame, or "none observed"
```

The important distinctions are observed versus assumed, pipeline success versus learned-skill success, and a mutable filename versus a content hash.

10. Anatomy of a Microduck Environment

An environment is an executable task specification. It defines what the robot can observe and do, how the world changes, what counts as progress, and which experience Proximal Policy Optimization (PPO) collects.

This chapter uses `Mjlab-Velocity-Flat-MicroDuck` as the main worked example.

In mathematical language, an environment implements a Markov Decision Process (MDP): state space, action space, transition law, reward, initial-state distribution, discount, and termination rules. In engineering language, it is also an interface contract among simulator, learner, exporter, and runtime. Those views line up as follows:

MDP object	Microduck implementation	Question to audit
hidden state s_t	MuJoCo bodies, joints, actuator and sensor state	What affects the future?
observation o_t	ordered actor/critic observation terms	What may each network know?
action a_t	14 joint-position offsets	What can the policy command?
transition p	physics, Better Actuator Models (BAM), delays, events, randomization	What dynamics generate the next state?
reward r_t	weighted reward manager terms	What behavior is optimized?
initial distribution ρ_0	reset events and command sampling	Where does experience begin?
terminal flags	timeout, fall, bounds, nonfinite state	When does return stop or bootstrap?

The simulator state is richer than the actor observation, so the deployed problem is more precisely a Partially Observable Markov Decision Process (POMDP). Previous action, angular velocity, projected gravity, and joint state form a compact *information state*: not a perfect reconstruction of physics, but enough history and sensing for a useful reactive controller.

The environment is therefore a hypothesis about sufficiency. If obstacle distance is absent, no optimizer can make the actor condition its first step on an unseen obstacle. If motor temperature changes future torque but is neither observed nor inferable from history, the actor must learn a robust compromise rather than temperature-specific control.

10.1 Start from the factory

The task registry points to:

```
make_microduck_velocity_env_cfg(play=False, rough=False)
```

The factory begins with `mjlab`'s velocity template and then specializes it for Microduck. This is an important design pattern:

```
cfg = make_velocity_env_cfg()
cfg.scene.entities = {"robot": MICRODUCK_WALK_ROBOT_CFG}
cfg.scene.sensors = (
    feet_ground_cfg,
    self_collision_cfg,
    foot_height_scan_cfg,
)
```

The resulting object is still just configuration. Managers copy and resolve it when an environment instance is constructed.

The live sources for this walkthrough are the `velocity` configuration, the custom MDP functions, and the task registry. Follow references between them: the configuration names *which* function and parameters are active, while `mdp.py` defines the tensor calculation.

Starting from the closest maintained template is software reuse with a scientific benefit. Closely related tasks inherit the same observation noise, delay, termination, and actuator assumptions. Copy-pasted standalone environments tend to drift invisibly, turning an intended reward comparison into a physics comparison.

10.2 Scene: the physical question

The scene selects:

- the robot's MuJoCo Extensible Markup Language (XML) model format (MJCF) file and its initial state;
- terrain;
- collision rules;
- contact and ray sensors;
- environment count and spacing; and
- viewer camera.

Flat velocity uses a plane. Rough velocity uses a generator with robot-scale terrain; step heights are capped around 1.5 cm because carrying human-scale terrain assumptions to a 25 cm robot would create a different task.

"Robot-scale" has a physical meaning. For motions dominated by gravity, the dimensionless Froude number compares inertial speed with gravitational speed:

$$\mathrm{Fr} = \frac{v^2}{gL},$$

where v is forward speed, g is gravitational acceleration, and L is a representative leg length. Two geometrically similar robots with comparable Fr have speed scaling approximately as $v \propto \sqrt{L}$, not directly with height. Terrain should likewise be expressed relative to foot size, clearance, and leg length. A 1.5 cm step is 6% of a 25 cm robot's height; the same relative obstacle for a 1.7 m human would be about 10 cm.

Dynamic similarity is only a starting point: motor torque density, foot geometry, joint range, controller frequency, and contact compliance do not all scale geometrically. The practical lesson is to measure the actual robot and state normalized terrain ratios rather than borrowing a benchmark's absolute meters.

The walking model has deliberately simplified fall contacts. Tasks that begin or finish on the ground use the all-collisions model. Model choice is part of the task definition, not a rendering preference.

Collision geometry and visual geometry serve different jobs. Visual meshes make videos recognizable; collision primitives define the contact problem the physics engine must solve. A foot that looks flat but has a rounded or misaligned collision shape changes the support polygon and reward-relevant contacts. Inspect both.

Contact capacity and solver iterations are computational parameters with behavioral effects. The rough task raises the contact limit and solver work because a fallen robot touching several terrain boxes can overflow a walking configuration sized for two feet on a plane. Dropped or poorly converged contacts can appear as explosive motion or nonfinite state; that is a physics model failure, not exploration.

Try it: compare resolved scenes

```
uv run play Mjlab-Velocity-Flat-MicroDuck --agent zero --num-envs 1
uv run play Mjlab-Velocity-Rough-MicroDuck --agent zero --num-envs 1
```

A zero agent is useful for inspecting physics and initial conditions without attributing motion to a policy.

For a target pose, extend the test: hold its corresponding joint targets for several seconds from perturbed resets and record trunk height *and tilt*. Settling at the expected height while lying sideways is not equilibrium validation. A pose-based task whose goal is not mechanically supportable asks learning to fight gravity forever.

10.3 Actions: the controllable question

The velocity task has one action term with dimension 14:

```
joint_pos_action = cfg.actions["joint_pos"]
joint_pos_action.scale = 1.0
```

The action manager interprets each network output as an offset from the default joint pose. Better Actuator Models (BAM) then turns the position target into voltage/torque behavior.

The conceptual map for servo j is

$$q_{t,j}^{target} = q_j^{HOME} + s_j a_{t,j},$$

where $s_j = 1$ radian per policy unit in this task. The target is not the next joint angle. The actuator's feedback law, voltage limit, motor constants, friction, load, and physics determine how much motion occurs during the next 20 milliseconds. This separation is fundamental:

```
policy action -> target -> actuator dynamics -> torque -> acceleration -> state
```

For a simplified proportional–derivative servo,

$$\tau_{t,j}^{cmd} = K_{p,j}(q_{t,j}^{target} - q_{t,j}) - K_{d,j}\dot{q}_{t,j},$$

followed by motor-voltage/torque saturation. The real BAM path is richer, but the equation explains why a large target offset may produce a saturated motor rather than an instantaneous large pose change.

Action design choices include:

- position, velocity, torque, or residual targets;
- scale and offset;
- clipping;
- ordering; and
- delay/filter behavior.

An action space should match what the runtime can reproduce. Training with an action low-pass filter and deploying without it creates a different plant; deploying a filter that training never saw does the same. This policy family is intentionally unfiltered.

The alternatives embody different assumptions:

Policy output	Benefit	Main burden
joint position target	compatible with smart servos; bounded posture intent	inner-loop dynamics are hidden from policy
joint velocity target	direct motion intent	needs matched velocity loop and limits
torque/current	maximum dynamic authority	model fidelity and hardware safety become harder
residual over classical target	preserves a known controller	baseline controller constrains discoverable behavior

Action ordering is as important as dimension. A 14-value vector can have the right shape and command the wrong legs. A deployment conformance test applies one small nonzero component at a time in simulation and on an unloaded or otherwise safe actuator fixture, then verifies the named joint, sign, and units. This tests the permutation that shape checks cannot see.

The policy is stochastic during PPO training. If its Gaussian sample is later clipped to a legal interval, all samples beyond the bound produce the same command. Extensive saturation therefore wastes probability mass and hides the size of the requested change. Log both pre-limit action statistics and

applied targets; a policy that lives on limits is often exploiting an overly wide interface or compensating for insufficient actuator authority.

10.4 Observations: the information question

An observation term is a function plus optional noise, scale, clipping, delay, and history. Actor terms are concatenated in order.

The main actor gets 48 proprioceptive values plus a 13D command block. The critic receives additional simulator-only signals. Two principles matter:

1. **Availability:** actor inputs must be reproducible on hardware.
2. **Meaning:** training and runtime must use the same units, frame, sign, ordering, offset, delay, and filtering.

Formally, the actor receives a possibly corrupted measurement function

$$o_t = h(s_t, \epsilon_t, \tau_t),$$

where ϵ_t represents sensor noise/bias and τ_t represents sample age or delay. The policy computes $a_t \sim \pi_\theta(\cdot \mid o_t)$, not from the simulator state s_t directly. Treating $o_t = s_t$ when designing the runtime is a common source of sim-to-real failure.

The ordered actor vector is:

```
3 angular velocity + 3 projected gravity
+ 14 relative joint positions + 14 joint velocities
+ 14 previous actions
+ 3 twist command + 4 head command + 6 body command
= 61 values
```

“Projected gravity” means the world gravity direction expressed in the robot body frame. It gives roll/pitch orientation without exposing a globally meaningful yaw. Relative joint position means measured joint angle minus the documented HOME angle. Previous action helps the memoryless network infer part of the actuator/delay state.

The construction can be viewed as typed concatenation:

```
parts = [
    angular_velocity_body_rad_s(),       # shape (N, 3)
    projected_gravity_body(),            # shape (N, 3)
    relative_servo_positions_rad(),       # shape (N, 14)
    servo_velocities_rad_s(),             # shape (N, 14)
    previous_policy_action(),             # shape (N, 14)
    command_twist_head_body(),            # shape (N, 13)
]
observation = concatenate(parts, axis=-1)
assert observation.shape == (num_envs, 61)
assert isfinite(observation).all()
```

Comments supply human meaning; automated tests must lock shape, ordering, values at named probe states, and the featurewise normalizer. Otherwise two programs can agree on 61 while disagreeing on every joint.

For example, “angular velocity” is incomplete documentation. You must know:

```
world or body frame?
rad/s or deg/s?
which axis signs?
raw, calibrated, or filtered?
current sample or delayed sample?
```

The current actor intentionally removes perfect base linear velocity and the general terrain height scan. A per-foot ray sensor supports foot-height features and rewards, but there is no forward obstacle representation in the actor.

Using simulator-only variables for the critic is called an asymmetric actor–critic. The critic can use true velocity or contact facts to estimate return more accurately during training, while the actor remains deployable. At inference the critic is discarded. Privilege must not leak into actor features, command generation, or a runtime preprocessing branch.

Observation normalization changes the function represented by saved weights:

$$\tilde{o}_i = \frac{o_i - \mu_i}{\sqrt{v_i + \epsilon}}.$$

The actor consumes $\tilde{o}$, so exporting only network matrices but not (μ, v, ϵ) exports a different policy. The mandatory Microduck export path bakes this transform into the Open Neural Network Exchange (ONNX) graph.

For each feature, write a contract row containing semantic name, unit, frame, sign, order, source sensor, calibration, expected range, timestamp, delay, noise model, and fallback. That row is more useful than a bare neural-network index during hardware integration.

10.5 Commands: the intention question

Commands are targets sampled by the training environment and supplied by an operator or planner at deployment.

This makes a commanded policy a conditional controller:

$$a_t \sim \pi_\theta(a \mid o_t^{robot}, c_t),$$

where c_t is intention and o_t^{robot} is measured robot state. A planner chooses *what* local motion is desired; the locomotion policy chooses *how* to realize it under contacts and disturbances. Confusing a command with a sensor would make the architecture circular: requested forward speed does not reveal actual forward speed or an obstacle.

The velocity task's twist command uses these ranges:

```
forward velocity vx       -0.4 .. +0.4 m/s
lateral velocity vy       -0.3 .. +0.3 m/s
yaw rate                  -1.0 .. +1.0 rad/s
resampling interval        3.0 .. 8.0 s
```

It also creates explicit buckets:

- standing/zero-command environments, because continuous uniform sampling almost never produces exactly zero;
- forward-command environments; and
- turn-in-place environments with zero linear velocity and a meaningful yaw command.

Why are buckets mathematically necessary? A continuously uniform random variable takes any exact value with probability zero. Thus

$$\Pr(v_x = 0, v_y = 0, \omega_z = 0) = 0$$

under independent continuous sampling, even though exact zero is the most important deployed idle command. Near-zero is also rare. With tolerance $\epsilon = 0.02$, the chance that independently sampled translation satisfies $|v_x| < \epsilon$ and $|v_y| < \epsilon$ is

$$\frac{0.04}{0.8}\frac{0.04}{0.6} \approx 0.0033,$$

only about 0.33%, before requiring a useful yaw rate. Data imbalance, not an optimizer defect, explains why spontaneous turn-in-place training can fail.

The implemented distribution is a mixture:

$$p(c) = p_{stand}p_{stand}(c) + p_{turn}p_{turn}(c) + p_{general}p_{general}(c),$$

with probabilities summing to one. This converts capability priorities into experience frequency. It also means evaluation should report each bucket separately; an overall mean can hide complete failure on a small but important slice.

The turn-in-place code is a useful real example of fixing the data distribution rather than tuning PPO:

```
# Abridged from VelocityCommandCommandOnly._resample_command
turn_ids = env_ids[random_values < rel_turn_in_place_envs]
self.vel_command_b[turn_ids, 0:2] = 0.0
sign = where(random_values < 0.5, -1.0, 1.0)
mag = uniform(0.4 * max_yaw_rate, max_yaw_rate)
self.vel_command_b[turn_ids, 2] = sign * mag
```

Independent uniform sampling made useful spin-in-place examples too rare. An explicit 15% bucket gives PPO enough on-policy data.

The head command is four joint deltas from HOME. Its range widens by curriculum. The six-dimensional body-pose slot is retained, but its tracking reward has weight zero in the main velocity policy; do not expect this checkpoint to obey body-pose commands reliably.

A nonzero command input with zero reward can keep network connections from remaining exactly unused, but it does not create a reason to obey that input. Capability requires the complete causal chain:

```
command varies -> actor observes it -> outcomes depend on action
-> reward distinguishes correct response -> training samples the region
-> evaluation tests it
```

At deployment, command timing joins the contract. The local twist should have a timestamp, validity duration, rate/acceleration limits, and stale-command fallback. A cloud planner that pauses cannot be allowed to leave an old forward command active indefinitely; the real-time layer should decay or replace it with the trained exact-zero command.

10.6 Rewards: the objective question

The main velocity reward stack includes:

Term	Purpose	Typical configured weight
linear velocity tracking	follow requested planar velocity	+2.0
angular velocity tracking	follow requested turn rate	+2.0
upright	keep trunk orientation useful	+2.0
pose	retain a workable leg posture	+1.0
air time	encourage stepping when commanded	+3.0
head-pose tracking	follow four head targets	+2.0
body angular velocity	discourage unnecessary rotation	-0.05
angular momentum	reduce excess whole-body motion	-0.02
action rate	reduce command jitter	starts at -0.1, ramps stronger
foot slip	reduce sliding without blocking turning	-0.1
self-collision	avoid invalid body contact	-1.0

Weights cannot be compared in isolation. A term's scale, frequency, gate, and the total positive reward mass determine its influence.

For configured feature functions f_i and weights w_i, the environment returns

$$r_t = \sum_{i=1}^{K} w_i f_i(s_t, a_t, s_{t+1}, c_t).$$

The learner optimizes expected discounted sums of r_t; it never reads the English purpose column. Every claim such as “discourage slip” is true only if the function, sign, units, contact selector, and gate implement it.

Define the empirical reward mass of term i over an evaluation set of T steps as

$$M_i = \frac{1}{T} \sum_{t=1}^{T} w_i f_{i,t}.$$

M_i is expressed in reward per step and includes actual behavior. A weight of `-1.0` on a function usually near zero can be weaker than `-0.05` on a large function active every step. Compare M_i, its distribution across command slices, and behavior videos before copying a weight to another task.

Reward terms also interact. If positive tracking terms sum to roughly eight units on a good step, a `-0.1` smoothness term has a different relative price than it would in a task whose positive stack sums to two. This is why “standard regularizer weights” are not portable constants.

Real code example: Gaussian head tracking

The custom reward compares the commanded head delta with measured joint delta:

```
# Abridged from mdp.head_pose_tracking
cmd = env.command_manager.get_command("head_pose")       # (N, 4)
measured = joint_pos[:, neck_ids] + backlash_position
actual = measured - default_joint_pos[:, neck_ids]
error = actual - cmd
per_joint = torch.exp(-((error / std) ** 2))
return per_joint.mean(dim=-1)                            # (N,)
```

At `abs(error) == std`, one joint contributes $e^{-1} \approx 0.37$. A very wide standard deviation weakens the gradient near small errors; a very narrow one can make far states nearly zero-reward and can tax unavoidable walking oscillation.

For one scalar error e, the reward and its derivative are

$$g(e) = \exp\left(-\frac{e^2}{\sigma^2}\right), \qquad \frac{dg}{de} = -\frac{2e}{\sigma^2} \exp\left(-\frac{e^2}{\sigma^2}\right).$$

The gradient is zero at perfect tracking, grows for small errors, reaches its largest magnitude at $|e| = \sigma/\sqrt{2}$, then decays toward zero far from the target. Therefore σ describes the error region where learning signal is concentrated. It is not merely a cosmetic tolerance.

Suppose $\sigma = 0.5$ radian. At $e = 0.1$, $g \approx 0.961$; at $e = 0.5$, $g \approx 0.368$; at $e = 1.0$, $g \approx 0.018$. Tightening to 0.1 makes the same 0.5-radian error score e^{-25}, effectively removing its gradient from early training. A curriculum can widen reachable commands only while the current policy remains inside a gradient-bearing region.

The implementation reads through simulated backlash so the tracking reward and encoder observation describe the same physical angle.

Separate avoidable bias from necessary motion

Microduck’s head is a large fraction of the robot’s mass, so walking naturally causes oscillatory head error. A tight instantaneous penalty priced that necessary motion so heavily that standing still became a better solution. The remedy was not “more PPO”; it was to penalize only the slow bias component.

An exponential moving average (EMA) of error obeys

$$m_t = (1 - \alpha) m_{t-1} + \alpha e_t, \qquad \alpha = \frac{\Delta t}{\tau + \Delta t}.$$

With time constant $\tau = 1$ second and policy interval $\Delta t = 0.02$ second, $\alpha \approx 0.0196$. Alternating zero-mean gait oscillation largely cancels, while a persistent downward droop remains in m_t and can be penalized by $-|m_t|$. This is a theory-to-code example of asking which part of an error is actually controllable.

Reward sign audit

There are two penalty styles in this project:

```
# Style A: function returns nonnegative cost
cost = error.square()
# Configure a NEGATIVE weight.

# Style B: helper returns a nonpositive penalty
penalty = -error.abs()
# Configure a POSITIVE weight.
```

A negative weight on Style B makes violations profitable. The reliable runtime check is:

```
every Episode_Reward/<penalty> contribution must be <= 0
```

Reward gates

A gate defines when a reward is meaningful. Foot air time should activate when walking is commanded, not while standing. A recovery reward should pay for progress toward upright rather than pay continuously for remaining fallen.

Hard state-based gates are often more effective than small penalties because they define what counts as the maneuver. For a roll, support contact and orientation axes can distinguish a real forward roll from a shoulder flop.

Write a gate explicitly as $z_t \in \{0, 1\}$:

$$r_t^{air} = z_t^{walking} f_{air}(s_t), \qquad z_t^{walking} = \mathbf{1}[\|c_t^{twist}\| > \epsilon].$$

Without the gate, stepping in place could earn air-time reward during a stand command. With a poorly chosen gate that pays only while fallen, remaining fallen may become profitable. For recovery progress, a potential difference is safer:

$$r_t^{progress} = \gamma\Phi(s_{t+1}) - \Phi(s_t).$$

Holding one bad state then pays approximately zero instead of a jackpot each step. Chapter 1 showed why discounted potential shaping preserves the optimal policy under its assumptions; here the engineering job is to choose a bounded, measurable potential such as uprightness.

Reward-design unit tests

Reward code deserves tests at named states, not only whole training runs. For each term, construct at least:

1. a perfect state and a clearly worse state;
2. gate-open and gate-closed states;
3. left/right mirrored states when symmetry is intended;
4. values at physical limits; and
5. a nonfinite input for terms on risky sensor paths.

Then assert direction and sign rather than overfitting to one floating-point constant:

```
assert tracking(perfect) > tracking(offset)
assert air_time(standing_command) == 0.0
assert weighted_penalty(violation) <= 0.0
assert isfinite(reward(extreme_but_valid_state))
```

10.7 Events and domain randomization: the variation question

Event terms run in modes:

Mode	Example
startup	foot friction, encoder bias fields, mass/inertia setup
reset	root pose, joints, center of mass (CoM), actuator friction scale, armature
interval	velocity pushes every few seconds

Domain randomization (DR) trains one policy over a distribution of plausible robots:

$$\theta_{physics} \sim p(\theta)$$

More completely, training maximizes an average over physical parameters, sensor corruption, commands, and initial state:

$$J(\pi) = \mathbb{E}_{\theta,\epsilon,c,s_0}\left[\sum_{t=0}^{H-1} \gamma^t r(s_t, a_t, s_{t+1}, c; \theta)\right].$$

This objective does not guarantee success for every sampled robot. A broad distribution can lower average performance or produce a conservative policy; a narrow or biased distribution may omit the real robot. Choose ranges from measurement, datasheets, system identification, calibration residuals, and known operating conditions. Label guessed ranges as hypotheses to be updated.

Distribution shape also encodes belief. A uniform range says every interior value is equally plausible and hard bounds are real. A truncated normal says central values are more plausible. A discrete mixture can represent known hardware revisions. Correlations matter: payload mass and center-of-mass shift should not always be sampled independently if one physical payload causes both.

Microduck randomizes selected quantities such as friction, mass/inertia, CoM, battery voltage/sag, armature, encoder bias, inertial measurement unit (IMU) alignment, delays, and pushes.

DR is not a substitute for calibration. Zero-centered IMU orientation variation teaches tolerance to uncertain mounting magnitude; it cannot remove a fixed systematic mounting bias. The runtime must calibrate that bias.

Custom randomization must restore defaults before applying a sampled change. If a +5% mass change is added to the already-randomized mass every reset, the distribution drifts outside the intended range during long training.

The accumulation bug is visible algebraically. If the intended rule is

$$m_k = m_0(1 + u_k), \qquad u_k \sim \mathcal{U}[-0.05, 0.05],$$

then every reset remains within 5% of m_0. The wrong recursive rule

$$m_k = m_{k-1}(1 + u_k)$$

forms a multiplicative random walk whose spread grows with reset count. A test should perform hundreds of resets and assert all realized values remain inside the configured support, then verify that the distribution is not collapsed to one value.

Randomization order should mimic the physical causal path. Encoder mounting bias changes a measurement, not the true joint. Battery sag changes available actuator voltage, not the desired command. Command latency holds an old target; it is not interchangeable with independent noise added to the current target. Placing uncertainty at the wrong layer may yield a robust policy for a machine that cannot exist.

10.8 Terminations: the data-recycling question

The velocity task can terminate for timeout, falling, terrain bounds, or a nonfinite state. A **not-a-number (NaN) guard** checks joints, root state, and named sensor data:

```
bad = ~torch.isfinite(data.joint_pos).all(dim=1)
bad |= ~torch.isfinite(data.joint_vel).all(dim=1)
bad |= ~torch.isfinite(data.root_link_pos_w).all(dim=1)
bad |= ~torch.isfinite(data.root_link_quat_w).all(dim=1)
bad |= ~torch.isfinite(data.root_link_lin_vel_w).all(dim=1)
bad |= ~torch.isfinite(data.root_link_ang_vel_w).all(dim=1)
```

Sensor-derived critic terms are separately sanitized so one invalid contact or ray value does not poison the value network before the environment resets.

Do not reuse walking terminations blindly. A recovery task needs time on the ground. An episodic trick may need a short horizon and a landing criterion.

Keep two meanings distinct:

- **termination**: the task's future has ended, such as an unrecoverable fall;
- **truncation**: collection stopped at a time or spatial boundary even though a valid continuation conceptually exists.

For a truncated transition, the return target normally bootstraps from the critic:

$$y_t = r_t + \gamma V(o_{t+1}).$$

For a truly terminal transition, future value is zero:

$$y_t = r_t.$$

Treating every timeout as terminal teaches an artificial value cliff at the horizon. Treating catastrophic nonfinite physics as a normal bootstrap target can inject meaningless values. Inspect how the environment and learner pass these flags rather than assuming the words are interchangeable.

Termination also changes the data distribution. If falling resets immediately, the learner sees many reset states and little self-recovery. That is appropriate for a walking-only controller intended to switch to a separate recovery policy, but wrong for a single policy expected to stand up.

10.9 Curricula: the pacing question

A curriculum changes the problem as training progresses. In this repository, steps mean environment steps:

```
curriculum step = PPO iteration * num_steps_per_env
                = PPO iteration * 24
```

This counter is *per environment rollout depth*, not the total number of parallel transitions. Increasing from 64 to 4,096 environments increases data per iteration, but stage 1,000 still begins after 1,000 PPO iterations unless the configuration says otherwise. Wall-clock arrival may change because throughput changes.

A scheduled curriculum creates a sequence of data distributions $p_0, p_1, \ldots$ and objectives $r_0, r_1, \ldots$. Learning is therefore nonstationary: the policy is chasing a moving problem. The purpose is to keep useful experience and gradients available, not to make a plot look smoothly difficult.

The action-rate curriculum uses discrete stages:

```
weight_stages = [
    {"step": 0,         "weight": -0.1},
    {"step": 500 * 24,  "weight": -0.2},
    {"step": 750 * 24,  "weight": -0.4},
```

```
    {"step": 1000 * 24, "weight": -0.6},
    {"step": 1250 * 24, "weight": -0.8},
    {"step": 1500 * 24, "weight": -1.0},
]
```

The helper mutates the live manager copy:

```
term_cfg = env.reward_manager.get_term_cfg(reward_name)
term_cfg.weight = selected_stage["weight"]
```

Writing to `env.cfg.rewards` after manager initialization is a silent no-op because managers copied their configurations.

A stage should follow demonstrated competence. If the main skill metric drops exactly at every boundary, the schedule is outpacing the policy.

Three common curriculum axes solve different shortages:

Curriculum	Changes	Use when
command range/mix	requested behaviors	rare or difficult commands lack data
randomization range	physical uncertainty	nominal skill exists before robustness
reward/regularization weight	optimization priorities	smoothness would suppress early exploration

A fourth method, reverse curriculum, changes reset states. If a maneuver learns the beginning but never encounters successful endings, initialize some worlds near the end, then move the reset frontier backward. This supplies on-policy experience in the missing region without dictating a full waypoint trajectory.

Iteration schedules are simple and reproducible, but competence-triggered schedules can be safer conceptually: widen only after a held-out success metric exceeds a threshold for several evaluations. They require hysteresis and logged state so noise cannot flip difficulty repeatedly. Whichever method is used, record the active stage beside every metric.

Trace one stage all the way to behavior

At iteration 1,000, `step=1000*24=24,000`. The action-rate weight becomes `-0.6`, the standing fraction becomes `0.15`, and the head command range also widens. Because several changes coincide, a metric discontinuity cannot be attributed to one of them without an ablation. Staggered boundaries improve causal diagnosis; synchronized boundaries simplify an intended phase change. This is an experimental-design tradeoff, not a formatting detail.

10.10 What the velocity policy can and cannot do

Capability	In the current task?	Reason
commanded forward/lateral walk	yes	command input and active tracking reward
commanded turn or stand	yes	explicit command buckets and tracking
commanded head pose	yes	4D input and active reward
commanded body pose	not reliably	slot exists, reward weight is zero
global waypoint navigation	no	no waypoint/map state or navigation objective
obstacle avoidance	no	no forward obstacle observation or avoidance objective
visual decision making	no	camera pixels/features are not actor inputs

Putting a box in the rendered scene does not create perception. An actor can only condition behavior on information in its observation or inferable from contact after collision.

The changing arrows seen in a default viewer can look like a random walk, but that description is misleading. The environment randomly resamples desired local velocities every 3–8 seconds; PPO trains a *conditional velocity tracker*. Randomness supplies a training distribution. The learned intention interface is twist plus four head targets. It does not include a destination, path, obstacle map, or active body-pose objective in this checkpoint.

A capability claim needs five linked facts:

Link	Velocity tracking example	Obstacle avoidance requirement
representation	twist command in 61D actor input	range/depth/local-map feature or safe planner command
variation	sampled velocity commands	varied obstacle geometry and relative motion
consequence	actions change measured velocity	actions change clearance/collision outcome
objective	velocity tracking rewards	clearance/progress/collision/safety objective
evaluation	fixed command buckets	unseen layouts, near misses, collision and progress metrics

If any link is absent, optimization cannot establish the capability.

Two sound extension architectures are:

1. a perception/planning system detects obstacles and sends safe local twist commands to the unchanged 61D locomotion policy; or
2. a deliberately versioned policy family receives exteroceptive features and is trained on obstacle-rich scenes with a new runtime contract.

The first preserves a small, fast, hot-swappable motor policy. The second may learn tighter perception-action coupling but requires new training data, network design, inference budgets, tests, and deployment plumbing.

For Microduck and JumpRover, the first architecture is the safer initial handoff: perception estimates local obstacles and robot pose, a planner emits a bounded timestamped twist, and the 50 hertz local policy tracks it. Cloud reasoning can propose goals or plans, but a real-time local process must check freshness and limits. A network outage must degrade to stop or a locally safe behavior rather than freezing the last plan.

The second architecture becomes attractive when local geometric features must couple tightly to foot placement, for example stepping over a narrow obstacle. Then version the observation schema rather than silently inserting features into the 61D vector, and train/evaluate the complete sensor-to-action latency.

10.11 Exercises: read configuration as a scientific claim

Use the live `microduck_velocity_env_cfg.py` and `mdp.py` rather than answering only from this chapter.

1. What command causes standing? Why is a near-zero uniform sample not an equivalent training case?
2. Why is turn-in-place sampled in its own bucket? Calculate the probability that uniform translation lies within ±0.02 m/s on both axes.
3. Which head joints are commandable, and in what order?
4. Why does `body_pose` remain in the observation when its reward is zero? What capability can and cannot be claimed?
5. Which randomization changes the actuator rather than MuJoCo joint friction? Why would randomizing the wrong field silently fail?
6. At which PPO iteration does the action-rate weight first become `-0.6`? Name two other scheduled changes close enough to confound interpretation.

7. For $g(e) = \exp(-(e/0.5)^2)$, compute $g(0)$, $g(0.5)$, and $g(1.0)$. Explain why tightening the standard deviation can hurt early learning.
8. A helper returns `-abs(error)`. Which weight sign makes it a penalty? What runtime log sign should result?
9. Write pseudocode for a 500-reset test that catches accumulating mass randomization.
10. A 20-second horizon ends while the robot is upright and walking. Is that naturally a termination or truncation, and should the value target bootstrap?
11. Design a one-hot action-order conformance test. What error can it catch that a `(14,)` shape assertion cannot?
12. A reward term has weight `-0.05` and unweighted mean 8. A second has weight `-1.0` and unweighted mean 0.1. Compute their mean reward masses and compare their actual influence.
13. With $\Delta t = 0.02$ second and EMA time constant $\tau = 1$ second, calculate α. What type of tracking error passes through this slow state?
14. List the minimum environment and runtime changes required for a policy to avoid obstacles directly rather than through planner-provided safe twists.
15. Run a zero-agent scene inspection. Separate facts it can verify from learned capabilities it cannot verify.
16. Select one reward term and trace: configuration entry → function → tensor inputs → gate → returned sign → configured weight → logged weighted mass. Write the trace as a seven-line audit record.

10.12 Folded solutions

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show solutions to Exercises 1–8

1. Standing is an **exact zero twist command** selected by the command term's `rel_standing_envs` bucket. A near-zero command still asks for motion and can activate walking gates; continuous sampling also gives exact zero probability zero.
2. Independent sampling makes zero translation plus meaningful yaw rare. The translation probability is $(0.04/0.8)(0.04/0.6) = 1/300 \approx 0.0033$, or 0.33%. The explicit `rel_turn_in_place_envs` bucket provides a controlled fraction instead.
3. The 4D head command controls `neck_pitch`, `head_pitch`, `head_yaw`, and `head_roll`, in that order. The dimensions are deltas from HOME.
4. The body slot preserves the shared 61D hot-swap interface and exposes the same schema to task variants. The small changing inputs can avoid a wholly dead connection, but a zero tracking weight supplies no incentive to obey them. Schema compatibility may be claimed; reliable body-pose control may not.
5. `randomize_joint_friction` changes BAM's per-environment `friction_scale`. BAM computes actuator friction and overwrites/does not use MuJoCo's ordinary `dof_frictionloss` as a conventional actuator would, so a random number in that unused field changes no transition the policy sees.
6. The `-0.6` stage starts at `1000*24=24,000` environment steps, which the project maps to PPO iteration 1,000. Standing fraction becomes 0.15 there, and the head-command range widens there as well; either can contribute to a coincident metric change.
7. $g(0) = 1$, $g(0.5) = e^{-1} \approx 0.368$, and $g(1) = e^{-4} \approx 0.0183$. A narrower standard deviation moves early, inaccurate states into the almost-zero tail, leaving little gradient. It can also overprice physically necessary oscillation.
8. The function is already nonpositive, so a **positive** weight preserves it as a penalty. A negative weight would double-negate it into positive reward. The weighted `Episode_Reward/<term>` contribution should be nonpositive.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show solutions to Exercises 9–16

9. Save the compile-time/default mass m_0, reset 500 times, collect each realized mass, and assert every value lies in $[0.95m_0, 1.05m_0]$. Also assert the sample standard deviation is nonzero. A compact sketch is:

```
samples = []
for _ in range(500):
    env.reset()
    samples.append(read_mass())
assert all(0.95 * m0 <= m <= 1.05 * m0 for m in samples)
assert stdev(samples) > 0
```

   A restore-then-randomize implementation passes; a multiplicative random walk eventually escapes the interval with high probability.

10. A time limit interrupts a physically valid continuation, so it is naturally a truncation. The target should normally include $\gamma V(o_{t+1})$. A task-specific finite-horizon objective could define time as terminal, but that must be intentional and time remaining may then belong in the observation.

11. Start at HOME and apply a small positive offset to action index j, all other entries zero. Verify only the documented servo moves in the positive convention, then repeat for all 14 indices. This catches permutation and sign errors that preserve vector shape.

12. The masses are –0.05(8) = –0.4 and –1(0.1) = –0.1 reward per step. The first term has four times the observed mean influence despite its 20-times smaller absolute weight.

13. $\alpha = 0.02/(1 + 0.02) \approx 0.0196$. The EMA retains slow or persistent bias and attenuates fast, roughly zero-mean oscillation. It is therefore a stateful filter, not a new measurement.

14. Add a physically plausible obstacle sensor/model, a versioned actor feature and normalizer contract, obstacle-rich scene/reset distributions, rewards or constraints for clearance/collision and task progress, a model architecture with adequate temporal/spatial context, matching runtime preprocessing and latency, and evaluation on held-out layouts. A box in the viewer alone satisfies none of those causal links.

15. A zero actor can verify loading, gravity, collision shapes, passive-joint motion, reset states, action plumbing at zero, and numerical finiteness. It cannot verify tracking, gait quality, recovery, obstacle avoidance, robustness, or hardware transfer because no learned decisions are used.

16. One acceptable record for action rate is:

```
config: rewards[action_rate_l2]
function: inherited action-rate L2 cost
inputs: current and previous 14D applied policy actions
gate: active each policy step
return: nonnegative squared change
weight: negative, scheduled -0.1 toward -1.0
log: weighted episode contribution must remain <= 0
```

   A different term is correct if every link is tied to the live code and its sign is reasoned from the function's actual return.

Continue with training, evaluation, and debugging.

11. Training, Evaluation, and Debugging

Training is not "start Proximal Policy Optimization (PPO) and wait." A useful run is a controlled experiment with a hypothesis, resolved configuration, checkpoints, per-term metrics, and rollouts that test the intended behavior.

Three processes must remain separate:

```
optimization: use training experience to change parameters
selection:    use validation cases to choose a checkpoint/configuration
assessment:   use untouched test cases to estimate final capability
```

If the same ten trials are repeatedly watched while rewards and checkpoints are tuned, those trials have become validation data. Reporting them as an unseen test exaggerates confidence even though no supervised labels were used.

Deep reinforcement learning (RL) is unusually variable because data depends on the changing policy, resets and disturbances are random, and nonlinear optimization amplifies early differences. The evaluation methods in this chapter build on Deep Reinforcement Learning at the Edge of the Statistical Precipice and its `rliable` reference implementation. The repository was archived in 2025, but the paper's lessons—intervals, performance profiles, robust aggregate metrics, and paired comparisons—remain valuable. Robot evaluation adds command slices, physical failure taxonomies, and explicit safety tails.

11.1 The end-to-end loop

```
physics assumption check
        v
CPU config/reward tests
        v
64-env, 5-iteration smoke train
        v
full vectorized run
        v
metric and checkpoint inspection
        v
fixed evaluation battery + video
        v
one evidence-backed change
        +-------------------------> repeat
```

For a target/rest pose, add a physics-only check before training: hold its control target from noisy initial states for several seconds and measure both height and tilt. A fallen body can have a stable height, so height alone cannot prove equilibrium.

11.2 Before launching a full run

Write a short experiment note:

```
hypothesis:
task ID and commit:
baseline run/checkpoint:
one intentional change:
expected main metric effect:
expected visible behavior:
failure criterion:
training budget:
```

Then run:

```
uv run --with pytest pytest tests/
```

```
uv run train <TASK_ID> \
    --env.scene.num-envs 64 \
    --agent.max_iterations 5
```

Do not combine reward redesign, new observations, stronger domain randomization (DR), a new robot model, and new PPO hyperparameters in one experiment. Even a successful result will not tell you which change mattered.

Think in terms of variables:

- the **treatment** is the one intended change;
- controlled variables include commit, task, simulator, training budget, and evaluation cases;
- nuisance variables include learner seed, reset randomness, device timing, and nondeterministic kernels; and
- outcomes are predeclared metrics and failure classes.

For a paired seed design, baseline and treatment use the same seed set:

```
baseline: 101, 202, 303, 404, 505
treatment: 101, 202, 303, 404, 505
```

Analyze per-seed differences. Pairing does not make five seeds "large," but it removes some variation caused by comparing unrelated initializations. Never discard a failed seed because its curve is ugly unless the predeclared rule identifies an infrastructure-invalid run and is applied equally to both arms.

Training budget should be expressed in at least three units:

$$N_{transition} = N_{env} N_{step} N_{iteration},$$

wall-clock time, and compute/device type. Equal iterations with different environment counts are not equal sample budgets; equal transitions on devices with different speed are not equal wall budgets. State which comparison the hypothesis requires.

11.3 Read the training dashboard in layers

Layer 1: pipeline health

Check:

- iteration advances;
- simulation and learning frames per second (FPS) are plausible;
- losses and Kullback–Leibler (KL) divergence are finite;
- no not-a-number (NaN) termination spike appears;
- checkpoints continue to save; and
- graphics processing unit (GPU) memory remains stable.

Layer 2: episode behavior

The main velocity episode lasts 20 seconds, or 1,000 policy steps at 50 Hz. A mean episode length near 1,000 suggests most episodes reach timeout. It does not prove walking; standing still may also survive.

A rising episode length can be good for locomotion and irrelevant for a fixed short trick. Interpret it against the task.

Layer 3: reward signs

For every penalty contribution:

```
Episode_Reward/<penalty> <= 0
```

If a penalty is positive, stop and inspect its function/weight sign. PPO will farm a double-negated violation.

Layer 4: the main skill

Track the term that directly represents the task:

```
walking      linear/angular tracking and air time
stand-up     upright/height completion and landing quality
sit-stand    commanded transition completion
spin         commanded yaw-rate profile and grounded support
```

Total reward can rise because action rate fell while locomotion remained absent. Main-task metrics and video must agree.

Layer 5: policy distribution

Watch entropy/noise scale, KL, and action statistics. Early entropy collapse can freeze a do-nothing solution. Persistently large action noise can hide a useful mean policy. These are diagnostic signals; do not tune them before checking the task specification.

The Kullback–Leibler (KL) divergence estimates how far the updated policy moved from the behavior policy that collected the rollout. A small value does not prove learning, and a large value does not identify whether the environment is correct; it only diagnoses the optimization step. Read it beside clipping fraction, value loss, entropy, gradient norm, and task performance.

Learning curves are temporally correlated: checkpoint 1,001 descends from checkpoint 1,000 and is not an independent replicate. A moving average can make a plot readable, but it does not create more independent evidence. Keep raw data, state the smoothing window, and compare independent seeds for uncertainty.

When an anomaly begins, locate the first iteration rather than inspecting only the final corrupted state. Save:

```
last known-good checkpoint and metrics
first bad checkpoint and metrics
active curriculum stage
first nonfinite tensor/term if relevant
representative rollout immediately before and after
```

This brackets the causal event and makes a reduced reproduction possible.

11.4 Weighted logs and reward mass

`Episode_Reward/<term>` is the weighted contribution. If a term's weight is zero, its log is zero even when the underlying behavior is poor. If a curriculum changes the weight, a jump in the log may reflect the coefficient, not behavior.

Compare reward mass:

$$M_i = \mathbb{E}[w_i f_i], \qquad A_i = \mathbb{E}[|w_i f_i|].$$

M_i is signed mean contribution; A_i is absolute activity. Cancellation can make M_i small even when a signed shaping term is very active, so both are useful. A rough relative share is

$$S_i = \frac{A_i}{\sum_j A_j + \epsilon}.$$

This is a diagnostic, not a causal attribution: changing one term changes the policy and therefore every f_j. Still, it catches orders-of-magnitude mistakes that raw weights hide.

Suppose one task has 8 points/step of positive reward and another has 2. An action-rate weight of `-0.1` is four times weaker relative to the first task, even though the text of the configuration is identical.

Log unweighted features or behavior metrics when a curriculum can set a term's weight to zero. Otherwise the dashboard erases the measurement exactly when it would reveal whether the policy is ready for that term to turn on.

11.5 Evaluation is a separate experiment

Do not judge only the final checkpoint or one random viewer episode. Create a battery that covers the state distribution you care about.

Example velocity battery:

Case	Command	What to measure
idle	`[0, 0, 0]`	stable stance, head bias, jitter
forward	`[+0.2, 0, 0]`	achieved speed, drift, falls
backward	`[-0.2, 0, 0]`	tracking and foot clearance
lateral	`[0, +0.15, 0]`	side tracking and collisions
turn in place	`[0, 0, +0.6]`	yaw rate, translation drift
combined	`[+0.2, 0, +0.5]`	curve tracking
head command	fixed twist + head deltas	per-joint error and gait effect
disturbance	selected push cases	recovery and settling time

The default `play` command generator is useful for qualitative variety, but a fixed battery is better for checkpoint comparison. The deployment rehearsal in `scripts/infer_policy.py` allows explicit initial velocity commands and keyboard changes.

11.5.1 Define the sampling hierarchy

Do not confuse four sample counts:

```
training seeds
  checkpoints within a seed
    evaluation scenarios/commands
      repeated episodes within a scenario
```

Twenty episodes from one checkpoint measure conditional rollout variability; they are not twenty independently trained policies. For a claim about the algorithm or reward recipe, the independent unit is normally the training seed. For a claim about one chosen deployable checkpoint, repeated physical conditions may be the relevant unit, but the claim must stay checkpoint-specific.

Use a table with one row per raw episode, not only an aggregate dashboard:

```
policy_hash, train_seed, checkpoint_iteration, scenario_id,
eval_seed, command, reset_class, disturbance, success,
tracking_error, fall_time, energy_proxy, failure_class, video_id
```

This structure permits regrouping without rerunning and prevents a mean from hiding which command failed.

11.5.2 Continuous metrics

For command c_t and measured velocity v_t, useful tracking metrics include

$$\mathrm{MAE} = \frac{1}{T}\sum_{t=1}^{T} |v_t - c_t|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{T}\sum_{t=1}^{T}(v_t - c_t)^2}.$$

Mean absolute error (MAE) is easy to interpret and less dominated by spikes. Root mean squared error (RMSE) emphasizes large errors. Report both only when each answers a stated question; a dozen redundant metrics increases the chance of selecting a flattering one after seeing results.

Robot-specific measures can include:

$$\text{drift} = \| p_T - p_0 - \text{desired displacement} \|_2,$$

$$E_{mechanical} = \sum_{t,j} |\tau_{t,j}\dot{q}_{t,j}|\Delta t,$$

plus peak tilt, slip distance, impact impulse, settling time, action variation, and deadline misses. $E_{mechanical}$ is an absolute joint-work proxy; real battery energy additionally depends on voltage/current, regeneration, motor loss, and electronics.

Average metrics can conceal unsafe tails. Report a high quantile such as the 95th percentile tracking error and a worst-slice result. For a "larger is better" score X, lower-tail conditional value at risk at fraction α is the mean of the worst α fraction:

$$\mathrm{CVaR}_{\alpha}^{lower}(X) = \mathbb{E}[X \mid X \le q_{\alpha}],$$

where q_{α} is the α quantile. Conditional value at risk (CVaR) makes repeated near-failures visible even when mean tracking is strong.

11.5.3 Binary success and uncertainty

If k of n trials succeed, the estimate is $\hat{p} = k/n$. The familiar standard error

$$\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

is useful intuition but a normal interval behaves badly near zero/one and at small n. The Wilson interval is a better elementary default. With standard normal critical value z, its center and half-width are

$$c = \frac{\hat{p} + z^2/(2n)}{1 + z^2/n},$$

$$h = \frac{z}{1 + z^2/n}\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}.$$

Report $[c - h, c + h]$. Eighteen successes in 20 trials means 90%, but the approximate 95% Wilson interval is about 70%–97%. This is far less certainty than the point estimate visually suggests. Zero failures is also not proof of zero risk: with 20/20 successes the lower Wilson bound is only about 84%.

For robot safety, a success label should be accompanied by failure classes and severity. One gentle stop and one high-energy face impact are both "failures" in a binomial count but demand different engineering responses.

11.5.4 Means, medians, interquartile means, and bootstrap intervals

The arithmetic mean uses every value but can be dominated by rare extreme runs. The median is robust but discards much rank information in small samples. The interquartile mean (IQM) averages the middle 50% after sorting, combining some robustness with better sample efficiency than the median on multi-task benchmarks:

$$\mathrm{IQM}(x) = \mathrm{mean}\{x_i : q_{0.25} \le x_i \le q_{0.75}\}.$$

Do not mechanically replace every robot metric with IQM. For one homogeneous command slice, show all seed values, mean/median, and an interval. IQM is most useful when aggregating comparable normalized scores over multiple tasks or conditions.

A nonparametric bootstrap approximates estimator uncertainty:

1. sample the independent units with replacement;

2. compute the statistic on that resample;
3. repeat many times; and
4. take suitable percentiles of the bootstrap statistics.

If episodes are nested within training seeds, resample seeds as clusters or use a hierarchical bootstrap. Resampling every episode as independent produces an interval that is too narrow because all episodes from one checkpoint share the same learned policy.

For baseline A and treatment B trained with matching seeds, bootstrap the paired differences $d_s = B_s - A_s$. The probability of improvement estimate

$$\hat{P}(B > A) = \frac{1}{S}\sum_{s=1}^{S} \mathbf{1}[B_s > A_s]$$

is intuitive, but with few seeds its uncertainty must also be shown.

Run the standard-library example:

```
python examples/evaluation_statistics.py
```

It computes mean, median, IQM, a seeded bootstrap interval, a paired probability of improvement, and a Wilson success interval. Alter one outlier and observe which summaries move.

11.5.5 Stratify before aggregating

An overall score answers performance under a particular mixture of cases. If 90% of trials are easy forward motion and 10% are turn-in-place, the overall mean mostly answers the forward question. Preserve named slices:

```
command bucket x reset class x terrain x disturbance x hardware condition
```

Report per-slice sample count and uncertainty. Then, if one deployment-weighted aggregate is needed, publish its weights:

$$J_{deploy} = \sum_k \omega_k J_k, \qquad \sum_k \omega_k = 1.$$

The worst credible slice can be more important than J_{deploy} for safety. Test values inside the training support, near its boundaries, and just outside it. This distinguishes interpolation, edge robustness, and extrapolation; the word “robust” should not combine them.

Random evaluation and adversarial search are complementary. Random trials estimate performance under a declared distribution. A grid or optimization over push direction, timing, friction, delay, and command can locate failure boundaries. Once found and used for tuning, those cases become validation cases; retain a fresh final assessment set.

11.6 Watch video and compute state metrics

Video answers geometric questions metrics can miss:

- Which body or collision geometry touched?
- Did a “forward roll” go over the head or shoulder?
- Did the feet step or slide?
- Is the head tracking target or serving as a counterweight?

State metrics answer questions video can mislead:

- exact trunk tilt and height;
- achieved versus commanded speed;
- peak angular rate and acceleration;
- contact timing and impact force;
- per-spawn success rate; and
- end-state clusters.

Use both. A smooth camera angle can hide lateral drift, and a scalar threshold can split one visibly identical behavior cluster.

Synchronize video frames with state/action logs using a shared simulation step or timestamp. A useful review panel places commanded/measured twist, trunk tilt, contacts, action extrema, reward terms, and curriculum state under the video. Then "the foot slipped here" becomes a queryable event rather than a memory of one playback.

Create a failure taxonomy before counting:

```
F0 no failure
F1 initial-state collapse
F2 tracking loss without fall
F3 slip then fall
F4 self-collision
F5 joint/action saturation
F6 numerical/contact-solver failure
F7 deadline or stale-command safety stop
```

Keep raw notes so classes can be refined, but freeze the taxonomy before the final test. Separate policy behavior (`F2`) from invalid simulation (`F6`) and runtime infrastructure (`F7`); pooling them obscures which subsystem needs a change.

11.7 Checkpoint selection

The last checkpoint is not automatically best. A policy can discover the skill and later trade it away when a curriculum adds difficulty or regularization.

Evaluate checkpoints around:

- first visible skill discovery;
- every curriculum boundary;
- sudden main-metric changes;
- entropy or KL events; and
- the final iteration.

Load an exact local checkpoint:

```
uv run play <TASK_ID> \
    --checkpoint-file logs/rsl_rl/<experiment>/<run>/model_2000.pt \
    --num-envs 1
```

Preserve the resolved `params/` directory with the selected model.

Selecting the maximum of many noisy checkpoint estimates creates a winner's curse. If 100 equivalent checkpoints are each evaluated on five noisy trials, the best observed one is likely lucky. A disciplined protocol is:

1. define a validation battery and selection rule, such as lowest validation tracking error subject to zero severe failures;
2. choose once using that battery;
3. lock the checkpoint content hash; and
4. evaluate it on a larger untouched test battery.

When training several seeds, decide whether the deployed artifact is the best validation seed or a representative fixed-seed recipe. Report algorithm-level seed distribution separately from the selected-checkpoint test. They answer different questions.

Learning performance also has a time axis. Compare curves at matched transition budgets, not just final return. The area under a learning curve up to budget B,

$$\mathrm{AUC}(B) = \int_0^B J(n)\, dn,$$

summarizes sample efficiency but depends on metric scaling and interpolation. Always show the curve and final performance beside the area under the curve (AUC), because a fast weak plateau and a slow strong result can have similar areas.

11.8 Common failure patterns

The robot does nothing

Likely causes:

- attempt penalties or smoothness costs dominate before skill discovery;
- positive task rewards are too sparse or flat;
- command distribution rarely includes the desired case;
- a tracking standard deviation is too tight at initial errors; or
- standing earns most of the same positive stack as moving.

Measure per-term contributions before increasing entropy or learning rate.

The robot moves violently

Check whether an arrival state pays a per-step jackpot, whether impact/contact is under-specified, and whether action-rate regularization has begun. For a small robot, 3.5–5.5 rad/s body rotation may be physically natural during a tumble; penalize impact and thrash rather than importing human-scale angular speed intuition.

It finds the beginning but not the finish

The successful frontier may almost never appear in on-policy data. Add reverse-curriculum resets near later maneuver states and consolidate each slice before widening.

It camps at a waypoint

Per-step waypoint rewards create local jackpots. For an episodic landing task, prefer a fixed final target with progress/landing shaping and let reinforcement learning (RL) discover the path.

It reaches the goal too fast and crashes

An early-arrival state that keeps paying makes speed profitable. Use a slewed target or rate-limited progress so being ahead of the intended transition does not pay extra.

Joints park at hard limits

The default limit cost may activate only near the last portion of the range. Add a qpos-side limit-proximity penalty for the offending joints rather than shrinking the wide command range needed by low-stiffness servos.

Training becomes NaN

Do not merely lower the learning rate. Determine whether the first nonfinite value originated in physics, a sensor, reward math, observation normalization, or PPO. Inspect `Episode_Termination/nan_state`, contact conditions, and a minimal reset/step reproduction.

Simulation succeeds and hardware fails

Audit the interface before redesigning rewards:

```
joint names/order and signs
HOME offsets
units and coordinate frames
observation order and normalization
sensor delay/noise/filtering
action scaling and filtering
policy frequency and missed deadlines
actuator voltage, friction, saturation, backlash
```

```
command-slot writes
```

In-sim playback applies the checkpoint normalizer and can hide an Open Neural Network Exchange (ONNX) export mistake. Runtime all-zero commands can select “stand” when an application expected a trick flag. These interface failures look like bad learning.

Debug with competing hypotheses

Do not stop at the first plausible story. For “turning policy falls,” write alternatives and discriminating evidence:

Hypothesis	Prediction	Cheap test
turn commands were too rare	forward strong, turn slice weak across seeds	command-count and sliced evaluation
yaw observation sign is wrong	response consistently turns opposite	named-state observation/action probe
slip penalty blocks pivoting	applied feet remain planted while target yaw rises	contact/slip/action trace and weight ablation
actuator saturation	target error and voltage/torque limit co-occur	saturation telemetry
checkpoint mismatch	metrics cannot reproduce saved video	artifact hashes and manifest replay

Prefer a test whose outcomes separate hypotheses. Increasing training time tests none of these cleanly: it may mask data scarcity, intensify a reward exploit, or simply waste budget.

For numerical failures, bisect along two dimensions:

- **time**: find the first bad iteration/step; and
- **complexity**: reduce worlds, terrain, events, sensors, or reward terms while preserving the failure.

Remove one causal branch at a time. If disabling a reward hides a NaN only because it no longer reads a bad sensor, the origin may still be the sensor; trace the first nonfinite value rather than the final stack frame.

11.9 Curriculum diagnosis

Plot main metrics against curriculum stages. If a metric drops at exactly the same step every run, inspect the stage before changing PPO.

Example reasoning:

```
observation: air-time reward rises through iteration 700
event: action-rate cost doubles at iteration 750
result: air-time collapses and never recovers

hypothesis: smoothness tax arrived before stepping consolidated
experiment: delay only that stage, preserve every other setting
```

Curricula should introduce challenge after competence, not according to an arbitrary desire to finish sooner.

11.10 A disciplined reward change

Use this sequence:

1. Name the visible failure precisely: “left hip roll remains within 3 degrees of its hard limit for 70% of forward steps.”
2. Confirm it from state data, not one frame.
3. Identify the cheapest exploit in the current objective.
4. Add or change one term/gate.
5. Write a pure-function test and a configuration wiring/sign test.

6. Run the five-iteration smoke test.
7. Compare a fixed checkpoint battery with the baseline.
8. Keep or revert based on the intended behavior, not total reward alone.

11.11 Exercises and evidence lab

1. A study trains 2,048 worlds for 3,000 iterations with 24 rollout steps. Calculate transitions. What additional quantities are needed to compare its resource budget with another study?
2. A penalty helper returns a nonpositive value, has weight `-0.2`, and its logged weighted contribution is positive. Diagnose the error.
3. Term A has signed mean contribution `+1.5` and absolute activity `1.5`. Term B has signed mean `0` but absolute activity `2.0`. What can and cannot be concluded?
4. Why do 100 episodes from one training seed not equal 100 independent algorithm runs? Which question can those episodes still answer well?
5. Using $z = 1.96$, calculate or approximate the Wilson interval for 18 successes in 20 trials. Interpret it in plain language.
6. Baseline seed scores are `[0.4, 0.7, 0.5, 0.8, 0.2]`; treatment scores are `[0.5, 0.6, 0.7, 0.9, 0.6]` in matching seed order. Compute paired differences and the observed probability of improvement.
7. Explain why selecting the best of 100 checkpoints on five trials and reporting those same five trials is biased. Propose a repair.
8. Design strata for evaluating idle, forward, turning, and push recovery. Give one continuous metric, one binary criterion, and one failure class for each.
9. A mean tracking score improves while worst-10% score falls sharply. What engineering conclusion is justified? What is not?
10. A smoothed learning curve has 1,000 plotted points. How many independent training replicates does that imply?
11. Construct three competing hypotheses for a policy that walks in the viewer but fails after ONNX export. Give one discriminating test for each.
12. When should a numerical-failure run be excluded from a comparison, and what must be reported?
13. Run `python examples/evaluation_statistics.py`, change one score to an extreme outlier, and explain the response of mean, median, and IQM.
14. Choose one trained checkpoint and produce this report:

```
task, commit, resolved config, checkpoint hash:
selection rule and validation battery:
untouched test battery and raw-data path:
train seeds and evaluation seeds:
per-slice sample counts:
main estimates with uncertainty:
severe failures and synchronized videos:
reward signs and main reward masses:
observed capability boundary:
next hypothesis and falsifying result:
```

Avoid “it works.” Prefer “tracks +0.2 m/s on 18/20 flat starts, but falls in 6/20 turn-in-place trials after the first direction change.” The second form states a population, condition, numerator, denominator, and failure trigger.

11.12 Folded solutions and report rubric

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show solutions to Exercises 1–7

1. The count is 2048 × 3000 × 24 = 147,456,000 transitions. Also report device, wall time, power/accounting assumptions if relevant, simulator and policy rates, environment count, update/minibatch schedule, and whether both studies count the same kind of transition. Equal transitions do not imply equal wall time or optimization updates.
2. Multiplying a nonpositive helper by a negative weight double-negates it into reward. Use a positive weight for that helper or rewrite it as a nonnegative cost with a negative weight. The episode contribution should then be nonpositive on violations.
3. A contributes positive reward consistently on average. B is active but its signed values cancel; it may be potential shaping or vary across cases. B's larger absolute activity means it strongly changes instantaneous returns, but neither statistic alone proves causal influence on the learned policy.
4. All 100 episodes share one learned parameter vector and training history; they estimate rollout/reset variability conditional on that checkpoint. They can estimate that checkpoint's success probability under a declared deployment scenario well, but not variability of the training algorithm.
5. Substitution gives center about 0.838 and half-width about 0.134, hence approximately `[0.70, 0.97]`. "18/20" is compatible with a much lower true success rate than 90%; more independent trials are needed for a narrow operational guarantee.
6. Differences are `[+0.1, -0.1, +0.2, +0.1, +0.4]`. Four of five are positive, so the observed probability of improvement is 4/5 = 0.8. With only five pairs the uncertainty is large; show all values or a paired interval rather than claiming an 80% universal probability.
7. Noise helps one of many checkpoints win, and reusing the same five trials conditions the report on that luck. Select with a fixed validation battery, lock the checkpoint hash, then assess once on a larger untouched test battery. Report the selection procedure and number of candidates.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show solutions to Exercises 8–13

8. One valid design is:

Slice	Continuous	Binary	Failure class
idle	action variation/head bias	stays upright 20 s	jitter-induced slip
forward	velocity MAE/drift	reaches timeout within error bound	forward fall
turn	yaw MAE/translation drift	completes commanded angle	pivot slip
push	peak tilt/settling time	recovers without fall	unrecovered fall

   Thresholds, command values, pushes, horizon, and sample counts must be fixed before final evaluation.
9. The change improved central/average behavior while worsening tail behavior under the tested distribution. That is a tradeoff requiring severity and slice investigation; it does not justify "better" or "safer" without an application-defined utility/constraint.
10. None. The 1,000 points may come from one correlated trajectory through parameter space. Independent replicate count is the number of independently trained seeds, not checkpoints or smoothing samples.
11. Examples: missing normalizer predicts mismatched normalized features—compare training-side and ONNX outputs on fixed observations. Wrong action order predicts named joints respond to the wrong indices—run one-hot probes. Wrong command-slot writes predict a correct policy response to unintended intention—log and compare the complete 61-vector. Artifact mismatch is a fourth hypothesis tested by hashes.
12. Exclude only under a predeclared, treatment-independent infrastructure rule, such as a confirmed machine outage before training began. Algorithmic NaNs are outcomes, not inconvenient data.

Report every exclusion, raw count, reason, affected arm/seed, evidence, and analysis with failures included when possible.

13. The mean moves directly with the outlier. The median often does not move until rank order crosses the center. IQM drops the outer quarters and is robust if the changed score remains outside the middle half. This robustness is useful, but a catastrophic robot failure must still be reported rather than trimmed out as statistical inconvenience.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show a reference evidence-backed review rubric

An acceptable answer names a reproducible case, reports uncertainty, and separates observations from the next hypothesis. For example:

```
Task/checkpoint:       exact task + Secure Hash Algorithm 256-bit (SHA-256)
                       checkpoint digest
Selection:             predeclared validation rule and candidate count
Test battery:          20 untouched flat +0.2 m/s; 20 turn-in-place cases
Observed result:       18/20 forward; 14/20 turn, with Wilson intervals
Raw failure classes:  4 turn falls after reversal; 2 initial slips
Penalty signs:        every weighted penalty <= 0 (attach metric export)
Episode length:       timeout is success only for continuous-walking cases
Visible compromise:  large translation drift while tracking yaw
State evidence:       median/p90 yaw error and drift, synchronized to video
Capability boundary: tested commands/terrain only; no obstacle claim
Hypothesis:           turn-in-place samples remain underrepresented
Single next change:   change only its explicit command-bucket fraction
Falsifier:            paired turn metric/tail does not improve across seeds
```

Those numbers are illustrative. A strong submission includes raw episode rows, representative success and failure video, resolved configuration, seed list, evaluator commit, slice definitions, and the result that would falsify the proposed hypothesis.

Continue with export, deployment, and sim-to-real.

12. Export, Deployment, and Sim-to-Real

A training checkpoint is not the deployed policy. Deployment is a contract between the exported graph, observation producer, command source, timing loop, action mapping, actuator interface, and safety system.

This is where a statistically good policy becomes a real-time software component. The key question changes from "does expected return improve?" to "does this exact byte-level artifact receive the same physical meanings, on time, within a bounded authority envelope?"

12.1 Artifact flow

```
model_<iteration>.pt
  actor + critic + normalizers + optimizer + training state
        |
        | scripts/export.py
        v
policy.onnx
  actor inference graph + actor observation normalizer + metadata
        |
        | scripts/infer_policy.py
        v
CPU MuJoCo rehearsal using deployment-style observations and commands
        |
        | real runtime integration and staged safety tests
        v
14 joint targets at 50 Hz on Microduck
```

The critic, reward functions, terminations, randomizers, and Proximal Policy Optimization (PPO) optimizer are not deployed. Their job was to shape the actor during training.

Track four distinct artifacts:

Artifact	Contains	Main use
training checkpoint	actor, critic, optimizer, normalizers, counters	resume and exact export
exported actor	deterministic inference graph and actor normalization	runtime inference
policy manifest	schema, joint order, units, rate, source/config hashes	integration and audit
release bundle	actor, manifest, tests, calibration compatibility, rollback	physical deployment

Filenames are mutable labels. Compute a Secure Hash Algorithm 256-bit (SHA-256) digest for every immutable artifact and place the digest in the release record. If a runtime log says only `walk.onnx`, it cannot prove which bytes moved the robot.

12.2 Export through the mandatory path

Export a local checkpoint:

```
uv run scripts/export.py Mjlab-Velocity-Flat-MicroDuck \
    --checkpoint-file logs/rsl_rl/velocity/<run>/model_<iteration>.pt \
    --onnx-file walk.onnx
```

Or export from W&B:

```
uv run scripts/export.py Mjlab-Velocity-Flat-MicroDuck \
```

```
    --wandb-run-path <entity>/mjlab_microduck/<run_id> \
    --onnx-file walk.onnx
```

The script:

1. loads the play configuration for the exact task identifier (ID);
2. constructs the actor and critic architecture;
3. restores the checkpoint;
4. obtains the deterministic inference policy;
5. exports the actor with its `EmpiricalNormalization` submodule; and
6. attaches metadata such as joint names, defaults, action scale, commands, and observation names.

Never hand-convert the checkpoint. In-simulator playback normalizes observations and can conceal a missing-normalizer deployment bug.

During PPO training, actions are sampled from a distribution. Deployment usually exports the deterministic mean action:

$$a_t^{deploy} = \mu_\theta(\tilde{o}_t),$$

not a fresh Gaussian sample. Evaluation must state whether it used the mean or stochastic actor. Comparing a stochastic checkpoint rollout with deterministic export adds sampling noise to an interface test.

The embedded normalizer means the exported function is conceptually

$$f_{export}(o) = f_{actor}\left(\frac{o - \mu_{obs}}{\sqrt{v_{obs} + \epsilon}}\right).$$

Its public input is the **raw 61D contract**. Pre-normalizing in the runtime applies the transformation twice. Conversely, exporting only f_{actor} and feeding raw measurements omits it. Both graphs accept the same tensor shape, which is why numerical parity tests are mandatory.

After export, bind provenance:

```
sha256sum logs/rsl_rl/velocity/<run>/model_<iteration>.pt
sha256sum walk.onnx
git rev-parse HEAD
```

Store the export command, exporter commit, task ID, checkpoint digest, output digest, and resolved configuration together. Metadata inside the graph is helpful for portability; an external signed/read-only manifest remains useful because metadata can itself be rewritten.

12.3 Inspect the Open Neural Network Exchange (ONNX) interface

```
uv run python - <<'PY'
import numpy as np
import onnxruntime as ort

path = "walk.onnx"
session = ort.InferenceSession(path, providers=["CPUExecutionProvider"])
input_info = session.get_inputs()[0]
output_info = session.get_outputs()[0]

print("input:", input_info.name, input_info.shape, input_info.type)
print("output:", output_info.name, output_info.shape, output_info.type)
print("metadata:", session.get_modelmeta().custom_metadata_map)

obs = np.zeros((1, 61), dtype=np.float32)
action = session.run([output_info.name], {input_info.name: obs})[0]
print("action shape:", action.shape)
print("finite:", np.isfinite(action).all())
PY
```

For this policy family, expect one batch of 61 float inputs and 14 float outputs. A finite result for an all-zero synthetic observation proves graph execution, not meaningful robot behavior.

Build a frozen parity corpus rather than testing only zero:

```
HOME and level gravity, zero command
small positive/negative perturbation of every joint feature
representative forward, lateral, turn, head, and idle commands
states near but inside documented observation limits
recorded simulation observations from success and failure trajectories
```

For each raw 61-vector, compare deterministic checkpoint output with exported output before converting either to joint targets. Record maximum absolute, root mean squared, and per-action error. Then separately test the target map:

$$q^{target} = q^{HOME} + s \odot a,$$

where $\odot$ is elementwise multiplication. This separates graph error from HOME, scale, sign, or permutation error.

A one-step parity tolerance should come from numeric precision and runtime provider evidence. Long contact rollouts are sensitive: tiny accepted action differences can produce different contact sequences. Therefore trajectory divergence does not automatically falsify export parity, while one-step output disagreement does.

If lower-precision or quantized inference is introduced, calibrate it on this corpus and on closed-loop rollouts. An action error δa_j becomes target error $s_j \delta a_j$; the actuator gain can amplify it into torque. Smaller files are not free if they move a high-gain joint across a contact boundary.

12.4 Rehearse the deployment loop

Current policies use the unified 13D command block, so include `--new-cmd-obs`:

```
uv run scripts/infer_policy.py \
    --walking walk.onnx \
    --new-cmd-obs \
    --lin-vel-x 0.20
```

This script runs ONNX Runtime on the central processing unit (CPU) and builds the same observation order the robot runtime must build. It also applies action delay and maps actions as:

```
observation = np.concatenate([
    base_angular_velocity,       # 3
    projected_gravity,           # 3
    relative_joint_position,     # 14
    joint_velocity,              # 14
    previous_action,             # 14
    command,                     # 13
]).astype(np.float32)

action = onnx_session.run(..., {input_name: observation[None, :]})[0]
target_position = default_pose + action * action_scale
```

The terminal, not the viewer window, receives keys:

```
UP/DOWN           forward command
LEFT/RIGHT        lateral command for the walking model
A/E               turn command
SPACE             zero velocity commands
H                 enter/leave head-command mode
B                 enter/leave body-command mode
P                 apply a random push
Q                 quit
```

The body-pose user interface (UI) exists for policy families that train those slots. The main velocity checkpoint has body-pose reward weight zero, so UI availability is not proof of learned body control.

12.5 The exact 61D runtime contract

Slice	Values	Runtime source
base angular velocity	3	calibrated inertial measurement unit (IMU), body frame, rad/s
projected gravity	3	calibrated orientation projected into body frame
relative joint position	14	servo encoders minus exact HOME offsets
joint velocity	14	encoder-derived velocity in matching order/units
previous action	14	last actor output, not measured joint target
twist	3	application/planner command
head pose	4	requested joint deltas from HOME
body pose	6	requested body deltas or zeros

For every slice, verify:

```
dimension
ordering
dtype
units
frame
sign
offset
latency
noise/filter behavior
reset/initial value
```

The ONNX graph normalizes the assembled raw observation. Do not pre-normalize again in the runtime.

The contract should be machine-readable as well as documented. One source of truth can generate:

- simulator observation-order tests;
- runtime constants/enumerations;
- graph metadata;
- a human-readable table; and
- recorded-log column names.

Duplicating a 61-entry list by hand in several languages invites silent drift. A schema version changes whenever meaning, order, scale, frame, or timing changes—even if dimension remains 61.

Calibration is a transform, not a vague offset

Let the inertial sensor frame be I and the policy body frame be B. If R_{BI} rotates a vector from sensor coordinates into body coordinates and b_I is gyro bias, then

$$\omega_B = R_{BI}(\omega_I - b_I).$$

Applying the inverse rotation, subtracting bias after rotation with a bias in the wrong frame, or using degrees per second instead of radians per second all produce plausible finite numbers with the wrong meaning.

Encoder conversion likewise needs direction $d_j \in \{-1, +1\}$, scale k_j, raw zero z_j, and policy HOME q_j^{HOME}:

$$q_j^{rel} = d_j k_j (c_j - z_j) - q_j^{HOME}.$$

Test named poses and rotations. Holding a robot level should yield projected gravity near the training convention; rotating it gently about one documented axis should change the expected component and sign. Save calibration identity and checksum with each hardware log.

Velocity estimation is also part of the policy. A finite difference

$$\dot{q}_t \approx \frac{q_t - q_{t-1}}{t_t - t_{t-1}}$$

must use measured timestamps, not assume every packet arrived exactly on the nominal period. Filtering reduces noise but adds phase delay; match the filter or cover its effect during training.

12.6 Timing is part of the policy

Microduck training assumes:

```
physics model step        5 ms
policy period            20 ms (50 Hz)
action decimation         4
unfiltered policy output
modeled sensor/action delays
```

A real loop should use a monotonic clock, timestamp sensor snapshots, detect missed deadlines, and choose a documented safe response. Running inference "whenever Linux schedules it" without observing jitter changes the effective delay distribution.

Do not silently reuse stale commands forever. High-level commands need an owner, sequence/timestamp, and expiry behavior. The local safety controller must be able to stop or hold the robot without a cloud round trip.

12.6.1 End-to-end age, not only inference latency

Suppose a sensor sample is captured at t_s, observation assembly starts at t_o, inference finishes at t_i, communication finishes at t_c, and the actuator applies the target at t_a. The age of information at actuation is

$$A = t_a - t_s.$$

Optimizing only $t_i - t_o$ can leave A large because a sample waited in a queue. Measure the whole path and its distribution: median is useful for throughput; high percentile and maximum observed value matter for control.

For policy period T_p = 20 ms, a simple execution budget is

$$C_{sense} + C_{encode} + C_{infer} + C_{safety} + C_{send} + C_{margin} \le T_p.$$

Each C should be a measured worst-case or high-confidence bound under representative system load, not an idle-machine average. Worst-case execution time (WCET) is the longest time a component can take under its stated assumptions. General-purpose Linux rarely gives a formal WCET guarantee, so the runtime should at least record percentiles, maxima, deadline misses, and the load conditions used.

Schedule against absolute deadlines to avoid drift:

```
period_ns = 20_000_000
next_release = monotonic_ns()
while enabled:
    next_release += period_ns
    sample = latest_sensor_snapshot()        # timestamp travels with sample
    command = latest_valid_command_or_zero()
    observation = encode_contract(sample, command, previous_action)
```

```
    action = policy(observation)
    target = safety_limit(home + scale * action)
    send_with_sequence_and_deadline(target, next_release)
    record_timing_and_contract_values()
    sleep_until_monotonic(next_release)
```

If the loop finishes late, sleeping “20 ms from now” makes phase drift. An absolute schedule exposes lateness and tries to return to the intended phase. The safety policy must define whether a late cycle holds the last safe target, uses a fresh observation immediately, ramps toward HOME, or disables torque.

12.6.2 Queue semantics are control semantics

For state feedback, processing every old sample in first-in, first-out order can create an ever-growing lag during overload. A bounded latest-value mailbox usually fits a reactive policy better: overwrite stale unread samples and count drops. For event logs, in contrast, losing intermediate records may be unacceptable. Choose queue behavior per data meaning.

Commands need a validity envelope:

```
source identity, sequence, source timestamp, receive timestamp,
valid-until time, requested mode, bounded values
```

Reject out-of-order or expired commands. A cloud planner may send low-rate intent, but only the local safety authority decides whether it is fresh enough to enter the 50 hertz loop.

Run the deterministic queue demonstration:

```
python examples/realtime_data_age.py
```

It compares first-in, first-out processing with a latest-value mailbox under a temporary consumer slowdown and prints the resulting sample ages.

12.6.3 Communication has a schedulable bit budget

Whether the real transport is a serial servo protocol, Controller Area Network (CAN), CAN with Flexible Data Rate (CAN-FD), or EtherCAT, estimate utilization before hardware integration. If message class i sends b_i on-wire bits at frequency f_i, ideal utilization is

$$U = \frac{\sum_i b_i f_i}{R_{link}},$$

where R_{link} is usable bit rate. Include framing, identifiers, checksums, start/stop bits, inter-frame gaps, device turnaround, retries, synchronization, diagnostics, and firmware-update traffic. Then measure; the ideal equation does not capture arbitration and driver latency.

As an intentionally naive serial example, suppose each of 14 devices costs 20 response bytes plus 10 command/protocol bytes at 50 hertz, with 10 on-wire bits per byte. The raw budget is

$$14(20 + 10)(10)(50) = 210{,}000 \text{ bit/s}.$$

That is already 21% of a 1 megabit/s link before turnaround and retries. A group read/write protocol can change the overhead substantially. The purpose of the estimate is to reveal feasibility and headroom early, not to replace a bus trace.

Clock alignment matters when measurements come from different devices. A vector assembled from “latest” joint samples and an older inertial sample is not truly simultaneous. Preserve source timestamps, measure clock offset/drift, and either align/interpolate within justified limits or train against the observed skew distribution.

12.7 Sim-to-real gap

The sim-to-real gap is the difference between the transition distribution used for training and the real robot:

$$P_{sim}(s_{t+1} \mid s_t, a_t) \neq P_{real}(s_{t+1} \mid s_t, a_t)$$

Microduck reduces this gap with:

- measured computer-aided design (CAD) geometry and mass properties;
- the Better Actuator Models (BAM) voltage-controlled XL330 model;
- friction, armature, mass/inertia, and center of mass (CoM) randomization;
- battery voltage and load-sag variation;
- sensor noise, bias, IMU misalignment, and latency;
- command delay;
- backlash models with encoder-consistent observations; and
- pushes and varied initial states.

Randomization should cover plausible uncertainty, not every imaginable value. An unrealistically wide distribution can teach an overly conservative policy or make the task unsolvable.

The gap is easier to act on when classified:

Gap	Example	Primary response
structural	missing backlash/contact/thermal mode	improve model or controller architecture
parametric	mass, friction, gain, voltage differ	measure, identify, randomize uncertainty
observation	bias, frame, filtering, missing data	calibrate and match sensor pipeline
temporal/computational	delay, jitter, packet loss	real-time design and matched timing randomization
scenario	surfaces/pushes/commands absent in training	expand task distribution and evaluation
objective	simulated reward tolerates unsafe exploit	redesign reward/constraint and acceptance test

Domain randomization usually optimizes an average:

$$\max_{\pi} \mathbb{E}_{\theta \sim p(\theta)}[J(\pi; \theta)].$$

A safety-critical design may care about a lower tail or bounded set instead:

$$\max_{\pi} \min_{\theta \in \Theta} J(\pi; \theta).$$

Worst-case robust optimization can be overly conservative and depends on a credible set Θ; expected randomization can hide rare failures. A practical compromise trains on a measured distribution, evaluates explicit boundary/adversarial cases, and enforces hard runtime limits outside learned authority.

Alternatives and complements include:

- **system identification**: fit nominal parameters from input/output data;
- **online adaptation**: infer a latent environment/motor context from recent history and condition the policy on it;
- **residual modeling/control**: learn only the discrepancy around a trusted nominal model or controller;
- **real-data fine-tuning**: update with carefully collected physical data; and

- **teacher–student distillation**: train a privileged/adaptive teacher, then transfer behavior to a deployable observer.

None removes interface verification. A sophisticated adaptive policy with the wrong joint permutation is still wrong.

Validate from open loop to closed loop

Model comparison should progress through:

1. **static calibration**: zero/HOME, gravity, masses, geometry, frame signs;
2. **one-step response**: voltage/target to immediate current, velocity, torque;
3. **multi-step open-loop trajectory**: replay identical target sequence;
4. **closed-loop subsystem**: matched controller on a supported joint/fixture;
5. **whole-robot closed loop**: bounded policies and scenarios.

One-step error exposes local dynamics without policy feedback hiding it. Long-horizon open-loop error reveals accumulated phase/model mismatch. Closed-loop success is the final goal but can conceal compensating errors, so retain lower-layer evidence.

12.8 Calibrate before randomizing

Use a parameter workflow:

```
measure real subsystem
        v
fit nominal simulator parameter
        v
validate on held-out motion
        v
estimate repeatability and uncertainty
        v
randomize around the nominal range
```

Examples:

- identify actuator friction and torque response on a bench;
- measure battery sag under representative load;
- compare commanded and measured step responses;
- measure encoder bias and delay;
- weigh assemblies and estimate their CoM/inertia; and
- verify foot contact friction on real surfaces.

`scripts/testbench_sim2real.py` and `scripts/validate_bam_testbench.py` support the actuator side of this process.

For measured outputs y_t and simulated predictions $\hat{y}_t(\theta)$, a basic fit minimizes

$$\theta^* = \arg\min_{\theta} \sum_{t \in \mathcal{D}_{fit}} \| W(y_t - \hat{y}_t(\theta)) \|_2^2,$$

where W scales quantities with different units or importance. Validate on a held-out excitation $\mathcal{D}_{test}$; fitting and evaluating on the same step response can overfit sensor noise or one operating point.

Parameters must be identifiable from the experiment. Slow, small motion may reveal static friction but not voltage saturation; one direction may not reveal asymmetry; a fixed load cannot distinguish motor gain from inertia. Design safe excitations that separately cover position, speed, acceleration, load, direction, voltage, and temperature regions needed by the policy.

Fit residuals are evidence for randomization. If a parameter varies between repeated trials, randomize that repeatability range. If one nominal model has a systematic frequency-dependent residual, merely widening a scalar parameter may not cover the missing dynamic mode; revise the model or observation/history instead.

12.9 Staged physical acceptance

The exact hardware procedure belongs to the Microduck runtime/hardware project, but a safe conceptual progression is:

1. **Static interface test:** no torque; verify joint order, signs, HOME, sensors, command zeros, ONNX shapes, and loop timing.
2. **Supported low-authority test:** robot secured; conservative target/current limits; verify action direction and emergency stop.
3. **Supported policy test:** run short windows with zero or tiny commands; inspect saturation, delay, temperature, and observation ranges.
4. **Tethered behavior test:** flat surface, local operator, hard timeout, recorded telemetry/video.
5. **Bounded untethered test:** only after earlier acceptance criteria pass.
6. **Command and disturbance battery:** expand one measured region at a time.

Never make the cloud, Wi-Fi, or a high-level planner responsible for the fast emergency path.

12.9.1 Authority must decrease with latency

A practical robot has several control planes:

```
cloud/remote agent: goals, semantic plan, slow perception assistance
brain system-on-chip: local perception, planning, policy inference, logging
real-time control board: timestamped I/O, limits, watchdog, safe state
motor drive: current/voltage/position protection and commutation/servo loop
mechanical system: passive stops, compliance, stable power-off behavior
```

The system-on-chip (SoC) is the main compute module. The lower a layer and the shorter its deadline, the less it may depend on upper layers. For JumpRover, whose brain SoC is ahead of the unfinished real-time board and mechanics, this is a design requirement for the eventual handoff—not evidence that those unfinished layers already satisfy it.

Neural actions are proposals inside a safety envelope. Hard checks can include:

$$q_{min} \le q^{target} \le q_{max}, \qquad |q_t^{target} - q_{t-1}^{target}| \le \Delta q_{max},$$

plus current, velocity, temperature, tilt, communication age, power, and mode limits. Clipping alone can create sustained saturation; log every intervention and enter a safe state when interventions persist.

A watchdog asks not “is the process alive?” but “has a valid, fresh, ordered command satisfying the current mode arrived before its deadline?” Exercise it with fault injection before free motion:

- stop the inference process;
- delay or reorder command packets;
- freeze one sensor timestamp;
- corrupt one value beyond range;
- disconnect the planner/network; and
- request a policy/schema/calibration mismatch.

Each case needs an expected bounded transition and a recorded recovery/reset procedure. An emergency stop must be reachable independently of the policy and tested under the same power conditions as deployment.

12.10 Hot-swapping policies

The runtime can switch walking, standing, recovery, and trick policies because they share the 61D input and 14D output contract. Hot-swapping still requires:

- the same joint/action metadata;
- correct command-slot semantics for each skill;
- clearing stale command values when a skill expects zero padding;
- defined previous-action behavior at the boundary; and
- a safe arbitration state machine.

For example, sit/stand uses the first twist slot as a posture flag, while an all-zero twist means stand. Feeding zeros because an application forgot the flag can look like a policy that ignores the button.

Shape compatibility is necessary, not sufficient. At switch time, policy B receives the physical state produced by policy A. If B was trained only from a narrow reset distribution, that state may be out of distribution. Define a guarded transition:

```
request skill B
 -> verify policy/schema/calibration identity
 -> command A toward a documented handoff set
 -> verify pose, speed, contacts, command freshness, and safety state
 -> initialize B's previous-action/history rule
 -> activate B with bounded authority
 -> monitor acceptance window or fall back
```

Blindly blending actions,

$$a = (1 - \beta)a^A + \beta a^B,$$

can smooth targets but produces actions neither policy trained to own and does not guarantee contact compatibility. Use blending only when its states and transients are trained or explicitly validated. A discrete state machine with safe handoff sets is easier to reason about initially.

Version command semantics per skill. Reusing twist index 0 as a posture flag is valid only when the active-policy manifest declares that interpretation and the arbiter clears unrelated slots atomically. Stale head/body commands should not leak across skill changes.

Rehearse combinations before hardware:

```
uv run scripts/infer_policy.py \
    --walking walk.onnx \
    --standing stand.onnx \
    --sitstand sitstand.onnx \
    --roulade roulade.onnx \
    --new-cmd-obs
```

12.11 Deployment acceptance record

For each released policy, retain:

```
source commit and task ID
resolved env/agent YAML
checkpoint digest
export command and ONNX digest
ONNX input/output and metadata inspection
fixed simulation evaluation results
CPU rehearsal results
runtime version and observation/action contract version
hardware calibration version
staged physical test logs and video
known operating envelope and known failures
rollback artifact
```

“The file loads” is a format check. “The viewer walks” is a simulation check. Neither is physical release evidence.

Make release compatibility executable. The runtime should refuse to arm when policy schema, joint map, robot hardware revision, calibration version, action rate, or safety-envelope version is incompatible with its manifest. A warning that scrolls past while torque enables is not enforcement.

Rollback means more than retaining an old file. Keep the previous known-good bundle, its compatible runtime/configuration, and a tested atomic activation mechanism. Record why rollback occurred and preserve the failed bundle/logs for diagnosis. Never overwrite the only known-good artifact during an experiment.

Telemetry needed for postmortem should be bounded and prioritized. At minimum, retain monotonic/source timestamps, mode/policy hash, raw 61D observation, action and applied target, command identity/age, safety interventions, actuator status, deadline metrics, and faults. High-rate storage may use a ring buffer that freezes recent history on a trigger so logging cannot exhaust memory.

12.12 Exercises and deployment lab

1. Name the information present in a training checkpoint but normally absent from the deployed actor. Why is the checkpoint still needed after export?
2. Derive what happens when a runtime pre-normalizes an observation and feeds it to a graph that already embeds the same normalizer.
3. Design a frozen 61D parity corpus with at least five semantically different cases. Why is all-zero alone insufficient?
4. If action scale for joint j is 1 radian/unit and export error is 0.003, what target-position error results in radians and degrees?
5. A sensor captures at 100.000 ms, observation assembly starts at 104.000 ms, inference ends at 108.500 ms, send finishes at 111.000 ms, and the actuator applies at 113.000 ms. Compute inference latency and age of information.
6. In a 20 ms period, sensing costs 2.5 ms, encoding 1.0 ms, inference 4.5 ms, safety 0.5 ms, and sending 3.0 ms. Compute nominal slack. Why is this not a WCET proof?
7. Run `python examples/realtime_data_age.py`. Explain why the first-in, first-out queue becomes stale even without packet loss.
8. Repeat the chapter's serial-link calculation at 100 hertz. What utilization does it imply on 1 megabit/s before unmodeled overhead?
9. Write the correct body-frame angular-velocity transform from an inertial sensor measurement, mounting rotation, and sensor-frame bias. Name two plausible but wrong implementations.
10. Two joint samples are 0.020 radian apart and their measured timestamps are 15 ms apart, although nominal period is 20 ms. Calculate velocity using measured and assumed time. What error does the assumption cause?
11. Classify each as structural, parametric, observation, temporal, scenario, or objective gap: missing backlash; wrong gyro sign; low battery voltage; rare 35 ms inference; unseen wet tile; reward permits head impacts.
12. Why must actuator identification use held-out excitations? Give one example of a parameter that a slow low-load trajectory cannot identify well.
13. Specify the expected safe response to three injected faults: expired cloud command, frozen sensor timestamp, and repeated joint-target saturation.
14. Design a guarded handoff from walking to stand-up recovery. Include the previous-action rule and fallback.
15. Explain why a cloud agent may choose a goal but must not own the emergency stop or 50 hertz deadline.
16. Prove checkpoint-to-ONNX parity: export one exact checkpoint, inspect the interface/metadata, compare deterministic one-step actions over a frozen corpus, rehearse deployment-style commands, and write a signed/hash-based evidence record.

12.13 Folded solutions

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show solutions to Exercises 1–8

1. The checkpoint includes critic, optimizer state, observation-normalizer state, counters/curriculum progress, and usually stochastic-policy training state in addition to actor weights. It remains the reproducible source for re-export, parity, audit, and training resume; the smaller actor cannot recreate all of that state.
2. If $N(o) = (o - \mu)/\sigma$, double normalization computes $N(N(o)) = ((o - \mu)/\sigma - \mu)/\sigma$, not $N(o)$. Only special accidental values of μ, σ hide the error. Shape and finiteness still pass.

3. Include level HOME/zero command, positive and negative one-feature probes, forward/lateral/turn/head commands, near-limit valid observations, and recorded success/failure states. Zero does not exercise joint ordering, command routing, realistic normalization ranges, or nonlinear activation regions.
4. The target error is 1(0.003) = 0.003 radian. In degrees it is $0.003(180/\pi) \approx 0.172^{\circ}$. Whether that is acceptable depends on the joint, gains, contact state, and closed-loop evidence.
5. Inference latency is 108.5 – 104 = 4.5 ms. Information age at application is 113 – 100 = 13 ms. Reporting only 4.5 ms omits sensing wait, communication, and actuation timing.
6. Used time is 2.5 + 1 + 4.5 + 0.5 + 3 = 11.5 ms, so nominal slack is 8.5 ms. These are single/average measurements unless bounded under representative worst load; scheduling, allocation, cache, transport retries, and contention can consume the apparent slack.
7. The producer creates four 200 hertz samples per 50 hertz controller period, but FIFO consumes only one. Its unread backlog grows, so selected capture times move farther behind wall time. Latest-value semantics intentionally drop obsolete state and retain bounded age; drop counts remain diagnostic.
8. Doubling frequency doubles the raw budget to 420,000 bit/s, or 42% of a 1 megabit/s link. Protocol turnaround, retries, diagnostics, and scheduling still require measured headroom.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show solutions to Exercises 9–15

9. Use $\omega_B = R_{BI}(\omega_I - b_I)$. Wrong variants include using R_{IB} without transposing/inverting it, subtracting a body-frame bias as though it were sensor-frame, applying degrees as radians, or reversing an axis sign.
10. Measured-time velocity is 0.020/0.015 = 1.333 rad/s. Assumed-time velocity is 0.020/0.020 = 1.0 rad/s, underestimating by 0.333 rad/s or 25% relative to the measured-time value.
11. Missing backlash is structural; wrong gyro sign is observation; low battery voltage is parametric/operating-condition; rare 35 ms inference is temporal; unseen wet tile is scenario (and friction parameter once modeled); a reward permitting head impacts is objective. Boundaries can overlap, so name the causal layer the proposed fix addresses.
12. Fitting and testing the same excitation measures memorization of one operating trajectory, not predictive model quality. Slow low-load motion poorly identifies voltage/current saturation, high-speed back electromotive force, inertia under acceleration, or thermal effects.
13. An expired cloud command should be rejected and replaced with trained zero or a local safe mode. A frozen sensor timestamp should invalidate the observation and hold/ramp/disable according to the tested state machine, not reuse it silently. Repeated saturation should raise telemetry and leave neural authority—ramp to a bounded pose or disable as hardware permits.
14. Verify the recovery policy bundle/schema/calibration, detect a valid fallen state within its trained reset support, zero unrelated commands, initialize previous action according to its manifest (often last applied action or a documented zero), switch with bounded current/target rate, monitor progress and impacts, and fall back to torque-off/supported intervention on timeout or contract violation. Do not switch while the walking policy is still commanding motion without an explicit transition.
15. Cloud/network latency, outages, reordering, and security boundaries are incompatible with the fast physical deadline. The remote agent may provide expiring semantic intent; local real-time layers validate it and retain independent watchdog, limits, stop, and safe-state authority.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show the expected parity evidence and comparison code for Exercise 16

The result should include model/checkpoint hashes, exact raw observation corpus, normalizer provenance, action transform, runtime providers, tolerances, and per-element error—not only “both looked similar.” Compare deterministic actors on identical 61D raw inputs:

```
import numpy as np

# Shape: (number_of_frozen_cases, 14). Both arrays must come from the same
# raw 61D observations; the checkpoint path must apply its normalizer once and
# the exported graph must contain/apply that same normalizer once.
```

```
checkpoint_actions = np.load("checkpoint_actions.npy")
onnx_actions = np.load("onnx_actions.npy")

assert checkpoint_actions.shape == onnx_actions.shape
assert checkpoint_actions.shape[1] == 14
error = np.abs(checkpoint_actions - onnx_actions)
print("max_abs", error.max())
print("p99_abs", np.quantile(error, 0.99))

# Derive tolerances from numeric precision/runtime evidence; do not copy these
# illustrative values blindly for a quantized model.
np.testing.assert_allclose(
    onnx_actions,
    checkpoint_actions,
    rtol=1e-5,
    atol=1e-6,
)
```

First compare one-step outputs; long contact trajectories can diverge from tiny floating-point differences even when the interface is correct. If one-step outputs disagree, inspect observation order, raw versus normalized input, actor mean versus stochastic sample, data type, HOME/action scaling, and stale previous action before blaming physics. The final record binds checkpoint, export, corpus, result, and rehearsal log by hashes and includes the known operating envelope and rollback bundle.

Continue with Microduck customization labs.

13. Customization Labs

These labs turn the book into a project course. Work on a branch, keep one question per experiment, and preserve tests/configuration with every result.

The labs are ordered from reproduction to architecture change. Do not skip the early ones because later work needs their habits: artifact identity, pure tests, fixed evaluation, and deployable contracts.

Every lab notebook should contain seven fields:

```
claim:        one falsifiable sentence
change:       exact code/config treatment
controls:     what remains fixed
mechanism:    why the change should affect the claim
measurement: raw cases plus aggregate/uncertainty
acceptance:   threshold chosen before seeing the result
decision:     keep, revise, or —rejectwith evidence
```

Use the live `tasks` package, `tests` directory, and `scripts` alongside these instructions. Start a focused branch and record the baseline:

```
git switch -c study/<short-lab-name>
git rev-parse HEAD
git status --short
uv run --with pytest pytest tests/
```

Do not mix unrelated local changes into a lab patch. A result is reproducible only if its source commit, dirty diff, lockfile, command, seeds, and artifacts are recoverable.

Lab 0: reproduce before modifying

Goal: prove that you can run and explain the existing system.

1. Run all central processing unit (CPU) tests.
2. Run a 64-environment, five-iteration velocity smoke train.
3. Load an existing trained velocity checkpoint.
4. Record a 10-second rollout.
5. Export Open Neural Network Exchange (ONNX) and run the CPU rehearsal with a fixed forward command.
6. Draw the 61D observation and 14D action layout from memory.

Deliverable: the lab report template from Chapter 9 plus one paragraph explaining why playback direction changes are command sampling, not random actions.

Acceptance evidence must include:

- clean test summary and five-iteration log;
- resolved environment/agent configuration;
- checkpoint and export digests;
- one frozen 61D observation with checkpoint/ONNX 14D action comparison;
- a finite 10-second video with task/command identity; and
- measured policy-loop throughput in the CPU rehearsal.

Explain each arrow in this chain from memory, then verify it in code:

```
task ID -> factory -> managers -> 61D raw observation
-> embedded normalizer -> deterministic actor -> 14D action
-> HOME/scale/order map -> actuator target
```

If one arrow cannot be explained, reproduction is incomplete even if the viewer looks convincing.

Lab 1: change a command distribution

Question: does more turn-in-place data improve turning without hurting forward walking?

The current setting is:

```
TURN_IN_PLACE_FRACTION = 0.15
```

If one rollout batch contains $N = N_{env}N_{step}$ transitions and command buckets were independent per transition, expected turn samples would be Np_{turn}. Commands persist for seconds, so transitions inside one command segment are correlated; the true number of independent command episodes is much smaller. Log both transition count and resample/episode count.

Changing p_{turn} also changes the mixture objective:

$$J = p_{turn}J_{turn} + p_{stand}J_{stand} + p_{general}J_{general}.$$

An improvement in J_{turn} may accompany regression elsewhere because training capacity and samples are reallocated. This is a multi-objective trade, not a free augmentation.

Experiment:

1. Keep a baseline run and fixed evaluation battery.
2. Change only this fraction, for example to `0.25`.
3. Add/update a configuration test that checks the expected value.
4. Run the CPU suite and five-iteration smoke test.
5. Train both settings for the same budget and seed policy.
6. Compare turn-in-place success, translation drift, forward speed tracking, and total command coverage.

Do not conclude that 25% is universally better. Increasing one bucket reduces the fraction of all other experience.

Use paired training seeds and a table such as:

Variant	Train seed	Turn success	Turn drift	Forward MAE	Severe failures
0.15	101	measure	measure	measure	classify
0.25	101	measure	measure	measure	classify

Repeat over several seeds. Predeclare a decision such as “retain 0.25 only if the paired turn-success estimate improves, its uncertainty is reported, and forward mean absolute error (MAE) plus severe-failure rate remain within chosen non-inferiority margins.” The margins must come from use requirements, not be moved after results appear.

Also assert the mixture is valid:

```
def test_command_bucket_probabilities_are_valid():
    cfg = make_microduck_velocity_env_cfg()
    twist = cfg.commands["twist"]
    assert 0.0 <= twist.rel_turn_in_place_envs <= 1.0
    assert 0.0 <= twist.rel_standing_envs <= 1.0
    assert (
        twist.rel_turn_in_place_envs + twist.rel_standing_envs
    ) <= 1.0
```

Adapt field names to the live configuration and test the resolved value; the remaining probability belongs to general command sampling.

Lab 2: add a small custom reward safely

Goal: learn the pure-helper → environment wrapper → configuration → test pattern.

Suppose a new task needs a self-negating trunk-height error:

```python
# In tasks/mdp.py
def height_l1_from_values(height: torch.Tensor, target: float) -> torch.Tensor:
    """Return a nonpositive penalty; zero only at the target."""
    return -torch.abs(height - target)


def trunk_height_l1_penalty(
    env: ManagerBasedRlEnv,
    target: float,
    asset_cfg: SceneEntityCfg = SceneEntityCfg("robot"),
) -> torch.Tensor:
    asset = env.scene[asset_cfg.name]
    origin_z = env.scene.terrain.env_origins[:, 2]
    height = torch.nan_to_num(
        asset.data.root_link_pos_w[:, 2] - origin_z,
        nan=0.0,
    )
    return height_l1_from_values(height, target)
```

Wire it in the relevant task factory:

```python
cfg.rewards["trunk_height_l1"] = RewardTermCfg(
    func=microduck_mdp.trunk_height_l1_penalty,
    weight=0.2,  # positive: the function already returns <= 0
    params={"target": 0.095},
)
```

Test the math and sign without a simulator:

```python
def test_height_l1_penalty():
    h = torch.tensor([0.095, 0.105, 0.075])
    out = height_l1_from_values(h, target=0.095)
    torch.testing.assert_close(out, torch.tensor([0.0, -0.01, -0.02]))


def test_height_penalty_weight_is_positive():
    cfg = make_my_task_env_cfg()
    assert cfg.rewards["trunk_height_l1"].weight > 0.0
```

Trace units and sign. The helper returns meters with a negative sign. A weight of 0.2 reward units per meter gives

$$r_h = -0.2|h - h^*|.$$

At a 2 cm error the contribution is –0.004 per step. Over 1,000 undiscounted logged steps that could sum to about –4, but Proximal Policy Optimization (PPO)'s effect depends on competing terms and state visitation. Compute observed reward mass after the smoke/full run rather than assuming `0.2` is large or small.

The absolute-value subgradient is constant away from zero:

$$\frac{\partial r_h}{\partial h} = -0.2\,\mathrm{sign}(h - h^*), \qquad h \neq h^*.$$

The policy gradient does not differentiate directly through simulator height; the equation still reveals objective geometry: unlike a narrow Gaussian, L1 error does not vanish far from target. That can help recovery but can also dominate phases where target height is inappropriate.

Before keeping the reward, answer:

- Is the target measured from the actual current robot model?
- Is height meaningful in every task phase?
- Could a fallen pose match the height?
- Does this term block motion the task physically requires?
- How large is its weighted mass relative to the positive objective?

This example teaches wiring; it is not a recommendation to add a height penalty to the main walking task.

Extend the test matrix:

```
def test_height_penalty_is_symmetric_and_monotone():
    near = height_l1_from_values(torch.tensor([0.090, 0.100]), 0.095)
    far = height_l1_from_values(torch.tensor([0.075, 0.115]), 0.095)
    torch.testing.assert_close(near[0], near[1])
    torch.testing.assert_close(far[0], far[1])
    assert torch.all(far < near)
```

Add a configuration-level test that the selected trunk and terrain-relative height are correct. Then audit stable counterexamples: prone, supine, and side poses can sometimes match height. If the true goal is *progress to upright*, a bounded potential difference in tilt plus a terminal pose criterion may be harder to farm than continuous absolute height.

Lab 3: design a new task from a template

Choose the closest family:

Intended behavior	Starting point
locomotion	velocity
episodic trick ending in a pose	stand-up
commanded two-state transition	sit-stand
dynamic maneuver	roulade

Before code, complete a task specification:

```
initial-state classes and probabilities:
command and resampling semantics:
actor-observable variables:
critic-only variables:
14D action interpretation:
success state and minimum hold time:
true termination versus truncation:
positive objective terms:
cost/constraint terms:
five cheapest plausible exploits:
curriculum frontier:
deployment owner and skill-handoff states:
```

Translate it into a task-specific Markov Decision Process objective:

$$J(\theta) = \mathbb{E}_{s_0, c, \xi, \pi_\theta}\left[\sum_{t=0}^{H-1} \gamma^t \sum_i w_i f_i(s_t, a_t, s_{t+1}, c; \xi)\right],$$

where ξ contains randomized physical/sensor conditions. Circle every quantity unavailable to the actor and confirm it appears only in critic, reward, reset, or evaluation logic—not in deployed observation.

Implementation checklist:

1. Write the task's observable success criterion and plausible exploits.
2. Verify target states in physics before Proximal Policy Optimization (PPO).
3. Create `microduck_<name>_env_cfg.py` by building on the closest factory.
4. Put custom Markov Decision Process (MDP) functions in `tasks/mdp.py`, grouped with the task.
5. Preserve the 61D actor command layout and 14D action layout.
6. Select the correct walk/all-collisions/roller model.
7. Give the runner a distinct `experiment_name`.
8. Register train/play variants in `tasks/__init__.py`.
9. Add a backlash twin only if it mirrors the base model correctly.
10. Write config tests for dimensions, signs, selectors, gates, and task registration.

11. Run CPU tests and the five-iteration smoke train.
12. Evaluate the main behavior, every stable flop, and likely reward hacks.

Minimal registration shape:

```
register_mjlab_task(
    task_id="Mjlab-MyTask-Flat-MicroDuck",
    env_cfg=make_microduck_my_task_env_cfg(),
    play_env_cfg=make_microduck_my_task_env_cfg(play=True),
    rl_cfg=MicroduckMyTaskRlCfg,
    runner_cls=MicroduckOnPolicyRunner,
)
```

Add invariant tests before reward tuning:

```
def test_my_task_family_contract():
    cfg = make_microduck_my_task_env_cfg()
    assert expected_actor_observation_dimension(cfg) == 61
    assert expected_action_dimension(cfg) == 14
    assert all_penalty_weights_have_expected_sign(cfg)
    assert selectors_resolve_on_configured_robot(cfg)

def test_my_task_is_registered():
    assert "Mjlab-MyTask-Flat-MicroDuck" in list_registered_task_ids()
```

The helper names above are pseudocode; use existing repository test utilities and patterns instead of inventing a second configuration system. A valuable test fails when someone changes the selected model, command order, or reward sign—not only when the file fails to import.

Define an acceptance ladder:

```
physics target is supportable
-> manager construction and 61/14 contracts pass
-> five-iteration run is finite and exports
-> skill appears in nominal cases across seeds
-> stable-flop/exploit battery is rejected
-> randomization boundaries retain performance
-> deployment rehearsal meets timing/interface contract
```

Lab 4: add a curriculum from evidence

Do not begin by inventing ten stages. First train the easier slice and identify the measured frontier.

Example goal: widen a pose command only after small commands track well.

```
cfg.curriculum["pose_range"] = CurriculumTermCfg(
    func=microduck_mdp.pose_command_range_curriculum,
    params={
        "command_name": "head_pose",
        "range_stages": [
            {"step": 0, "ranges": ((-0.05, 0.05),) * 4},
            {"step": 500 * 24, "ranges": ((-0.2, 0.2),) * 4},
        ],
    },
)
```

For the real head, use per-joint ranges rather than the identical illustrative ranges above because yaw, pitch, and roll have different limits.

Test that:

- stage steps use environment-step units;
- the live command-manager term changes;
- the first and last ranges are intended;
- the policy sees nonzero samples before a reward is introduced; and
- metrics do not collapse at the stage boundary.

The schedule above is iteration-based because repository curriculum steps are $k = 24 \times$ iteration. At 500 iterations, the stage begins at step 12,000 regardless of environment count. Log the active range and empirical command histogram so a metric jump can be separated from a logging-weight jump.

A configuration test should exercise boundary values:

```
def test_pose_range_curriculum_boundaries(env):
    # Pseudocode: invoke the curriculum through the initialized manager.
    apply_curriculum(env, step=500 * 24 - 1)
    before = env.command_manager.get_term_cfg("head_pose").ranges
    apply_curriculum(env, step=500 * 24)
    after = env.command_manager.get_term_cfg("head_pose").ranges
    assert before == ((-0.05, 0.05),) * 4
    assert after == ((-0.2, 0.2),) * 4
```

Use the live helper signature and per-joint physical ranges in production. Critically, mutate/read the manager copy after initialization; changing `env.cfg` may not change the running term.

For a competence-triggered alternative, define a held-out metric m_k, entry threshold u, exit threshold $l < u$, and patience P:

```
advance after m_k >= u for P consecutive evaluations
do not regress unless m_k <= l for P consecutive evaluations
```

The gap between u and l is hysteresis; it prevents noisy oscillation between stages. Save curriculum state in checkpoints. Compare scheduled and competence-triggered designs at equal transition budgets rather than assuming adaptive pacing is automatically superior.

Lab 5: obstacle avoidance—choose the architecture first

The current velocity policy cannot avoid obstacles. There are two different projects hidden inside the phrase “add obstacle avoidance.”

Option A: perception/planning above locomotion

```
camera/depth/ToF
      v
local obstacle model + planner
      v
safe local [vx, vy, yaw_rate] command
      v
existing 61D Microduck walking policy
```

Depth cameras, range rays, and time-of-flight (ToF) sensors differ in field of view, minimum range, sunlight sensitivity, latency, and missing-data behavior. Choose a sensor the physical robot can reproduce; a perfect simulator depth map is not a deployable observation.

This is the recommended first project. It preserves the proven fast motor policy and observation contract. The planner can be classical, learned, or cloud-assisted, but a local collision/stop layer must remain available when network service is absent or stale.

Build a simulation harness that varies obstacles and checks:

- planner command expiry;
- stop distance;
- minimum clearance;
- collision rate;
- progress to goal; and
- behavior on missing/stale perception.

The locomotion policy still needs robustness to the planner’s command changes, but it does not need pixels.

Start with stopping geometry. For speed v, end-to-end reaction delay τ, conservative deceleration $a_{stop} > 0$, and margin m, a one-dimensional stopping bound is

$$d_{stop} = v\tau + \frac{v^2}{2a_{stop}} + m.$$

The first term is distance traveled before braking begins; the second follows from $v_f^2 = v^2 - 2a_{stop}d$ with $v_f = 0$. Measure a_{stop} across surfaces, battery states, and command transitions. Perception range must exceed this bound plus uncertainty at every allowed speed.

A small local planner can score candidate twists without modifying the motor policy:

```
def choose_twist(candidates, local_map, goal, now):
    valid = []
    for twist in candidates:
        path = rollout_kinematic_model(twist, horizon_s=1.0)
        clearance = minimum_clearance(path, local_map)
        if clearance < stopping_margin(twist, now):
            continue
        score = goal_progress(path, goal) - turn_cost(twist)
        valid.append((score, twist))
    return max(valid)[1] if valid else ZERO_TWIST
```

This resembles a sampled Model Predictive Control (MPC) or dynamic-window planner. Its model need not predict joint contacts; it proposes a safe local velocity inside the locomotion policy's trained command envelope. Verify that command acceleration is also within training experience.

Do not use time-to-collision alone at zero relative velocity or with invalid depth. Missing/old perception must produce an explicit conservative state, not an infinity that looks safe. Record source timestamp and map age in every planner decision.

Option B: an end-to-end exteroceptive policy

```
proprioception + command + depth/rays/features
      v
new actor architecture
      v
joint actions
```

This requires a new versioned policy family because extra features break the 61D hot-swap contract. Define:

- sensor model and hardware availability;
- feature dimensions/order/units and missing-data semantics;
- model architecture and inference budget;
- obstacle/goal scene generator;
- collision, clearance, progress, and deadlock objectives;
- domain randomization for sensor noise/dropout and obstacle geometry;
- runtime contract/version negotiation; and
- safety behavior independent of learned perception.

Do not merely add boxes and a collision penalty. Without pre-contact observation, the policy learns only collision reaction.

One compact exteroceptive representation is K body-frame range rays plus a validity mask:

$$x_i^{ray} = \mathrm{clip}\left(\frac{d_i}{d_{max}}, 0, 1\right), \qquad m_i \in \{0, 1\}.$$

Never encode "missing" as ordinary maximum distance unless training and safety semantics intentionally mean "assume clear." A mask lets the network distinguish no return from a measured far surface. Include ray angles/frame, height, update rate, age, clipping, and self-geometry exclusions in the schema.

Possible network architectures include:

- concatenate a small ray vector with proprioception before a multilayer perceptron;
- encode a depth image with a convolutional/transformer encoder, then fuse a compact feature with proprioception; or

- train a perception encoder separately and freeze/distill it for bounded real-time inference.

The simple concatenated baseline is easiest to test. Image encoders can use richer geometry but demand more data, augmentation, compute, and latency matching.

An illustrative objective is

$$r_t = w_p\left(\Phi_{goal}(s_{t+1}) - \Phi_{goal}(s_t)\right) - w_c\mathbf{1}[collision] - w_n \max(0, d_{safe} - d_{min}) - w_a \| a_t - a_{t-1} \|_2^2.$$

Progress difference avoids paying forever for standing near a goal. Collision is a severe event; near-clearance shaping supplies earlier signal; action change discourages jitter. Audit the shortcut of freezing far from obstacles: without progress or a time cost, it may be safest and highest-return.

Train with procedurally varied widths, shapes, heights, materials, motion, sensor dropout, latency, and spawn/goal relationships, then reserve generator seeds and geometry families for test. Curriculum can begin with sparse static obstacles, but evaluation must include clutter, narrow passages, dead ends, dynamic crossings, missing data, and the exact stop behavior.

Lab 5 acceptance matrix

Case	Required evidence
clear path	progress/tracking is not materially degraded
static obstacle	no collision; minimum clearance and stop/progress pass
narrow passage	bounded success/failure, no oscillatory deadlock
dynamic crossing	prediction/reaction remains within latency envelope
stale/missing sensor	local safe stop occurs before stopping bound
unseen geometry	uncertainty and failure classes reported
cloud/network loss	local planner/safety behavior remains available

Option A and B should be compared on end-to-end latency, failure severity, data/compute cost, schema complexity, and physical evidence—not on novelty.

Lab 6: measure one sim-to-real parameter

Choose one subsystem, such as a single XL330 step response.

1. Define a safe bench input and collect timestamped command, position, velocity, current/voltage, and load conditions.
2. Replay the same input in the Better Actuator Models (BAM) testbench simulation.
3. Compare rise time, overshoot, steady-state error, and decay/friction.
4. Fit a nominal parameter using training data.
5. Validate it on a different input.
6. Estimate a realistic randomization range from repeated trials.
7. Record the model version and evidence.

The lesson is broader than one actuator: fit first, randomize uncertainty second.

Use at least two excitation families, for example small bidirectional steps for friction/deadband and larger safe sweeps for speed/voltage behavior. Split by trajectory, not individual adjacent samples, because neighboring timestamps are strongly correlated.

For candidate parameters θ, define a weighted residual:

$$L(\theta) = \sum_t \left(w_q(q_t - \hat{q}_t(\theta))^2 + w_v(\dot{q}_t - \hat{\dot{q}}_t(\theta))^2 + w_i(i_t - \hat{i}_t(\theta))^2\right).$$

Weights nondimensionalize signals or express priorities; otherwise a large-numeric-unit channel dominates. Report each physical residual separately even when optimizing one scalar loss.

A transparent coarse-to-fine search is a good beginner baseline:

```
best = None
for friction in friction_grid:
    for motor_gain in gain_grid:
        prediction = replay_sim(command, friction, motor_gain)
        loss = weighted_residual(measurement, prediction)
        if best is None or loss < best.loss:
            best = Result(loss, friction, motor_gain)
```

Then validate `best` on held-out command shapes, directions, loads, and battery states. Inspect residual versus time; one scalar score can hide a phase lag that is disastrous in closed loop. If many parameter pairs fit equally well, the experiment did not identify them separately—design a new excitation rather than reporting false precision.

Lab 7: create a policy acceptance battery

Write a headless evaluator for one task that runs a matrix of seeds and command cases. Output a machine-readable summary:

```
{
  "task": "Mjlab-Velocity-Flat-MicroDuck",
  "checkpoint": "model_2000.pt",
  "cases": 100,
  "successes": 92,
  "falls": 5,
  "nonfinite": 0,
  "mean_linear_tracking_error_mps": 0.04,
  "mean_yaw_tracking_error_radps": 0.09
}
```

Define success before running. Keep the raw per-case data so an aggregate number can be audited. Add video sampling for human-visible quality.

Prefer newline-delimited JavaScript Object Notation (JSONL) for raw rows and a separate summary. Each row should include policy/config hashes, train/evaluation seeds, scenario identifier (ID), command, reset/disturbance parameters, success, continuous metrics, failure class, and video/log reference. JavaScript Object Notation (JSON) is a portable structured-text format; JSONL keeps one independently parseable record per line.

Evaluator pseudocode:

```
for case in fixed_case_manifest:
    env.reset(seed=case.seed, overrides=case.reset)
    apply_command(case.command)
    trace = rollout(policy, horizon=case.horizon)
    row = {
        **case.identity(),
        "success": success_rule(trace),
        "metrics": compute_metrics(trace),
        "failure_class": classify_failure(trace),
    }
    append_jsonl("episodes.jsonl", row)
```

The manifest must be read-only during final assessment. Validate the evaluator itself with synthetic traces: known success, exact threshold, fall, timeout, and nonfinite cases. An off-by-one time or frame convention can reverse labels.

Report Wilson intervals for per-slice success, seed-level values for algorithm comparisons, and a lower-tail metric for severe behavior. A hundred simulation episodes from one checkpoint do not equal a hundred independent training seeds; state which uncertainty is estimated.

Lab 8: build a real-time budget and stale-data test

Goal: prove that the deployment loop implements the timing distribution the policy expects.

1. Draw the path from sensor capture through observation, inference, safety, transport, and actuator application.
2. Instrument each boundary with a monotonic timestamp and sequence number.
3. Run at representative central processing unit load and record at least several minutes of cycles.
4. Report median, 95th, 99th, maximum observed duration, missed deadlines, and information age—not only average inference time.
5. Calculate on-wire bus utilization including framing/turnaround/retries and compare it with a captured trace.
6. Inject a missed inference cycle, frozen sensor timestamp, late command, and lost cloud connection.
7. Verify the real-time board/local safety state reaches the predeclared safe response without cloud help.

Use `python examples/realtime_data_age.py` to explain why a latest-value mailbox is appropriate for reactive state. Then implement the same semantics with explicit overwrite/drop counters in the runtime.

The deliverable is a timing table:

Stage	Nominal budget	p50	p99	max observed	deadline action
sensor and transport	choose	measure	measure	measure	invalidate stale
observation	choose	measure	measure	measure	safe fallback
inference	choose	measure	measure	measure	hold, ramp, or disable
safety/send	choose	measure	measure	measure	local stop
end-to-end age	`< trained/tested` bound	measure	measure	measure	reject cycle

“No misses observed” is evidence over a stated duration/load, not proof that a deadline can never be missed. A hard real-time claim needs a stronger platform and worst-case analysis; the safety state is still required.

Lab 9: design and test one skill handoff

Choose walking → standing, walking → recovery, or recovery → walking. Write a finite-state machine with explicit guards:

```
ACTIVE_A
  -> HANDOFF_REQUESTED
  -> ENTER_HANDOFF_SET
  -> VERIFY_BUNDLE_AND_STATE
  -> ACTIVE_B
  -> ACCEPT or FALLBACK
```

Define the handoff set $\mathcal{H}_{A\to B}$ by measurable bounds such as pose, speed, contacts, action magnitude, sensor age, and command validity:

$$\mathcal{H}_{A\to B} = \{s : |\text{tilt}(s)| < \epsilon_R, \|\dot{q}\| < \epsilon_v, \text{contacts}(s) \in C, \text{age}(s) < \tau_{max}\}.$$

State how policy B initializes previous action/history and how unrelated command slots are cleared. Test:

- request at a valid handoff state;
- request outside the set;
- wrong policy schema/hash;
- stale command during transition;
- B fails to make progress before timeout; and
- repeated switch requests.

Measure target discontinuity, saturation, peak tilt, contacts, transition time, success, and fallback. Do not hide a discontinuity with an untrained blend; train or validate any transition trajectory the system applies.

Capstone: a complete evidence-backed customization

Your capstone should include:

```
problem statement and non-goals
architecture choice
physics assumptions and measurements
observation/action/command contract
reward and termination rationale
likely exploits
unit and configuration tests
five-iteration smoke evidence
training configuration and checkpoints
fixed evaluation battery and videos
ONNX export/interface inspection
CPU deployment rehearsal
sim-to-real risk register
known limitations and next experiment
```

The capstone is complete when another person can reproduce the result and understand where it is safe to generalize—not when one attractive video exists.

Recommended milestones:

Review	Required artifact	Stop condition
problem review	specification, non-goals, hazard/reward-hack list	capability is not observable/trainable
physics review	equilibrium/actuator/sensor evidence	target or model is implausible
software review	schema, pure/config tests, registered task	invariant or clean sync fails
smoke review	finite five-iteration export/reload	integration failure
learning review	multi-seed curves and fixed validation	main skill absent or exploit dominates
assessment review	locked checkpoint and untouched battery	acceptance or severe-tail criterion fails
deployment review	parity, timing, safety/fault injection	contract/deadline/safe state fails

Grade evidence, not novelty:

```
20% problem/task specification and causal reasoning
15% physics/model measurement
15% tests and reproducible software
20% evaluation design and statistics
15% deployment/safety contract
10% diagnosis of limitations and alternatives
 5% clarity of handoff
```

A negative result with a well-falsified hypothesis, preserved artifacts, and a clear next experiment can score higher than a cherry-picked apparent success.

Where to continue learning

After this book, read code and experiments in this order:

1. the velocity task and its focused tests;
2. sit-stand for command-conditioned state transitions;
3. stand-up for reset distributions and recovery shaping;
4. roulade for dynamic-task reward lessons;
5. backlash for observation/physics consistency; and

6. roller tasks for passive joints and architecture-specific indices.

Return to the book index whenever a new experiment crosses from task design into training or deployment.

Folded reference outcomes

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show reference outcomes for Labs 0–4

Lab 0 — reproduction. A complete result binds a pinned source/lock and resolved configuration to the checkpoint, exported graph, frozen observation corpus, parity output, rehearsal log, and video by content hashes. It explains that the environment's sampled command changes intention while the deterministic policy conditionally responds; it does not call the learned action random. CPU tests and smoke training answer different claims and both are retained.

Lab 1 — command mixture. A strong analysis first proves the resolved probability changed and other task configuration did not. It logs actual command episode counts, pairs baseline/treatment seeds, and reports turn plus forward/idle slices with uncertainty and severe failures. A result that improves turn success but violates the predeclared forward non-inferiority margin is a tradeoff, not an unqualified improvement. Exact-zero training remains explicit.

Lab 2 — reward helper. The pure tensor tests establish zero at target, symmetry, monotonic magnitude, nonfinite handling policy, and returned sign. The configuration test establishes a positive weight for the self-negating helper and correct trunk/terrain selectors. The run audit shows every weighted penalty nonpositive and reports its observed mass. The write-up checks prone, supine, and side-pose counterexamples and rejects the term if matching height rewards a stable flop or blocks required motion.

A pure reward helper and wiring test can follow this pattern:

```
def limit_proximity_cost(q, lower, upper, margin):
    distance = minimum(q - lower, upper - q)
    return maximum(0.0, margin - distance) / margin

def test_limit_cost_is_zero_away_from_limits():
    assert limit_proximity_cost(0.0, -1.0, 1.0, 0.1) == 0.0

def test_limit_cost_grows_near_upper_limit():
    assert limit_proximity_cost(0.95, -1.0, 1.0, 0.1) > 0.0
```

Adapt this to Torch and repository conventions. The helper returns a nonnegative cost, so the configured reward weight should be negative. This contrasts deliberately with the self-negating height helper.

Lab 3 — new task. The submission contains the pre-code worksheet, measured supportability of target states, the closest-template rationale, registered train/play configuration, 61D/14D or explicitly versioned contract, selectors resolved on the actual model, sign/gate tests, and five-iteration export/reload. Evaluation includes intended success, every plausible stable flop, reset classes, and a named exploit taxonomy. A novel task with no deployment owner or handoff state remains incomplete.

Lab 4 — curriculum. Boundary tests show stage selection immediately before and at each step, using the initialized manager rather than a stale config copy. Logs contain active range/weight and empirical sample distribution. Skill metrics are aligned to boundaries. If collapse repeats at a boundary, the outcome is "schedule outpaces competence"; delaying/staggering one stage is the next controlled test. A competence-triggered variant persists hysteresis, patience, and stage state in checkpoint provenance.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show reference outcomes for Labs 5–9

Lab 5 — obstacles. Option A retains the 61D motor-policy contract and demonstrates timestamped perception, an explicit invalid-data state, commands inside the trained envelope, measured stopping-distance assumptions, local expiry/stop behavior, and held-out-layout metrics. Option B versions the schema, documents every ray/image feature and validity mask, matches inference latency, trains pre-

contact information with progress/clearance/collision objectives, and tests unseen geometries plus sensor dropout. In either option, rendering a box without a causal observation and objective is rejected. Cloud loss must not remove local stop authority.

Lab 6 — identification. Raw timestamped bench traces, safe excitation, calibration, and load/battery/temperature conditions are preserved. Candidate parameters are fit on complete trajectories and tested on different command families. The report shows position/velocity/current residuals over time, not only one loss. Correlated or unidentifiable parameters are labeled; the next excitation is designed to separate them. Randomization centers on validated nominal behavior and covers measured residual/repeatability rather than an arbitrary wide interval.

Lab 7 — evaluator. A locked case manifest and tested success classifier produce raw JSONL rows, a reproducible summary, per-slice counts/intervals, training-seed identity, failure severity, and synchronized sample videos. A nonfinite simulator case is classified separately from a policy fall. The summary is regenerated from raw rows by one command, and the final assessment manifest was not used for tuning.

Lab 8 — timing. The trace measures capture-to-actuation age and every intermediate stage under representative load. It reports distribution tails, deadline misses, bus trace versus analytic budget, data drops/overwrites, and the exact response to injected stalls/staleness/disconnects. A latest-value mailbox keeps feedback age bounded, while a separate lossless/logging path preserves events. The local safe response works with cloud and brain process unavailable.

Lab 9 — handoff. The finite-state machine rejects incompatible bundles and requests outside the handoff set, atomically resets command semantics/history, and transitions only after measurable guards pass. Success and fallback paths are exercised. Logs show target discontinuity, saturation, state/command age, transition time, contacts, and final state. Any action blend is included in the trained or explicitly validated transition distribution.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show the capstone completion rubric

A passing capstone lets an independent learner reproduce the software pipeline from a clean checkout, rerun a bounded evaluation, inspect raw evidence, and state exactly what was *not* demonstrated. Its chain is:

```
measured/declared assumptions
-> executable task and invariant tests
-> finite smoke/export evidence
-> multi-seed learning and locked checkpoint selection
-> untouched per-slice assessment with uncertainty/failures
-> one-step export parity and deployment rehearsal
-> timing/safety/fault-injection record
-> operating envelope, rollback, and next falsifiable hypothesis
```

Automatic failure conditions include an unrecoverable artifact, absent raw evaluation rows, an observation/action schema mismatch, positive weighted penalty due to sign error, capability claim without causal observation/reward, unsafe severe failure hidden by an average, or physical testing before the declared interface/safety gates pass.

A high-quality negative capstone might show that an obstacle planner cannot stop within range once measured perception age and wet-surface deceleration are included. Rejecting deployment, documenting the boundary, and proposing a lower speed or better sensor is successful engineering and successful learning.

14. Demonstrations, Imitation, and Offline Robot Learning

Much modern robot learning does not begin with random exploration and a reward. It begins with demonstrations, logs, or a pretrained policy. This chapter shows how those settings relate to—and differ from—reinforcement learning.

The family has several distinct questions:

Setting	Data	Learns from	Main danger
supervised imitation	(o, a) demonstrations	action labels	covariate shift/multimodality
inverse reward learning	expert trajectories	inferred objective	reward ambiguity
offline reinforcement learning	fixed (s, a, r, s') data	reward and bootstrapping	out-of-support value error
offline-to-online	prior data plus new interaction	both	unsafe/unstable distribution transition
sequence policy pretraining	multi-task trajectory corpora	conditional prediction	data semantics and deployment mismatch

Historically, behavior cloning imported supervised pattern recognition into control; inverse reinforcement learning asked which reward could explain an expert; Dataset Aggregation (DAgger) addressed learner-induced state shift; adversarial imitation matched occupancy distributions; offline reinforcement learning (RL) added pessimism or in-data constraints to value learning; and modern generative/transformer policies model multimodal action sequences. These are complementary tools, not a single ladder on which the newest name always wins.

14.1 A transition dataset is an interface

A robot-learning dataset may contain

$$D = \{(o_t, a_t, r_t, o_{t+1}, d_t, m_t)\},$$

where m_t is optional metadata such as task language, camera calibration, timestamps, episode identifier (ID), or robot embodiment.

Before selecting an algorithm, audit:

- exact observation fields, units, frames, and timestamps;
- action semantics: torque, target, delta, end-effector pose, or action chunk;
- who/what generated each action;
- whether failed episodes are present;
- whether reward is measured, inferred, sparse, or absent;
- termination versus time-limit truncation;
- task and environment coverage;
- sensor/action rate and missing samples; and
- train/validation/test split by episode, scene, object, and operator.

A large dataset with inconsistent action semantics can be less useful than a small coherent one.

14.1.1 Time alignment creates the supervised label

Suppose an image was exposed at t^I, joint state sampled at t^q, the teleoperator chose a command using information available near t^H, and the actuator applied it at t^a. Writing them in one row does not make them simultaneous. The learning example is really

$$(o(t^I, t^q, \ldots), a(t^a), \Delta t, \text{ages}).$$

If image latency is 100 ms and control runs at 20 hertz, a naive nearest-row join can pair an image with an action two control steps into its future. The policy then appears accurate offline but cannot reproduce that acausal mapping online.

Use source and receive timestamps, document clock synchronization, and align by the information the demonstrator actually had. Preserve raw timing so a later learner can test alternative alignment. A dataset created only after resampling to a neat fixed grid may erase the evidence of delays and drops.

Action semantics require the full control path. For a target position log,

$$a_t = q_t^{target}$$

differs from a delta action

$$a_t = q_t^{target} - q_t,$$

and both differ from the policy-normalized offset $a_t = (q_t^{target} - q^{HOME})/s$. A learner trained on one and deployed as another can remain numerically finite while commanding the wrong magnitude.

14.1.2 Split by causal unit, not shuffled frames

Neighboring frames from one trajectory share scene, object, operator, and history. Random frame splitting puts nearly identical observations in train and test, producing leakage. Split complete episodes first. To test stronger generalization, hold out entire objects, scenes, operators, robot revisions, or collection days. State the unit because each split answers a different question.

Dataset size has several meanings:

```
frames/transitions, control hours, episodes, tasks,
unique scenes/objects/operators, robots/embodiments
```

Millions of adjacent frames may contain fewer independent decisions than thousands of diverse episodes. Report all relevant axes.

14.2 Behavior cloning

Behavior cloning (BC) treats demonstration actions as labels. For a continuous action and deterministic policy, a simple objective is

$$\min_\theta \ \mathbb{E}_{(o,a)\sim D}\left[\|\pi_\theta(o) - a\|_2^2\right].$$

Plain language: make the policy predict the demonstrator's action for each recorded observation.

The more general maximum-likelihood objective is

$$\min_\theta -\mathbb{E}_{(o,a)\sim D}[\log \pi_\theta(a \mid o)].$$

If π_θ is a Gaussian with fixed isotropic variance $\sigma^2 I$ and mean $\mu_\theta(o)$, then

$$-\log \pi_\theta(a \mid o) = \frac{1}{2\sigma^2} \|a - \mu_\theta(o)\|_2^2 + C.$$

Thus mean-squared error is not arbitrary: it is maximum likelihood under a single Gaussian noise model. Learning per-action variance, a mixture, discrete codes, or diffusion changes the assumed conditional distribution.

Action normalization matters. If one gripper dimension spans 0–1 while an arm joint spans only 0.02 radians, raw squared error gives the gripper far more numerical weight. Standardize by a documented training-set statistic or choose physical loss weights:

$$L_{BC} = \frac{1}{d_a} \sum_j \left(\frac{a_j - \hat{a}_j}{s_j} \right)^2.$$

Do not compute normalization statistics from test episodes. Save them with the policy, just as Microduck saves observation normalization.

The core training loop maps directly to the equation:

```
for observation, expert_action in loader:
    predicted_action = policy(observation)
    normalized_error = (predicted_action - expert_action) / action_scale
    loss = normalized_error.square().mean()
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Training loss measures prediction on demonstrated states. Closed-loop rollout measures whether those predictions keep the robot near such states; both are needed.

BC is supervised learning, not reinforcement learning. It does not need reward, next-state values, or online interaction. It is often a strong baseline because it avoids reward design and difficult exploration.

Why mean-squared error can average incompatible actions

Suppose a demonstrator passes an obstacle equally often on the left and right. At the decision point the action distribution has two modes. A deterministic mean-squared-error policy may average them and drive straight toward the obstacle.

For squared loss, the optimal deterministic predictor is the conditional mean:

$$f^*(o) = \mathbb{E}[a \mid o].$$

To see why, write $a = \mathbb{E}[a \mid o] + \epsilon$ with $\mathbb{E}[\epsilon \mid o] = 0$; expanding $\mathbb{E}[\|a - f(o)\|^2 \mid o]$ leaves a constant noise term plus $\|\mathbb{E}[a \mid o] - f(o)\|^2$. For equally likely scalar actions −1 and +1, the optimum is zero even if zero never appeared in data.

Responses include:

- provide more context so the modes become distinguishable;
- predict a multimodal distribution;
- use a mixture model or discrete latent choice;
- predict an action sequence/chunk; or
- use a generative policy such as diffusion.

A mixture-density policy represents

$$\pi(a \mid o) = \sum_{k=1}^{K} \rho_k(o) \mathcal{N}(a; \mu_k(o), \Sigma_k(o)).$$

It can sample a coherent left or right mode, but component collapse and unstable switching remain possible. A latent choice should persist long enough to finish the maneuver; action chunks or a recurrent/skill-level latent can provide that temporal consistency.

14.3 Covariate shift and compounding errors

Training observations come from the demonstrator distribution $d_{expert}(o)$. Deployment observations come from the learned policy $d_\pi(o)$. A small prediction error changes the robot state; the next observation may never occur in demonstrations; another error follows.

This is **covariate shift**. In sequential control, errors can compound over time.

In a simple worst-case argument, let the policy have error probability ϵ on expert-distribution states over horizon T. By step t, the chance that at least one earlier mistake occurred is at most about $t\epsilon$ by a union bound. Summing exposure to off-expert states across the horizon gives order

$$\sum_{t=1}^{T} t\epsilon = \frac{T(T+1)}{2}\epsilon = O(T^2\epsilon).$$

This is not a precise prediction for every robot; it explains why a small one-step validation error can coexist with poor long-horizon control.

The Dataset Aggregation (DAgger) idea alternates:

1. run the current learner;
2. ask an expert for the correct action on states the learner visits;
3. aggregate those labeled states into the dataset; and
4. retrain.

The primary DAgger paper analyzes this interactive reduction. On hardware, expert intervention and safe rollout collection must be engineered; a human may not label a 1 kHz recovery action.

Because the aggregated dataset includes learner-visited states, the analysis can reduce the horizon scaling toward $O(T\epsilon)$ under its assumptions. The price is interaction and expert labels exactly where the learner may be unsafe. Practical variants use intervention: a human or safety controller takes over, and the pre-intervention/recovery segment becomes corrective data. This changes the data distribution and must be logged; intervention-filtered datasets are not ordinary expert demonstrations.

Run the dependency-free illustration:

```
python examples/behavior_cloning_shift.py
```

It shows both an invalid conditional mean for two action modes and quadratic growth of a simple uncompensated drift cost. The toy model is not a robot benchmark; it makes the failure mechanisms inspectable.

14.4 Inverse rewards and occupancy matching

Sometimes actions are available but the transferable object of interest is the objective. **Inverse reinforcement learning** asks for a reward under which the expert appears (near-)optimal. The problem is inherently ambiguous: many rewards induce the same policy, including positive rescalings and some shaping transformations. A recovered reward is an explanatory model under assumptions, not the expert's uniquely true intent.

Maximum-entropy inverse learning assigns expert-like trajectories probability roughly

$$p_\theta(\tau) \propto \exp\left(\sum_t r_\theta(s_t, a_t)\right),$$

balancing reward fit with distributional uncertainty. It requires dynamics or sampling machinery and careful feature/identifiability assumptions.

Generative Adversarial Imitation Learning (GAIL) instead trains a discriminator to distinguish expert from policy occupancy and updates the policy to confuse it. One convention is

$$\min_{\pi} \max_{D} \mathbb{E}_{(s,a)\sim\rho_E}[\log D(s,a)] + \mathbb{E}_{(s,a)\sim\rho_\pi}[\log(1 - D(s,a))] - \lambda\mathscr{H}(\pi),$$

where ρ_E and ρ_π are discounted state–action occupancy measures and $\mathscr{H}$ encourages policy entropy. Matching occupancy addresses more than one-step action regression, but GAIL normally needs fresh policy rollouts in a simulator/environment and can inherit adversarial-training instability. It is imitation from demonstrations, not necessarily *offline* learning.

Use inverse/adversarial methods when reward discovery or distribution matching is genuinely required. For a well-covered task with actions, BC is a simpler baseline; for a real robot without safe interaction, a rollout-hungry method may be unsuitable regardless of benchmark performance.

14.5 Action chunking

Instead of predicting one action, an **action-chunk** policy predicts a short sequence:

$$\pi(o_t) \to (a_t, a_{t+1}, \dots, a_{t+H-1}).$$

Benefits can include temporal consistency and fewer high-level inference calls. But open-loop execution for the entire chunk delays correction. Practical systems often use receding horizon: predict a chunk, execute a small prefix, observe again, and replan.

Let prediction horizon be H_p, executed prefix H_e, and control period Δt. Maximum open-loop time is $H_e\Delta t$. Larger H_p gives the model temporal structure, while smaller H_e preserves feedback. These are separate knobs; a policy can predict 100 steps but execute only one or several.

When a new chunk is predicted every step, several earlier chunks contain a proposal for current action a_t. Temporal ensembling averages them with recency weights:

$$\bar{a}_t = \frac{\sum_{i=0}^{K-1} w_i \hat{a}_i^{(t-i)}}{\sum_{i=0}^{K-1} w_i}, \qquad w_i = \exp(-ki).$$

This can smooth inconsistent predictions but adds a stateful filter and delay. Training, evaluation, and runtime must use the same rule. At a contact transition, averaging an old “continue closing” proposal with a new “stop” proposal may be unsafe, so inspect chunk age and discontinuities rather than assuming smooth is always better.

The Action Chunking with Transformers (ACT) paper uses action chunking with a transformer-style conditional variational autoencoder for low-cost bimanual manipulation. Its demonstration-based success should not be generalized to fast balance loops without latency and disturbance tests.

A conditional variational autoencoder (CVAE) uses an encoder distribution $q_\phi(z \mid o, a_{t:t+H})$, a decoder $p_\theta(a_{t:t+H} \mid o, z)$, and a prior $p(z)$. A common training objective minimizes reconstruction plus regularization:

$$L_{CVAE} = -\mathbb{E}_{z\sim q_\phi}[\log p_\theta(a_{t:t+H} \mid o, z)] + \beta D_{KL}(q_\phi(z \mid o, a) \,\|\, p(z)).$$

The Kullback–Leibler (KL) term makes latent samples usable from the prior at inference. Too much regularization can make the decoder ignore z; too little can leave a latent distribution that does not match inference. ACT’s exact architecture and temporal ensembling should be read in its official code, not reduced to “transformers solve manipulation.”

14.6 Diffusion Policy

A diffusion model learns to turn noise into a structured sample through iterative denoising. Diffusion Policy conditions that process on robot observations to generate action sequences.

Conceptually:

```
random action sequence
       |
       v
denoise using image/state condition
       |
       v
more coherent action sequence
       |
       v
execute first action(s), observe, repeat
```

Why this can help:

- several valid action modes can be represented rather than averaged;
- an action horizon captures temporal structure;
- image-conditioned manipulation has genuinely multimodal choices.

Costs include several denoising evaluations, runtime latency, sensitivity to data coverage, and no automatic recovery outside demonstrations.

Let x_0 denote a demonstrated action chunk. A forward diffusion process adds Gaussian noise at level k:

$$x_k = \sqrt{\bar{\alpha}_k} x_0 + \sqrt{1 - \bar{\alpha}_k}\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

A conditional network learns to predict the injected noise from noisy action, noise level, and observation context c:

$$L_{diff} = \mathbb{E}_{x_0,k,\epsilon}\left[\|\epsilon - epsilon_\theta(x_k, k, c)\|_2^2\right].$$

At inference, begin from Gaussian x_K and apply a scheduler's reverse steps until a structured x_0 remains. The scheduler, number of steps, chunk horizon, normalization, and executed prefix are part of the policy—not interchangeable implementation details.

Why can diffusion represent two routes? Training does not force one deterministic output to be the conditional mean; different initial noise can denoise toward different high-density action sequences. But mode diversity is useful only when samples remain temporally coherent and safe. Evaluate repeated samples from the *same* observation and examine mode frequency, collision, and latency.

The primary Diffusion Policy paper reports evaluation across 12 tasks in four manipulation benchmarks and public code/data. Those results support the method in that protocol; they do not imply diffusion is required for all robots.

Important alternatives illustrate the design space:

- Vector-Quantized Behavior Transformer (VQ-BeT) learns discrete latent action codes and a transformer, targeting multimodality with fewer iterative inference steps;
- 3D Diffusion Policy (DP3) conditions a diffusion policy on compact point-cloud features for spatial generalization;
- BAKU studies modular observation trunks, transformer fusion, chunks, and several action heads for multi-task policies; and
- deterministic/recurrent BC remains the latency and implementation baseline.

These papers report different tasks, data counts, robots, rates, and protocols. Their headline success rates are not a common leaderboard. Reproduce a simple BC/chunk baseline on your dataset before attributing gains to a generative head.

14.7 Offline reinforcement learning: improvement without new interaction

Offline RL uses a fixed dataset but has rewards and sequential transitions. It tries to find a policy better than the data-collection behavior while avoiding unsupported actions.

This creates a tension:

```
stay close to data -> reliable estimates but limited improvement
move beyond data   -> possible improvement but uncertain values
```

Ordinary off-policy algorithms can overestimate an unseen action, then train the actor to select it. Because no online rollout corrects the mistake, the error can reinforce itself.

Let dataset behavior be $\beta(a \mid s)$ and candidate policy be $\pi(a \mid s)$. A standard critic target samples/evaluates the next policy action:

$$y = r + \gamma Q_{target}(s', a'), \qquad a' \sim \pi(\cdot \mid s').$$

If $\beta(a' \mid s') \approx 0$, the dataset provides little evidence for that value. Function approximation may extrapolate a high number; the actor then prefers it; bootstrapping propagates it backward. This feedback loop is the offline form of extrapolation error.

One coverage diagnostic is a state–action occupancy ratio

$$C = \sup_{s,a} \frac{d_\pi(s,a)}{d_\beta(s,a)}.$$

If the denominator is zero where the learned policy goes, C is unbounded and no finite-data guarantee based on behavior coverage can help. In continuous high-dimensional robotics, estimating this ratio is itself hard, so practical algorithms use proximity, uncertainty, pessimism, or in-sample backups as proxies.

Offline data also cannot identify counterfactual dynamics from one action at a state. Seeing an expert close a gripper does not reveal what would have happened under a high-speed sideways motion. Generalization comes from model inductive bias and nearby coverage, not information magically added by an offline objective.

14.8 Conservative Q-Learning

Conservative Q-Learning (CQL) adds pressure for values of actions outside the dataset to be lower than values of observed actions. One conceptual form is

$$\min_Q \underbrace{L_{Bellman}(Q)}_{\text{fit transitions}} + \alpha \left(\underbrace{\mathbb{E}_{s,a\sim\mu}[Q(s,a)]}_{\text{candidate actions}} - \underbrace{\mathbb{E}_{s,a\sim D}[Q(s,a)]}_{\text{dataset actions}} \right).$$

μ is an action proposal distribution. The added term discourages the critic from assigning unjustified high value broadly.

The CQL paper provides theoretical and benchmark evidence for conservative value estimation. Conservative does not mean physically safe; it describes value estimation relative to data support.

For a finite action set, a common conservative gap resembles

$$L_{CQL}(Q) = \alpha \mathbb{E}_{s\sim D} \left[\log \sum_a \exp Q(s,a) - \mathbb{E}_{a\sim D(\cdot|s)} Q(s,a) \right].$$

The log-sum-exp is a smooth maximum over candidate actions. Minimization pushes down broadly high values while the dataset-action term pushes observed actions up relative to them. Continuous-action

implementations approximate the first term using sampled actions and importance corrections; reading those sampling details is essential for mapping paper to code.

Pseudocode:

```
bellman = mse(q_data, reward + gamma * target_q_next)
q_candidates = q(states_repeated, sampled_candidate_actions)
conservative_gap = logmeanexp(q_candidates) - q_data.mean()
critic_loss = bellman + alpha * conservative_gap
```

Large α can become too pessimistic and prevent improvement; small α may leave unsupported optimism. Some variants tune a Lagrange multiplier toward a target gap. Treat it as a dataset/task-sensitive control, not a universal safety coefficient.

14.9 Implicit Q-Learning

Implicit Q-Learning (IQL) avoids evaluating the Q-function at policy actions outside the dataset during its main value-learning stage.

Its key stages are:

1. fit a state value using **expectile regression** toward the upper portion of dataset action values;
2. fit Q with a Bellman target using that state value; and
3. extract a policy with advantage-weighted behavior cloning.

An expectile parameter $\tau > 0.5$ weights positive residuals more heavily, making $V(s)$ reflect better actions present in the dataset without taking an explicit max over unseen actions.

For residual $u = Q(s, a) - V(s)$, asymmetric squared loss is

$$L_V = \mathbb{E}_{(s,a)\sim D}\left[|\tau - \mathbf{1}[u < 0]|u^2\right].$$

If $\tau = 0.7$, positive residuals ($Q > V$) receive weight 0.7 while negative residuals receive 0.3, pulling V toward the upper part of values among *dataset actions*. It is an expectile, not a percentile/quantile: squared residual magnitude matters.

The critic then uses

$$L_Q = \mathbb{E}_D\left[Q(s, a) - (r + \gamma(1 - d)V(s'))\right]^2.$$

Policy extraction weights demonstrated actions approximately by

$$w(s, a) = \exp(\beta(Q(s, a) - V(s))),$$

usually with clipping. Better-than-baseline dataset actions receive more weight.

The actor loss maps directly to weighted likelihood:

```
advantage = q_data.detach() - value.detach()
weight = exp(beta * advantage).clamp(max=max_weight)
actor_loss = -(weight * policy.log_prob(dataset_action)).mean()
```

`detach` expresses the algorithmic intent that this actor update not change the critic through the weight. Clipping prevents a few estimated advantages from dominating. Increasing β makes extraction greedier and more sensitive to critic error.

The IQL paper reports strong Datasets for Deep Data-Driven Reinforcement Learning (D4RL) results and online fine-tuning. A beginner should retain the design principle: improve toward the best supported data without trusting arbitrary unseen actions.

The official IQL implementation is compact enough to trace from `value_net.py`, critic updates, and advantage-weighted policy code back to these equations.

14.10 Other offline and offline-to-online lenses

CQL and IQL are influential, not exhaustive. Several alternatives clarify what an algorithm assumes.

Behavior-regularized actor–critic

Twin Delayed Deep Deterministic Policy Gradient (TD3) plus behavior cloning, usually written TD3+BC, uses an offline critic and an actor objective schematically like

$$\max_{\theta} \mathbb{E}_{s\sim D}[\lambda Q(s, \pi_\theta(s))] - \mathbb{E}_{(s,a)\sim D}[\|\pi_\theta(s) - a\|_2^2].$$

The Q term seeks improvement; cloning keeps actions near data. Its importance is methodological: a carefully normalized simple baseline can rival elaborate methods, so algorithm comparisons must control implementation details.

Return-conditioned sequence modeling

Decision Transformer represents a trajectory as desired return-to-go, states, and actions, then autoregressively predicts actions:

```
(return-to-go, state, action, return-to-go, state, action, ...)
```

At step t, return-to-go updates as $R_{t+1} = R_t - r_t$ in the undiscounted form. The model is trained by sequence likelihood rather than an explicit Bellman backup. It can exploit long context and transformer scaling, but asking for a return never represented by compatible trajectories does not create the missing behavior. Return scale, context length, inference caching, action semantics, and dataset quality remain critical. The official code is a useful reference for the token/interleaving pipeline.

Offline pretraining followed by controlled interaction

Pure offline conservatism can make a poor starting point for online improvement if Q-values have an unsuitable scale. Calibrated Q-Learning (Cal-QL) studies conservative but calibrated pretraining for fine-tuning. Reinforcement Learning with Prior Data (RLPD) studies how an online off-policy learner can mix prior and new data efficiently; its official repository exposes the replay and update design.

This phase change is risky on a robot:

```
offline policy determines initial visited states
-> new data distribution enters replay
-> critic scale/uncertainty changes
-> actor may move beyond demonstrator support
```

Use bounded authority, human/local safety intervention, a rollback policy, separate offline/online replay metrics, and staged interaction budgets. “Uses offline data” does not make later exploration safe.

Selection guide

Method	Main inductive bias	Attractive when	Audit first
BC/recurrent BC	imitate conditional action	demonstrations strong	rollout shift, multimodality
chunk/generative BC	model temporal/multiple modes	action choices are structured	latency, mode safety
TD3+BC	improve while cloning	rewards exist; simple baseline desired	Q extrapolation and BC scale
CQL	pessimistic values	broad mixed-quality data	conservative penalty/sampling
IQL	select good in-data actions	useful actions already in data	expectile/weight sensitivity
Decision Transformer	conditional sequence modeling	long context and varied returns	unsupported return commands

Method	Main inductive bias	Attractive when	Audit first
RLPD/Cal-QL	prior data then interaction	safe online budget exists	transition and safety envelope

14.11 Dataset coverage is the real boundary

Imagine a walking dataset containing only forward motion on high-friction floor. No offline objective can identify the correct recovery action for an unseen sideways slip purely from that data unless learned structure generalizes correctly. The dataset does not contain counterfactual evidence.

Audit coverage along dimensions that matter physically:

- commands and task goals;
- initial poses and failure/recovery states;
- surfaces, payloads, voltage, and temperature;
- sensor noise/dropout and latency;
- successful and unsuccessful behavior;
- humans/operators and camera viewpoints; and
- action saturation and safety boundaries.

Build a coverage cube rather than one histogram. Example rows are command buckets, columns are surface classes, and layers are reset/failure classes. Store count, duration, success/failure, action extrema, and sensor validity per cell. Empty cells are explicit unsupported claims.

For continuous observations, nearest-neighbor distance or learned density can flag novelty, but neither proves safety. High-dimensional image distances may reflect lighting rather than control relevance; a state can look familiar while an unseen action is requested. Combine representation-based diagnostics with semantic slices and closed-loop held-out evaluation.

Dataset quality is not identical to expert success rate:

- proficient demonstrations provide clean targets;
- varied suboptimal data can reveal alternatives and reward ranking;
- failures identify boundaries and recovery, if safely collected/labeled;
- interventions show where autonomy became unsafe, but censor what would have happened afterward; and
- duplicate or temporally dense data may overweight one operator/style.

For Microduck, a viewer log becomes useful offline data only if it records the raw 61D observation before embedded normalization, 14D action and applied target, command, reward/termination reconstruction, next observation, timestamps/delays, randomization parameters, contacts, task/config/artifact identity, and failure labels. A compressed video plus joint positions is not a transition dataset.

14.12 D4RL and benchmark literacy

D4RL introduced standardized offline-RL datasets with behavior mixtures and evaluation protocols. A normalized score commonly has the form

$$100\frac{J_\pi - J_{random}}{J_{expert} - J_{random}}.$$

This makes scores more comparable within a benchmark, but the reference policies, environment version, termination handling, and dataset composition remain part of the result. A score of 100 is not “100% physically safe” or “solved for every robot.”

Normalization is affine. If $J_{expert} - J_{random}$ is small or reference values change across versions, the normalized scale becomes unstable or incomparable. Scores above 100 and below 0 are possible. Report raw return, normalized score, environment/dataset version, evaluation episodes, seed-level values, and uncertainty.

D4RL deliberately includes expert, medium-quality, replay, and mixed datasets, plus navigation/manipulation domains. That variety tests different coverage failures. An algorithm ranking on one dataset type need not carry to another, and a simulated benchmark result does not establish hardware robustness.

Offline evaluation has a special model-selection problem: training loss and estimated Q can improve while the actual policy worsens. Benchmarks permit online simulator rollouts for validation, but a real deployment may not. Reserve safe validation trials, use behavior/value diagnostics cautiously, and avoid selecting hyperparameters on the final test environment.

14.13 Open robot-data ecosystems

Hugging Face LeRobot provides an open-source ecosystem for robot datasets, policies, and real-hardware tools. Its value for study is the full data-to-policy workflow and common dataset format—not an assumption that every supported policy is RL.

`robomimic` is a focused framework for learning from robot demonstrations, with reproducible datasets, BC/recurrent/transformer/generative and offline-RL baselines. Its versioned dataset notes are also a lesson: simulator/binding changes can prevent exact reproduction even when a dataset name looks unchanged.

Distributed Robot Interaction Dataset (DROID) reported 76,000 demonstration trajectories (350 hours) across 564 scenes and 84 tasks, collected by 50 people, and released data, policy-learning code, and hardware documentation. This scale supports studies of scene diversity; it does not eliminate action/calibration or task-distribution boundaries.

Open X-Embodiment standardized data across 22 robot embodiments and studied Robotics Transformer X (RT-X) cross-robot models. Cross-embodiment pooling creates a harder interface problem:

```
different cameras/frames/rates/action spaces/grippers
  -> embodiment-specific decoding and normalization
  -> common semantic/task representation
  -> per-robot safety and evaluation
```

Data volume is valuable only after these meanings are aligned. A universal tensor field named `action` can still mix Cartesian deltas, absolute poses, and joint targets.

Generalist manipulation projects such as Octo and OpenVLA train on large collections of robot demonstrations. They belong primarily to pretrained imitation/foundation-policy research. Chapter 16 studies their architecture and safety boundary.

As of this book's September 2026 review, no single public dataset, format, or policy family is a universal standard for locomotion, manipulation, mobile navigation, and every embodiment. The durable skills are schema inspection, split design, baseline reproduction, licensing/provenance, and deployment contract testing. Treat ecosystem version and paper date as part of every claim.

14.14 Choosing among BC, offline RL, and online RL

Start with behavior cloning when:

- demonstrations are strong and cover deployment;
- reward is missing or unreliable;
- a simple, auditable baseline is needed.

Consider offline RL when:

- transitions and meaningful rewards exist;
- the dataset contains a range of behavior quality;
- policy improvement beyond average demonstration behavior matters;
- you can evaluate conservatively before hardware.

Consider online fine-tuning when:

- a safe simulator or controlled hardware protocol exists;
- deployment states differ from logged data;

- reward is reliable; and
- rollback and exploration limits are enforced.

A common progression is

```
demonstrations -> behavior cloning -> offline RL (optional)
               -> safe simulation/real fine-tuning -> frozen deployment
```

Each arrow needs its own evaluation gate.

14.15 A practical dataset card

Record at least:

```
robot: exact hardware revision
task: operational success definition
episodes: total / success / failure
observation_schema: fields, shapes, units, frames
action_schema: meaning, range, rate, delay
sensors: models, calibration, timestamps
collection_policy: human / scripted / learned, versions
environment_distribution: objects, surfaces, lighting, payload
safety_interventions: what was filtered or stopped
splits: episode/scene/object/operator separation
known_gaps: unsupported conditions
license_and_consent: provenance and permitted use
```

Without provenance, “more data” can mean more untraceable error.

Automate dataset invariants before training:

```
for episode in dataset:
    assert episode.observation.shape[1:] == observation_schema.shape
    assert episode.action.shape[1:] == action_schema.shape
    assert len(episode.next_observation) == len(episode.action)
    assert timestamps_are_monotonic(episode)
    assert terminal_and_truncation_are_consistent(episode)
    assert finite_or_explicitly_masked(episode)
    assert action_within_recorded_interface(episode)
```

Also compute per-field min/quantiles/max, missingness, sample intervals, action saturation, episode length, task/operator counts, and near-duplicate hashes. Review images for privacy/consent obligations and strip secrets or bystanders according to the dataset license/governance plan. A public robot-data release is both a machine-learning artifact and a record of real environments/people.

Version raw and derived data separately. A derived reward, resized image, time-aligned action, or filtered episode is code-dependent transformation, so store source dataset digest, transformation commit/config, and output digest.

14.16 Exercises

1. Give an example where squared-error BC averages two valid actions into an invalid one. Derive the optimal scalar prediction for equally likely actions `-1` and `+1`.
2. An image arrives 100 ms after exposure in a 20 hertz policy. How many nominal control periods old is it? Why can joining it to the action in its receive row create future-information leakage?
3. Explain covariate shift using a robot that drifts 2 cm left per step. What is drift after 25 uncompensated steps, and why is the real failure potentially worse than this linear arithmetic?
4. Explain the $O(T^2\epsilon)$ worst-case BC intuition and what additional resource DAgger uses to improve state-distribution coverage.
5. Is GAIL offline because it starts from expert demonstrations? Explain using its occupancy objective.
6. For a 20 hertz chunk policy that executes 6 predicted actions before replanning, calculate open-loop time. Give one task where this is acceptable and one where it is dangerous.
7. In the diffusion equation, what does $\tilde{\alpha}_k \to 0$ do to x_k? Distinguish the training noise-prediction pass from inference.

8. Distinguish ordinary off-policy online RL from offline RL. Why does the same Bellman target become more dangerous offline?
9. Explain the sign of CQL's dataset-action term. What would happen if both candidate and dataset Q-values were pushed down equally without a relative term?
10. For IQL with $\tau = 0.8$, what weights apply to residuals $u > 0$ and $u < 0$? Why is this an expectile rather than a quantile?
11. A Decision Transformer dataset's best return is 50. Does conditioning on return 100 guarantee a twice-as-good policy? Why not?
12. Baseline and treatment each use 100,000 prior transitions, but treatment begins collecting online robot data. List five new safety/statistical facts the comparison must report.
13. Design train/validation/test splits for three tables, ten objects, and two human demonstrators. Which generalization question does each split answer?
14. A dataset contains only successful trials. What information about failure recovery and interventions is missing?
15. Build a coverage matrix for Microduck commands, surfaces, and reset classes. Which empty cell would invalidate a claim of general slip recovery?
16. Write a dataset card for a Microduck playback log. Which additional fields are needed before it could support offline learning?
17. Choose among deterministic BC, generative chunk BC, IQL, and offline-to- online RLPD for: (a) excellent demonstrations with no reward, (b) mixed reward-labeled logs, and (c) sparse demonstrations plus a safe simulator. Defend baselines and escalation gates.
18. Compare LeRobot, robomimic, DROID, and Open X-Embodiment as study resources. Do not rank them with one number; state the distinct question each enables.

Continue with modern robot locomotion, adaptation, and sim-to-real research.

14.17 Folded solutions

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show solutions to Exercises 1–9

1. Left/right obstacle passing gives actions $-1, +1$. Expected squared error is $L(x) = \frac{1}{2}(x+1)^2 + \frac{1}{2}(x-1)^2 = x^2 + 1$, so $dL/dx = 2x = 0$ at $x = 0$. Zero is the conditional mean and can be invalid even though both labels are safe modes. Add disambiguating context or a mode-preserving distribution/latent.
2. At 20 hertz, one period is 50 ms, so the image is two periods old. Joining by receive row pairs the old visual scene with an action selected from newer information. At deployment the policy cannot see that newer scene inside the old image, so offline accuracy contains acausal leakage.
3. Simple drift is $25(0.02) = 0.50$ m. The real error can be worse because the learner enters unseen states, changes contact/viewpoint, and may no longer make the same 2 cm error; dynamics can amplify it nonlinearly or cause a fall.
4. Error probability accumulates over earlier steps, giving at most roughly $t\epsilon$ probability of prior mistake at step t; summing gives $\epsilon T(T+1)/2$. DAgger queries an expert on learner-visited states and aggregates labels, paying for interactive rollout, annotation, and safety.
5. Usually no. GAIL must sample the current policy occupancy ρ_π to train its discriminator and policy. Unless those rollouts come from a fixed model or special offline reformulation, it interacts with an environment/simulator despite starting from expert trajectories.
6. Open-loop time is $6/20 = 0.30$ s. It may be acceptable for a slow free-space arm segment with local low-level protection; it is dangerous for biped balance, dynamic contact, or human-proximate motion where state changes within tens of milliseconds.
7. As $\bar{\alpha}_k \to 0$, the signal coefficient vanishes and x_k approaches Gaussian noise. Training samples a demonstrated chunk, noise level, and noise once and learns to predict that noise. Inference starts from noise and iterates multiple scheduler/model steps without knowing a demonstrated x_0.
8. Online off-policy learning reuses replay but can collect corrective data under its evolving policy. Offline RL never observes consequences of new actions. A target action outside behavior support can receive arbitrary extrapolated Q; bootstrapping and actor maximization then reinforce it without correction.

9. Minimizing `candidate log-sum-exp - dataset Q` pushes broad candidate values down *relative to observed actions*. If every value were lowered equally, action ranking/unsupported optimism could remain while Q scale simply shifted; the Bellman term and relative contrast establish useful values.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show solutions to Exercises 10–18

10. For $u > 0$, weight is $|0.8 - 0| = 0.8$; for $u < 0$, weight is $|0.8 - 1| = 0.2$. Squared magnitude enters the objective, so this is asymmetric least squares/expectile regression. Quantile regression uses asymmetric absolute (pinball) loss.
11. No. The model learned conditional correlations within dataset support. A return token of 100 may be out of distribution and is not an optimizer that constructs missing twice-as-good trajectories. It may ignore, extrapolate, or fail under the unsupported condition.
12. Report online interaction count/time and schedule, initial/offline policy, authority/safety/interventions, replay mixing and update-to-data ratio, failures/severity, environment resets/changes, seed design, rollback, and which evaluation data remained untouched. Five of these is the minimum; all are useful.
13. Hold out a complete table for scene transfer, selected objects across seen tables for object transfer, and one operator for style/operator transfer. A combined held-out table/object/operator test asks compositional transfer. Validation holds separate examples for selection; never shuffle frames from the same episode across splits.
14. Missing evidence includes warning states, unsuccessful actions, recoverability boundaries, corrective actions, intervention policy, censored post-intervention outcomes, and severity. Success-only BC may have no label for states caused by its own mistakes.
15. Make rows `{idle, forward, lateral, turn, combined}`, columns `{high-friction, low-friction, uneven}`, and layers `{nominal, biased pose, push/slip}`. If all low-friction push/slip cells are empty, the dataset cannot directly support a general low-friction slip- recovery claim even if nominal forward data are abundant.
16. Record robot revision, task/commit/config/checkpoint, raw 61D schema and normalizer identity, 14D action/applied target, HOME/scale/order, commands, timestamps/delays, next observations, rewards, terminal versus truncation, resets/randomization/contacts, episode and failure/intervention labels, split units, transforms, hashes, license, and known gaps. Video alone lacks the causal transition fields.
17. (a) Start deterministic/recurrent BC; escalate to generative chunks only if measured multimodality/temporal inconsistency warrants latency. (b) Start BC and a simple reward-aware baseline, then IQL if mixed quality contains better supported actions and offline evaluation exists. (c) Pretrain BC or value/policy from demonstrations, validate in simulator, then consider RLPD with bounded online budget and safety. In all cases the escalation gate is a named baseline failure, not fashion.
18. LeRobot teaches an integrated open dataset/policy/hardware workflow; robomimic supports controlled learning-from-demonstration/offline baselines and reproducibility studies; DROID supports in-the-wild scene/diversity and standardized collection questions on one broad platform family; Open X-Embodiment supports cross-robot semantic/action alignment and transfer. Dataset/task/robot/protocol differences make one scalar ranking invalid.

15. Modern Robot Locomotion, Adaptation, and Sim-to-Real Research

Recent locomotion systems combine ideas rather than relying on one magical algorithm: graphics processing unit (GPU)-parallel simulation, calibrated actuator models, domain randomization, privileged training information, histories or latent adaptation, curricula, and strict deployment interfaces.

This chapter is an annotated research pathway. Dates and evaluated scope are included because "state of the art" is meaningful only relative to a protocol.

15.1 The recurring locomotion architecture

Many systems have this shape:

```
terrain / navigation / operator
            |
      command or short plan
            |
  learned locomotion policy -(20200 Hz)
            |
 joint targets or torques
            |
 embedded motor control and safety (faster)
            |
        physical robot
```

The learned policy is usually a local motor skill. It does not necessarily choose a destination, recognize an object, build a map, or enforce hard current limits.

15.1.1 Locomotion is a hybrid dynamical system

During one fixed contact mode, rigid-body dynamics can be written

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q) = S^T\tau + J_c(q)^T\lambda_c,$$

where q is generalized position, M inertia, C velocity-dependent terms, g gravity, τ actuator torque, S selects actuated joints, and contact Jacobian J_c maps contact force λ_c into generalized force.

Touchdown and liftoff change constraints. An impact can reset velocity:

$$x^+ = \Delta(x^-),$$

with pre-impact state x^- and post-impact state x^+. Continuous flows plus discrete contact transitions make locomotion *hybrid*. A neural policy need not explicitly label every mode, but contacts, history, and physics still determine which transition is possible.

This explains several practical facts:

- a small action difference near touchdown can cause a large trajectory change;
- average one-step model error can hide wrong contact timing;
- a smooth action penalty cannot define a valid gait by itself; and
- evaluation must stratify contact failures, not only average velocity.

15.1.2 Classical models remain useful mental instruments

For slow static balance, the center-of-mass projection should remain inside the support polygon. Dynamic walking intentionally violates this static condition, so a stronger simplified model is the linear inverted pendulum. At constant center-of-mass height z_0, horizontal motion relative to support point p obeys

$$\ddot{x} = \omega_0^2(x - p), \qquad \omega_0 = \sqrt{g/z_0}.$$

Define the capture point

$$\xi = x + \frac{\dot{x}}{\omega_0}.$$

If a foot/support point can be placed at $p = \xi$ in the ideal model, divergent motion is arrested asymptotically. This is not an exact biped controller for Microduck—height changes, angular momentum, finite feet, torque limits, and impact matter—but it gives a physical diagnostic: a policy falling forward may lack reachable support ahead of its capture point.

Periodic gait stability can be studied with a Poincaré return map. Observe a state x_k at the same gait event each cycle, such as left touchdown:

$$x_{k+1} = P(x_k).$$

A periodic gait is fixed point $x^* = P(x^*)$. Locally, perturbations evolve as

$$\delta x_{k+1} \approx A_P \delta x_k, \qquad A_P = \left.\frac{\partial P}{\partial x}\right|_{x^*}.$$

If every relevant eigenvalue magnitude of A_P is below one, perturbations shrink cycle to cycle in the local model. Reinforcement learning (RL) does not make this theory obsolete; push recovery and touchdown-to-touchdown error are empirical ways to probe the same stability question.

15.1.3 What RL contributes

Classical models expose structure and constraints. RL can optimize a nonlinear, contact-rich feedback law over a broad state distribution without solving a trajectory optimization online. In practice the strongest systems combine:

```
measured mechanics and actuator loops
+ simulator/contact model
+ task/reward and curriculum
+ learned local feedback
+ model-based safety/navigation around it
```

The learning problem is not “replace control theory.” It is “which feedback mapping is hard to hand-design, and how will physical structure constrain, diagnose, and protect it?”

15.2 Milestone: actuator-aware sim-to-real locomotion

The 2019 Learning agile and dynamic motor skills for legged robots paper trained policies in simulation and transferred them to ANYmal. A key lesson is that actuator behavior and dynamics identification are first-class parts of the learning system.

What to study:

- how actuator behavior was represented;
- what observations/actions reached the deployed network;
- which parameters were randomized;
- how real tests were evaluated; and

- which skills used separate policies or objectives.

What not to conclude: every robot can copy ANYmal's parameter values or reward weights. Morphology, actuator, sensor, rate, and task differ.

Microduck's Better Actuator Models (BAM) XL330 model belongs to the same engineering tradition: train through a more realistic actuator interface instead of an unlimited ideal position source.

An ideal position source assumes q_{t+1} can follow a target regardless of load. A physical actuator is closer to

$$\tau_t = f(q_t, \dot{q}_t, q_t^{target}, V_t, T_t, \eta),$$

where voltage V_t, temperature T_t, and parameters η affect available torque, friction, saturation, and delay. The environment transition therefore depends on actuator internal/operating state. A policy trained on impossible torque can learn a gait no hardware controller can realize.

Actuator-aware transfer has two complementary parts:

1. identify a nominal f from safe measured excitations and held-out traces;
2. randomize remaining uncertainty and operating variation around it.

Randomizing a bad functional form cannot create missing hysteresis or thermal dynamics; fitting one trace cannot establish robustness. This nominal-plus- residual view is a recurring theme from the 2019 work through newer system- identification and residual-alignment methods.

15.3 Milestone: massively parallel simulation

The 2021 Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning work showed that thousands of GPU-parallel environments can greatly reduce wall-clock Proximal Policy Optimization (PPO) training time and introduced a game-inspired terrain curriculum. It accompanied the open `legged_gym` ecosystem.

The important distinction is:

$$\text{environment steps} \neq \text{wall-clock seconds}.$$

Parallelism produces more samples per second; it does not make each sample more informative. It changes optimization dynamics too: batch size, update count, and policy lag must be understood.

For N_e environments, rollout depth H, and policy rate f_p, one update contains

$$B = N_e H$$

transitions and

$$t_{aggregate} = \frac{N_e H}{f_p}$$

simulated robot-seconds, but only H/f_p seconds of temporal depth per world. Increasing N_e cannot reveal a 10-second consequence if every fragment is 0.48 seconds and bootstrapping/history cannot represent it.

At fixed total transition budget, changing N_e can change:

- batch size and stochastic-gradient variance;
- number of policy updates;
- diversity versus temporal correlation inside a batch;
- normalization statistics;
- policy lag for off-policy replay; and

- curriculum progress if keyed to iterations rather than total transitions.

Thus a throughput benchmark and a learning comparison are separate. Plot transitions per wall second versus environment count to find device saturation; plot performance versus transitions and wall time to study sample and compute efficiency.

Microduck's 4,096 environments × 24 steps is this pattern applied through MuJoCo Warp and the Robotic Systems Lab reinforcement learning (RSL-RL) library.

15.4 Open training stacks

Useful open projects occupy different layers:

Project	Main role	Study value
`rsl_rl`	robot-focused PPO and related training components	readable runner/algorithm/storage separation
`legged_gym`	Isaac Gym locomotion environments	influential vectorized locomotion recipe; pin compatible versions
Isaac Lab	Isaac Sim robot-learning framework	sensors, randomization, reinforcement learning (RL), and imitation workflows
`mjlab`	manager-based MuJoCo Warp robot learning	Microduck's environment layer
MuJoCo Playground	GPU-accelerated MuJoCo robot-learning suite	open locomotion, manipulation, vision, and sim-to-real examples
Genesis	general robotics physics/simulation platform	alternative parallel simulator; inspect released features, not roadmap claims

These are not drop-in equivalents. Compare physics, renderer, task application programming interface (API), algorithm integration, supported hardware, license, version compatibility, and reproducibility before migrating.

The 2025 MuJoCo Playground paper documents an open MuJoCo-based stack across locomotion and manipulation. Its breadth makes it a valuable comparison project for the Microduck/mjlab design.

15.5 Domain randomization: train a distribution, not one simulator

Let physical parameters be ξ, such as mass, friction, delay, or motor strength. Instead of optimizing one nominal environment, domain randomization optimizes

$$J(\theta) = \mathbb{E}_{\xi \sim p(\xi),\ \tau \sim \pi_\theta, \xi}\left[\sum_t \gamma^t r_t\right].$$

Plain language: sample a plausible robot/world, run the policy, and optimize average performance over that population.

Early primary demonstrations include visual domain randomization and dynamics randomization for control.

What to randomize

Randomize uncertainty that exists and affects behavior:

- link mass, inertia, and center of mass;
- ground friction/restitution/compliance;
- actuator gain, strength, friction, damping, delay, and voltage;
- encoder/inertial measurement unit (IMU) bias, noise, mounting error, and dropout;
- command delay and control-period jitter;

- initial state, terrain, disturbances, and payload.

Three failure modes

1. **Wrong center**: randomization surrounds an inaccurate nominal model.
2. **Too narrow**: the real robot lies outside the training distribution.
3. **Too broad**: policy becomes unnecessarily conservative or cannot find a behavior valid across contradictory physics.

Calibrate, quantify residual uncertainty, randomize, then validate held-out corners. Do not select ranges by folklore.

Expected robustness is not worst-case robustness

The expectation objective can sacrifice a rare parameter region to improve the majority. Two alternatives are a lower-tail objective and a minimax objective:

$$\max_{\theta} \ \mathrm{CVaR}_{\alpha}^{lower}(J(\theta;\xi)),$$

$$\max_{\theta} \ \min_{\xi\in\Xi} J(\theta;\xi).$$

Conditional value at risk (CVaR) emphasizes the worst-performing fraction. Minimax emphasizes the worst member of a declared set. Both can become conservative or chase simulator artifacts, so physical plausibility of tails and failure classification remain essential.

Parameters are often correlated. Adding a payload changes mass, center of mass, and inertia together; battery voltage and motor torque limit co-vary; delay and packet loss may increase under bus load. Independent uniform sampling can create impossible combinations and miss real correlated ones. Represent known hardware modes as joint distributions or discrete mixtures:

$$p(\xi) = \sum_{k} p(k)\, p(\xi \mid \text{hardware mode } k).$$

Use held-out *combinations*, not only held-out one-dimensional endpoints. A policy robust to low friction alone and weak motor alone may still fail when both coincide.

Randomization curricula start near nominal physics, then widen after skill appears. This can solve exploration, but it risks overfitting to a nominal gait that cannot adapt. Compare against full-range training and record performance at every range, not only the final average.

15.6 System identification is becoming more systematic

The open PACE sim-to-real project uses measured encoder data and optimization to identify actuator/joint dynamics for legged robots. It is a useful modern study of the workflow

```
safe excitation -> measured trajectory -> parameter fit
                -> held-out validation -> policy training/transfer
```

The reusable idea is measurement-driven identification. Its exact tooling and models are not automatically suitable for XL330 servos or a wheeled-leg robot; the excitation, parameters, and safety protocol must match the hardware.

Given real trajectory $y_{0:T}$ and simulated prediction $\hat{y}_{0:T}(\eta; u)$ under input u, a basic estimator is

$$\eta^{*} = \arg\min_{\eta} \sum_{t} \| W(y_t - \hat{y}_t(\eta; u)) \|_2^2 + \lambda R(\eta).$$

W balances units/measurement confidence; R encodes plausible parameter ranges. Optimization does not guarantee identifiability. If changing inertia and motor gain produces the same slow response, many

η fit equally well. The input must be persistently exciting enough—within safety limits—to expose the dynamics being estimated.

Validate on different amplitudes, frequencies, directions, loads, battery states, and temperatures. Then inspect residual structure:

- zero-mean random residual suggests uncertainty/noise randomization;
- constant bias suggests calibration/nominal error;
- phase-dependent residual suggests missing delay/dynamics;
- state-dependent residual suggests an omitted nonlinear mode; and
- cross-joint residual suggests coupling not captured by independent models.

Do not squeeze all residuals into wider scalar friction bounds. Model the layer that causes the pattern.

15.7 Privileged learning

Simulation exposes information unavailable on the robot: exact base velocity, terrain height, contact force, friction, mass, and external disturbance. **Privileged learning** uses this information during training without requiring it at deployment.

Three patterns are common:

Asymmetric critic

```
actor  <- deployable observations only
critic <- actor observations + privileged state
```

The better-informed critic can estimate value with less ambiguity while the actor remains deployable.

Let deployed observation be o_t and privileged state be p_t. The actor is $\pi_\theta(a \mid o)$, while critic is $V_\phi(o, p)$. The policy-gradient estimate uses

$$\hat{g} = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid o_t) \hat{A}_t(o_t, p_t).$$

Privilege enters the training baseline/advantage, not the actor input. A more accurate critic can reduce variance, but a critic that overfits exact simulator state may generalize poorly across randomization and destabilize advantages. Compare actor-identical ablations across seeds and inspect value error by domain, not merely final return.

Teacher-student distillation

```
privileged teacher -> target actions or latent representation
deployable student -> imitates using sensor-accessible history/vision
```

A basic student loss is

$$L_{distill} = \mathbb{E}\left[\|\pi_S(h_t) - \pi_T(o_t, p_t)\|_2^2\right],$$

where h_t is deployable history. Training only on teacher trajectories can repeat behavior cloning (BC) covariate shift; roll out the student in simulation and label its visited states with the teacher when safe, or fine-tune with task reward. Teacher action may also be impossible for the student's delayed/noisy sensors to infer; no distillation loss can recover information absent from h_t.

Privileged experience for later RL

A privileged policy generates useful trajectories that warm-start a visual or off-policy learner.

The non-negotiable audit is actor information at deployment. A simulator result with height-map truth does not prove a depth-camera student works.

Test for leakage by replacing every privileged channel with nonsense during actor-only export and verifying actor output is unchanged. Trace dataflow, not only network signatures: a command generator or preprocessing feature derived from simulator truth can leak privilege outside the nominal actor tensor.

15.8 Online adaptation and Rapid Motor Adaptation (RMA)

Fixed domain randomization asks one policy to be robust across all sampled physics. An adaptive system tries to infer the current environment.

Rapid Motor Adaptation (RMA) separates:

- a base policy conditioned on an environment latent; and
- an adaptation module that infers that latent from recent proprioceptive history.

Conceptually:

$$z_t = g_\psi(o_{t-H:t}, a_{t-H:t-1}), \qquad a_t = \pi_\theta(o_t, z_t).$$

z_t is not necessarily a human-readable friction coefficient. It is a control-useful summary inferred from how the robot has responded.

One two-stage construction is:

1. train a base policy with privileged environment encoder $z_t^* = e(p_t)$ across randomized domains;
2. freeze or retain the base and train history encoder g_ψ to predict the teacher latent:

$$L_{adapt} = \mathbb{E}\left[\|g_\psi(h_t) - z_t^*\|_2^2\right].$$

The latent is useful only insofar as the base policy's action depends on it and history identifies it. Two robots with different friction can produce identical history while standing still; excitation through motion reveals the difference. This is **identifiability under the policy's own data distribution**.

History length H spans physical time $H\Delta t$. Too short misses slow motor or terrain effects; too long increases compute and can average over a recent change. A recurrent neural network can summarize variable history, a temporal convolution can expose a fixed receptive field, and an explicit estimator can produce interpretable parameters. Compare latency, reset state, uncertainty, and failure recovery—not only nominal return.

Questions for a new robot:

- Which physical changes leave an observable trace in the history?
- How long must the history be relative to the dynamics?
- How quickly does the estimate react versus amplify noise?
- What happens immediately after reset, before enough history exists?
- Was the adaptation distribution broad enough to include hardware?

Also ask what happens when conditions change abruptly. Evaluate friction or payload switches at controlled times and plot latent/action/recovery transient. A good final steady state with a dangerous two-second adaptation lag is not a good locomotion controller.

For Microduck, previous action already provides one step of actuator history. A fair adaptation experiment would add a versioned history/latent path while holding command, reward, randomization, network budget, and evaluation constant. First test whether explicit voltage sensing or improved BAM calibration solves the same failure more simply.

15.9 Robust perceptive locomotion

Proprioception tells what the robot feels now; **exteroception** senses the external environment before contact, using depth, light detection and ranging (LiDAR), or vision.

The 2022 Learning robust perceptive locomotion for quadrupedal robots in the wild work integrates exteroceptive and proprioceptive information with a recurrent encoder designed to cope with unreliable terrain perception.

A perceptive controller is a state-estimation problem as well as a policy. A recurrent belief can be written

$$h_t = F_\psi(h_{t-1}, o_t^{prop}, o_t^{exo}, m_t, \Delta t_t), \qquad a_t = \pi_\theta(o_t^{prop}, h_t, c_t),$$

where m_t indicates validity/age. Proprioception can reject an exteroceptive hallucination after contact, while exteroception anticipates terrain. Training must include corrupted, missing, delayed, and geometrically miscalibrated input so the fusion rule learns when not to trust it.

Coordinate transforms are part of perception. A depth point measured in camera frame C becomes body/world point through calibrated transforms such as

$$p_B = T_{BC} p_C.$$

An incorrect camera pitch makes a flat floor look like a ramp. Domain randomization around calibration uncertainty teaches tolerance; systematic mounting bias still requires calibration.

Representation choices include local height maps, point clouds, range rays, latent image features, and foothold candidates. Each trades geometry, occlusion, compute, and deployment reproducibility. A height map convenient in simulation may require mapping/localization that a small robot cannot run.

This addresses a different task than Microduck velocity tracking. Obstacles or terrain must appear in the actor input (or in a planner producing safe commands). Rendering an obstacle without sensing it does not create perceptive locomotion.

Evaluate perception/control jointly by obstacle class, sensor condition, speed, and stopping/recovery outcome. Pixel or depth reconstruction error alone does not state whether the robot selected a feasible foot placement.

15.10 From locomotion to parkour

Agile terrain tasks need more than a flat-ground velocity reward:

- terrain/obstacle observations;
- goal/contact representation;
- curricula or privileged teachers;
- collision and constraint definitions;
- recovery from perception error; and
- evaluation by obstacle class and failure mode.

Extreme Parkour with Legged Robots studies low-cost quadruped parkour with a front depth camera. The 2024 SoloParkour work studies constrained visual locomotion, using privileged experience to warm-start depth-based off-policy learning on Solo-12.

Compare what each deploys—not only the video:

- sensor and frame rate;
- observation history;
- teacher information;
- action/control rate;
- constraint definition;
- obstacle distribution; and
- number and type of real trials.

Parkour illustrates three distinct learning problems:

1. **motion feasibility**: can the body produce a jump/climb under torque and contact limits?
2. **perception/estimation**: where is the obstacle and how uncertain/stale is that estimate?
3. **selection/planning**: which skill, timing, and foothold make progress?

One end-to-end policy may contain all three, but ablations and interfaces should still separate them. A privileged teacher can solve feasibility/selection from perfect geometry; a student then learns deployable perception. If the student fails, compare teacher-on-noisy-observation, perception labels, and motor tracking before retuning every reward.

Collision penalties are not hard constraints. When success reward outweighs them, the policy may accept impacts. Report collision/impact distributions and use runtime feasibility/stop guards where violation cannot be tolerated.

15.11 Hierarchical wheeled-leg locomotion

The 2024 Learning Robust Autonomous Navigation and Locomotion for Wheeled-Legged Robots integrates adaptive locomotion, a learned navigation controller, and large-scale path planning. The paper is especially relevant to Jump Rover because it demonstrates the architectural distinction between local wheel-leg control and navigation.

It does not imply that a new wheeled-leg platform can train before its mechanical, actuator, sensing, and realtime interfaces are characterized. Architecture transfers more readily than weights.

Hierarchy separates horizons and authority:

```
global planner (seconds/metres): route through mapped world
local navigation (hundreds of ms): collision-aware short trajectory/twist
locomotion -(520 ms): contact/actuator feedback for requested motion
drive firmware (sub-ms to few ms): current/velocity/position and protection
```

The local command must lie inside the locomotion policy's trained capability set $\mathscr{C}(o)$. A command governor can project a planner request onto an estimated safe set:

$$c^{applied} = \arg\min_{c \in \mathscr{C}(o)} \| c - c^{planner} \|_W^2.$$

Estimating $\mathscr{C}$ is hard; start with conservative measured bounds on speed, turn rate, acceleration, terrain, tilt, sensor age, and stopping distance. Log every projection so the planner is not falsely credited for commands the local layer rejected.

For JumpRover, a cloud agent can interpret goals, retrieve maps, or propose a task plan. The brain system-on-chip should perform local perception/planning, and the future real-time board must retain watchdog, bus timing, actuator limits, and safe-stop authority. Training can begin meaningfully only after the mechanical model and board/runtime contracts provide credible actions, observations, delays, and safety behavior.

15.12 From motion tracking to reusable whole-body skills

Commanded velocity rewards discover a useful gait but do not specify style or rich whole-body coordination. Motion-reference methods import kinematic examples and use physics/RL to make them robust and dynamically feasible.

DeepMimic established an influential example-guided recipe. A tracking reward often combines exponential errors:

$$r_t = w_q e^{-k_q \| q_t - q_t^{ref} \|^2} + w_v e^{-k_v \| \dot{q}_t - \dot{q}_t^{ref} \|^2} + w_e e^{-k_e \| p_t - p_t^{ref} \|^2} + w_c r_{task}.$$

Reference phase/index must be observed or inferred. Retargeting human motion to a robot is itself an optimization over different limb lengths, joints, limits, contacts, and balance; copying angles is rarely valid.

Adversarial Motion Priors (AMP) learn a style reward from unstructured motion clips, reducing hand-crafted reference tracking/clip selection. Adversarial Skill Embeddings (ASE) learn a reusable latent repertoire for downstream tasks. MaskedMimic frames diverse partial control conditions as masked motion

inpainting for physics-based characters. These works expand control expressivity, but many results are in simulated characters; physical robot transfer adds actuator, impact, sensing, and safety evidence.

The 2025 preprint ASAP studies two-stage whole-body humanoid skill transfer with a learned residual/delta action model from real data. The model is integrated into simulation for policy fine-tuning. This is a modern instance of structured residual alignment: learn what the nominal simulator misses rather than only widening every randomization.

The 2025 BeyondMimic preprint reports a motion-tracking foundation plus guided diffusion for test-time whole-body task control and real humanoid deployment. A 2026 preprint on whole-body humanoid locomotion via motion generation and tracking combines terrain-aware diffusion reference generation with an RL tracker and closed-loop fine-tuning for onboard perceptive locomotion. These are frontier systems reviewed in September 2026, not settled universal recipes; inspect revisions, released code/checkpoints, real-trial counts, and failure protocols.

The conceptual progression is:

```
single reference tracking
-> unstructured motion prior
-> reusable skill latent / partial-condition controller
-> task-guided motion generation
-> physical tracker with explicit sim-to-real alignment
```

At every stage, a generated kinematic motion is only a reference. The low-level physics policy, actuator limits, contacts, and safety system decide whether the robot can execute it.

15.13 Recent off-policy scaling is a research direction

The 2025 preprint Learning Sim-to-Real Humanoid Locomotion in 15 Minutes reports massively parallel FastSAC/FastTD3 recipes on humanoids. It is valuable evidence that off-policy continuous-control methods can be engineered at GPU parallel scale, challenging a simplistic "PPO is always the locomotion choice" belief.

Treat it as a recent, reproducible direction with stated hardware and compute, not a universal 15-minute promise. Reproduction must match environment count, GPU, simulator, update-to-data ratio, task, randomization, and success criteria.

On-policy PPO discards/restricts reuse after several epochs so updates remain close to the collection policy. Soft Actor-Critic (SAC) and Twin Delayed Deep Deterministic Policy Gradient (TD3) learn from replay and can reuse each transition, improving sample efficiency in some settings. Massive parallelism creates a very fast incoming data stream, so off-policy training must balance replay freshness, critic updates, target networks, value overestimation, and device utilization.

Define an update-to-data ratio clearly. One useful convention is critic minibatch samples consumed per new environment transition:

$$\mathrm{UTD} = \frac{N_{gradient}\, B_{minibatch}}{N_{new\ transitions}}.$$

Some papers/code use "gradient steps per collection step" instead, so the same number can mean something different. State numerator and denominator. High UTD extracts more optimization from data but can overfit critic errors; low UTD may leave the learner unable to keep up with collection.

A fair PPO/off-policy comparison matches more than wall time:

Axis	Why it matters
transitions	sample efficiency
wall time/device/power	compute efficiency
actor/critic capacity	representation budget
simulator worlds/rate	data throughput/correlation
reward/randomization/curriculum	task difficulty
evaluation seeds and physical trials	outcome uncertainty

Axis	Why it matters

Algorithm family should be an experimental variable after the environment and transfer stack are credible, not the first explanation for a broken task.

15.14 A research-to-project comparison card

For any locomotion paper, fill this before copying code:

```
Paper/project and version:
Robot morphology and mass:
Actuator and low-level controller:
Policy rate / physics rate:
Observation (train actor / deploy actor / critic):
Action representation:
Command/goal representation:
Algorithm and network memory:
Simulator and environment count:
System identification:
Domain randomization ranges:
Curriculum/reset distribution:
Baselines:
Number of training seeds:
Sim evaluator:
Real trials and failure categories:
Public code/checkpoint/data:
What is directly transferable:
What must be re-measured:
```

15.15 Microduck research extensions

High-value controlled studies include:

1. **Robustness versus adaptation**: compare fixed history-free PPO with an adaptation latent under held-out actuator/friction changes.
2. **Calibration versus randomization width**: measure actuators, fit BAM, then ablate narrow/medium/wide residual randomization.
3. **Privileged critic ablation**: keep actor identical and compare critic information across seeds.
4. **PPO versus off-policy at matched cost**: match environment steps, GPU time, evaluator, and model capacity.
5. **Perceptive hierarchy**: compare a local obstacle planner commanding the unchanged walk policy against a new perceptive locomotion actor.

Each is a paper-shaped question because it changes one conceptual factor and defines evidence before training.

Turn them into controlled designs:

Study	Treatment	Fixed control	Primary evidence	Falsifying result
robustness vs adaptation	history/latent encoder	same physics range, actor budget, seeds	held-out parameter switches and recovery time	no paired tail/recovery gain
calibration vs domain randomization (DR) width	measured nominal + three residual widths	same reward, algorithm, and budget	real/held-out sim tracking and worst slice	width not predictive of transfer
privileged critic	add exact velocity/terrain only to critic	identical actor/input/evaluator	seed-level sample efficiency and final score	advantage noise/value error not improved
PPO vs off-policy	algorithm/replay	matched transitions, compute record, networks/task	performance vs transitions and wall time	claimed benefit disappears when matched

Study	Treatment	Fixed control	Primary evidence	Falsifying result
perceptive hierarchy	planner+61D vs versioned perceptive actor	same sensor/scenes/safety	held-out obstacles, latency, collision/progress	complexity adds no robust benefit

For adaptation, include abrupt tests: nominal → low voltage at t = 5 s, high → low friction on a marked patch, or payload attachment between episodes. Plot latent, action saturation, tracking, tilt, and recovery; final averages alone hide dangerous transients.

For JumpRover, the first research artifact should be a *measurement and interface dataset*, not a long RL run: motor/drive response, encoder/inertial timing, bus age, battery/load behavior, geometry/inertia, contact/wheel friction, and safe-state transitions. Once the mechanics and real-time board exist, fit a nominal model, reproduce the control path in simulation, and only then select robustness/adaptation studies.

15.16 Exercises

1. What makes locomotion a hybrid dynamical system? Give one continuous mode and one discrete event for Microduck.
2. For center-of-mass height z_0 = 0.10 m, position x = 0.01 m, velocity $\dot{x}$ = 0.20 m/s, and g = 9.81 m/s², calculate ω_0 and capture point ξ. State why it is only a diagnostic.
3. A Poincaré linearization has eigenvalues `0.6`, `-0.8`, and `1.05`. Is the fixed gait locally stable in that model? Interpret the negative eigenvalue.
4. Explain why parallel simulation improves wall-clock throughput but not necessarily sample efficiency. Calculate batch transitions and per-world temporal depth for 4,096 worlds × 24 steps at 50 hertz.
5. Explain why an ideal position actuator can produce an untransferable policy. Name four arguments of a more realistic torque model.
6. Compare expected domain-randomized, lower-tail CVaR, and minimax objectives. Which failure does each emphasize?
7. Give two physically correlated randomization variables. Why can independent sampling create impossible robots?
8. A parameter fit matches training steps but has phase lag on held-out sine sweeps. What does this residual suggest, and why is wider mass randomization not the first fix?
9. Give one train-only privileged variable and a way a deployment student could infer its effect. How would you test for leakage?
10. Contrast robustness through domain randomization with online adaptation. What observation-identifiability condition must adaptation satisfy?
11. A history encoder uses 50 steps at 50 hertz. What physical duration does it cover? Give one benefit and one risk of doubling it.
12. Design a failure taxonomy for perceptive stair climbing that separates perception, feasibility, control, timing, and safety.
13. Explain why a generated human-motion reference is not directly a safe humanoid action. Name the tracker/retargeting responsibilities.
14. Contrast reference tracking, adversarial motion priors, reusable skill latents, and masked/conditional motion controllers.
15. Define update-to-data ratio in your own numerator/denominator. Why can “UTD=4” be ambiguous across codebases?
16. Design the command-governor interface between a cloud/local planner and JumpRover locomotion. Which authority remains on the real-time board?
17. From one paper in this chapter, fill the research-to-project comparison card using only the paper and official repository. Mark omissions rather than guessing.
18. Propose one Microduck experiment whose negative result would still teach something specific; include treatment, controls, primary metric, tail case, and falsifier.

Continue with vision, foundation policies, and hierarchical autonomy.

15.17 Folded solutions

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show solutions to Exercises 1–9

1. Dynamics flow continuously while a contact set is fixed, then touchdown, liftoff, impact, or mode switching changes constraints/reset state. For Microduck, single-foot support is a continuous mode and the other foot's touchdown is a discrete event.
2. $\omega_0 = \sqrt{9.81/0.10} \approx 9.90\ \mathrm{s}^{-1}$. Thus $\xi = 0.01 + 0.20/9.90 \approx 0.0302$ m, about 3.0 cm from the reference origin. The model assumes constant height, point-like dynamics, simplified support, and ignores angular momentum/actuator/contact limits, so it diagnoses rather than exactly commands Microduck.
3. No. Local asymptotic stability requires all relevant magnitudes below one; `1.05` grows about 5% each cycle. `-0.8` shrinks in magnitude while flipping sign each cycle, an alternating decaying perturbation.
4. More worlds increase transitions per wall second, not information gained per transition. Batch size is 4096(24) = 98,304 transitions. Each world spans 24/50 = 0.48 s; aggregate simulated time is 1,966.08 s.
5. It allows instantaneous/unlimited target tracking and hides voltage, load, friction, saturation, and delay, so the learned gait may demand impossible torque. A realistic map can depend on position, velocity, target/error, voltage, temperature, load, delay/history, and identified motor/friction parameters; any four earn the point.
6. Expectation optimizes average performance under $p(\xi)$ and can sacrifice rare cases. Lower-tail CVaR focuses on the worst chosen fraction under that distribution. Minimax focuses on the single worst member of a declared set, often most conservative and most sensitive to simulator artifacts.
7. Payload mass–center-of-mass–inertia, battery voltage–torque limit, or bus load–delay–loss are correlated. Independent extremes can describe a payload whose inertia/center is physically inconsistent or a high-voltage motor with an unrelated weak torque limit, wasting capacity on impossible combinations.
8. Frequency-dependent phase residual suggests missing delay, filtering, or a dynamic mode. Mass width changes inertial scale but does not reproduce a consistent phase lag; measure timing/model the causal layer first.
9. True friction can be privileged. A student may infer its effect from recent commands, foot motion, body response, and slip. Export the actor, replace privileged channels with arbitrary values, and verify outputs are unchanged; also trace command/preprocessing dataflow for indirect leakage.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show solutions to Exercises 10–18

10. Domain randomization trains one robust mapping across a distribution without explicitly identifying its member. Adaptation infers a latent/current context and changes actions. Different domains must cause distinguishable deployable histories under the policy; standing still may not identify friction.
11. Duration is 50/50 = 1 s. Doubling may expose slower voltage/payload effects and reduce ambiguity, but increases memory/compute, startup state, and risk of averaging over an abrupt recent change.
12. Example classes: missing/wrong/stale stair geometry (perception/timing), correct geometry but unreachable foothold (feasibility), feasible target but wrong action/tracking (control), toe/body collision or slip/saturation (physical execution), recovery versus fall, and local safety intervention. Report counts per stair/sensor/speed slice and synchronized traces.
13. Human joints/morphology do not match robot links/limits/contacts. Retargeting maps bodies while enforcing reachable poses and contacts; the RL tracker supplies feedback and dynamic feasibility under torque/impact/randomization; runtime safety limits authority. A kinematic reference has no guarantee of balance or safe torque.
14. Tracking follows an indexed clip with explicit pose/velocity rewards. A motion prior learns style/distribution reward from unstructured clips. A skill latent exposes reusable behavioral modes for downstream tasks. Masked/conditional controllers fill missing motion from partial goals or modalities. Expressivity grows, along with data, inference, and validation burden.

15. One definition is minibatch samples consumed by critic gradients divided by newly collected transitions. Another codebase may count gradient calls per vectorized collection call and omit minibatch size. Therefore “4” is ambiguous unless numerator, denominator, batching, actor/critic updates, and time interval are stated.
16. The planner sends timestamped, expiring goal/local twist with source and sequence. A local command governor clamps/projects it into measured speed, acceleration, terrain, age, and stopping bounds; locomotion tracks it. The real-time board retains fresh-I/O validation, watchdog, current/position/ temperature limits, deadline handling, emergency stop, and safe state without cloud or brain cooperation.
17. A passing card quotes morphology/mass, actuator/controller, rates, train/deploy/critic observations, action, simulator/worlds, algorithm, identification/randomization, curricula, seeds/evaluation, real trials, failures, and released artifacts. “Not reported” is the correct answer when primary sources omit an item.
18. Example: test history-conditioned adaptation versus an actor of matched parameter count under held-out voltage drops. Hold task, reward, randomization, seeds, transitions, and deployment rate fixed. Primary metric is recovery time/tracking after the drop; tail is falls/saturation at lowest safe voltage. Falsifier: paired tail/recovery does not improve while nominal quality/latency is non-inferior. A negative result bounds the value of history and motivates explicit voltage sensing or better actuator fit.

16. Vision, Foundation Policies, and Hierarchical Autonomy

A robot that can balance is not automatically a robot that knows where to go. This chapter connects pixels to geometry, geometry to safe local motion, and language goals to typed robot skills. It also explains what modern vision-language-action (VLA) policies learn, what they do not learn, and where reinforcement learning (RL) still belongs.

By the end, you should be able to:

- derive a 3D point from a calibrated depth pixel;
- explain why timestamps and uncertainty are part of a perception output;
- compare modular, end-to-end, and hybrid visual-control architectures;
- map action regression, tokens, chunks, diffusion, and flow matching to code;
- read current generalist-policy papers without treating model size as proof;
- derive the option-value equation used in hierarchical RL; and
- specify a safe onboard/cloud architecture for JumpRover.

The central rule is simple: **semantics may run slowly, but physical stability cannot wait**.

16.1 Perception turns state into belief

Proprioception measures the robot itself: joint encoders, an inertial measurement unit (IMU), motor state, and battery voltage. **Exteroception** measures the surrounding world: a red-green-blue (RGB) camera, depth camera, light detection and ranging (LiDAR), radar, microphone, or tactile array.

An image is not a state vector. The same image can be compatible with many worlds: a small nearby obstacle may occupy the same pixels as a large distant one; a hidden person may still be moving; and camera motion may resemble object motion. A useful formal model is therefore a **Partially Observable Markov Decision Process (POMDP)**. The hidden state is s_t, the observation is o_t, and the robot acts from a belief $b_t(s)$, a probability distribution over possible states.

A Bayes-filter update has two steps. First predict through the dynamics:

$$\bar{b}_t(s_t) = \int p(s_t \mid s_{t-1}, a_{t-1}) b_{t-1}(s_{t-1})\, ds_{t-1}.$$

Then incorporate the new sensor measurement:

$$b_t(s_t) = \eta\, p(o_t \mid s_t) \bar{b}_t(s_t),$$

where η normalizes the distribution to integrate to one. In plain language, prediction says where the world could have moved; measurement says which predictions agree with the sensor. A learned recurrent policy may encode this belief implicitly in hidden activations. A geometric system may represent it explicitly as pose covariance, object tracks, and an occupancy map.

This distinction matters for Microduck. Its present 61-dimensional actor observation contains proprioception and commands, not pixels or range. A box rendered in the viewer changes neither o_t nor the reward unless the environment explicitly connects it. The policy may collide because, from its information, the box does not exist.

16.2 From a pixel to a point in the world

The pinhole camera model

A camera maps a 3D camera-frame point (X, Y, Z) to pixel coordinates (u, v). For the ideal pinhole model,

$$u = f_x X/Z + c_x,$$
$$v = f_y Y/Z + c_y.$$

Here f_x, f_y are focal lengths measured in pixels and (c_x, c_y) is the principal point. Collect them in the intrinsic matrix

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}.$$

If a depth sensor supplies Z, back-projection recovers the camera-frame point:

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = ZK^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}.$$

Suppose $f_x = f_y = 400$, $(c_x, c_y) = (320, 240)$, and pixel $(360, 220)$ has depth $Z = 1.2$ m. Then

$$X = 1.2(360 - 320)/400 = 0.12 \text{ m},$$

$$Y = 1.2(220 - 240)/400 = -0.06 \text{ m}.$$

The point is 12 cm to one camera-side axis, 6 cm to the other, and 1.2 m forward. Axis signs depend on the declared frame convention; never guess them.

Intrinsics, distortion, and extrinsics

Intrinsic calibration estimates focal lengths, principal point, and lens distortion. Real lenses bend rays, commonly with radial and tangential terms; undistortion must use the calibration for the actual resolution and crop.

Extrinsic calibration estimates the rigid transform between sensor and robot frames. With homogeneous coordinates,

$$^{B}p = {}^{B}T_{C}\, {}^{C}p,$$

where ${}^{B}T_{C}$ maps a point from camera frame C into robot-base frame B. The superscripts answer "expressed in which frame?" A transform in the wrong direction can look numerically plausible while placing every obstacle behind the robot.

Calibration is not a one-time ceremonial file. A loose bracket, changed focus, new image crop, or mechanical impact changes the mapping. Store calibration revision, image mode, temperature assumptions, and validation residual with the deployment artifact.

Error propagation

Depth and calibration errors grow into position error. For $X = Z(u - c_x)/f_x$, a first-order variance approximation is

$$\sigma_X^2 \approx \left(\frac{u - c_x}{f_x}\right)^2 \sigma_Z^2 + \left(\frac{Z}{f_x}\right)^2 \sigma_u^2 + \left(\frac{Z(u - c_x)}{f_x^2}\right)^2 \sigma_{f_x}^2.$$

This is a Jacobian covariance rule: sensitivity squared times input variance. It explains why distant depth, edge pixels, and uncertain focal length should not yield the same clearance margin as a clean central measurement.

16.3 Time is part of every measurement

A perception message needs at least:

```
value + frame + capture timestamp + covariance/confidence + validity
```

The timestamp should describe photon capture, not when a neural network finished. Define action-time age as

$$A_t = t_{\text{act}} - t_{\text{capture}}.$$

At speed v, age alone creates approximately vA_t of unobserved travel. A rover moving at 0.5 m/s with a 220 ms-old obstacle map has moved 11 cm since the represented scene. That is substantial on a small robot.

Important sources of age and misalignment include:

- exposure and sensor readout;
- rolling-shutter rows captured at different instants;
- camera/IMU clocks with offset or drift;
- frame queues that process old images in order;
- preprocessing, inference, transport, and planning; and
- an action chunk continuing after the observation that produced it is stale.

A latest-value queue is often safer than an unbounded first-in-first-out queue: drop an old frame when a newer one is available. Log capture time, inference start/end, planner time, and action-application time separately. Average frames per second (FPS) can look excellent while the tail age is unsafe.

A minimal message and age check might be:

```python
from dataclasses import dataclass

@dataclass(frozen=True)
class LocalMap:
    capture_ns: int
    frame: str
    min_clearance_m: float
    clearance_std_m: float
    valid: bool

def usable(msg: LocalMap, now_ns: int) -> bool:
    age_ms = (now_ns - msg.capture_ns) / 1e6
    return msg.valid and msg.frame == "base_link" and age_ms <= 150.0
```

The number 150 ms is an example, not a universal safe threshold. Derive the real bound from speed, braking, compute tails, and mechanical tests.

16.4 Three visual-control architectures

Modular perception and planning

A modular stack may use **simultaneous localization and mapping (SLAM)** to estimate robot pose while constructing a map:

```
camera/depth -> detection + tracking + local map
             -> collision-aware planner -> bounded velocity command
             -> locomotion policy -> realtime controller
```

Intermediate contracts are inspectable. A developer can replay a camera log, measure map error without moving motors, then test the planner against a known map. Geometric collision constraints and deterministic fallbacks fit naturally. The cost is interface engineering: frame, units, latency, and confidence errors can propagate across otherwise-correct modules.

End-to-end visual policy

```
images + proprioception + language/goal -> neural policy -> robot actions
```

The representation is optimized for task success and can use visual cues a hand-designed map discards. But data requirements, causal diagnosis, sensor shift, and safety validation become harder. “End-to-end” never literally removes firmware, motor limits, synchronization, or emergency-stop logic.

Hybrid architecture

```
learned visual encoder -> terrain/object latent or metric local map
proprioception + goal  -> learned local policy
                       -> geometric governor -> realtime controller
```

Most deployable systems are hybrids. The engineering question is not which label is fashionable, but **where uncertainty becomes explicit and where authority is bounded**.

A useful contract table is:

Boundary	Required fields	Rejection example
perception to planner	frame, time, covariance, valid region	stale map
planner to skill	goal, bounds, expiry, stop rule	unreachable goal
skill to controller	schema, units, rate, sequence	wrong joint order
cloud to onboard	exact skill and typed parameters	unknown skill

16.5 Obstacle avoidance is a causal chain

Putting obstacles into a simulator teaches nothing by itself. Successful avoidance requires the whole chain:

```
vary obstacles
    -> sense before contact
    -> encode usable geometry/history
    -> make safe progress preferable to collision
    -> expose recovery decisions
    -> evaluate held-out layouts and failures
```

A stopping-distance derivation

Let forward speed be v, end-to-end reaction delay be τ, and verified braking magnitude be $a_b > 0$. During delay the robot travels $v\tau$. Under constant braking, $0 = v^2 - 2a_b d$, so braking distance is $v^2/(2a_b)$. Add a geometric and estimation margin m:

$$d_{\text{stop}} = v\tau + \frac{v^2}{2a_b} + m.$$

For $v = 0.4$ m/s, $\tau = 0.18$ s, $a_b = 0.8$ m/s^2, and $m = 0.12$ m:

$$d_{\text{stop}} = 0.4(0.18) + \frac{0.4^2}{2(0.8)} + 0.12 = 0.292 \text{ m}.$$

This is already 29.2 cm. If the robot cannot reliably detect, command, brake, and settle within that space, 0.4 m/s is outside the validated envelope.

If clearance estimate D is approximately Gaussian with mean μ_D and standard deviation σ_D, a one-sided conservative clearance can be

$$D_{\text{safe}} = \mu_D - k\sigma_D.$$

Choosing $k = 2.33$ corresponds to roughly a 1% lower-tail probability under a well-calibrated Gaussian assumption. Real perception errors are often non-Gaussian and correlated, so measure empirical coverage rather than trusting that interpretation automatically.

Solve $D_{\text{safe}} \geq d_{\text{stop}}$ for the maximum permitted speed. Since the equation is quadratic,

$$v_{\max} = -a_b\tau + \sqrt{(a_b\tau)^2 + 2a_b(D_{\text{safe}} - m)}.$$

This converts uncertainty into authority: greater age or variance lowers the speed command before collision.

The runnable camera and stopping example computes the projection, age distance, and speed envelope without third-party packages.

Occupancy is not certainty

An occupancy grid stores a probability per cell, often in log-odds form:

$$\ell_t = \log \frac{p_t}{1 - p_t}.$$

Independent measurement updates become additions in log-odds, with a prior correction. The independence assumption is imperfect when adjacent depth pixels share the same failure. Do not interpret 100 correlated pixels as 100 independent confirmations.

For Microduck, the lowest-risk first project is to preserve its locomotion actor:

```
depth -> local occupancy/height map -> local planner
      -> [forward speed, lateral speed, yaw rate]
      -> existing 61D-input velocity policy
```

A second research project can train perceptive locomotion from depth/history and proprioception directly to 14 joint targets. That may coordinate footsteps with terrain, but it creates a new observation contract, dataset, runtime, and sim-to-real validation problem. It is not a drop-in upgrade.

16.6 Learning a visual representation

A visual encoder maps image I_t to a compact latent $z_t = f_\psi(I_t)$. What makes z_t useful depends on its training objective.

Four common objectives

1. **Supervised prediction.** Predict depth, segmentation, object labels, or traversability. Labels make semantics explicit but are expensive and can omit unlabelled control cues.
2. **Reconstruction.** Encode enough information to reconstruct masked pixels or video. This uses unlabeled data, but pixel detail is not automatically relevant to control.
3. **Contrastive learning.** Make matching views/text close and nonmatching samples far apart.
4. **Task loss.** Fine-tune the encoder through behavior cloning or return. It focuses features on the current task but may forget reusable information.

A common contrastive loss for an image anchor i, positive text or view i^+, negatives j, similarity s, and temperature T is

$$L_i = -\log \frac{\exp(s(z_i, z_{i^+})/T)}{\sum_j \exp(s(z_i, z_j)/T)}.$$

The loss teaches relative association, not metric depth or collision safety. A representation can retrieve "chair" perfectly while confusing whether its legs block the planned path.

Probes test information; interventions test use

A **linear probe** freezes the encoder and trains a small linear head to predict a property such as depth. High probe accuracy says information is decodable; it does not prove the policy uses it. Stronger tests intervene:

- mask the obstacle region and measure action change;
- keep geometry fixed while changing texture;
- move the camera while keeping the robot state fixed;
- inject timestamp delay independently of image corruption; and
- compare success on held-out objects, rooms, and lighting.

Separate representation failure ("the latent lacks clearance") from policy failure ("clearance is encoded but ignored") and calibration failure ("the camera-to-base transform is wrong").

16.7 Anatomy of a vision-language-action policy

A VLA policy consumes visual observations, a language instruction, and usually robot state, then predicts one action or an action sequence:

$$a_{t:t+H-1} \sim \pi_\theta(I_{t-k:t},\, q_t,\, \text{instruction},\, e).$$

Here q_t is proprioception, H is action-chunk length, and e encodes the embodiment or robot-specific schema. Most VLA pretraining is supervised imitation on demonstrations, not online RL. A neural policy is not RL merely because its output moves a robot.

Tokenizing observations

An image encoder turns patches or learned regions into visual tokens. A text tokenizer converts the instruction to language tokens. Proprioception may be projected by a multilayer perceptron (MLP) into the same hidden width. A transformer mixes these tokens with attention. For queries Q, keys K, and values V, one attention head computes

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

This is content-dependent weighted averaging. It does not by itself impose a coordinate frame, causality, or physical feasibility; training data and masks must teach those relations.

Five action parameterizations

1. Direct regression. Predict a continuous action and minimize, for example, absolute error:

$$L_{\text{L1}} = \frac{1}{Hd_a}\sum_{h=0}^{H-1}\sum_{j=1}^{d_a} |a_{t+h,j} - \hat{a}_{t+h,j}|.$$

It is simple and fast. With multiple equally valid demonstrations, a single mean prediction can average incompatible behaviors.

2. Per-dimension action tokens. Normalize each action component and assign it to one of B bins. An autoregressive language-model-style head predicts categorical symbols:

$$L_{\text{token}} = -\sum_{h,j} \log p_\theta(k_{h,j} \mid I, q, \text{instruction}, k_{<h,j}).$$

Tokens reuse language-model machinery, but naïve quantization introduces bin error and creates long output sequences at high control rates.

3. Action chunks. Predict H future actions at once. If policy inference is 10 Hz and hardware control is 50 Hz, a five-step chunk can fill the interval. Chunks represent temporal coordination and reduce calls, but a long open-loop chunk reacts slowly to disturbance. Receding-horizon execution predicts H steps and executes only the first $E < H$ before replanning.

4. Diffusion or flow action generation. Start with noise and iteratively transform it into a continuous action chunk. This can represent multiple modes without averaging them. It costs multiple numerical steps and introduces sampling variation.

5. Compressed trajectory tokens. Frequency-space Action Sequence Tokenization (FAST) transforms action chunks into a frequency representation, quantizes coefficients, and compresses them. Smooth robot trajectories often concentrate energy in low frequencies, so a chunk can require far fewer tokens than independent timestep bins. The FAST paper reports that this improves high-frequency dexterous-action modeling in its evaluated settings.

Flow matching from equation to pseudocode

Let x_1 be a demonstrated action chunk and x_0 Gaussian noise. A simple linear probability path at interpolation time $\tau \in [0, 1]$ is

$$x_\tau = (1 - \tau)x_0 + \tau x_1.$$

Its target velocity is $u = x_1 - x_0$. Train a conditional vector field v_θ from vision-language context c:

$$L_{\text{flow}} = \mathbb{E}_{x_0, x_1, \tau}\left[\|v_\theta(x_\tau, \tau, c) - (x_1 - x_0)\|_2^2\right].$$

At inference, sample $x(0) \sim \mathcal{N}(0, I)$ and solve

$$\frac{dx}{d\tau} = v_\theta(x, \tau, c)$$

numerically to $\tau = 1$. An Euler implementation is conceptually:

```
x = gaussian_noise(action_chunk_shape)
for step in range(num_flow_steps):
    tau = step / num_flow_steps
    velocity = model(x, tau, images, instruction, robot_state)
    x = x + velocity / num_flow_steps
action_chunk = denormalize(x)
```

Real implementations choose a path, sampler, masks, padding, and numerical schedule carefully. The pseudocode maps the mathematics to computation; it is not a production controller.

Normalization and embodiment adapters

Robot datasets disagree about action coordinates and scale. A value of 0.1 can mean 0.1 radians, 0.1 m/s, a normalized gripper command, or an end-effector delta. A robust adapter declares:

- joint, task-space, or base-velocity action meaning;
- absolute versus delta coordinates;
- reference frame and axis order;
- units, control rate, and chunk horizon;
- bounds and normalization statistics;
- absent dimensions and masks; and
- conversion to the local safety-controlled command.

Cross-embodiment data is useful only after this schema work. File-format compatibility is not semantic compatibility.

16.8 A research map from 2022 through 2026

This area changes quickly. The table is a map of design ideas, not a leaderboard. Results across papers use different robots, data, tasks, and success definitions. Chapter 18 contains detailed paper seminars.

The names below include Robotics Transformer 1 (RT-1), Robotics Transformer 2 (RT-2), and Generalist Robot 00 Technology (GR00T). They are project names, not certifications of general capability.

Work	Main design lesson	Evidence boundary
RT-1	scale a token policy over diverse real tasks	one fleet/task family
RT-2	co-train web semantics and robot action tokens	closed models/setup
Open X-Embodiment	normalize and mix many robot datasets	manipulation-heavy mix
Octo	open generalist transformer for adaptation	fine-tuning still needed
OpenVLA	open 7B visual-language backbone to action tokens	large runtime footprint
π_0	flow-matched continuous action chunks	adapter/data overlap matter
GR00T N1	reasoning backbone plus continuous action expert	mainly manipulation evidence
$\pi_{0.5}$	heterogeneous co-training for open-world tasks	reported platform/protocol
SmolVLA	smaller open model and asynchronous inference	staleness must be measured
FAST	compress smooth chunks into action tokens	tokenizer is one component
OpenVLA-OFT	continuous chunks and parallel optimized fine-tuning	benchmark-specific comparison

Robotics Transformer 1 (RT-1) and **Robotics Transformer 2 (RT-2)** made large-scale language-conditioned robot action tokenization influential. Open X-Embodiment and Robotics Transformer X (RT-X) then made cross-robot data normalization a central research problem.

Octo's official code, OpenVLA's official code, and Physical Intelligence's `openpi` make different parts of the stack inspectable. OpenVLA also studies Low-Rank Adaptation (LoRA), which updates a low-rank correction $W' = W + BA$ instead of every element of W. It reduces trainable parameters; it does not guarantee low inference latency.

Generalist Robot 00 Technology (GR00T) N1 separates a semantic reasoning system from a continuous action system. SmolVLA, distributed through the LeRobot project, emphasizes a smaller open model and asynchronous inference. Both reinforce the same systems lesson: model architecture does not remove action schemas, observation age, or local safety.

OpenVLA-OFT reports large gains in its LIBERO and real-robot protocols by combining continuous actions, chunks, parallel decoding, and an absolute-error loss. This is more educational than quoting its final success number: several apparently small action-head decisions can dominate adaptation quality and throughput.

As of this book's September 2026 review, 2026 papers are still recent preprints. For example, Green-VLA proposes staged visual-language, multi-embodiment, embodiment-specific, and RL-alignment phases plus out-of-distribution detection. Treat it as a current research signal whose claims require reproduction, not as settled superiority. Publication date and model name are never substitutes for matched trials.

How to choose a starting point

Ask in this order:

1. Does the released action schema match the robot and task?
2. Does the pretraining data contain related viewpoints, objects, and motions?
3. Are weights, adapters, normalization, and evaluator all available?
4. Can onboard hardware meet memory and tail-latency requirements?
5. Is there enough target data to fine-tune and validate held-out conditions?
6. Does the license permit the intended use and redistribution?

A small behavior cloning (BC) policy with correct data can beat an enormous foundation policy with the wrong embodiment. Run that baseline.

16.9 Hierarchical reinforcement learning and options

In hierarchical RL, a higher policy chooses a goal, latent, or temporally extended skill; a lower policy executes it. An **option** consists of:

- an initiation set $\mathcal{I}_o$: states where it may start;
- an internal policy $\pi_o(a \mid s)$; and
- a termination probability $\beta_o(s)$.

Because an option can last k primitive steps, the high level forms a semi-Markov decision process. If rewards during the option are $r_t, \dots, r_{t+k-1}$, its option return is

$$R_o = \sum_{i=0}^{k-1} \gamma^i r_{t+i}.$$

The option-value Bellman equation is

$$Q(s, o) = \mathbb{E}\left[R_o + \gamma^k \max_{o'} Q(s', o') \mid s, o\right].$$

The factor is γ^k, not merely γ, because physical time passed inside the option. Long skills accumulate more reward and more risk before the high level can reconsider.

A robot option should be a typed contract:

```
option: dock
preconditions:
  dock_track_age_ms_max: 150
  localization_std_m_max: 0.08
parameters:
  dock_id: dock.main
  approach_speed_mps: 0.08
success: charging_contact_confirmed
abort: [track_lost, bumper_hit, timeout, tilt_limit]
timeout_s: 30
fallback: stop_balanced
```

The initiation set becomes preconditions, termination becomes success/abort, and the internal policy becomes a versioned skill. This connects RL theory to a practical application programming interface (API).

16.10 Language planning needs physical affordance

A language model (LM) can score whether a skill sounds useful. It cannot infer current physical feasibility from words alone. SayCan formalizes the combination for candidate skill o:

$$\text{score}(o) \propto p_{\text{LM}}(o \mid \text{instruction}, \text{history})\, p_{\text{afford}}(\text{success} \mid b, o).$$

The first factor asks “does this step help the request?” The second asks “can the robot do it from the current belief?” If the requested mug is meaningful but unreachable, semantic probability can be high while affordance is low.

An affordance estimator may be a learned skill value, geometric feasibility check, or conservative combination. Calibrate it on attempted starts, including failures. A planner that sees only successful demonstrations will overestimate what is executable.

A cloud large language model (LLM) should emit a constrained proposal such as:

```
{
  "request_id": "mission-184-step-3",
  "skill": "follow_person",
  "subject_id": "person.bruce",
  "following_distance_m": 1.2,
  "max_speed_mps": 0.15,
  "expires_at_monotonic_ms": 8842120,
  "stop_if": {
    "track_age_ms_gt": 150,
    "clearance_m_lt": 0.35,
    "localization_std_m_gt": 0.10
  }
}
```

JavaScript Object Notation (JSON) syntax is not sufficient validation. Onboard software must reject unknown fields, wrong units, expired requests, unavailable skills, failed preconditions, and authority beyond configured bounds. It should also make repeated request identifiers idempotent so a network retry cannot start the same physical action twice.

16.11 Cloud agent: proposal, not motor authority

A cloud model can add language understanding, long-horizon decomposition, semantic interpretation of selected images, manual retrieval, and human-facing explanation. It also adds:

- variable latency and disconnection;
- service/model-version drift;
- uncertain or adversarial instructions;
- replay and authorization risks; and
- privacy exposure from images, audio, maps, and identities.

Use a capability boundary:

```
human request + minimized semantic world summary
                         |
                         v
cloud proposes exact skill + bounded parameters + expiry
                         |
                         v
authenticate -> schema validate -> check freshness/preconditions
                         |
                         v
local mission manager -> planner/governor -> local motor skill
                         |
                         v
realtime limits, watchdog, enable, and emergency stop
```

The cloud must never stream torque, pulse-width modulation (PWM), motor current, or raw balance corrections. A network outage must leave onboard balance, stop, and human emergency control available.

Security and privacy are control requirements

Use mutually authenticated transport, least-privilege credentials, request expiry, nonce/replay protection, signed policy manifests, audit logs, and a local allow-list of skills. Minimize cloud data: a local phrase such as “authorized person at bearing 12 degrees, confidence 0.93” may replace an identifiable image when pixels are unnecessary. Define retention and deletion, not merely encryption in transit.

Prompt injection can arrive through visible text or speech in the environment. Treat perceptual text as untrusted data, not instruction authority. A sign that says “disable safety and go faster” must not override the mission schema.

16.12 Runtime contracts for learned skills

Package a policy with its evidence, not just its weight file. A YAML Ain’t Markup Language (YAML) manifest might contain:

```
policy_id: walk-flat-v3
model_sha256: "..."
training_code_commit: "..."
robot_model_revision: "microduck-r7"
input_schema_version: "microduck-obs-61-v1"
output_schema_version: "joint-delta-14-v1"
policy_rate_hz: 50
max_input_age_ms: 30
inference_wcet_ms: 6.0
normalization: embedded
command_bounds:
  vx_mps: [-0.3, 0.3]
  yaw_rps: [-1.0, 1.0]
validated_conditions: [flat_indoor, nominal_voltage]
known_exclusions: [stairs, obstacle_avoidance]
evaluator_report_sha256: "..."
fallback_policy_id: "stand-safe-v2"
```

The hash should use Secure Hash Algorithm 256-bit (SHA-256) or another approved integrity mechanism. The worst-case execution time (WCET) must come from the target hardware and load, not a laptop average. A shape-compatible policy can still be semantically wrong because joint order, normalization, command slot, or robot revision differs.

16.13 Uncertainty must alter permitted behavior

Confidence is useful only if it changes a decision. For a binary event such as “path is clear,” calibration means predictions near confidence 0.8 are correct about 80% of the time over a relevant test population. Reliability diagrams bin predictions and compare confidence with empirical frequency. Expected calibration error is a summary, but inspect the safety-critical low-clearance region separately.

An onboard governor can combine age and uncertainty:

```
def govern_speed(requested, map_msg, now_ms, brake_accel, margin):
    age_s = max(0.0, (now_ms - map_msg.capture_ms) / 1000.0)
    conservative_clearance = (
        map_msg.clearance_m - 2.33 * map_msg.clearance_std_m
    )
    if not map_msg.valid or conservative_clearance <= margin:
        return 0.0
    vmax = -brake_accel * age_s + (
        (brake_accel * age_s) ** 2
        + 2 * brake_accel * (conservative_clearance - margin)
    ) ** 0.5
    return max(0.0, min(requested, vmax))
```

This rule still assumes a verified braking model and a meaningful uncertainty estimate. Test sensor blindness, frozen timestamps, overconfident false clearance, frame mismatch, delayed inference, and controller rejection. A fallback that was never fault-injected is only a design intention.

16.14 Worked JumpRover architecture

JumpRover is a wheeled-leg platform whose brain system-on-chip is partly verified while mechanics, realtime board, and physical plant are not yet accepted. That status changes the correct RL plan: architecture and interfaces can progress, but dynamics training should not pretend unknown hardware is final.

Proposed multi-rate layers

Layer	Example rate	Owns	Must not own
motor electronics	10–40 kHz	current/commutation, hard trips	semantics
realtime board	0.5–2 kHz	sensors, limits, enable, watchdog	cloud calls
stabilizer/skill	50–200 Hz	balance, contact, bounded motion	mission text
local planner	10–30 Hz	collision-aware commands	motor current
perception	5–30 Hz	tracks, map, covariance, age	actuator enable
mission manager	1–10 Hz	options, recovery, lifecycle	raw torque
cloud agent	asynchronous	semantic proposal/explanation	hard deadlines

Rates are hypotheses until measured. A brushless direct-current motor (BLDC) stage may use field-oriented control (FOC) on the realtime board. The board's microcontroller unit (MCU) must retain enable, limits, heartbeat, and safe-stop authority even if the SG2002 brain crashes.

Interface sequence

```
camera/IMU/encoders
       |
       +--> timestamped state estimator --> local belief
       |                                      |
       +--> visual perception ------------+--> local planner
                                          |
cloud proposal --> mission validator --> option command + bounds
                                          |
                                  policy/command governor
                                          |
                                  realtime board + watchdog
                                          |
                                  power stage/motors
```

Every arrow needs units, frame, rate, timestamp, sequence number, validity, timeout, and behavior on loss. The diagram is not an implementation until those contracts and tests exist.

Hardware gates before dynamics learning

Do not begin expensive locomotion training against guessed mechanics. Require:

1. accepted mass, inertia, center of mass, wheel radius, linkage geometry, and joint limits from the physical build;
2. measured command-to-torque or command-to-force behavior across speed, voltage, and temperature;
3. measured latency, jitter, saturation, dead zone, friction, backlash, and braking distance;
4. synchronized encoder and IMU logs under controlled excitation;
5. proven watchdog, disable, overcurrent, overtemperature, tilt, and communication-loss behavior; and
6. a deterministic non-learning baseline that can safely exercise the plant.

These measurements identify a simulator distribution. Domain randomization then covers credible residual uncertainty; it should not conceal absent mechanical knowledge.

Three staged learning projects

Stage A: semantic autonomy without learned dynamics. Use local mapping, geometric planning, and conservative hand-engineered drive/stabilization. Connect cloud planning only through typed options. This validates the entire authority and freshness path.

Stage B: bounded local RL skill. Train a proprioceptive stabilizer or command tracker after plant identification. Keep the planner and collision governor outside. Verify simulation, processor-in-loop, hardware-in-loop, then tethered physical trials.

Stage C: perceptive mobility research. Add height maps, visual history, or latent terrain observations. Compare against Stage B under identical held-out terrain and faults. The new policy must beat the modular baseline in more than average reward: success, collision, intervention, tail latency, energy, and recovery all matter.

Acceptance evidence for handoff

A team receiving the RL stack should get:

- versioned mechanical/electrical parameters and calibration;
- simulator and system-identification residuals;
- logged observation/action schema with replay tooling;
- policy manifest, training commit, seed/config, and dataset provenance;
- checkpoint-selection protocol and held-out evaluation report;
- timing traces on target compute;
- fault-injection and fallback results;
- physical operating envelope and known exclusions; and
- rollback image plus named release owner.

This is how Microduck's learning transfers: not by copying its 14-joint network, but by copying disciplined contracts, measurement, reward audits, staged training, artifact provenance, and sim-to-real tests.

16.15 Worked design: person following

Consider "follow Bruce to the workshop." Decompose the problem before choosing a model.

1. The cloud or onboard semantic model resolves the request to an authorized `follow_person` option and subject identifier (ID).
2. Onboard perception detects and tracks the subject, returning bearing, range, identity confidence, covariance, capture time, and track age.
3. A local planner combines that track with obstacles and produces a bounded velocity command.
4. The local locomotion policy tracks the command; a governor clips it using stopping distance and tilt limits.
5. The realtime board enforces communication timeout and motor limits.
6. Track loss, identity switch, low clearance, excessive age, or human stop causes local stop; reacquisition does not occur silently outside consent rules.

Evaluate the chain with a factorial test matrix:

- known and unseen clothing/backgrounds;
- crossing people and partial/full occlusion;
- bright, dim, backlit, and motion-blurred video;
- static, approaching, departing, and abruptly stopping subject;
- narrow passages and moving obstacles;
- injected perception delay, dropped frames, and network loss; and
- authorized versus unauthorized people.

Report false-follow rate, missed-follow rate, identity switches, range error, map age, minimum clearance, stop latency, intervention rate, and mission success. A single success percentage cannot reveal whether the robot followed the wrong person dangerously.

16.16 Exercises

1. A camera has $f_x = 500$, $c_x = 320$. A pixel at $u = 370$ has depth 2 m. Compute X in the camera frame.
2. Explain the difference between intrinsic and extrinsic calibration, and give one failure caused by each.
3. A rover moves at 0.6 m/s and its map is 250 ms old. How far has it traveled since image capture, before adding braking distance?
4. Derive the stopping-distance equation from constant-acceleration motion.
5. With mean clearance 0.8 m, standard deviation 0.1 m, and $k = 2$, what is conservative clearance? Why might the claimed tail probability be wrong?
6. Explain why a visible simulated obstacle is not necessarily an actor observation.
7. Compare modular and end-to-end obstacle avoidance for Microduck.
8. What does a linear probe establish, and what does it fail to establish?
9. Why can direct squared-error action prediction be poor for two equally valid ways around an obstacle?
10. If an action chunk has $H = 20$ at 50 Hz, how much future time does it cover? Give one benefit and one risk.
11. Explain flow-matching training and inference without using the phrase "it denoises."
12. Why can FAST use fewer tokens than independent timestep quantization for smooth robot motion?
13. Distinguish VLA imitation pretraining from online RL fine-tuning.
14. An option lasts four primitive steps with rewards $(1, 1, 0, 2)$ and $\gamma = 0.9$. Compute its internal discounted return and the discount on the next-option value.
15. Construct a case in which a language planner gives a skill high semantic probability but the affordance probability should be near zero.
16. List six checks an onboard validator must perform on a cloud skill request.
17. Design a policy manifest for one Microduck behavior, including one exclusion that prevents a semantic mismatch.
18. Propose the first JumpRover learning experiment that is valid before the final mechanics exist, and one experiment that must wait.

Continue with research literacy and capstone projects.

16.17 Folded solutions

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show worked answers to Section 16.16

1. Back-project along the horizontal axis:

$$X = Z(u - c_x)/f_x = 2(370 - 320)/500 = 0.20 \text{ m}.$$

   The point is 20 cm along the camera's positive horizontal axis under the declared convention.

2. Intrinsics describe projection inside the camera: focal lengths, principal point, and distortion. Wrong intrinsics can bend a straight wall or give incorrect lateral position. Extrinsics describe the rigid transform between camera and robot/world frames. A reversed or stale extrinsic can put a correctly detected obstacle on the wrong side of the rover.

3. Age distance is

$$d = v\tau = 0.6(0.250) = 0.15 \text{ m}.$$

   The rover traveled 15 cm before considering planner delay or braking.

4. During reaction delay, constant speed contributes $v\tau$. During braking, use $v_f^2 = v_i^2 + 2ad$ with $v_f = 0$ and $a = -a_b$, yielding $d = v^2/(2a_b)$. Add a residual margin m, giving $d_{\text{stop}} = v\tau + v^2/(2a_b) + m$.

5. The conservative clearance is 0.8 – 2(0.1) = 0.6 m. A Gaussian tail interpretation may be wrong because errors can be skewed, heavy-tailed, correlated, shifted by domain change, or miscalibrated. Check empirical coverage on relevant held-out scenes.
6. Rendering changes the pixels a human sees. An actor responds only to values included in its observation tensor or commands derived from sensors. Microduck's current 61D actor has no obstacle range, pixels, or map.
7. The modular route converts perception into safe velocity commands for the existing actor, preserves the observation contract, and enables component replay tests. End-to-end depth-to-joints may coordinate foot placement more tightly but needs a new policy, sensor simulation/randomization, runtime schema, much more data, and separate safety validation. Begin with the modular baseline so the research comparison is meaningful.
8. A linear probe shows that a property is linearly decodable from a frozen representation on the probe distribution. It does not show that the control policy uses that property, that the correlation is causal, or that it survives deployment shift. Use masking/intervention and held-out tests.
9. If demonstrations turn left and right with equal probability, squared error favors their conditional mean. The mean may command straight ahead—the only colliding action. A multimodal head, explicit route variable, or planner can preserve the alternatives.
10. The chunk covers 20/50 = 0.4 s. It reduces inference frequency and learns coordinated trajectories. It may continue executing stale intent for up to that horizon unless receding-horizon replanning or interruption is used.
11. Training samples a demonstrated chunk, a noise chunk, and an interpolation time. The model learns the velocity that transports the interpolated point along a path toward the data distribution, conditioned on observations. Inference samples noise and numerically integrates this learned velocity field to produce a continuous action chunk.
12. Smooth trajectories have correlated timesteps and most energy in a few low-frequency coefficients. A frequency transform exposes that structure, allowing compression to represent a whole chunk with fewer symbols than quantizing every component at every time independently.
13. VLA imitation fits demonstrated actions conditioned on observations and language using supervised likelihood/regression/generative losses. Online RL collects reward-bearing interaction under the current policy and changes behavior to improve expected return. A later RL-alignment phase does not retroactively make all pretraining RL.
14. The internal return is

$$R_o = 1 + 0.9(1) + 0.9^2(0) + 0.9^3(2) = 3.358.$$

    The next-option value is multiplied by $\gamma^4 = 0.6561$.
15. For "clean the spill," `pick_up_sponge` is linguistically useful, but its affordance should approach zero if the sponge is behind a locked door or absent from the map. Semantic relevance does not create reachability.
16. At minimum validate authentication/authorization, exact schema and units, request ID/replay status, expiry/freshness, skill allow-list and version, parameter bounds, current preconditions/affordance, and an available fallback. Six are requested; production should perform all of these.
17. A valid Microduck manifest declares model hash, training commit, robot revision, 61D input schema, 14D output order/scale, embedded observation normalization, 50 Hz rate, measured inference bound, command limits, evaluator report, and fallback. `known_exclusions: [obstacle_avoidance]` prevents a planner from assuming that a flat-ground velocity tracker sees obstacles.
18. Before final mechanics, validate the typed cloud-to-onboard option path in a software simulator with stale/replayed/invalid messages and prove local stop under disconnection. Training a dynamics-sensitive jump or balance policy must wait for accepted geometry, mass/inertia, actuators, realtime timing, braking, synchronized logs, and safety interlocks.

17. Research Literacy, Reproduction, and Capstone Projects

Being current in reinforcement learning does not mean collecting paper titles. It means being able to state what was tested, reproduce a meaningful slice, measure uncertainty, and know which conclusion the evidence does not support.

By the end of this chapter, you should be able to:

- turn an abstract claim into a falsifiable experiment;
- distinguish the training run, evaluation episode, and physical robot as different experimental units;
- find confounds in an apparently fair algorithm comparison;
- calculate and interpret seed-level uncertainty without pseudoreplication;
- trace one equation into source code, configuration, and logged evidence;
- preserve a reproduction artifact another person can actually exercise; and
- propose a capstone whose stopping rule and acceptance criteria exist before its results.

Research literacy is not habitual skepticism. It is disciplined curiosity: **what exact observation would make you revise the current explanation?**

17.1 A source hierarchy

Prefer sources in this order:

1. peer-reviewed paper or clearly versioned preprint;
2. official project page and repository from the authors;
3. released configuration, code, checkpoints, data, and evaluation scripts;
4. independent reproductions that disclose deviations; and
5. summaries, talks, or social posts only as pointers.

An arXiv identifier proves a document exists; it does not prove peer review, correctness, reproducibility, or relevance to your robot. An open repository proves some artifact is visible; it does not prove it matches the paper result.

Publication status is evidence metadata

A document can be an unreviewed preprint, a revised preprint, a workshop paper, an accepted conference paper, a journal article, or a withdrawn/corrected work. Record status and version rather than using "paper" as if all stages were equal. A later version may change the method, tasks, or conclusions while retaining the same arXiv identifier.

For fast-moving robot foundation policies, make a dated statement:

```
Verified 2026-09-01:
- paper version: arXiv v3, dated YYYY-MM-DD
- official repository: organization/project at commit <hash>
- checkpoint: exact model identifier and digest
- evaluation claim used: table/figure/section
- publication status: preprint / venue + year
```

"State of the art" is not a permanent property stored in a model name. It is a comparison under a protocol at a date. If data, robot, action space, or test distribution differs, reproduce the comparison before carrying the label into your project.

Artifact openness has levels

Do not collapse these statements:

- source code is visible;

- training code runs;
- exact configuration and dependencies are present;
- weights are downloadable;
- training data and mixture are accessible;
- the official evaluator reproduces a reported number; and
- a new team independently obtains a compatible result.

Each is stronger than the previous one in a different way. A code license can also differ from a base-model or dataset license. Record all three before calling a policy reusable.

The Association for Computing Machinery artifact review and badging policy separates artifact availability, artifact evaluation, and result validation. This is a useful mental model even when a robotics venue does not award badges.

17.2 Read a paper in three passes

Pass 1: scope

Read title, abstract, figures, evaluation tables, limitations, and conclusion. Write one sentence:

```
The paper claims X on task/benchmark Y under protocol Z.
```

If you cannot fill Y and Z, do not repeat X yet.

Add the population and comparator:

```
Under protocol Z, method X improved metric M over baseline B
on evaluated population Y; the evidence does not yet cover U.
```

The final clause *U* is the **exclusion boundary**. For example: "The paper reports manipulation success on two arms; it does not evaluate dynamic biped balance." Writing the boundary prevents an impressive adjacent result from silently becoming evidence for your robot.

Pass 2: mechanism

Read method and algorithm. Identify:

- inputs available during training and deployment;
- outputs/action representation;
- objective and constraint;
- data collection policy;
- model architecture and memory;
- simulator/hardware and rates; and
- differences from the strongest baseline.

Trace the data flow, not just named blocks:

```
raw sensor -> preprocessing -> observation schema -> model -> action adapter
           -> safety/controller -> plant -> metric logger
```

Ask which block owns normalization, history, privileged information, action clipping, and termination. A paper diagram may omit these because they are implementation details; in sim-to-real work they can determine the result.

Pass 3: evidence

Read experiment details, appendices, and repository configuration. Record:

- tasks and split;
- environment steps, data size, and compute;
- number of seeds and hardware trials;
- metric aggregation and uncertainty;
- hyperparameter tuning budget;
- ablations;

- failure examples; and
- artifact availability.

Now write three sentences:

1. **Support:** "The experiment supports ..."
2. **Non-support:** "It does not establish ..."
3. **Discriminator:** "The next experiment that separates explanations A and B is ..."

This forces reading to produce a testable project decision instead of a longer summary.

Worked claim dissection

Suppose an abstract says, "Our policy robustly transfers to real robots." A research note should unpack it:

Word	Question needed before reuse
policy	actor only, estimator plus actor, or full system?
robustly	mean success, worst case, intervention, or no falls?
transfers	zero policy updates, or calibration and real adaptation?
real robots	how many units, revisions, trials, surfaces, and operators?

If the study used one robot, 20 hand-reset trials, and nominal voltage, a fair restatement is valuable but narrower. Precision is not hostility to the paper; it is how its evidence becomes usable.

17.3 Claim taxonomy

Separate these claim types:

- **theoretical**: follows from stated assumptions and proof;
- **algorithmic**: mechanism is defined and implemented;
- **benchmark empirical**: result holds on evaluated benchmark/protocol;
- **sim-to-sim**: transfers between simulators;
- **sim-to-real**: evaluated on stated hardware conditions;
- **generalization**: evaluated on a specified held-out distribution;
- **scaling**: behavior changes with data/model/compute range;
- **systems**: latency, throughput, reliability, or resource result.

A benchmark empirical claim is not a theoretical guarantee. A ten-minute hardware video is not a reliability distribution. "Zero-shot sim-to-real" means no real-world policy fine-tuning under that setup; it does not mean no hardware calibration, system identification, or engineering.

Quantifiers are part of the claim

Compare:

- "the policy avoided obstacles";
- "the policy avoided all 12 training layouts";
- "across 200 preregistered held-out trials, it avoided contact in 184"; and
- "its lower 95% confidence bound exceeded the accepted baseline by 5 percentage points."

The first can describe one video. The later statements expose population, count, uncertainty, and decision rule. Prefer words such as **on the evaluated conditions**, **for the sampled seeds**, and **within the measured operating envelope**. Avoid "always," "safe," "general," and "solved" unless the evidence supports their unusually broad quantifiers.

Turn a question into variables

"Does history help?" is not yet an experiment. Define:

- **treatment:** observation history length 4 instead of 1;
- **outcomes:** held-out success, falls, energy, and inference time;
- **controlled factors:** simulator, reward, network budget, optimizer, steps, seeds, and evaluator;

- **population:** declared command/physics distribution;
- **unit:** independently trained seed, not each timestep;
- **hypothesis:** history improves partial observability enough to offset added optimization and latency cost; and
- **decision rule:** a predeclared effect and uncertainty threshold.

Draw a small causal sketch before training:

```
history length ---> state information ---> success
       |                    |
       +--> model size -----+--> optimization
       +--> inference time ------> stale action ---> success
```

If increasing history also increases parameter count and latency, success cannot automatically be attributed to memory. Match capacity, measure timing, or explicitly label those pathways as part of the treatment.

17.4 Baselines answer "better than what?"

A useful baseline should be:

- relevant to the task and information setting;
- implemented competently;
- given a comparable tuning/compute budget;
- evaluated with the same metric and held-out cases; and
- simple enough to reveal whether complexity is necessary.

Robot baselines can include:

- zero/random policy for plumbing only;
- hand-designed or proportional–derivative (PD) controller;
- Model Predictive Control (MPC) or state machine;
- behavior cloning;
- standard Proximal Policy Optimization (PPO) or Soft Actor-Critic (SAC) implementation;
- prior method with authors' configuration; and
- an oracle using privileged information, clearly labeled as an upper bound.

A new reinforcement learning (RL) controller that has not beaten an accepted scripted balance baseline is not ready for hardware because its training reward is high.

Fairness is more than equal environment steps

Algorithm A and B can receive equal environment interactions yet unequal:

- gradient updates and replay reuse;
- network parameters and observation history;
- wall time and graphics processing unit memory;
- expert demonstrations or offline pretraining;
- privileged critic inputs;
- hyperparameter search trials; and
- checkpoint-selection opportunities.

There is no single universally fair budget. Report at least environment steps, wall-clock time, compute hardware, gradient updates, and external data. Then say which resource the comparison holds fixed and why that matches the intended deployment question.

Use two baseline roles:

1. a **sanity baseline** establishes that task wiring and metric are meaningful;
2. a **competitive baseline** asks whether the new method adds value beyond a credible existing choice.

An oracle with privileged terrain may expose the cost of partial observability, but it is not deployable. A hand controller may be deployable but incapable of the full task. Label each role instead of ordering every method on one axis.

Baseline integrity checklist

Before accepting a comparison, verify that every method:

- sees the declared observation and no accidental privilege;
- uses the same action bounds and control frequency;
- encounters the same reset, termination, and evaluation seeds;
- has a competent hyperparameter source or tuning allocation;
- is evaluated from a frozen checkpoint, without online test adaptation unless that adaptation is the treatment; and
- is allowed to fail visibly rather than having failures filtered from video.

17.5 Ablation studies identify causes

An **ablation** removes or changes one component while holding other factors fixed.

Examples:

- Better Actuator Models (BAM) actuator model on versus ideal actuator;
- observation history 1 versus 4;
- privileged critic on versus off;
- action-rate curriculum versus full penalty from iteration 0;
- calibrated randomization versus arbitrary wide randomization;
- perception confidence supplied versus omitted.

An ablation needs multiple seeds when training is variable. Comparing one lucky run with one unlucky run does not identify a cause.

One-at-a-time ablations can miss interactions

Suppose actuator realism A and domain randomization D each have two levels. A 2×2 factorial design measures:

Actuator realism	Randomization	Mean held-out success
off	off	measure
on	off	measure
off	on	measure
on	on	measure

The effect of realism when randomization is off is

$$\Delta_{A|D=0} = \bar{y}_{1,0} - \bar{y}_{0,0},$$

and when randomization is on it is

$$\Delta_{A|D=1} = \bar{y}_{1,1} - \bar{y}_{0,1}.$$

Their difference is an interaction:

$$I_{A,D} = \Delta_{A|D=1} - \Delta_{A|D=0}.$$

If realism helps only with calibrated randomization, a one-at-a-time study from the “both off” corner may reach the wrong design conclusion. Factorial studies cost more; use them for interactions your mechanism actually predicts.

Mechanistic and deletion ablations differ

Deleting a module asks whether the complete system needs it. A mechanistic ablation changes the quantity the explanation depends on. If a paper says history helps because it estimates velocity, useful tests include:

- history length while matching model capacity;
- explicit velocity with no history;
- shuffled or time-reversed history;
- sensor delay swept independently; and
- a probe for velocity plus a control intervention.

These distinguish "more parameters" from "temporal information." A good ablation attacks the causal story, not merely a software checkbox.

17.6 Random seeds and uncertainty

A seed initializes stochastic components: network weights, environment resets, commands, minibatch order, and sometimes graphics processing unit (GPU) kernels. Deep RL can produce meaningfully different outcomes across seeds.

For returns $x_1, \ldots, x_n$, the sample mean is

$$\bar{x} = \frac{1}{n}\sum_i x_i.$$

The sample standard deviation is

$$s = \sqrt{\frac{1}{n-1}\sum_i (x_i - \bar{x})^2}.$$

Neither says everything about skewed failures. Also report median, quantiles, success rate, and named failure categories.

First identify the experimental unit

If one trained policy runs 1,000 evaluation episodes, those episodes estimate that policy's conditional performance. They are not 1,000 independent training runs. Treating every episode as an independent algorithm replicate is **pseudoreplication** and produces falsely narrow uncertainty.

A useful hierarchy is:

```
algorithm/configuration
  -> independent training seed
       -> checkpoint selected by fixed rule
            -> evaluation scenario
                 -> repeated episode / hardware trial
```

Variation can arise at every level. Report seed-to-seed variability and, for physical robots, unit/day/operator variation when those are in the intended population. Ten retries on the same robot in the same marked start pose do not establish reliability across builds and rooms.

The standard error of a seed mean is

$$\mathrm{SE}(\bar{x}) = \frac{s}{\sqrt{n}},$$

but a normal interval is fragile for few, skewed seeds. It also answers only about the mean under the sampled training process, not the probability of a rare catastrophic hardware outcome.

Pair seeds and scenarios when possible

If methods A and B use the same environment seeds and evaluation scenarios, analyze paired differences

$$d_i = x_i^{(A)} - x_i^{(B)}.$$

Common scenario difficulty then cancels. Report the distribution and interval of d_i, not two unrelated error bars. Pairing is valid only when the pairing is meaningful and does not couple training in a way that changes the method.

For physical tests, construct a balanced trial order so battery temperature, floor wear, and operator learning do not all favor the method tested second. Randomize or counterbalance order and log it.

Success rates are binomial only under assumptions

For k successes in n independent Bernoulli trials,

$$\hat{p} = k/n.$$

A Wilson interval behaves better than the simple $\hat{p} \pm 1.96\sqrt{\hat{p}(1-\hat{p})/n}$ near 0, 1, or small n. But neither fixes correlated trials, changing conditions, or post-selected retries. Define what counts as a trial, success, intervention, and excluded run before testing.

Report numerator and denominator—"18/20"—not only 90%. State whether a human reset, safety catch, or retry converts an attempt into failure or exclusion.

Bootstrap confidence interval intuition

To estimate uncertainty without assuming a normal distribution:

1. sample n results with replacement from the observed n results;
2. compute the chosen statistic;
3. repeat many times; and
4. take appropriate percentiles of the bootstrap statistics.

This quantifies sampling uncertainty given the observed runs. With only three seeds, the interval will be uncertain because the evidence is genuinely weak.

For a paired comparison, resample paired seed/scenario records together. For a hierarchical robot study, a **stratified bootstrap** may first resample robots or seeds, then episodes within them. Resampling 10,000 correlated timesteps as if independent manufactures certainty.

Minimal seed-level code is:

```
import random

def bootstrap_mean_interval(seed_scores, repeats=20_000, rng_seed=0):
    rng = random.Random(rng_seed)
    n = len(seed_scores)
    estimates = []
    for _ in range(repeats):
        sample = [rng.choice(seed_scores) for _ in range(n)]
        estimates.append(sum(sample) / n)
    estimates.sort()
    return estimates[int(0.025 * repeats)], estimates[int(0.975 * repeats)]
```

The percentile bootstrap is pedagogically simple, not universally optimal. With very few seeds it cannot reveal modes never sampled.

The RLiable evaluation paper explains why point estimates from a few RL runs can mislead and proposes interval estimates, performance profiles, and robust aggregate metrics.

The interquartile mean (IQM) sorts scores and averages the middle 50%. It is less dominated by extreme runs than the mean while using more information than the median. Across multiple tasks, normalize only with defensible reference scores; arbitrary normalization can reverse comparisons.

A performance profile asks, for threshold τ, what fraction of task-run scores exceed it:

$$F(\tau) = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}[x_i > \tau].$$

One curve reveals whether improvement is broad or concentrated in easy tasks. The official `rliable` repository is now archived, so pin its commit if using it and keep a small independent statistic check in your analysis tests.

Effect size comes before significance

Ask whether the difference matters physically. A 0.5% mean reward gain may be statistically detectable yet irrelevant; one fewer fall per 100 missions may be operationally important. Predeclare a **smallest effect of practical interest** in task units: success points, centimeters of clearance, joules, or interventions per hour.

Power calculations require an expected effect and variability. If those are unknown, run a labelled pilot to estimate logistics, not to make a definitive claim. Do not repeatedly inspect results and stop when a favorable interval first appears unless a valid sequential design was declared.

17.7 Best checkpoint bias

If you evaluate 100 checkpoints on the same test conditions and report only the best, you have tuned on the test set. Similarly, choosing the best of many seeds and hiding the rest estimates luck, not expected performance.

Define selection before final testing:

```
training set: optimizer updates
validation set: checkpoint/hyperparameter selection
test set: one final locked evaluation
hardware acceptance: separately defined safety/performance protocol
```

The winner's curse

Suppose every checkpoint has the same true performance μ, but measured validation score is

$$\hat{J}_j = \mu + \epsilon_j.$$

Selecting $j^* = \arg\max_j \hat{J}_j$ also selects a checkpoint with unusually positive noise. Its validation score is biased upward even though the individual measurements were unbiased. More checkpoints, seeds, and hyperparameter trials create more opportunities to select luck.

The remedy is not to ban checkpoint selection. Define it on validation data, then estimate the chosen procedure once on a locked test set. The unit being evaluated is the entire procedure—training, selection, and adaptation—not an imaginary checkpoint chosen with test knowledge.

Multiple questions create false discoveries

If you test many metrics, tasks, seeds, and subgroup cuts, some favorable result can occur by chance. Mark one or a small number of primary outcomes before running. Treat exploratory findings as hypotheses for a new confirmatory run. Where formal hypothesis tests are appropriate, disclose the family and apply a multiple-comparison method; do not hide the unsuccessful comparisons.

Robot learning often benefits more from intervals, effect sizes, failure clusters, and replication than a ritual threshold on one p-value. The key is that analysis choices cannot secretly depend on which story looks best.

17.8 Reproducibility levels

Repeatability

Same team, code, data, environment, and procedure obtains consistent results.

Reproducibility

Another team uses provided artifacts/procedure and obtains a compatible result.

Replicability

Another team independently implements the idea and obtains evidence supporting the claim.

Terminology varies by field, so define what you mean. In all cases preserve:

- source commit and dirty diff;
- lockfile/container and driver/NVIDIA Compute Unified Device Architecture (CUDA) versions;
- exact resolved environment/agent configuration;
- seeds and command lines;
- raw and aggregated metrics;
- checkpoints and model hashes;
- evaluator source/version;
- rollout video and failure labels; and
- hardware/calibration revisions.

The definitions above follow the current Association for Computing Machinery convention: repeatability is same team/setup, reproducibility is different team with the same setup/artifacts, and replicability uses an independently developed setup. State the convention because older literature sometimes swaps the last two words.

Provenance is a directed graph

A policy result depends on more than one commit:

```
source commit + dirty diff + dependency lock + simulator asset revision
          + resolved config + seed + dataset/checkpoint inputs
                                |
                                v
                          training log
                                |
                checkpoint-selection rule
                                |
                                v
                    exported deployment artifact
                                |
                 evaluator + scenario manifest
                                |
                                v
                       report + raw trials
```

Hash large immutable artifacts and record human-readable identifiers. A hash proves byte identity, not correctness. Preserve the conversion program that made an Open Neural Network Exchange artifact, including observation normalization, because a checkpoint hash alone cannot reconstruct deployment.

A runnable artifact needs an outer and inner loop

The **outer loop** creates the environment, downloads/verifies assets, and runs a smoke test. The **inner loop** reproduces one table/figure from existing logs without retraining. A reviewer with limited compute should be able to exercise the inner loop and verify artifact consistency before attempting the expensive outer run.

At minimum include:

```
README with exact commands and expected output
LICENSES and data/model usage conditions
dependency lock or immutable container recipe
small test fixture and checksum
resolved experiment configurations
raw per-seed/per-trial data
```

```
analysis script that regenerates the reported table
known deviations, failures, and compute estimate
```

The machine-learning reproducibility program report documents why checklists, code submission, and independent reproduction are complementary. A completed checklist is evidence of disclosure, not proof that the result is true.

The classic Deep Reinforcement Learning That Matters paper documents sensitivity to implementation and experimental choices. The lesson remains current: algorithm labels alone do not define an experiment.

17.9 Reproduction card template

```
# Reproduction: <paper/result>

## Claim being tested
One scoped sentence.

## Primary sources
Paper version, official repository commit/release, model/data identifiers.

## Original protocol
Task, observations, actions, data, algorithm, compute, seeds, metrics.

## Our deviations
Every hardware/software/config difference and expected consequence.

## Success criterion chosen before running
Numerical metric, uncertainty, and qualitative failure boundary.

## Baselines and ablations
What is held fixed and what changes.

## Results
All seeds/trials, not only best; confidence intervals and failure categories.

## Interpretation
What evidence supports, does not support, and next discriminating experiment.

## Artifacts
Commits, lockfiles, commands, configs, logs, checkpoints, videos, hashes.
```

Claim-to-evidence ledger

Maintain a ledger while reading and experimenting:

Claim identifier	Evidence	Assumptions	Status	Next discriminator
C1: history reduces falls	seeds, test set, interval	matched latency	open	delay sweep
C2: policy fits target	replay and timing log	normalizer exact	supported	hardware gate

Use statuses such as `proposed`, `implemented`, `smoke-tested`, `supported in simulation`, `supported on hardware`, `contradicted`, and `inconclusive`. “Implemented” is not evidence of performance. “No significant difference” is not evidence of equivalence unless the interval excludes effects that matter.

Trace an equation into code

For one central method equation, make a four-column trace:

Mathematical symbol	Tensor/config	Source location	Logged check
ϵ clip	`clip_param`	loss function	clip fraction
γ	`gamma`	return estimator	return fixture

Mathematical symbol	Tensor/config	Source location	Logged check
r_t	reward sum	environment manager	per-term mass
a_t	actor output	action adapter	min/max/histogram

Then answer:

1. Does the implementation use the same sign, reduction, and normalization as the equation?
2. Is a symbol scheduled or mutated during training?
3. Does the deployment path add filtering, clipping, or coordinate conversion?
4. Which unit test or log would expose a mismatch?

A paper may describe one policy-gradient update while the code additionally normalizes advantages, clips value loss, schedules learning rate, and limits gradient norm. Those additions do not automatically invalidate the paper, but they are part of the reproducible algorithm.

A cheap reproduction ladder

Do not begin by spending the full paper budget. Climb:

1. **Static audit:** schemas, shapes, signs, dependencies, and config resolution.
2. **Unit analogy:** a tiny script that demonstrates the central equation.
3. **Released evaluation:** official checkpoint with official evaluator.
4. **Smoke train:** short run proving the update and artifacts execute.
5. **Small matched study:** enough seeds to expose gross effects.
6. **Full reproduction:** declared budget and protocol.
7. **Extension:** only after the reproduction boundary is understood.

At each rung define expected output and a stop condition. This saves compute while making negative results interpretable.

Rules shared by every capstone

Before the first result exists, commit a short protocol containing:

- question, mechanism, primary outcome, and practical effect threshold;
- baseline, treatment, controlled factors, and known confounds;
- experimental unit, seeds/scenarios/trials, and selection rule;
- compute/hardware budget and early-stop conditions;
- failure/intervention definitions and safety authority;
- artifact paths, licenses, and expected reproducibility level; and
- what result would falsify the favored explanation.

A capstone can pass with a negative result. It cannot pass by changing its goal after seeing an attractive video.

17.10 Capstone A: tabular foundations

Goal: show you understand learning before deep networks.

1. Modify `examples/tabular_q_learning.py` to implement State–Action–Reward–State–Action (SARSA).
2. Compare SARSA and Q-learning in a grid with a costly cliff.
3. Sweep ϵ and three seeds.
4. Plot or tabulate return, cliff entries, and convergence episodes.
5. Explain why on-policy exploration can produce a safer route during learning.

Deliverable: two-page report plus runnable code and raw seed results.

Acceptance gate: a fresh clone runs both algorithms, regenerates the table, and shows paired seed results. The discussion must separate learning return from cliff-entry risk; “SARSA is safer” is too broad if only one exploration schedule was tested.

17.11 Capstone B: PPO mechanism study

Goal: connect the clipped objective to observed optimization.

1. Use `examples/ppo_clip_demo.py` to map advantage sign and probability ratio to the surrogate objective.
2. Add cases at ratios 0.5 through 1.5.
3. Predict the flat regions before running.
4. In a small standard control environment, compare two clip values while holding seeds and budget fixed.
5. Record approximate Kullback–Leibler (KL) divergence, entropy, return, and update stability.

Deliverable: reproduction card explaining what clipping does and does not guarantee.

Acceptance gate: the report predicts the clipped piecewise objective before showing optimizer results, records all seeds, and does not describe clipping as a hard Kullback–Leibler constraint. At least one case should show that network updates can move the aggregate policy farther than an individual sample's flat surrogate suggests.

17.12 Capstone C: reproduce the Microduck pipeline

Goal: demonstrate end-to-end robot-RL understanding without claiming a five-iteration policy is trained.

1. run central processing unit (CPU) tests and task registry;
2. run a 64-environment, five-iteration smoke train;
3. inspect resolved environment and agent configuration;
4. play zero agent and one available checkpoint;
5. classify the observation, action, command, and reward terms;
6. export via the official normalizer-baking path;
7. inspect Open Neural Network Exchange (ONNX) shape and run CPU deployment rehearsal; and
8. state why random-looking viewer direction is command sampling, why obstacles are not avoided, and why body-pose intent is not trained by the main recipe.

Deliverable: exact commands, log/artifact paths, one annotated rollout, and a contract diagram.

Acceptance gate: start from a clean dependency sync, run the central processing unit tests, preserve the resolved configuration, and label the five-iteration run only as a smoke test. The exported artifact must pass a one-step observation/action comparison against the simulation path. Record any missing checkpoint or external credential as a scoped limitation rather than substituting an unrelated model.

17.13 Capstone D: modern locomotion ablation

Choose one:

- actuator calibration versus randomization width;
- privileged critic versus symmetric actor/critic information;
- observation history versus feed-forward policy;
- PPO versus SAC at matched environment steps and wall time; or
- existing planner + velocity policy versus perceptive end-to-end policy.

Before running:

1. identify one primary paper and official codebase;
2. freeze the baseline contract;
3. define at least three training seeds;
4. define held-out physics/commands;
5. define human-visible failure categories; and
6. estimate compute and stopping rule.

Deliverable: a mini-paper with negative results preserved.

Acceptance gate: the treatment changes one declared factor or reports an explicit factorial interaction. Evaluate a frozen selection procedure on locked conditions, include timing/resource cost, and publish per-seed raw values. A conclusion such as "no evidence at this budget" is valid; "methods are equal" requires an equivalence margin and adequate evidence.

17.14 Capstone E: Jump Rover staged handoff

This capstone starts with evidence, not RL installation.

Phase 1: hardware truth

- finish and measure mechanics;
- validate motor/servo sign, range, current, and thermal behavior;
- validate realtime watchdog, limits, stop, and communication;
- accept a tethered classical/scripted balance/drive baseline.

Phase 2: digital twin

- record mass, inertia, geometry, contacts, actuator response, delay;
- create simulator and four-action environment;
- replay hardware excitation in simulation;
- define held-out identification trials.

Phase 3: learning

- train bounded residual or command policy in simulation;
- compare against the frozen baseline;
- export a versioned model with schema and worst-case execution time (WCET);
- perform sim-to-sim, processor-in-loop, bench, tether, then floor gates.

Phase 4: autonomy

- onboard perception produces confidence-bearing world state;
- local planner/behavior tree selects exact skills;
- cloud agent proposes semantic plans only;
- realtime microcontroller unit (MCU) retains motor safety and network-loss response.

Deliverable: signed gate evidence and rollback artifact at every phase. The project-specific handoff document in the Jump Rover repository contains the current interfaces and blockers.

Acceptance gate: no phase inherits a green status from a presentation or simulation video. Each requirement points to a dated measurement, test log, revision, owner, and expiration/retest condition. A failed realtime or mechanical gate blocks dynamics-sensitive RL but does not block software-only schema, replay, cloud-timeout, and mission-validation tests.

17.15 Threats to validity and research ethics

Before interpreting a result, audit five kinds of validity.

Internal validity: did the treatment cause the change?

Confounds, inconsistent tuning, changing simulator assets, lucky selection, and different data can explain an apparent improvement. A controlled ablation, paired conditions, and preserved provenance strengthen internal validity.

Construct validity: did the metric represent the goal?

Training reward may rise while the maneuver fails. “Upright” based only on body height can count a tilted robot. “No collision” can ignore a human safety catch. Define observable success and failure from the physical intent, then test the metric against adversarial examples and human-labelled videos.

External validity: where should the result generalize?

A policy tested on one simulator, robot, floor, lighting setup, or operator has not established performance outside that population. Name the target population, sample held-out factors from it, and avoid claiming beyond them. Generalization to novel textures is not generalization to novel dynamics.

Statistical-conclusion validity: is the evidence strong enough?

Few seeds, correlated trials, unstable metrics, multiple selection, and unreported exclusions can make the estimated effect unreliable. Show raw points, intervals, denominators, selection, and sensitivity to reasonable aggregation choices.

Systems validity: was the deployable system evaluated?

A graphics processing unit benchmark with preloaded images does not establish camera-to-actuator latency. A simulation actor with privileged state does not establish onboard perception. Evaluate the exported artifact, preprocessing, normalizer, action adapter, queue, safety governor, and target compute as one timed path.

Ethics and safety affect the dataset

Stopping unsafe trials protects people and hardware, but it also censors what would have happened. Predefine intervention criteria and count every intervention as an outcome; never delete it as an "incomplete episode." Obtain consent and define access/retention for identifiable camera, audio, and operator data. Report environmental and financial compute cost when it affects who can reproduce the work.

A **preregistration** is a time-stamped protocol committed before results. It does not freeze all scientific thought; it separates confirmatory analysis from later exploration. Amendments are allowed when disclosed with time and reason.

17.16 A weekly research-study routine

For one paper per week:

1. Monday: scope pass and claim sentence.
2. Tuesday: derive one central equation in your own words.
3. Wednesday: inspect official code/config corresponding to that equation.
4. Thursday: run one tiny reproduction or dependency-free analogy.
5. Friday: write evidence, limitations, and one project-relevant experiment.

Reading ten abstracts is less useful than tracing one claim through equation, implementation, configuration, and result.

Add a sixth step: preserve the paper version, repository commit, and dated claim ledger. On the next week, begin by checking whether a new version changed the result you relied on.

17.17 Final self-assessment

You are ready to begin independent robot-RL research when you can:

- reject an impossible task definition before training;
- derive a Bellman or policy-gradient update and implement a small version;
- justify algorithm choice from interaction/data constraints;
- separate simulator truth, actor observation, critic privilege, and runtime sensor data;
- design baselines and ablations before seeing results;
- report uncertainty and failure clusters;
- preserve a deployable observation/action/timing contract; and
- say "the evidence does not establish that" without losing enthusiasm for the next experiment.

17.18 Exercises

1. Classify this claim and narrow it: "Our learned controller is robust because it succeeds in 47 of 50 trials on one robot and one floor."
2. One trained policy is evaluated for 500 episodes. Another seed is never trained. What is the experimental unit for an algorithm-training claim, and what do the 500 episodes estimate?

3. Explain why treating 24 rollout steps from each of 4,096 parallel environments as 98,304 independent algorithm replicates is pseudoreplication.
4. Paired held-out scores for methods A and B are $A = [0.8, 0.6, 0.9]$ and $B = [0.7, 0.65, 0.75]$. Compute paired differences and their mean. What does three pairs fail to establish?
5. Explain the winner's curse when selecting the best of 100 checkpoints.
6. In a factorial ablation, outcomes for $(A, D) = (0, 0), (1, 0), (0, 1), (1, 1)$ are 0.40, 0.50, 0.55, 0.80. Compute both conditional effects of A and their interaction.
7. Why should a report say "18/20 successes" rather than only "90% success"? Name two trial-dependence problems that still invalidate a binomial model.
8. Trace Proximal Policy Optimization clip parameter ϵ from equation to configuration, source computation, and one logged diagnostic.
9. Method A and B use equal environment steps, but B performs ten times as many gradient updates and uses demonstrations. Is the comparison invalid? Give a precise way to report it.
10. Draft a primary outcome and falsification rule for the claim "adding a depth map teaches Microduck obstacle avoidance."
11. A repository releases a checkpoint but omits observation normalization and the evaluator. Which openness claims are justified, and what cannot yet be reproduced?
12. A new 2026 preprint claims state-of-the-art vision-language-action control. List the minimum checks before adding that claim to a JumpRover decision.

Continue with the detailed paper seminars, then use Chapter 20 for terminology and worked checks before returning to the capstone whose deliverable you cannot yet produce.

17.19 Folded capstone reference checks

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show reference mechanisms and completion criteria for Capstones A–E

These are not single "correct" experiment results. They are reference checks that make an incomplete or unsupported submission visible.

Capstone A — SARSA. The on-policy target uses the action actually selected by the exploratory policy at the next state:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\,[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]\,.$$

A minimal update helper is:

```
def sarsa_update(q, s, a, reward, next_s, next_a, alpha, gamma):
    target = reward + gamma * q[next_s][next_a]
    q[s][a] += alpha * (target - q[s][a])
```

Q-learning instead uses `max(q[next_s])`. Near a costly cliff, SARSA learns about the consequences of its continuing exploration, so it can prefer a route with more clearance while training. Completion requires all requested seeds and cliff-entry counts; one attractive trajectory is not the result.

Capstone B — PPO clipping. For positive advantage, the surrogate becomes flat above ratio $1 + \epsilon$; for negative advantage it becomes flat below $1 - \epsilon$. Clipping the objective does not impose a hard bound on the final network update because all samples share parameters and the optimizer takes multiple minibatch steps. A complete report shows the predicted piecewise curve, measured approximate KL and entropy, both clip settings under the same budget, and every seed.

Capstone C — Microduck pipeline. A passing submission distinguishes a smoke test from learned locomotion, records the actor input as 61 values and the action as 14 values, uses the official export path that bakes observation normalization into ONNX, and rehearses that artifact on CPU. It explicitly states that the main actor has no obstacle sensor or global-goal input and that its body-pose reward has zero weight. Missing any one of configuration, checkpoint, normalizer, command ordering, or timing leaves the deployment contract unproved.

Capstone D — locomotion ablation. The causal comparison changes exactly one named factor, holds training and evaluation budgets fixed, uses at least three seeds, and reports held-out conditions plus failure categories. If PPO and SAC receive different wall time, replay warm-up, observations, or reward code, label those as confounds rather than attributing the entire difference to the algorithm name. A negative result with complete artifacts passes; a best-seed claim without uncertainty does not.

Capstone E — Jump Rover. Each phase needs a measurable exit gate. Examples are a recorded watchdog stop bound and thermal envelope for hardware truth; held-out excitation error for the digital twin; matched-baseline success and processor-in-loop worst-case execution time for learning; and demonstrated safe behavior under cloud timeout, stale perception, and invalid skill identifiers (IDs) for autonomy. A cloud-generated plan is never evidence that the real-time motor safety layer works. Every gate must name its artifact and rollback version.

17.20 Folded exercise solutions

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show worked answers to Section 17.18

1. It is a benchmark/hardware empirical claim on a narrow evaluated population, not general robustness. A precise version is: "On one robot revision and one floor under the stated 50-trial protocol, the controller succeeded in 47/50 attempts." Report intervention/failure definitions and an interval; robustness to other robots, surfaces, voltage, and disturbances remains open.
2. The independently trained seed is the unit for variation in the training procedure. The 500 episodes estimate that one frozen policy's conditional performance over sampled evaluation cases. They improve precision about that policy but do not measure how often retraining produces a good policy.
3. Parallel steps share one policy, optimizer history, simulator code, reward, and often correlated initializations. They are samples used to make one update, not independent re-runs of the algorithm. Seed-level conclusions require independently initialized training procedures.
4. Paired differences $A - B$ are $[0.10, -0.05, 0.15]$, with mean $(0.10 - 0.05 + 0.15)/3 \approx 0.0667$. Three pairs show the observed direction but provide weak uncertainty coverage and cannot establish robustness to unsampled training seeds, scenarios, or physical conditions.
5. Every validation score contains noise. Taking the maximum selects both good underlying performance and unusually favorable noise, so the selected validation score overestimates future performance. Select by a fixed validation rule, then evaluate the chosen procedure once on locked test data.
6. With $D = 0$, the effect of A is $0.50 - 0.40 = 0.10$. With $D = 1$, it is $0.80 - 0.55 = 0.25$. The interaction is $0.25 - 0.10 = 0.15$: actuator realism has a larger observed effect when randomization is on.
7. The fraction exposes sample size; 90% could mean 9/10 or 900/1,000 and those have different uncertainty. Dependence arises if repeated trials share an overheated battery/robot or if nearly identical reset scenes make outcomes correlated. Adaptive retries and changing conditions are additional issues.
8. In the equation, ϵ defines the ratio clip range $[1 - \epsilon, 1 + \epsilon]$. In configuration it may be `clip_param`; trace that field into the surrogate-loss `clamp`/`clip` call. Log clip fraction and approximate Kullback–Leibler divergence to check whether updates frequently hit the flat region and how far the aggregate policy moves.
9. It is a valid comparison of two complete procedures at equal interaction budget if the extra updates and demonstrations are disclosed as part of B. It is not a clean claim that the algorithmic update alone caused the gain. Report environment steps, gradient updates, wall time, compute, and external demonstration count; add matched-data/update ablations if that causal claim matters.
10. One primary outcome could be collision-free goal completion over a locked set of unseen obstacle geometries, with all attempts and interventions counted. Falsify the favored mechanism if the depth policy does not improve paired collision-free completion over the planner/velocity baseline

by the predeclared practical margin, or if improvement disappears when texture is changed while geometry is fixed. Also measure age and stopping clearance.

11. It justifies "checkpoint bytes are available," subject to working access and license. It does not establish an exercisable deployment artifact, exact inference behavior, or reproduction of reported metrics. Recover the normalizer/action adapter and official evaluation protocol before comparison.
12. Verify paper version/status/date; official author repository and commit; code, weights, data mixture, evaluator, and licenses; observation/action schema and embodiments; baselines, task splits, trials/seeds, uncertainty, and tuning budget; target compute latency/memory; and whether the reported protocol overlaps JumpRover's task. Label an unreproduced preprint claim as provisional and compare it with a small matched baseline.

18. Detailed Paper Seminars: Impactful Robot-Intelligence Research

This chapter is a guided reading course, not a list of fashionable titles. The selected works introduced ideas that materially shaped open robot-learning practice, provided unusually useful experimental evidence, or opened a major current direction. They span locomotion, world models, large-scale real-robot reinforcement learning (RL), imitation, cross-embodiment data, foundation policies, and language-guided planning.

"Most impactful" is necessarily a judgment. The selection criteria are:

- the work changed how important robot-learning problems are formulated;
- the central mechanism can be explained and tested;
- the evidence is substantial and scoped clearly;
- a primary paper and preferably official artifacts exist; and
- the idea teaches something transferable to Microduck, Jump Rover, or another real robot.

For each seminar, read the paper itself after this guide. This chapter explains the map; it does not replace methods, appendices, or code.

18.1 Seminar 1 — Massively parallel Proximal Policy Optimization (PPO) for locomotion

Primary work

- Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning (2021)
- Official `legged_gym`
- Official Robotic Systems Lab reinforcement learning (RSL-RL)

The problem before the paper

Legged locomotion policies could take hours or days to train, which made reward, curriculum, and dynamics experiments slow. Physics simulation was often split between central processing unit (CPU) environments while neural-network updates ran on a graphics processing unit (GPU), creating data movement and throughput bottlenecks.

Central idea

Run thousands of physics environments in parallel on one GPU and collect short on-policy fragments from all of them:

```
environment 1:     o0 a0 r0 ... o23 a23 r23
environment 2:     o0 a0 r0 ... o23 a23 r23
...
environment 4096:  o0 a0 r0 ... o23 a23 r23
                              |
                              v
                    one large PPO batch
```

The time horizon per environment can be short because the aggregate batch is large. For Microduck:

$$4096 \times 24 = 98,304 \text{ transitions per iteration.}$$

This changes wall-clock economics. It becomes reasonable to discard stale PPO data because almost 100,000 fresh transitions arrive each iteration.

Curriculum insight

The work also uses a terrain curriculum framed like a game: environments move toward harder terrain when the policy makes sufficient progress and toward easier terrain when it fails. This concentrates samples near the current competence frontier.

The general principle is not "always increase difficulty." It is:

sample where success is neither nearly zero nor nearly certain.

If every rollout fails instantly, the learning signal cannot distinguish useful partial behavior. If every rollout succeeds, it supplies little pressure to improve robustness.

What the evidence establishes

The paper reports dramatic wall-clock training speed in its ANYmal/Isaac Gym setup, analyzes components in the massively parallel regime, and demonstrates sim-to-real locomotion. The open stack influenced a broad family of legged-robot projects.

What it does not establish

- PPO is always more sample efficient than off-policy methods.
- Any 4,096-environment recipe fits a graphics processing unit (GPU) with 8 gigabytes (GB) of memory.
- A fast training curve transfers without actuator/system identification.
- Reward and curriculum values copy across robot size or morphology.

Code-reading route

In RSL-RL, trace:

```
runner -> rollout storage -> returns/advantages -> PPO minibatches
       -> actor/critic update -> new rollout
```

In Microduck, match this to `num_steps_per_env=24`, environment count, PPO epochs, and minibatches. Calculate samples and optimizer exposures before launching a run.

Reproduction exercise

Run the same small locomotion task at 64, 256, and the largest feasible environment count. Keep total environment transitions approximately fixed. Report:

- simulator frames/second;
- GPU memory;
- wall time;
- policy updates;
- return across at least three seeds; and
- whether optimization behavior changes with batch size.

This tests the throughput insight without pretending to reproduce ANYmal.

18.2 Seminar 2 — Rapid Motor Adaptation

Primary work

- Rapid Motor Adaptation (RMA) for Legged Robots (2021)
- Official project page

The problem

A robust policy trained under domain randomization must act reasonably across many dynamics. But the best action on ice differs from the best action on high friction. If the policy can infer the current condition, it can adapt rather than compromise.

The true environment parameter vector might contain friction, payload, motor strength, and terrain compliance:

$$e_t = [\mu, m_{payload}, k_{motor}, \ldots].$$

Those quantities are easy to read from simulation but usually unavailable on the real robot.

Architecture

RMA trains a base policy with a compact environment encoding:

$$a_t = \pi(o_t, z_t).$$

During a privileged training phase, an encoder can derive z_t from simulator environment information. A deployment adaptation module instead predicts it from recent observation/action history:

$$\hat{z}_t = g(o_{t-H:t}, a_{t-H:t-1}).$$

The history contains the consequence of prior actions. For example, the same motor command produces different acceleration under different payload or friction.

Why a latent instead of explicit parameter regression?

The policy needs a control-useful summary, not necessarily an accurate human parameter report. Several physical changes can have similar short-term effects, and some are not separately identifiable from proprioception. A learned latent can represent equivalence classes relevant to action.

Training logic

The idea is teacher-student distillation across information sets:

```
simulation parameters -> privileged encoder -> target latent
history                -> adaptation module -> predicted latent
                                          |
observation + latent -> base policy -> action
```

The deployment module learns to reproduce a latent whose use was already made valuable by the base policy.

Evidence and impact

The paper evaluates simulation-trained RMA on a Unitree A1 over diverse real terrains without real-world policy fine-tuning. The architecture popularized fast history-based adaptation as a practical alternative/complement to fixed robustness.

Limitations and audit questions

- Adaptation can only infer properties that affect observed history.
- The first moments after reset have insufficient history.
- Latent prediction can lag a sudden surface change.
- A training distribution missing a real failure mode cannot confer magic adaptation.
- History length, encoder latency, and sensor bias affect deployment.

Microduck experiment

Randomize one identifiable factor, such as actuator strength. Train:

1. a feed-forward robust baseline;
2. the same actor with observation history; and

3. a privileged-latent teacher plus history student.

Evaluate on held-out fixed strengths and a within-episode strength change. Plot tracking error versus time after the change. This distinguishes average robustness from adaptation speed.

18.3 Seminar 3 — Dreamer and physical robot world models

Primary works

- DayDreamer: World Models for Physical Robot Learning (2022)
- DreamerV3: Mastering Diverse Domains through World Models (2023)
- Official DreamerV3 code

The problem

Model-free RL may need many environment interactions. That is tolerable in a fast simulator but costly on physical hardware. A learned world model can use each transition to predict many aspects of future experience, then train a policy through imagined rollouts.

Recurrent state-space model

Dreamer-family methods use a compact latent state containing a deterministic memory and a stochastic component. At a high level:

$$h_t = f(h_{t-1}, z_{t-1}, a_{t-1}),$$

$$z_t \sim q(z_t \mid h_t, o_t).$$

- h_t carries recurrent history;
- z_t represents uncertain current information;
- posterior q incorporates the real observation.

The model learns to predict observations/features, rewards, and continuation. A prior predicts z_t without seeing o_t, which is what imagined rollouts need.

Model-learning objective in words

The objective balances:

1. reconstruct/predict information from observations;
2. predict reward and whether the episode continues; and
3. make the predictive prior agree with the observation-informed posterior without collapsing the representation.

The divergence term is commonly based on Kullback–Leibler (KL) divergence. Here it asks how different two latent distributions are—not whether the robot's physical path is close in meters.

Imagination

Starting from a posterior latent inferred from real data:

```
latent now -> actor samples action -> world model predicts latent next
           -> predicted reward/value -> repeat in imagination
```

Actor and critic learn from these compact imagined trajectories. No future pixels must be rendered during policy learning.

DayDreamer evidence

DayDreamer applies the approach online to four physical robot settings, including legged recovery/walking, visual manipulation, and visual navigation. Its importance is showing that world-model learning can operate directly on real robot experience rather than only simulated/game benchmarks.

DreamerV3 evidence

DreamerV3 emphasizes a single configuration across more than 150 evaluated tasks and introduces scale-robust transformations/normalization. It provides strong general evidence for the family, not proof that one model captures every contact-rich robot accurately.

Failure analysis

Inspect separately:

- reconstruction/prediction quality;
- reward prediction calibration;
- continuation/termination prediction;
- latent rollout error versus horizon;
- imagined policy success versus real rollout success; and
- uncertainty under out-of-distribution actions.

A visually plausible prediction can have the wrong contact timing, while an unrecognizable decoded image might still retain adequate control information.

Project exercise

Before training a world model on a robot, fit a one-step predictor to logged proprioception. Evaluate one-step, 5-step, and 25-step open-loop errors on held- out action sequences. Compare against a persistence baseline $\hat{s}_{t+1} = s_t$. This establishes whether learning dynamics adds signal before adding actor optimization.

18.4 Seminar 4 — Temporal Difference Learning for Model Predictive Control, second generation (TD-MPC2)

Primary work

- TD-MPC2: Scalable, Robust World Models for Continuous Control (2023)
- Official code

How it differs from Dreamer

Both learn latent dynamics, but TD-MPC2 emphasizes local trajectory optimization at decision time. It learns an encoder, latent dynamics, reward, value functions, and a policy prior. Candidate action sequences are refined and scored in latent space.

Conceptually, for horizon H:

$$\text{score}(a_{t:t+H-1}) = \sum_{k=0}^{H-1} \gamma^k \hat{r}(z_{t+k}, a_{t+k}) + \gamma^H \hat{V}(z_{t+H}).$$

The learned value estimates what happens after the short explicit planning horizon. This is a common model-based pattern: model near-term consequences, bootstrap the distant future.

Planning loop

1. encode current observation/history into latent state;
2. initialize a population of action sequences, helped by the learned policy;
3. roll sequences through latent dynamics;
4. score them with predicted reward and terminal value;
5. update the proposal distribution toward high-scoring sequences;

6. execute only the first action; and
7. observe and replan.

Evidence

The paper reports one set of hyperparameters across 104 online RL tasks in four domains and investigates scaling a multi-task agent. This supports the method's breadth within that suite.

Engineering tradeoff

A feed-forward PPO actor performs one network pass. TD-MPC2 performs several model/value evaluations over candidate sequences per action. Compare:

data efficiency versus runtime planning cost and model risk.

For a 20 Hz arm, planning may fit comfortably. For a 100 Hz balance loop on a small processor, measure worst-case execution before choosing it.

Reproduction exercise

Use the official code on one supported benchmark. Then halve and double the planning horizon while keeping training checkpoint fixed. Measure return and inference latency. Explain why a longer horizon can either help foresight or hurt through compounded model error and missed deadline.

18.5 Seminar 5 — QT-Opt and large-scale real-world RL

Primary work

- QT-Opt: Scalable Deep Reinforcement Learning for Vision-Based Robotic Manipulation (2018)

QT-Opt is the paper's method name, not a full term that the authors expand letter by letter. It combines learned Q-values with numerical action optimization, which is the mechanism worth remembering.

Why this older paper remains important

QT-Opt is an influential counterexample to the idea that robot policies must always learn in simulation or by imitation. It scales off-policy Q-learning to hundreds of thousands of real grasp attempts and closed-loop visual control.

Task formulation

The observation is camera-based; the continuous action describes gripper motion and gripper command; sparse success indicates a completed grasp. The learned Q-function estimates long-horizon success probability/value from observation and action.

Continuous action prevents enumerating $\arg\max_a Q(s, a)$. QT-Opt uses the Cross-Entropy Method (CEM), a sampling optimizer:

1. sample action candidates from a distribution;
2. evaluate each with Q;
3. retain an elite high-value fraction;
4. refit the action distribution to elites; and
5. repeat, then execute a high-value action.

This makes Q-learning possible without a separate deterministic actor, at the cost of several Q evaluations per control decision.

Distributed data and training

The system separates robot collection, replay storage, distributed training, and deployment. Off-policy learning lets old grasp attempts remain useful as new policies collect better data.

Evidence

The paper reports use of more than 580,000 real grasp attempts and high success on its unseen-object evaluation. It documents learned closed-loop behaviors such as regrasping and object repositioning that were not individually programmed.

Limitations and modern reading

- The data/robot fleet budget is far beyond a hobby project.
- Grasp success is narrower than general manipulation.
- The exact camera/action distribution defines generalization.
- A percentage on unseen objects does not describe every failure severity.
- CEM inference has a compute/latency cost.

The lasting lesson is a systems one: replay-based RL can turn a growing robot fleet log into policy improvement when task reward is reliable and collection is automated.

Small-scale exercise

In simulation, compare a discrete grid over a 2D continuous action with CEM optimization of the same learned/known Q surface. Measure solution quality versus Q evaluations. This isolates action optimization from the cost of training a deep visual Q-function.

18.6 Seminar 6 — Action Chunking with Transformers (ACT) and Diffusion Policy

Primary works

- ACT: Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware (2023)
- Official ACT code
- Diffusion Policy: Visuomotor Policy Learning via Action Diffusion (2023)
- Official Diffusion Policy code

Shared problem: demonstrations are temporally and multimodally structured

A one-step squared-error behavior-cloning policy assumes a single average action label. Manipulation demonstrations often have:

- temporally coordinated submotions;
- several valid approaches;
- pauses and contact transitions;
- human timing variability; and
- errors that compound when predicting one action at a time.

Both ACT and Diffusion Policy predict **action chunks**, but represent their distribution differently.

ACT architecture

ACT uses a conditional variational autoencoder (CVAE) with a transformer-style policy. During training, an encoder sees the demonstration action sequence and infers a latent style z. The policy/decoder predicts an action chunk conditioned on observation and z.

The CVAE objective has two parts:

$$L = L_{action} + \beta D_{KL}\big(q_\phi(z \mid o, a_{t:t+H}) \| p(z)\big).$$

- L_{action} makes predicted chunks match demonstrations;
- KL regularization makes the training latent distribution compatible with a simple prior available at deployment;
- β controls the tradeoff.

ACT also temporally ensembles overlapping chunk predictions. If several past inferences predicted the action for the current time, the controller combines them with age-dependent weights. This can smooth transitions without making the policy completely open loop.

Diffusion Policy architecture

Diffusion starts with a noisy action trajectory x_K and repeatedly predicts how to remove noise conditioned on observation:

$$x_{k-1} = \text{denoise}_\theta(x_k, o, k) + \text{scheduled noise}.$$

Training corrupts real action sequences with known Gaussian noise and asks the network to predict the noise (or an equivalent denoising target):

$$L(\theta) = \mathbb{E}_{x_0,k,\epsilon}\left[\|\epsilon - \epsilon_\theta(x_k, o, k)\|^2\right].$$

This is supervised generative modeling. It can retain distinct modes because sampling begins from different noise rather than forcing one mean action.

At deployment, receding-horizon control predicts a sequence but executes only a prefix before observing again.

Evidence

ACT reports fine-grained low-cost bimanual tasks with relatively small demonstration sets. Diffusion Policy reports results across 12 tasks from four manipulation benchmarks and real-robot evaluations. Each supports action sequence modeling in its task family.

Comparison

Question	ACT	Diffusion Policy
action distribution	latent-variable decoder	iterative generative denoising
inference passes	typically one policy decode	several denoising steps
temporal mechanism	chunks + temporal ensembling	chunks + receding horizon
strength	efficient fine-grained imitation	expressive multimodal trajectories
concern	latent use/collapse, chunk tuning	inference latency, denoising schedule

What to reproduce

On one official simulated task:

1. train a one-step behavior cloning (BC) baseline;
2. train action chunks at two horizons;
3. hold demonstration split and image encoder fixed;
4. report task success and correction after an injected perturbation;
5. measure inference latency; and
6. visualize predicted action sequences, not only final success.

The research question is whether temporal/multimodal representation explains the improvement—not whether one method has a newer name.

18.7 Seminar 7 — Robotics Transformer 1 (RT-1) and Open X-Embodiment

Primary works

- RT-1: Robotics Transformer for Real-World Control at Scale (2022)

- Open X-Embodiment: Robotic Learning Datasets and Robotics Transformer X (RT-X) Models (2023)
- Official Open X-Embodiment repository

RT-1 question: does robot policy performance scale with diverse data?

RT-1 studies a high-capacity transformer policy trained on a large real-robot dataset covering many language-described tasks. Images are converted into tokens; language conditions the network; actions are discretized into tokens.

An action dimension a^j within range $[l_j, u_j]$ can be placed into one of B bins. The model predicts a categorical token rather than a raw floating number. De-tokenization maps the selected bin back to a command.

This turns policy learning into sequence classification:

$$L = -\sum_t \sum_j \log p_\theta(\text{action-token}_{t,j} \mid I_{\le t}, \text{instruction}).$$

The model is trained by imitation. It does not learn from a scalar reward in the PPO sense.

TokenLearner and efficiency

Full image patches across time create many tokens. RT-1 uses a learned spatial token reduction so the transformer processes a smaller set of image features. The design highlights a recurring systems problem: a robot policy needs enough visual context while meeting inference rate.

RT-1 evidence

The work studies data/model scaling and generalization on its Everyday Robots fleet/task setup. Its importance is the large, diverse real-robot policy study, not a claim that tokenized actions dominate continuous outputs everywhere.

Open X-Embodiment question: can data transfer across robots?

Robot datasets differ in cameras, arms, grippers, action frames, rates, and task language. Open X-Embodiment assembled standardized data from 22 robot embodiments across 21 institutions, covering 527 skills (reported as 160,266 tasks/variations in the paper's terminology).

The central hypothesis is positive transfer:

target-robot performance after multi-robot pretraining
> target-only performance at comparable target-data volume.

RT-X models adapt the RT-1 and Robotics Transformer 2 (RT-2) policy families to the combined data.

The hard part is normalization, not file concatenation

Cross-embodiment training must reconcile:

- observation names, image views, and missing sensors;
- absolute versus delta actions;
- joint versus end-effector coordinates;
- action dimension and gripper semantics;
- frequency and temporal horizon;
- language/task taxonomy; and
- dataset imbalance.

A common action space may discard embodiment-specific capability. A very loose space may prevent shared learning. This is an interface research problem.

Evidence and limitations

The Open X paper reports positive transfer for multiple evaluated robots and makes standardized datasets/models available. But 22 embodiments do not uniformly cover all robot types: dataset volume and task diversity are imbalanced, mostly toward manipulation.

Microduck relevance

The main lesson is not to feed manipulation actions into a biped. It is to version a policy-family schema so related Microduck skills can share useful representations. Microduck's stable 61D actor contract enables hot swapping; multi-task pretraining would still need task labels, compatible actions, and balanced sampling.

Reproduction exercise

Choose two compatible datasets from Open X. Write a schema table before training. Compare target-only BC with joint pretraining plus target fine-tuning, matching target examples. Report per-task transfer, including negative transfer. A global average can hide one task getting worse.

18.8 Seminar 8 — From RT-2 to open vision-language-action (VLA) and flow policies

Primary works and artifacts

- RT-2 (2023)
- Octo (2024) and official code
- OpenVLA (2024) and official code
- π_0 (2024) and official `openpi`
- $\pi_{0.5}$ (2025)
- Generalist Robot 00 Technology (GR00T) N1 (2025) and official code
- SmolVLA (2025) and LeRobot

This is a fast-moving area. The purpose is to understand design axes, not name a permanent winner.

RT-2: action tokens inside a vision-language model

RT-2 co-fine-tunes a pretrained vision-language model on web vision-language tasks and robot trajectories. Robot actions are expressed as tokens in the same autoregressive vocabulary as text.

The hoped-for transfer is:

```
web vision/language knowledge (objects, concepts, relations)
                          +
robot action demonstrations (grounded motor behavior)
                          |
                          v
language-conditioned visual action tokens
```

The paper reports thousands of evaluation trials and improved semantic generalization in its robot setup. It demonstrates that web-scale semantic pretraining can influence robot action selection. It does not show that web text teaches unobserved robot dynamics.

Octo: reusable open policy initialization

Octo is trained on 800,000 trajectories from Open X-Embodiment and is designed to accept different observation/task modalities and adapt to new action spaces. Its transformer uses token groups and readout tokens so downstream users can attach suitable action heads.

Octo's scientific value includes open checkpoints/code and ablations on architecture/data choices. It frames a generalist policy as an initialization to fine-tune, not a universal zero-shot hardware controller.

OpenVLA: an open vision-language model (VLM)-to-action recipe

OpenVLA starts from a pretrained language backbone and visual encoders, then trains on 970,000 Open X robot trajectories. It predicts discretized action tokens autoregressively. The 7-billion-parameter model makes modern VLA study possible outside the organizations that developed closed RT-2 models.

Parameter-efficient fine-tuning such as Low-Rank Adaptation (LoRA) updates low-rank matrices rather than every full weight matrix:

$$W' = W + BA,$$

where A and B have much smaller rank than W. This reduces trainable parameters, but not necessarily base-model inference memory or latency.

π_0: continuous action chunks with flow matching

π_0 combines a pretrained vision-language backbone with an action expert that generates continuous action chunks using flow matching. Instead of classifying each action into a discrete token, it learns a velocity field that transforms noise into an action trajectory.

In a simplified conditional flow-matching view, interpolate between noise x_0 and data action x_1:

$$x_\tau = (1 - \tau)x_0 + \tau x_1.$$

Train a vector field $v_\theta(x_\tau, \tau, c)$, conditioned on vision/language c, to predict the direction toward data. At inference, numerically integrate the learned field from noise toward a structured action chunk.

The official `openpi` repository makes a particularly valuable systems lesson visible: model code is only one part. Data transforms, image masks, prompt tokenization, action padding, normalization statistics, embodiment config, checkpoint assets, inference server, and robot adapter define actual behavior.

$\pi_{0.5}$: heterogeneous co-training for open-world tasks

$\pi_{0.5}$ adds co-training on heterogeneous examples: robot actions, high-level semantic subtask prediction, object detections, multi-robot data, and web knowledge. The paper evaluates long-horizon manipulation in new homes.

The design suggests that low-level action data alone may not teach robust semantic decomposition. Mixing high-level prediction examples gives the model intermediate supervision. It also complicates attribution: ablations are needed to say which data source produces which capability.

GR00T N1: dual-system VLA for humanoid manipulation

GR00T N1 explicitly describes a dual system:

```
System 2: vision-language reasoning/understanding
                          |
System 1: diffusion-transformer continuous action generation
```

Training mixes real robot trajectories, human video, and synthetic data. The paper reports simulation benchmarks across embodiments and real humanoid bimanual tasks. “Humanoid foundation model” here centers primarily on language-conditioned manipulation; it should not be read as proof of autonomous locomotion safety.

SmolVLA: efficiency and asynchronous inference

SmolVLA studies a smaller, community-oriented model intended for affordable hardware. It also separates slower perception/action prediction from action execution using asynchronous inference.

Asynchrony introduces a control question: the predicted chunk may be based on an older observation. Measure

observation age at action execution = capture + queue + inference + transport delay.

Smooth throughput is not enough if stale actions meet a disturbed robot.

Frequency-space Action Sequence Tokenization (FAST): action encoding matters

- FAST: Efficient Action Tokenization for Vision-Language-Action Models (2025)
- Official `openpi` implementation

Frequency-space Action Sequence Tokenization (FAST) asks why an autoregressive model should emit one quantized symbol for every action dimension at every timestep. Robot trajectories are temporally correlated. Apply a discrete cosine transform to an action chunk, order/quantize its frequency coefficients, then compress the resulting integer sequence. Smooth motion often needs only a few large low-frequency coefficients.

If $A \in \mathbb{R}^{H \times d_a}$ is a chunk and $C \in \mathbb{R}^{H \times H}$ is a cosine-transform matrix, then

$$F = CA$$

contains temporal-frequency coefficients for every action dimension. Keeping or coding coefficients efficiently is a trajectory representation; it is not an assumption that high-frequency corrections never matter. Reconstruction error must be measured against task success, contacts, and actuator bandwidth.

The paper reports that naïve per-dimension/time binning performs poorly on its high-frequency dexterous tasks, while FAST enables efficient autoregressive training. FAST+ is trained as a reusable tokenizer over a large robot-data mixture. For a new embodiment, audit normalization, action rate, reconstruction error by joint, and whether safety-critical short impulses are compressed away.

OpenVLA-OFT: adaptation recipe can dominate the backbone

- Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success (2025)
- Official OpenVLA-OFT code

Optimized Fine-Tuning (OFT) replaces slow autoregressive per-action-token decoding with parallel continuous action chunks and a simple absolute-error objective. The paper reports a large throughput and success improvement over its OpenVLA baseline on the evaluated LIBERO and real-robot protocols.

The causal lesson is more important than the headline number. A VLA comparison changes at least four axes:

```
backbone representation
action target (tokens or continuous)
temporal output (one action or chunk)
decoder schedule (autoregressive or parallel)
adaptation scope and loss
```

An ablation must separate them. If a continuous chunk head wins, that is not automatically evidence that a larger vision-language backbone caused the gain.

A current frontier needs dated evidence

The 2025 Gemini Robotics report studies a closed generalist VLA plus an embodied-reasoning model, including semantic, spatial, and cross-embodiment evaluations. It is scientifically relevant but not reproducible from open weights and training code. The 2026 Green-VLA preprint proposes staged grounding, multi-embodiment pretraining, embodiment adaptation, and RL alignment. Because it is a recent preprint, treat its reported results as provisional until independently reproduced.

These examples prevent two opposite mistakes: ignoring closed work entirely, or treating an inaccessible reported result as a deployable open-source option. Record evidence and artifact openness on separate axes.

Comparing VLAs responsibly

Record:

Axis	Questions
pretraining	web data, robot hours/trajectories, embodiments, licenses?
action	tokens, regression, diffusion/flow, horizon, frame, rate?
adaptation	full fine-tune, LoRA, new head, frozen backbone?
evaluation	same tasks, splits, cameras, demonstrations, trials?
runtime	parameters, memory, average/worst-case execution time (WCET), asynchronous age?
openness	code, weights, data, exact training recipe, robot adapter?

Never compare one paper's percentage directly to another without protocol alignment.

Reproduction exercise

Use one official open VLA on a supported simulation or dataset. First run its released checkpoint/evaluator. Then fine-tune on a deliberately small target subset. Compare:

- from-scratch BC;
- frozen backbone + new action head;
- parameter-efficient fine-tuning; and
- full fine-tuning if compute permits.

Evaluate in-distribution and on one held-out object/layout. Report model memory, inference time, observation age, and task success.

18.9 Seminar 9 — SayCan, Code as Policies, and VoxPoser

Primary works

- SayCan: Do As I Can, Not As I Say (2022)
- Code as Policies (2022)
- Official Code as Policies code
- VoxPoser (2023)
- Official VoxPoser code

These works focus on high-level semantic planning/grounding. They complement, rather than replace, motor RL.

SayCan: combine usefulness with affordance

A large language model (LLM) can score which skill description is linguistically useful for an instruction. A learned value/affordance function estimates whether the robot can successfully execute that skill in the current situation.

For candidate skill k:

$$\begin{aligned}\text{score}(k) \propto\ & p_{\text{LM}}(k \mid \text{instruction}, \text{history}) \\ & \times p_{\text{afford}}(\text{success} \mid s, k).\end{aligned}$$

The product embodies "Say" times "Can." A sponge-related skill may be useful for cleaning, but its affordance should be low if no sponge is reachable.

This is a powerful architecture lesson: semantic plausibility and physical feasibility are different estimators. Exact skill choices constrain the LLM's output space.

Code as Policies: language models compose application programming interfaces (APIs)

Instead of selecting one listed skill, a code-capable LLM writes programs that call perception and control APIs. Programs can express loops, geometry, conditions, and reusable functions.

The strength is compositionality and inspectable structure. The danger is that generated code has real authority. A safe implementation needs:

- a restricted language/API;
- static validation and resource limits;
- no arbitrary shell/network/device access;
- typed physical units and bounded arguments;
- simulation/dry run;
- runtime monitor and timeout; and
- signed/approved primitives beneath it.

Generated Python with unrestricted motor access is not a safety architecture.

VoxPoser: language to spatial value maps

VoxPoser asks an LLM/VLM to compose 3D voxel maps encoding affordances and constraints. A model-based planner then optimizes a trajectory through those maps.

Example maps for "place the block in the bowl without crossing the laptop":

- attraction around bowl interior;
- grasp affordance around block;
- repulsion around laptop and table collision;
- orientation preference for gripper; and
- path smoothness/feasibility from the motion planner.

This grounds language into geometry rather than asking the LLM to emit every joint target.

Evidence boundary

These papers demonstrate important semantic/planning mechanisms in their robot settings. They do not make language-model output a hard guarantee, solve perception uncertainty universally, or remove the need for local feedback.

Worked Jump Rover design example

Define a fixed skill registry:

```
stop
turn_to_bearing(yaw_rad)
drive_local(distance_m, max_speed_mps)
follow_person(track_id, distance_m, max_speed_mps)
dock(dock_id)
```

For "follow Bruce," let a cloud agent propose `follow_person(track_id="person.bruce", ...)` only after local identity/ consent resolution maps language to an exact track identifier (ID). Local perception updates the track; local planning checks obstacles; realtime control executes bounded motion. If any layer becomes invalid, `stop` remains local.

Reproduction exercise

Build a simulation-only skill selector with ten exact skills. Compare:

1. language-model likelihood only;
2. affordance score only; and
3. their product.

Construct commands where a useful object is absent. Report inappropriate skill selection separately from motor execution failure.

18.10 Seminar 10 — Perceptive locomotion and visual parkour

Primary works

- Learning Robust Perceptive Locomotion for Quadrupedal Robots in the Wild (2022)
- Extreme Parkour with Legged Robots (2023)
- SoloParkour (2024)

The problem

Proprioceptive locomotion reacts after the feet/body experience terrain. Exteroception can see a step or gap before contact, enabling deliberate foot placement or body posture. But real depth is incomplete, noisy, reflective, occluded, and delayed.

Robust fusion rather than blind trust

Perceptive locomotion needs a belief over terrain, not a perfect height map. History and attention/gating can learn when exteroception agrees with proprioception and when to rely more on contact evidence.

Conceptually:

$$b_t = f_\psi(b_{t-1}, o_t^{prop}, o_t^{ext}, a_{t-1}),$$

$$a_t = \pi_\theta(o_t^{prop}, b_t, c_t).$$

b_t is a learned belief state carrying temporal context.

Privileged teacher and student

Simulation can train a teacher from exact terrain geometry. A student receives noisy depth/history and distills the teacher or learns from its experience. The gap to audit is:

```
teacher truth: exact terrain/contact/state
student input: rendered/noisy depth + proprioception
real input: sensor-specific artifacts + calibration + delay
```

Constrained visual learning

SoloParkour explicitly formulates task reward and physical constraints, then uses privileged experience to initialize visual off-policy learning. This is a useful combination of Chapter 6 (off-policy), Chapter 7 (constraints), Chapter 14 (experience), and this chapter's perception hierarchy.

Evidence

The cited works show increasingly demanding real quadruped terrain/parkour behaviors under their sensors and obstacle protocols. They establish that vision can participate in dynamic locomotion, including on low-cost platforms.

Failure taxonomy

Do not report only obstacle success. Label:

- perception missed obstacle;
- perceived geometry wrong;
- goal/contact plan infeasible;
- policy chose wrong motion despite adequate perception;
- actuator/contact mismatch;
- latency/stale frame;
- body collision other than intended contact;
- recovery succeeded/failed; and

- safety intervention.

Microduck experiment ladder

1. Add one obstacle and an oracle geometric local planner commanding the unchanged velocity policy.
2. Vary tracking accuracy and determine the locomotion controller's command response limit.
3. Add realistic perception noise/dropout to the planner input.
4. Only then compare a new depth-conditioned locomotion actor.
5. Evaluate held-out obstacle geometry and sensor faults.

This ladder determines whether failure belongs to perception, planning, command tracking, or joint-level control.

18.11 Seminar 11 — From motion imitation to general whole-body control

Primary works and open artifacts

- DeepMimic: Example-Guided Deep Reinforcement Learning of Physics-Based Character Skills (2018)
- Adversarial Motion Priors (AMP, 2021)
- Adversarial Skill Embeddings (ASE, 2022) and official code
- MaskedMimic (2024) and official ProtoMotions code
- ASAP: Aligning Simulation and Real-World Physics for Learning Agile Humanoid Whole-Body Skills (2025) and official code
- BeyondMimic (2025) and official tracking code

These works form a lineage, not a single benchmark ranking. DeepMimic asks how to track reference motion under physics. AMP replaces hand-written reference terms with a learned motion prior. ASE learns reusable latent skills. MaskedMimic treats diverse control signals as masked motion completion. ASAP targets real/simulation dynamics alignment, and BeyondMimic combines strong tracking with guided generative composition.

DeepMimic: reference motion becomes a task

A motion-capture clip supplies reference pose and velocity at phase $\phi_t \in [0, 1)$. The policy observes robot state and phase, then produces joint targets. A representative tracking reward combines exponential similarities:

$$\begin{aligned} r_t = {} & w_q \exp\left(-k_q \| q_t - \hat{q}(\phi_t) \|^2\right) \\ & + w_v \exp\left(-k_v \| \dot{q}_t - \hat{\dot{q}}(\phi_t) \|^2\right) \\ & + w_e \exp\left(-k_e \sum_j \| p_t^j - \hat{p}^j(\phi_t) \|^2\right). \end{aligned}$$

The hats are reference quantities and the end-effector index is j. The exponential gives a bounded smooth score: near the reference, small errors matter; far away, the term can saturate with almost no gradient. The weights and scales decide whether root balance, feet, or joint style dominate.

Early termination after an unrecoverable fall prevents the learner from spending most data far from the motion manifold. Reference-state initialization samples phases throughout the clip so training reaches every part rather than waiting for a novice policy to survive from frame zero.

The major limitation is specification: a phase-indexed tracker knows *which motion and time* to imitate. It is not yet a general skill selector.

Retargeting is an optimization problem

Human and robot skeletons have different limbs, limits, feet, and mass. One retargeting objective is

$$q^*(t) = \arg\min_q \sum_j w_j \| p_j(q) - \tilde{p}_j(t) \|^2 + \lambda \| q - q_{\text{rest}} \|^2,$$

subject to joint limits and contact constraints. Here $\tilde{p}_j$ is a scaled human target and $p_j(q)$ is robot forward kinematics. A low kinematic error does not imply dynamic feasibility. A pose can require impossible torque, self-collide, or move the center of pressure beyond support.

Always visualize retargeted clips, compute limit/contact/velocity violations, and identify which reference channels are physically meaningful before RL.

AMP and ASE: learn a motion manifold

Adversarial Motion Priors (AMP) trains a discriminator to distinguish short state transitions from reference motion and policy motion. A simplified discriminator objective is

$$\max_{D} \quad \mathbb{E}_{x\sim p_{\text{ref}}}[\log D(x)] + \mathbb{E}_{x\sim p_{\pi}}[\log(1 - D(x))].$$

The policy receives a style reward derived from confusing the discriminator, plus a task reward such as target speed. The discriminator learns what transitions look motion-like without a phase-aligned target at every step.

This removes much manual reward engineering but creates new failure modes: discriminator overfitting, reward nonstationarity, missing rare motions, and a policy that is stylistically plausible yet completes the task poorly. The reference dataset is now part of the reward definition.

Adversarial Skill Embeddings (ASE) conditions behavior on latent z and adds an information objective so motion makes z recoverable. Conceptually,

$$\max_{\pi,q} \quad \mathbb{E}[\log q(z \mid s_t, s_{t+1})] + \lambda\, J_{\text{style}}.$$

The latent becomes a reusable skill coordinate. A downstream task can select or adapt latents more cheaply than relearning low-level motion. Inspect whether different latents are genuinely controllable and useful, not merely separated in a visualization.

MaskedMimic: control as conditional inpainting

MaskedMimic randomly hides parts of motion information during training. The controller may receive sparse future poses, a trajectory, selected body targets, or text-derived motion constraints and must fill in a physically plausible whole-body behavior.

Let full motion condition be c and binary mask m. Training samples

$$\tilde{c} = m \odot c + (1 - m) \odot c_{\text{mask}},$$

then learns a controller consistent with visible constraints. Random masking unifies several interfaces: no mask is full tracking; sparse masks are partial control. The scientific question is whether the training mask distribution covers the combinations requested at deployment.

The official ProtoMotions model cards preserve resolved configuration, normalization, and mask layout—details without which a checkpoint name is not an executable control contract.

ASAP and BeyondMimic: crossing the hardware boundary

ASAP collects real/simulation paired behavior, learns a delta-action dynamics correction, and fine-tunes the tracker in the aligned simulation. The key concept is residual system identification: instead of asking broad randomization to cover every mismatch equally, learn structure from measured residuals.

Audit excitation coverage. A correction learned from gentle walking may not predict a landing impact. The aligned simulator is still a model with a validated region, not “the real world in software.”

BeyondMimic builds a robust high-quality motion-tracking layer and distills motion primitives into a diffusion controller that can be guided at test time by task costs such as waypoints or obstacle penalties. In a schematic guidance step,

$$x_{k-1} = f_\theta(x_k, c, k) - \lambda_k \nabla_{x_k} C(x_k),$$

where f_θ is the learned generative update and C is a differentiable task cost. Guidance composes behavior without retraining for every objective, but its cost and gradient do not guarantee contact feasibility or safety.

The paper reports impressive real humanoid motions and downstream behaviors in its setup. Some evaluations use motion capture for state/scene information; record that dependency before calling the result onboard-autonomous perception.

What this lineage teaches Microduck and JumpRover

Microduck's velocity policy does not become a general motion policy by adding a reference clip. A disciplined extension would:

1. retarget one feasible pose/motion and validate support, limits, and torque;
2. train a phase-conditioned tracking baseline;
3. compare exact tracking with an adversarial prior only after the baseline;
4. test reference starts, recovery, and held-out timing;
5. export through the same 61D observation contract only if command semantics can remain compatible; and
6. label motions unavailable on the physical robot.

JumpRover should wait for accepted mechanics before dynamics-sensitive retargeting. Its useful work now is motion schema, frame conventions, offline retargeting visualization, and safety/authority contracts.

Reproduction exercise

Use one official simulated humanoid or a simplified articulated model. Compare joint-only tracking with joint plus end-effector tracking under the same phase initialization. Report tracking terms, falls, torque, contact error, and held-out motion speed. Then remove phase or mask half the reference to observe which information the controller actually requires.

18.12 Seminar 12 — Fast off-policy learning in parallel simulation

Primary works and artifacts

- FastTD3: Simple, Fast, and Capable Reinforcement Learning for Humanoid Control (2025)
- Official FastTD3 code
- Learning Sim-to-Real Humanoid Locomotion in 15 Minutes (2025)
- Official Holosoma code

Massively parallel robot learning became closely associated with on-policy PPO, because thousands of simulators make fresh batches cheap. These works revisit whether replay-based off-policy methods can also exploit that throughput.

The parent algorithms

Twin Delayed Deep Deterministic Policy Gradient (TD3) uses two critics and a deterministic actor. A simplified target is

$$y = r + \gamma(1 - d) \min_{i \in \{1,2\}} Q_{\bar{\phi}_i}(s', \pi_{\bar{\theta}}(s') + \epsilon),$$

where target noise ϵ is clipped, d marks true termination, and bars denote target networks. Taking the smaller critic reduces positive value bias; delaying actor updates lets critics improve before the actor exploits them.

Soft Actor-Critic (SAC) instead trains a stochastic maximum-entropy policy. Its critic target includes entropy:

$$y = r + \gamma(1 - d)\left[\min_i Q_{\bar{\phi}_i}(s', a') - \alpha \log \pi_\theta(a' \mid s')\right].$$

The actor minimizes

$$J_\pi = \mathbb{E}\left[\alpha \log \pi_\theta(a \mid s) - \min_i Q_{\phi_i}(s, a)\right].$$

Temperature α prices entropy. More entropy encourages diverse action sampling, which may help exploration under broad domain randomization but can also inject motion the hardware should never see; deployment normally uses a deterministic statistic of the policy.

Why parallel off-policy learning is not automatic

Let N_{new} new transitions enter replay per collection cycle, and let G gradient steps each consume batch size B. Define update-to-data reuse

$$U = \frac{GB}{N_{\text{new}}}.$$

Very low U leaves the learner behind a flood of experience. Very high U can overfit replay errors, amplify critic bias, and spend more compute than a fresh on-policy update. Thousands of parallel environments also make adjacent collection batches highly synchronized. Replay capacity, warm-up, sampling, target updates, batch size, and network throughput must be designed together.

FastTD3 and FastSAC recipes

FastTD3 combines large-scale parallel simulation, large-batch updates, a distributional critic, clipped double-Q logic, and tuned optimization choices. A distributional critic predicts a return distribution or quantiles rather than one mean, providing a richer learning target. That does not automatically make behavior risk-sensitive; the actor's reduction of that distribution still defines its objective.

The FastTD3 paper reports solving high-dimensional HumanoidBench and other continuous-control tasks with competitive wall-clock time in its protocols. The later FastSAC/FastTD3 sim-to-real recipe reports training locomotion on a single RTX 4090 in 15 minutes and deployment on Unitree G1 and Booster T1, with domain randomization, terrain, pushes, and action-rate curriculum.

Those are end-to-end recipe claims. "Off-policy is faster than PPO" would be too broad: reward scale, simulator, batch/update geometry, networks, tuning, and hardware are all part of the result.

A 2026 signal, not a settled replacement

The June 2026 FastDSAC preprint and its official code study distributional SAC with constrained exploration and policy plasticity for scalable humanoid locomotion. It is relevant to the failure mode where early data and value estimates trap a policy, but at this book's review date it is a recent preprint. Reproduce its baseline protocol before treating the added components as generally necessary.

A fair Microduck experiment

Microduck currently has a proven PPO path, so preserve it as baseline. Before a long off-policy run:

1. implement the same 61D observation, 14D action, reward, resets, and actuator;
2. smoke-test not-a-number values, replay insertion, terminal masks, and export;
3. report both environment steps and U, critic/actor updates, wall time, and peak memory;

4. match evaluation checkpoints by environment interaction, then separately by wall time;
5. inspect critic values, target scale, replay age, action saturation, and penalty signs; and
6. use multiple seeds and the same held-out physics/commands.

If the off-policy method reaches the same return sooner but misses the 50 Hz inference contract or produces unsafe action chatter, it is not a deployment win. If it fails, distinguish algorithm mechanics from an implementation port that did not receive comparable tuning.

Reproduction exercise

On a supported official benchmark, compare PPO and FastTD3 or FastSAC at fixed environment steps and fixed wall time. Report return/success intervals, interactions, update ratio, replay size/age, gradient updates, memory, training throughput, and policy inference time. Ablate one advertised recipe component; do not compare only package defaults.

18.13 Synthesis: seven enduring research themes

Across these papers, the durable ideas are:

1. **Scale data generation carefully.** GPU environments, robot fleets, and cross-embodiment datasets change what is learnable, but interfaces and data balance still decide value.
2. **Exploit information asymmetry during training.** Privileged teachers, critics, and world models can improve learning while deployable actors use realistic inputs.
3. **Represent time explicitly.** History, recurrent belief, action chunks, and predictive models address partial observability and temporal coherence.
4. **Separate semantic intent from physical authority.** VLA/LLM planning belongs above bounded local skills and hard realtime safety.
5. **Convert demonstrations into controllable structure.** Reference tracking, motion priors, latent skills, masking, and generative guidance trade exact imitation for broader composition.
6. **Count optimization as well as interaction.** Parallel simulators make both on-policy freshness and off-policy replay practical; update/data geometry, wall time, and stability decide which helps.
7. **Evaluate the system, not the model name.** Data, normalization, control rate, actuator, planner, hardware limits, and failure recovery define robot intelligence in operation.

Continue with the open-source ecosystem and reproduction labs, which turns these seminars into executable project choices.

18.14 Folded seminar-exercise guidance

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show reference experiment designs for Seminars 1–12

These are solution **structures**, because reproducible measurements—not a predetermined winning number—answer the research exercises.

1. **Parallel PPO:** keep total transitions, task, network, optimizer, and evaluator fixed. Vary only environment count and corresponding iteration count. Report throughput, memory, updates, wall time, and three-seed uncertainty. If return changes, batch/update geometry changed learning even though transition count matched.
2. **RMA:** compare feed-forward robustness, history, and privileged-teacher/ history-student variants with matched actor capacity where possible. Use held-out fixed dynamics plus a mid-episode change; plot tracking error versus time since the change to distinguish robustness from adaptation speed.
3. **World model:** split complete trajectories before creating windows, fit on training trajectories, and roll out held-out action sequences open loop. Compare one-, five-, and 25-step state error against persistence. Never leak overlapping future windows into validation.
4. **TD-MPC2 horizon:** freeze one checkpoint and evaluator, then vary planning horizon. Record success/return together with median, p95/p99, and maximum inference time. A horizon is unusable if it misses the control deadline even when its simulator return improves.

5. **QT-Opt action search:** define a known 2D Q surface with multiple peaks. Compare grid and cross-entropy method (CEM) using equal Q-evaluation budgets; repeat random CEM seeds and report distance/value regret from the known optimum.
6. **ACT/Diffusion:** hold dataset split and visual encoder fixed. Compare one- step BC and two chunk horizons, inject the same perturbation, and measure success, recovery time, action discontinuity, and inference age. Visualize full predicted chunks so "smooth" cannot hide stale open-loop action.
7. **Open X transfer:** first harmonize action frames, units, rates, cameras, and missing fields in a schema table. Match target examples across target- only and pretrain/fine-tune runs; report every task so negative transfer is visible.
8. **Open VLA:** reproduce the released evaluator before fine-tuning. Compare from-scratch BC, frozen backbone, parameter-efficient adaptation, and full adaptation only when compute allows. Fix target splits and report memory, tail latency, observation age, in-distribution success, and held-out object/layout success.
9. **Language planning:** construct exact skill IDs and local preconditions. Evaluate language model (LM) relevance, affordance-only, and their product on present/absent objects. Score invalid selection separately from execution failure and prove timeout/network loss retains local `stop`.
10. **Perceptive locomotion:** progress from oracle geometry + existing velocity policy, through noisy/dropout planning, to a new visual actor. Use the same held-out obstacles and label perception, planning, command tracking, contact, actuator, recovery, and safety failures separately.
11. **Motion control:** validate retargeting limits and contacts before RL. Hold phase initialization and training budget fixed while comparing joint-only and joint-plus-end-effector rewards. Report each weighted term, tracking error, falls, torque, contact violations, and held-out time scaling. A masked condition is a separate information treatment, not just augmentation.
12. **Parallel off-policy:** match observation/action/reward and run both an interaction-budget and wall-time comparison. Record replay warm-up, update-to-data ratio, target updates, batch size, gradient count, replay age, critic statistics, memory, inference timing, and every seed. An ablation of one FastTD3/FastSAC recipe choice is needed before attributing a gain to the algorithm family.

One compact run table prevents a throughput exercise from becoming a story:

```
seed,num_envs,total_transitions,updates,fps,gpu_peak_mb,wall_s,eval_return
0,64,REPLACE,REPLACE,REPLACE,REPLACE,REPLACE,REPLACE
0,256,REPLACE,REPLACE,REPLACE,REPLACE,REPLACE,REPLACE
0,MAX_FEASIBLE,REPLACE,REPLACE,REPLACE,REPLACE,REPLACE,REPLACE
```

19. Open-Source Robot-Learning Ecosystem and Reproduction Labs

Open source makes a research claim inspectable, but the repository name is not the experiment. This chapter maps major projects to their layer, shows what to read, and gives a low-cost sequence of reproductions.

Project capabilities and versions change. Links were checked on 2026-09-01; pin a release or commit before reproducing anything.

By the end of this chapter, you should be able to:

- identify which layer an open repository actually supplies;
- distinguish source availability from an executable paper reproduction;
- inspect a repository from command line to metric without expensive training;
- write an observation/action/data adapter before choosing a large model;
- isolate incompatible simulator and accelerator stacks;
- record code, data, weights, assets, and licenses as separate dependencies;
- select a minimal stack for a Microduck or JumpRover question; and
- complete a reproduction lab with a one-command evaluator.

The links and active-project statements were rechecked on 2026-09-01. A dated check is a snapshot, not a promise that `main` will still behave identically.

19.1 Separate the layers before choosing software

```
dataset / demonstrations
          |
environment + simulator + robot/controller model
          |
algorithm + networks + replay/rollout storage
          |
experiment tracking + evaluator
          |
exported model + inference runtime
          |
hardware adapter + realtime safety
```

One repository may cover several layers but rarely all of them. Installing a reinforcement learning (RL) library does not provide a correct robot model; downloading a vision-language-action (VLA) checkpoint does not provide your camera calibration or action adapter.

Library, framework, benchmark, and artifact

These words are often mixed:

- a **library** provides reusable code such as losses or environment interfaces;
- a **framework** supplies conventions tying several layers together;
- a **benchmark** fixes tasks, splits, metrics, and usually a protocol;
- a **dataset** contains recorded examples under a schema and license;
- a **checkpoint** is one learned parameter state; and
- a **deployment artifact** packages the model with preprocessing, normalization, output conversion, schema, and runtime evidence.

A benchmark score cannot be reproduced from a library name. A checkpoint is not a deployment artifact. Write which noun you mean.

Four contracts cross every stack

1. **Semantic contract:** what observations/actions/commands mean, including frame and units.
2. **Temporal contract:** capture time, control rate, horizon, delay, and queue behavior.
3. **Statistical contract:** normalization, split, randomization, seed, and checkpoint selection.
4. **Authority contract:** which layer may command what, with timeout and fallback.

Two projects can expose arrays of identical shape while violating all four. For example, a 7D action may mean absolute end-effector pose in one repository and six deltas plus a binary gripper in another.

A layer-selection worksheet

Before installing, fill this table:

Need	Existing answer	Missing evidence
physics/robot	exact model and revision	identified residuals?
observation	keys, shapes, frames, rate	runtime sensor parity?
action	units, controller, bounds	hardware conversion?
learning	objective and data source	baseline/tuning budget?
evaluation	split, trials, metrics	failure labels/tails?
deployment	artifact and runtime	timing/fallback proof?

If your research question lives in one row, avoid replacing all six rows at once.

19.2 Small general-RL learning tools

Gymnasium

- Official repository
- Role: standardized environment application programming interface (API) and classic/control benchmarks.
- Read: reset/step return values, termination versus truncation, spaces, and seeding.
- Use: validate an algorithm idea cheaply before robot simulation.

CleanRL

- Official repository
- Role: single-file deep-RL implementations designed for readability and reproducibility.
- Read: one Proximal Policy Optimization (PPO) or Soft Actor-Critic (SAC) file end to end; map each equation to a code block.
- Caution: a concise benchmark implementation is not a robot deployment stack.

Stable-Baselines3

- Official repository
- Role: maintained PyTorch implementations and a convenient baseline API.
- Use: establish a competent algorithm baseline on Gymnasium-compatible tasks.
- Caution: defaults and wrappers are part of the experiment; inspect them.

19.3 Locomotion and graphics processing unit (GPU) simulation stacks

Robotic Systems Lab reinforcement learning (RSL-RL)

- Official repository
- Role: robot-focused runners, PPO and related algorithms, models, storage, export utilities, and teacher-student components.
- Read in order: runner → storage → PPO → actor/critic model.
- Microduck use: this is the optimizer/training loop beneath `mjlab` tasks.

`mjlab`

- Official repository
- Role: manager-based robot-learning environments on MuJoCo Warp.
- Read: environment configuration, observation/reward/event managers, and task registration.
- Microduck use: direct environment framework.

MuJoCo Playground

- Official repository
- Paper
- Role: GPU MuJoCo environments across locomotion, manipulation, and vision; multiple learning backends.
- Study: compare one task's observation/action/randomization/export path with Microduck's.

Isaac Lab

- Official repository
- Role: robot-learning framework on Isaac Sim with sensors, actuator models, environment design, randomization, RL, and imitation workflows.
- Study: manager-based versus direct environments, actuator configuration, and the current supported version matrix.
- Caution: Isaac Sim, driver, NVIDIA Compute Unified Device Architecture (CUDA), and extension versions form a large compatibility surface.

`legged_gym`

- Official repository
- Role: influential Isaac Gym vectorized locomotion tasks associated with Learning to Walk in Minutes.
- Study: reward preparation, terrain curriculum, command sampling, and task registry.
- Caution: it targets legacy Isaac Gym and a specific RSL-RL version. Follow its pinning instructions; do not combine newest packages by guesswork.

Genesis

- Official repository
- Role: Pythonic robotics physics platform with parallel simulation.
- Study: supported solvers, robot import, vectorized control API, benchmark scripts, and released—not announced—features.
- Jump Rover use: one candidate for the future isolated four-action sandbox, after measured hardware/digital-twin gates pass.

19.4 Manipulation environments and benchmarks

robosuite

- Official repository
- Paper
- Role: modular MuJoCo robot/controller/task framework with demonstrations and multi-modal sensors.
- Study: how action meaning changes among joint velocity, inverse kinematics, and operational-space control.

ManiSkill

- Official repository
- Role: GPU-parallel SAPIEN manipulation environments and baselines spanning RL, imitation, model-based learning, and VLA evaluation.
- Study: heterogeneous parallel scenes, state versus visual observations, and real-to-sim/sim-to-real protocols.

- Caution: asset licenses can differ from code license.

RLBench

- Official repository
- Paper
- Role: many language-described manipulation tasks in CoppeliaSim/PyRep.
- Study: task demonstrations, variation definitions, observation modes, and success conditions.

LIBERO

- Official repository
- Paper
- Role: lifelong/multitask manipulation benchmark with controlled spatial, object, goal, and task distribution shifts.
- Study: what a split is intended to test and whether a policy learns transfer or memorizes benchmark regularities.
- Caution: the official repository notes its main focus is imitation rather than sparse-reward RL.

19.5 Robot data and foundation-policy code

LeRobot

- Official repository
- Role: dataset format, hardware integrations, policy training/evaluation, and pretrained-policy ecosystem.
- Study: dataset features/video timing, normalization stats, policy config, robot adapter, and evaluation loop.

Open X-Embodiment

- Official repository
- Paper
- Role: standardized multi-institution robot datasets and Robotics Transformer X (RT-X) study.
- Study: embodiment/action transforms and dataset sampling weights.

Octo

- Official repository
- Role: open generalist policy/checkpoints intended for downstream adaptation.
- Study: token groups, task definitions, action heads, and fine-tuning config.

OpenVLA

- Official repository
- Role: open VLA code/checkpoints and fine-tuning path.
- Study: visual encoder fusion, action tokenization, normalization, Low-Rank Adaptation (LoRA), and inference serving.

`openpi`

- Official repository
- Role: official open implementations/checkpoints for π_0, Frequency-space Action Sequence Tokenization (FAST) variants, and $\pi_{0.5}$ support.
- Study: `Observation`/action tensor specs, image masks, data transforms, normalization stats, action horizon, config, and serving.
- Caution: the maintainers explicitly frame adaptation to new robots as an experiment, not a guaranteed drop-in result.

Isaac Generalist Robot 00 Technology (GR00T)

- Official repository
- GR00T N1 paper
- Role: open foundation-model artifacts/workflows oriented toward humanoid and generalist manipulation.
- Study: data format, embodiment config, dual-system inference, and simulation benchmark protocol.

SmolVLA

- Paper
- Implementation in LeRobot
- Role: smaller VLA and asynchronous inference designed for more accessible training/deployment.
- Study: observation age, action-chunk queueing, and actual central processing unit (CPU)/GPU latency.

Dataset-first tools beyond one foundation model

robomimic

- Official repository
- Role: demonstration datasets, observation encoders, behavior-cloning and offline-policy baselines, and reproducible manipulation evaluation.
- Study: dataset sequence sampling, normalization, train/validation masks, action representation, and rollout evaluator.
- Use: a smaller bridge from Chapter 14's imitation equations to code before a multi-billion-parameter policy.

DROID

- Dataset paper
- Official repository
- Role: large, diverse real-robot manipulation data collected across many scenes and institutions with a common hardware platform.
- Study: collection policy, camera/state/action timing, task language, quality filters, splits, and which portions are actually downloaded.
- Caution: dataset size does not guarantee coverage of a particular action, camera, object, or failure distribution.

Datasets for Deep Data-Driven Reinforcement Learning (D4RL)

- Datasets for Deep Data-Driven Reinforcement Learning (D4RL) paper
- Role: historically influential fixed offline-RL datasets and normalized scoring protocols.
- Study: behavior-policy quality/mixtures, dataset coverage, termination semantics, and benchmark-version warnings.
- Caution: a convenient normalized score can conceal what physical behavior improved; inspect raw return and failure trajectories.

Robot-data formats differ. Reinforcement Learning Datasets (RLDS), Hierarchical Data Format version 5 (HDF5)-based robomimic files, and LeRobot's episode-aware Parquet/video representation have different loading and metadata conventions. Conversion must preserve episode boundaries, capture timestamps, camera names, action semantics, language, and normalization provenance. A successful file conversion is not proof of a semantically lossless conversion.

Whole-body motion and fast control research

ProtoMotions

- Official repository
- Role: research framework and artifacts for adversarial motion priors, reusable skill embeddings, and MaskedMimic-style physics control.
- Study: motion retargeting, masks, reference-state initialization, reward terms, resolved model cards, and simulator/humanoid assumptions.

Aligning Simulation and Real-World Physics (ASAP) and BeyondMimic

- Official ASAP repository
- Official BeyondMimic tracking repository
- Role: current open pipelines for agile humanoid motion tracking, physics alignment, retargeting, simulation, and deployment artifacts.
- Study: released versus roadmap components, motion-data licenses, robot revisions, state-estimation dependencies, and real/simulation log pairing.

FastTD3 and Holosoma

- Official FastTD3 repository
- Official Holosoma repository
- Role: massively parallel off-policy continuous-control research and a current sim-to-real humanoid locomotion/motion-control stack.
- Study: replay warm-up, update-to-data ratio, large batches, distributional critics, curriculum, randomization, evaluator, and deployment adapter.
- Caution: "15 minutes" is a recipe result on stated tasks/hardware, not a portable training-time guarantee.

These projects are valuable study references even for a small robot, but their humanoid motion schemas and actuators are not Microduck or JumpRover drop-ins.

19.6 How to inspect an unfamiliar repository

Do not start by running its most expensive command. Answer these questions:

1. What license applies to code, weights, data, and assets separately?
2. Which commit/release matches the paper?
3. What Python, framework, CUDA/driver, and simulator versions are required?
4. Is the environment API Gymnasium-old, Gymnasium-new, or custom?
5. What are observation/action shapes, units, frames, rates, and normalization?
6. Where are reward, termination, reset, and randomization defined?
7. What command recreates a released evaluation without training?
8. Are pretrained artifacts content-addressed or mutable links?
9. What external downloads are executed?
10. Does the code send telemetry or require an account?

Then create an isolated environment. Do not install a second simulator stack into a working Microduck lock environment.

Start with identity, not installation

For a paper-matching study, record the immutable revision before following `main`-branch instructions:

```
git clone OFFICIAL_URL project-name
cd project-name
git rev-parse HEAD
git status --porcelain
git submodule status --recursive
git lfs ls-files
```

Git Large File Storage (Git LFS) pointers, submodules, downloaded assets, and model-hub revisions can all sit outside the top-level commit. Record them. A clean `git status` says tracked source is unchanged; it says nothing about mutable remote weights.

Map the repository before running it:

```
rg --files | sed -n '1,160p'
rg -n "def (main|train|evaluate)|class .*Env|register\(" .
rg -n "observation|action|reward|termination|truncation" src tests
rg -n "normalize|mean|std|checkpoint|export" src scripts
```

Use the project's documented directories instead of literally assuming `src` or `tests`. The goal is a route map, not a giant search transcript.

Trace one command end to end

When documentation says `train --task X`, follow:

```
console entry point
 -> argument/config parser
 -> task registry and resolved configuration
 -> environment constructor and wrappers
 -> observation/action transforms
 -> rollout or replay storage
 -> loss and optimizer update
 -> checkpoint writer
 -> evaluation entry point and metric aggregation
```

Write file/function names at every arrow. Default values can be changed by a configuration composition system, environment variables, command line, or checkpoint metadata. Preserve the **resolved** configuration after all overrides, not only the source template.

For a paper equation, trace symbol to code and diagnostic. For example:

Paper quantity	Code question	Runtime check
discount factor	where is it applied at truncation?	hand fixture
action at step t	before or after scaling/clipping?	minimum, maximum, and units
reward at step t	raw or weighted term?	per-term episode mass
normalizer	fit on which split/state?	stored stats and parity test
success	environment flag or postprocessor?	labelled rollout replay

Isolation is part of experimental design

Simulator stacks commonly constrain Python, compiler, graphics driver, CUDA, PyTorch/JAX, and native extensions differently. Use a separate lock/container per project. Do not solve one repository by upgrading shared packages beneath another.

A good isolation record includes:

- operating system and architecture;
- host driver and device model;
- environment lock or container image digest;
- language/runtime version;
- installed accelerator build and detected backend;
- simulator/asset version; and
- the exact smoke command and expected short output.

A container does not include the host graphics kernel driver. "Runs in Docker" does not erase driver/device compatibility. Test the actual accelerator backend; a package can import successfully while silently using the CPU.

A version range such as `torch>=2` is not a lock. A lock selects exact resolved artifacts for a platform. Preserve wheel/index source because two files with the same package version label may target different accelerator backends.

Treat external artifacts as a supply chain

Before executing a setup or model:

1. inspect scripts that download or execute remote content;
2. use the official author organization and a pinned revision;
3. verify published hashes where available and record your own digest;
4. inspect code, weight, dataset, and asset licenses separately;
5. run untrusted artifacts without credentials or hardware-device access;
6. use the safest restricted weight loader the framework provides; and

7. record telemetry, model-hub, and experiment-tracking network behavior.

A popular checkpoint can still be mislabeled or replaced. A checksum proves you used the same bytes later; provenance explains why those bytes were trusted.

Write the semantic adapter before the model adapter

For any cross-project policy, fill this schema:

```
observation:
  camera.front:
    shape: [3, 224, 224]
    color_order: RGB
    frame: camera_front_optical
    capture_rate_hz: 30
    normalization: checkpoint_stats_v2
  joint_position:
    order: [joint_0, joint_1, REPLACE]
    unit: rad
    reference: calibrated_zero
action:
  meaning: joint_position_delta
  order: [joint_0, joint_1, REPLACE]
  unit: rad
  rate_hz: 50
  horizon: 10
  bounds_source: runtime_manifest
timing:
  maximum_input_age_ms: REPLACE
  queue_policy: latest_value
termination:
  success: exact_definition
  failure: exact_definition
  timeout_is_truncation: true
```

Only after every `REPLACE` is resolved should shape conversion code exist. Unit-test zero, sign, axis, bounds, joint order, image crop/color, chunk timing, and normalization using a tiny recorded fixture.

19.7 Reproduction ladder

Use the cheapest rung that can falsify the current assumption:

```
read schema/config
      |
import/runtime smoke
      |
one environment + zero/random action
      |
released checkpoint evaluation
      |
short training smoke
      |
one benchmark reproduction across seeds
      |
one ablation
      |
new robot simulation
      |
processor-in-loop / hardware gates
```

Skipping from repository clone to hardware merges dependency, model, policy, timing, and safety failures into one mystery.

Each rung needs an observable pass condition:

Rung	Passing evidence	Common false pass
schema audit	units/frames/rates/splits written	shapes only

Rung	Passing evidence	Common false pass
import smoke	exact backend and versions printed	import on CPU
zero/random step	finite tensors and correct terminations	viewer opens
released eval	documented metric within declared tolerance	attractive video
train smoke	update, save, reload, evaluate	loss printed once
seeded reproduction	raw runs and uncertainty	best seed
ablation	one mechanism isolated	package defaults
new robot sim	identified model and held-out residual	plausible motion
processor/hardware	timing, faults, rollback, owner	untethered demo

Diagnose by the earliest failing rung

- Import failure is a dependency/packaging problem, not evidence against the algorithm.
- Released evaluation mismatch is an artifact/protocol problem; do not fine-tune yet.
- A smoke run that becomes not-a-number indicates numerical/configuration failure; more iterations do not cure it.
- Training success but export mismatch is a deployment-contract failure.
- Simulation success but processor timing failure is a systems failure.
- Hardware failure with simulator replay mismatch points toward model/adapter; matching replay but physical failure points toward unmodeled plant/sensing.

Keep the last passing artifact when moving up a rung. It becomes the reference when a later layer fails.

19.8 Lab sequence A — algorithm truth

1. Run this book's bandit, value iteration, Q-learning, and PPO clipping programs.
2. Read one matching CleanRL implementation.
3. Use a Gymnasium classic-control task and three seeds.
4. Reproduce a documented baseline before changing it.
5. Implement one controlled ablation.

Outcome: understand algorithm mechanics independent of robot complexity.

19.9 Lab sequence B — vectorized robot learning

1. Run Microduck CPU tests and zero-agent playback.
2. Run a 64-environment/five-iteration PPO smoke.
3. Trace one reward and observation through `mjlab` managers.
4. Inspect RSL-RL rollout batch shape and PPO update counts.
5. Export and run CPU Open Neural Network Exchange (ONNX) rehearsal.
6. Repeat one supported MuJoCo Playground task in a separate environment.
7. Compare environment and deployment contracts, not reward numbers.

Outcome: understand a complete simulator-to-runtime motor-policy pipeline.

19.10 Lab sequence C — robot imitation and data

1. Inspect one LeRobot dataset in a browser/tool without training.
2. Write its schema/dataset card.
3. Train a simple behavior cloning (BC) baseline on a small split.
4. Evaluate on fixed held-out episodes.
5. Compare one-step versus chunked action prediction.
6. Inject an observation perturbation and measure recovery.
7. Only then try Action Chunking with Transformers (ACT), Diffusion Policy, or a VLA.

Outcome: separate data quality/action representation from model scale.

19.11 Lab sequence D — foundation-policy adaptation

Choose Octo, OpenVLA, `openpi`, GR00T, or SmolVLA based on supported hardware, compute, and license.

1. Pin paper-matching repository commit and checkpoint hash.
2. Run the official evaluator on one supported task.
3. Measure model memory, average latency, p95/p99 latency, and action age.
4. Visualize observation and de-normalized action samples.
5. Fine-tune on a tiny target dataset with a fixed validation/test split.
6. Compare against target-only BC at matched target data.
7. Test one held-out object/layout and one sensor fault.
8. Document which “generalist” behavior transferred and which did not.

Outcome: evaluate pretraining as a hypothesis rather than an identity.

19.12 Lab sequence E — language planning with safe skills

1. Create a simulator-only registry of exact, typed skills.
2. Implement local preconditions and deterministic stop.
3. Let a large language model (LLM) rank/propose only registered skills.
4. Add an affordance/feasibility score.
5. Test absent objects, stale perception, conflicting instructions, timeout, and network loss.
6. Log proposal, validation decision, execution result, and fallback.
7. Do not connect generated code or free-form motor values to hardware.

Outcome: gain semantic planning without surrendering physical authority.

19.13 Bringing a new robot into an open stack

Do not begin by implementing the environment class. Begin with a frozen interface fixture representing one real or expected control cycle:

```
fixture = {
    "capture_ns": 8_400_000_000,
    "joint_position_rad": [0.0, 0.1, -0.2, 0.0],
    "joint_velocity_rad_s": [0.0, 0.0, 0.0, 0.0],
    "imu_gyro_rad_s": [0.01, -0.02, 0.0],
    "command": [0.1, 0.0, 0.0, 0.0],
}

def validate_action(action):
    assert len(action) == 4
    assert all(-1.0 <= value <= 1.0 for value in action)
    return action
```

Replace the dimensions and meanings with the actual robot. The fixture enables observation assembly, normalization, policy loading, action ordering, and timeout tests before motors or simulation are available.

Phase 0: selection

Choose the stack whose native abstractions require the fewest semantic conversions. Score candidates on:

- robot/model import and contact/sensor support;
- action/controller type;
- required learning algorithm and data path;
- accelerator/hardware availability;
- export/runtime path;
- license and artifact access;
- maintained tests/documentation; and
- team familiarity and debugging visibility.

Do not add the scores as if they were objective measurements. A missing realtime export path can be a veto even if every other row is strong.

Phase 1: interface without dynamics

Implement and test:

1. observation and command dataclasses/schema;
2. stable joint/sensor order and frame names;
3. timestamp, sequence, validity, and timeout behavior;
4. action bounds and unit conversion;
5. recorded-fixture replay; and
6. policy manifest plus rejection on mismatch.

For JumpRover, this phase is valid now even while mechanics and realtime board are unfinished. It should include cloud-option rejection and local-stop state machine tests, but no claim about physical balance.

Phase 2: measured digital twin

Once hardware exists, identify mass/inertia/geometry, actuator response, friction, backlash, delay, saturation, thermal/voltage behavior, contacts, and sensor noise. Replay the same excitation through hardware and simulation. Split identification and held-out validation trajectories by experiment, not by overlapping timesteps.

Use residual plots, not only one average error. A simulator that matches gentle motion and misses braking/impact is not validated for obstacle stopping or jumping.

Phase 3: deterministic baseline and learning

Build a bounded classical/scripted controller that proves sign, authority, timing, reset, and fallback. Then add the smallest learned component:

- residual action around a stable baseline;
- velocity/pose command tracker;
- state estimator or adaptation latent; or
- planner over an existing motor policy.

Freeze the baseline before comparison. A new stack must reproduce zero/random tests, train/save/reload smoke, held-out evaluation, and export parity before a long run.

Phase 4: deployment gates

Use simulation, alternate-simulator, processor-in-loop, unpowered bench, tethered, and bounded-floor tests in order. At every gate preserve input/output logs, target-compute latency tails, watchdog/fault results, operator/intervention rules, artifact hash, and rollback version.

This staged route separates “the open-source algorithm works” from “our adapter is correct” and “our robot is safe enough for this experiment.”

19.14 Dependency and provenance record

For every external project, preserve:

```
source_url: OFFICIAL_REPOSITORY_URL
commit: full_commit_sha
paper: exact_url_and_version
license: code / weights / data / assets
environment: lockfile_or_container_digest
accelerator: GPU, driver, CUDA/framework
command: exact_reproduction_command
config: resolved_config_path_and_hash
checkpoint: source_and_sha256
dataset: version/revision_and_split
local_changes: patch_or_clean
result: raw_metrics_and_evaluator_commit
```

This turns “I ran the GitHub project” into inspectable evidence.

A minimal lab report tree

```
reproduction/
  README.md                 # question, exact commands, expected output
  PROTOCOL.md               # predeclared comparison and stopping rule
  provenance.yaml           # revisions, hashes, licenses, hardware
  configs/resolved/         # one immutable config per run
  data/raw/                 # per-seed/trial records, never hand-edited
  analysis/                 # script and tested statistic helpers
  reports/                  # generated tables/figures and interpretation
  failures/                 # videos/logs plus category labels
```

Large artifacts may live in an external content-addressed store; keep their digests and retrieval instructions in the tree. One command should regenerate the report from raw results without retraining.

19.15 Choosing a first open-source project

Goal	Start with	Avoid starting with
learn RL equations/code	book examples + CleanRL/Gymnasium	billion-parameter VLA
learn robot PPO/sim-to-real	Microduck + RSL-RL/mjlab	unmeasured custom hardware
learn manipulation environments	robosuite or ManiSkill	real-arm online exploration
learn demonstrations	LeRobot + BC	offline RL before dataset audit
study lifelong transfer	LIBERO	claiming real-world generality from one split
study VLA adaptation	Octo/OpenVLA/SmolVLA on supported benchmark	raw motor deployment
study world models	DreamerV3 or Temporal Difference Learning for Model Predictive Control, second generation (TD-MPC2), official benchmark	long-horizon hardware planning first
study language planning	SayCan-style typed skill simulator	unrestricted generated code on robot

The smallest project that answers your question is the fastest route to genuine understanding.

19.16 Exercises

1. Classify Gymnasium, LIBERO, LeRobotDataset, and an exported Microduck policy as library/framework, benchmark, dataset, or deployment artifact. Explain where categories overlap.
2. Two policies both accept 14 floating-point values. Give four reasons their observations or actions may still be incompatible.
3. Why must a reproduction inspect termination separately from truncation?
4. Write a code-reading route from a command-line `train` entry point to the logged success metric.
5. Explain why `torch>=2` is not a reproducible accelerator environment.
6. A paper releases source and one weight file but no data mixture, normalization, or evaluator. What is open, and what claim cannot yet be reproduced?
7. List six fields that a conversion from RLDS to LeRobotDataset must preserve.
8. Design an isolation plan for Microduck, Isaac Lab, and an OpenVLA project on one workstation.
9. The official checkpoint evaluator produces a much lower result than the paper. What should happen before fine-tuning?
10. Choose a minimal open stack for adding modular obstacle avoidance to Microduck and name the baseline that must remain frozen.
11. Which JumpRover integration work is valid before mechanics/realtime board acceptance, and which work must wait?

12. Draft a provenance record for one external checkpoint, including code, weights, data, license, environment, evaluator, and local changes.

Continue with the glossary and worked problems, then select one reproduction ladder whose success criterion you can state before running it.

19.17 Folded lab completion rubric

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show reference evidence for Lab sequences A–E

- **A — algorithm truth:** the small implementation and reference implementation agree on hand-calculated updates; a documented baseline is reproduced across three seeds; one ablation changes a single mechanism and reports uncertainty.
- **B — vectorized robot learning:** tests, smoke training, resolved manager terms, rollout tensor shape, ONNX export, and CPU rehearsal are connected by exact paths/hashes. The five-iteration result is labeled pipeline evidence, not locomotion performance.
- **C — imitation and data:** a dataset card precedes training; splits prevent episode/scene leakage; one-step and chunked BC use matching data; the perturbation test reports recovery and failure, not only nominal loss.
- **D — foundation adaptation:** official checkpoint evaluation passes before fine-tuning; target-only BC and adaptation see the same target examples; model memory, tail latency/action age, in-distribution and held-out results, licenses, and failures are preserved.
- **E — safe language planning:** the model can select only offered exact identifiers (IDs); local schema/preconditions reject absent/stale cases; every effect has timeout/cancel/fallback; generated prose/code and raw motor values have zero execution authority.

For every sequence, commit a provenance record like Section 19.14 and a one-command evaluator. “Repository installed” is setup evidence, not a lab result.

19.18 Folded exercise solutions

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Show worked answers to Section 19.16

1. Gymnasium is primarily an environment-interface library plus a collection of reference environments. LIBERO is a benchmark with tasks, demonstrations, splits, and metrics. LeRobotDataset is a dataset format/class; a concrete repository using it is a dataset artifact. A correctly packaged exported Microduck policy, normalizer, schema, adapter, manifest, and evidence form a deployment artifact. LeRobot as a whole is also a framework, showing why the exact noun and scope matter.
2. The 14 values can differ in joint order, units, absolute versus delta meaning, normalization, reference pose/frame, scale/bounds, control rate, or actuator/controller interpretation. Shape checking catches none of those semantic mismatches.
3. True termination ends the task process, so a value target should usually not bootstrap past it. A time-limit truncation stops data collection while a valid future may still exist. Merging them changes return/value targets and can explain a reproduction gap even when every network parameter matches.
4. Follow console-script registration to argument/config parser, resolved task registry, environment constructor and wrappers, observation/action transforms, rollout/replay storage, loss/optimizer, checkpoint writer, evaluation loader, success computation, and aggregation/logger. Record the exact file/function and config field at each step.
5. The range can resolve to different releases over time and platforms. It omits Python, CUDA build/index, driver compatibility, companion packages, and native-extension versions. Preserve an exact platform lock or container digest and verify the detected accelerator backend.

6. The code bytes and checkpoint bytes are available under their stated licenses. Training-data provenance, exact preprocessing/inference, and the reported evaluation remain incomplete. One may inspect or perhaps run the model, but cannot claim reproduction of the paper result until the missing statistical and semantic contracts are reconstructed.
7. Preserve episode boundaries and identifiers, capture timestamps/rates, observation keys/camera names and image modes, state/action order/units/ frames, language/task labels, success/termination/truncation, calibration, normalization provenance, and split/group metadata. Six are required; a robust conversion records all of these and verifies a sampled episode.
8. Keep three project directories and three exact locks/containers. Share only explicitly versioned data through neutral files. Record the host driver as a common constraint, verify each PyTorch/JAX backend independently, and never upgrade Microduck's lock to satisfy Isaac Lab or OpenVLA. Give containers no motor devices or credentials during untrusted checkpoint inspection.
9. Stop at that rung. Verify paper/checkpoint/evaluator revisions, assets, dependencies, configuration, seeds, preprocessing/normalization, metric, and hardware. Preserve the mismatch and seek an official issue/revision if needed. Fine-tuning would erase the clean test of released reproducibility.
10. Keep the existing Microduck velocity policy and 61D contract frozen. Add a depth or range source, timestamped local occupancy map, geometric local planner, and uncertainty-aware command governor that outputs the existing bounded velocity command. Compare against oracle geometry first. A new end-to-end depth-to-joint policy is a later separate project.
11. Now: define schemas/frames/units, recorded fixtures, policy manifests, cloud typed-option validation, stale/replay rejection, local-stop state machine, simulator interfaces, and provenance/testing infrastructure. Wait to train or validate dynamics-sensitive balance/jump/locomotion until geometry, mass/inertia, actuators, sensors, timing, braking, watchdogs, and held-out system identification are accepted.
12. Record official source URL and commit/submodules; checkpoint identifier, revision, digest, and retrieval date; paper version; data mixture/version and license; code/weight/data/asset licenses; exact lock/container and accelerator; resolved configuration; evaluator commit/command; raw result; and a clean status or attached local patch. A digest establishes identity, while this context establishes provenance.

20. Glossary and Worked Problems

Use this chapter as both a reference and a self-test. Try each problem before reading its solution. The point is not to perform arithmetic quickly; it is to connect the arithmetic to a real robot decision.

20.1 Plain-language glossary

Action

The numeric decision produced by a policy at one time step. Microduck's main policy outputs 14 relative joint-position targets.

Actor

The neural network that selects actions. It is the part exported for deployment.

Actuator

Hardware that produces motion or force, plus its control electronics. A Dynamixel XL330 servo is an actuator. In simulation, an actuator model converts the policy's target into realistic force/torque behavior.

Advantage

An estimate of how much better or worse a sampled action was than the critic expected in that situation. Positive advantage encourages the action; negative advantage discourages it.

Better Actuator Models (BAM)

The actuator library/model used here to represent XL330 motor electrical behavior, firmware position control, friction, voltage, load sag, and delay more realistically than an ideal position actuator.

Behavior cloning (BC)

Supervised learning from demonstration pairs: given the observation, predict the demonstrator's action. BC does not require a reward or Bellman update and is therefore imitation learning, not by itself reinforcement learning.

Bellman backup

An update based on "immediate reward plus discounted value of what follows." Dynamic programming, temporal-difference (TD) learning, Q-learning, and many critics use different forms of this recursive idea.

Batch and minibatch

A batch is a collection of training samples processed together. Proximal Policy Optimization (PPO) collects a full rollout batch, shuffles it, and divides it into smaller minibatches for optimization.

Bootstrapping

Updating an estimate using another current estimate, such as training $V(s_t)$ toward $r_t + \gamma V(s_{t+1})$. It enables learning before an episode ends but can propagate estimation error.

Checkpoint

A saved training state such as `model_2000.pt`. It normally includes actor, critic, observation normalizers, optimizer state, and counters, so it can be evaluated or resumed.

Command

A requested behavior supplied to the policy as input, such as forward speed, turn rate, or head pose. The command expresses intention; the action expresses how the joints should move now.

Coordinate frame

The origin and axis directions used to describe a vector. A velocity in the robot body frame is different from the same three numbers in the world frame.

Critic

The training-only neural network that estimates expected future return. It helps estimate advantages and may use privileged simulator information.

Curriculum

A schedule that changes difficulty, command range, reset distribution, reward weight, or another task property after the policy learns an easier stage.

Decimation

The number of physics substeps for which one policy action is held. A 5 ms physics step with decimation 4 produces one policy decision every 20 ms.

Degree of freedom (DoF)

One independent direction of movement. A simple hinge joint has one rotational DoF. “14 actuated joints” means the policy controls 14 motorized DoFs.

Domain randomization (DR)

Sampling plausible physical parameters and disturbances during training—such as mass, friction, voltage, sensor bias, and delay—so the policy learns a family of robots rather than one perfect simulation.

Distribution shift

A difference between the data distribution used to train a model and the situations encountered during evaluation/deployment. An imitation policy can create states absent from expert data; an offline critic can be queried on actions absent from its log.

Diffusion policy

A generative policy that starts from noise and iteratively denoises a conditioned action or action sequence. This can represent several distinct valid action modes but requires multiple inference steps.

Entropy

A measure of uncertainty/spread in the policy’s action distribution. PPO uses an entropy bonus to preserve exploration early in learning.

Experience replay

A buffer of past transitions sampled for off-policy updates. Replay reuses expensive data and decorrelates adjacent samples, while creating distribution mismatch between older behavior and the current policy.

Environment

The complete executable task: simulated scene, robot, actions, observations, commands, rewards, reset events, terminations, and curricula.

Episode

One sequence from reset until timeout or another termination. The main walking episode lasts at most 20 simulated seconds.

Epoch

One complete optimization pass over a collected rollout batch. Microduck's main PPO configuration uses five epochs per rollout.

Generalized Advantage Estimation (GAE)

Generalized Advantage Estimation, a method that combines several temporal-difference errors. Its λ setting trades bias against variance.

Foundation policy / vision-language-action (VLA)

A broadly pretrained robot policy intended for adaptation across tasks or embodiments. A VLA model conditions actions on images and language. Most VLA pretraining is demonstration-based imitation rather than online reinforcement learning (RL).

Flow matching

A generative method that learns a vector field transporting samples from a simple noise distribution toward a data distribution. Some modern VLAs use it to generate continuous action chunks.

Gradient and backpropagation

A gradient says how a small change in each network parameter changes the loss. Backpropagation applies the chain rule through all layers to compute those gradients efficiently.

Hyperparameter

A setting chosen by the engineer rather than learned as a network weight—for example learning rate, PPO clip value, hidden-layer widths, γ, or λ.

Inference

Running a frozen actor to compute an action. No PPO update, critic training, or backpropagation occurs during inference.

Action chunk

A sequence of several future actions predicted at once. Chunks can improve temporal coherence; executing too much of a chunk open loop can delay feedback correction.

Kullback–Leibler (KL) divergence

Kullback–Leibler divergence, a measure of how one probability distribution differs from another. PPO monitors approximate KL to detect a policy update that moved too far from the rollout policy.

Low-Rank Adaptation (LoRA)

Low-Rank Adaptation, a parameter-efficient fine-tuning method that adds trainable low-rank updates to frozen or mostly frozen weight matrices. It reduces trainable parameter count but does not automatically make base-model inference small or realtime.

Markov Decision Process (MDP)

The mathematical model containing states, actions, transition probabilities, rewards, and a discount factor.

MuJoCo Extensible Markup Language (XML) model format (MJCF)

It describes robot bodies, joints, actuators, contacts, sensors, meshes, and related physics/model data.

Normalization

Transforming each observation feature using learned statistics so values with different units/scales are numerically comparable. The actor normalizer must be included in the deployed export graph.

Observation

The numeric information given to a policy. It may be only part of the complete environment state.

Open Neural Network Exchange (ONNX)

A portable neural-network graph format used to move the trained actor from Python training to the runtime.

On-policy

Learning from experience generated by the current or very recent policy. PPO collects a rollout, updates for a few epochs, discards it, then collects fresh experience.

Off-policy

Learning about a target policy from experience generated by a different behavior policy. Replay-based Soft Actor-Critic (SAC), Twin Delayed Deep Deterministic Policy Gradient (TD3), and a Deep Q-Network (DQN) are off-policy; offline RL is the special fixed-dataset setting with no new corrective interaction.

Partial observability / Partially Observable Markov Decision Process (POMDP)

A setting where the current observation omits information needed to predict the future. History, recurrent state, explicit estimation, or an adaptation latent can help when hidden properties leave observable traces.

Policy

The decision rule mapping observations (and commands) to action probabilities or inference actions. Here the policy is a multilayer perceptron (MLP) neural network.

PPO

Proximal Policy Optimization, the on-policy actor-critic algorithm used here. Its clipped objective reduces the incentive for one rollout to cause an excessively large policy change.

Privileged observation

Simulator information given to the training critic but withheld from the actor because the real robot cannot measure it equivalently.

System identification

Estimating physical model parameters from measured input/output trajectories. For sim-to-real work, identify and validate a nominal model before choosing randomization around its remaining uncertainty.

Teacher-student learning

A training scheme in which a teacher with richer information or capability provides targets for a deployable student. The student's sensor/input contract still determines what it can reproduce.

Reward and reward shaping

Reward is the scalar learning objective. Reward shaping adds measurable terms that make useful intermediate progress learnable. Poor shaping can create exploits or a different task.

Rollout or trajectory

A time-ordered sequence of observations, actions, rewards, and termination flags sampled by a policy. A rollout batch contains many short sequences from parallel environments.

Robotic Systems Lab reinforcement learning (RSL-RL)

The training library that provides PPO, neural-network models, rollout storage, normalization, checkpoints, and ONNX export integration in this stack.

Sim-to-real

Transferring a policy learned in simulation to a physical robot. Success requires consistent interfaces and a simulation distribution that covers the important real dynamics and uncertainties.

State

The full variables needed to describe the environment dynamics. The simulator state is richer than the actor observation.

Tensor

A multidimensional numeric array. Shapes matter: one actor observation is `(61,)`; 4,096 observations form `(4096, 61)`; actor actions form `(4096, 14)`.

Termination

A condition that ends and resets an episode, such as timeout, fall, terrain bounds, or nonfinite state.

Value function

The critic's estimate of expected discounted future reward from a state or critic observation.

World model / model-based RL

A world model predicts future state or a learned latent representation under actions. Model-based RL uses such predictions for planning or policy/value learning. Longer imagination can amplify model error.

Worst-case execution time (WCET)

Worst-Case Execution Time: the longest execution time under the defined conditions. A realtime loop must budget a defensible upper bound or percentile and deadline policy, not only average inference speed.

Warp and MuJoCo Warp

Warp is NVIDIA's Python/Compute Unified Device Architecture (CUDA) kernel framework. MuJoCo Warp implements graphics processing unit (GPU)-parallel MuJoCo-style simulation so thousands of environments can step together.

Action space

The set of actions a policy is allowed to produce. It includes dimension, bounds, and semantics—not just tensor shape. Joint targets, torques, end-effector deltas, and base velocities are different action spaces.

Actor-critic

An architecture with an actor choosing actions and a critic estimating value. The critic can reduce policy-gradient variance or train an off-policy actor; only the actor and required preprocessing usually deploy.

Affordance

An estimate of whether a skill or interaction is feasible in the current situation. A language instruction may make "pick up sponge" relevant while the affordance is low because no sponge is reachable.

Agent

The decision-making system interacting with an environment. Depending on scope, it may mean one policy network or a larger planner, estimator, memory, skill registry, and safety-governed runtime. State the boundary.

Algorithm versus implementation

An algorithm is an abstract update or planning procedure. An implementation also chooses network, optimizer, normalization, batching, numerical precision, wrappers, and defaults. Two projects bearing the same algorithm name can behave differently because those choices are part of the experiment.

Calibration

Estimating a sensor or actuator mapping against known references. Camera intrinsics map rays to pixels; camera extrinsics map between frames; encoder zero calibration maps readings to physical joints. Calibration uncertainty and revision belong in deployment evidence.

Confidence interval

An interval produced by a procedure designed to cover an unknown population quantity at a stated long-run rate under assumptions. It describes uncertainty of an estimate, not a range containing a fixed percentage of individual robot outcomes.

Covariance

A matrix describing the scale and joint variation of uncertain variables. Diagonal entries are variances; off-diagonal entries record correlation. A pose estimate needs frame and covariance because errors in position and orientation are often coupled.

Cross-entropy method (CEM)

A sampling optimizer that repeatedly evaluates candidates, keeps an elite fraction, and refits its proposal distribution. Model Predictive Control and QT-Opt use variants to search continuous action sequences.

Dataset, demonstration, and transition

A transition is one record such as $(o_t, a_t, r_t, o_{t+1}, d_t)$. A demonstration is a trajectory produced by an expert or teleoperator. A dataset is a versioned collection plus schema, provenance, splits, and license—not merely a directory of arrays.

Dataset Aggregation (DAgger)

An interactive imitation algorithm that rolls out the learner, asks an expert to label states the learner actually visits, aggregates those labels, and re-trains. It attacks behavior cloning's state-distribution shift, but requires safe expert access on learner-induced states.

Datasets for Deep Data-Driven Reinforcement Learning (D4RL)

An influential collection and protocol for studying fixed-dataset RL. The name does not imply that every current environment/version or score normalization is unchanged; pin the benchmark revision.

Digital twin

A versioned computational model connected to measured properties and validation data from a physical system. A visually similar robot model with guessed mass, actuators, contacts, and delay is a simulation asset, not yet a validated digital twin.

Discount factor

The number $\gamma \in [0, 1]$ multiplying later rewards. With constant timestep Δt, changing control rate while keeping γ changes the effective physical-time horizon. A continuous-time interpretation often chooses $\gamma = \exp(-\Delta t/\tau)$ for time constant τ.

Dynamics

The rule or probability distribution governing how state changes under action. Robot dynamics include rigid-body motion, contacts, actuators, delay, and unmodeled disturbances—not merely geometry.

Embodiment

The physical form and sensing/action interface of a robot: morphology, joints, actuators, cameras, controllers, frames, and rates. Cross-embodiment learning requires explicit adapters because the same task can use incompatible actions.

Exteroception and proprioception

Exteroception senses the outside world, such as camera, depth, or light detection and ranging (LiDAR). Proprioception senses the robot, such as encoders and inertial motion. A policy cannot avoid a pre-contact obstacle from proprioception alone unless another module converts exteroception into its commands.

Conditional variational autoencoder (CVAE)

A latent-variable generative model trained with reconstruction/action loss and a divergence regularizer toward a simple latent prior. Action Chunking with Transformers uses a CVAE-style latent to represent variation in demonstrated action sequences.

Action Chunking with Transformers (ACT)

An imitation architecture that predicts temporally coordinated action chunks with a transformer and latent-variable objective. Its acronym names the method; chunk length, temporal ensembling, action normalization, and latency are still deployment choices.

Conservative Q-Learning (CQL)

An offline RL method that penalizes high values on actions not sufficiently supported by the fixed dataset while fitting observed Bellman targets. The conservatism strength trades exploitation against pessimism.

Implicit Q-Learning (IQL)

An offline RL method that learns value through expectile regression on dataset actions, then extracts a policy by advantage-weighted imitation. It avoids querying arbitrary new policy actions during its central value target.

Generalization and out-of-distribution input

Generalization is performance on a specified held-out population. An out-of-distribution input lies outside or in a low-density region of the training distribution. Novel texture, novel geometry, novel dynamics, and a new embodiment are different generalization axes.

Hierarchical policy and option

A hierarchical policy selects goals or temporally extended skills. An option has start conditions, an internal policy, and termination. Because it consumes multiple primitive steps, its next-option value is discounted by the elapsed duration.

Inverse kinematics (IK)

Solving for joint configurations that place a link/end effector at a desired pose. IK handles geometry, not automatically dynamics, torque, collision, contact stability, or a time-optimal trajectory.

Latency, jitter, and age of information

Latency is a delay between named events; jitter is variation in that delay. Age of information is how old the source measurement is when an action uses it. A fast average inference time can coexist with unsafe queue age and tail jitter.

Model Predictive Control (MPC)

A controller that repeatedly predicts candidate future trajectories over a finite horizon, optimizes a cost, executes a prefix—often one action—and replans. Its behavior depends on model error, horizon, optimizer budget, constraints, and terminal value/cost.

Model-free RL

Reinforcement learning that does not explicitly learn/use a transition model for planning. A critic still predicts expected return, so "model-free" does not mean "learns no predictive quantity."

Occupancy grid

A map dividing space into cells with estimated occupied/free probability or log-odds. Cell probabilities are not independent guarantees, and the map needs a frame, timestamp, valid region, and uncertainty behavior.

Online and offline reinforcement learning

Online RL can collect new experience and correct its policy distribution. Offline RL must learn only from a fixed dataset, so unsupported actions and coverage become central. Off-policy online RL with replay is not the same as offline RL because it still gathers new data.

Policy gradient

The gradient of expected return with respect to policy parameters. A common estimator weights $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$ by an advantage. It is an expectation identity; finite batches, critics, clipping, and optimizer choices determine practical variance and bias.

Potential-based shaping

A reward of the form $F(s, s') = \gamma\Phi(s') - \Phi(s)$. Under its theorem's assumptions, it preserves optimal policies while rewarding progress in a potential Φ. Arbitrary dense bonuses do not inherit that guarantee.

Rapid Motor Adaptation (RMA)

A teacher-student locomotion architecture in which a base policy uses a compact environment latent and a deployment module predicts that latent from recent proprioceptive history. It adapts only to hidden properties that affect observed history within the trained distribution.

Random seed

An integer initializing pseudorandom processes such as network weights, resets, data order, and exploration. Multiple seeds sample training variability; many episodes from one seed do not replace independent trainings.

Return

The discounted sum of future rewards from a time step. Reward is one immediate signal; return is what the RL objective seeks to increase in expectation.

Reward mass

The observed weighted contribution a reward term accumulates, rather than its configuration weight alone. Compare reward mass when copying regularizers between tasks whose positive terms or episode lengths differ.

Risk and Conditional Value at Risk (CVaR)

Expected return averages outcomes. Conditional Value at Risk (CVaR) summarizes a chosen tail, such as the mean of the worst 10% returns. It can optimize tail performance only with a correctly defined loss orientation and enough tail samples; it is not a hard safety guarantee.

Semi-Markov decision process

A decision process whose high-level actions can last different durations. Option returns accumulate primitive rewards and bootstrap with γ^k after an option lasting k steps.

Simultaneous localization and mapping (SLAM)

Estimating robot pose while constructing or updating a map. SLAM output remains an uncertain, time-stamped estimate; it does not by itself choose a collision- free route or control motors.

Transformer, attention, and token

A token is one vector/symbol presented to a sequence model. Attention computes content-dependent weighted combinations of token values. A transformer stacks attention and feed-forward blocks; it does not automatically understand frames, physics, causality, or safety.

Frequency-space Action Sequence Tokenization (FAST)

A method that transforms and compresses continuous action chunks into frequency- space tokens for autoregressive vision-language-action models. Smooth temporal structure can use fewer tokens than independent per-timestep bins, subject to reconstruction and task validation.

Truncation

Stopping data collection because of a time or external limit while the modeled task could continue. It differs from true termination for value bootstrapping. Record both flags rather than collapsing them into one `done` value.

Uncertainty calibration

Agreement between predicted confidence and empirical frequency on a defined population. If events assigned confidence 0.8 occur about 80% of the time, that region is calibrated. Calibration can fail after distribution shift and must change permitted behavior to matter operationally.

Update-to-data ratio

The amount of optimizer sample consumption per newly collected transition in off-policy training. Too little reuse wastes data; too much can overfit replay and amplify critic error. Report the exact convention because projects count batches and gradient steps differently.

Vision-language model and large language model

A vision-language model (VLM) relates images and text. A large language model (LLM) predicts/represents language at scale. Neither term implies grounded motor skill. A VLA adds an action-learning path and still needs an embodiment adapter, local control, timing, and safety.

20.2 Worked problem 1: observation dimensions

Background

The actor concatenates proprioception (the robot's own measured motion/state) and commands. A wrong total means the network/runtime contract is broken.

Problem

Add the main terms: angular velocity 3, projected gravity 3, joint positions 14, joint velocities 14, previous actions 14, and command block 13.

20.3 Worked problem 2: timestep and policy rate

Background

Physics needs small steps for contacts. Neural policy inference can run less often using action decimation.

Problem

Physics timestep is 0.005 s and decimation is 4. Find the policy period and frequency. How long is a 500-step video?

20.4 Worked problem 3: discounted return

Background

Return adds current and future rewards, discounting each later step by γ. Use a short made-up reward sequence to see the calculation.

Problem

At time 0, the next three rewards are 1, 1, and 1. Let $\gamma = 0.99$ and stop after those three terms. What is G_0?

20.5 Worked problem 4: PPO rollout size

Background

One PPO iteration gathers the same number of time steps from every parallel environment.

Problem

How many transitions are collected with 64 environments and 24 steps? With 4,096 environments and 24 steps?

20.6 Worked problem 5: Gaussian tracking reward

Background

A common tracking term is $\exp(-(e/\sigma)^2)$, where e is error and σ is the error scale that still matters.

Problem

For head tracking with $\sigma = 0.5$ rad, calculate the per-joint reward at errors 0, 0.1, and 0.5 rad.

20.7 Worked problem 6: PPO clipping

Background

The probability ratio is new policy probability divided by old policy probability. For a positive advantage, PPO clips the benefit above 1.2 when $\epsilon = 0.2$.

Problem

Old action probability is 0.10, new probability is 0.13, and advantage is +2. Compare unclipped and clipped contributions.

20.8 Worked problem 7: penalty signs

Background

Configuration weight and function output multiply. Always reason about the product.

Problem

Function A returns squared error `+0.4`. Function B returns negative absolute error `-0.4`. What weight sign makes each a penalty?

20.9 Worked problem 8: why uniform sampling misses idle

Background

A continuous uniform random variable can produce values arbitrarily close to zero but has probability zero of producing one exact point.

Problem

Why does sampling each velocity uniformly from a range fail to train the exact deployment command `[0, 0, 0]`?

20.10 Worked problem 9: command versus action

Background

The planner states intent; the motor policy chooses immediate control.

Problem

Classify each value as command, actor observation, or actor action:

```
desired forward speed
measured head joint angle
new left-knee position target
desired head yaw delta
previous actor output
```

20.11 Worked problem 10: obstacle-avoidance architecture

Background

The renderer shows more than the actor receives. A visible object is not a numeric observation.

Problem

You add boxes to flat terrain and penalize collision, but keep the 61D actor unchanged. What can the policy learn, and what is missing?

20.12 Worked problem 11: actor versus critic information

Background

Asymmetric actor-critic training lets the critic see perfect simulator facts while the actor remains deployable.

Problem

Why can true base linear velocity improve the critic without being supplied to the actor? What would go wrong if the actor depended on it?

20.13 Worked problem 12: design one controlled experiment

Background

RL systems have many coupled variables. A good experiment changes one causal hypothesis and preserves a baseline.

Problem

Turn-in-place fails in 60% of trials. Propose a controlled experiment using the command distribution rather than changing PPO and rewards together.

20.14 Worked problem 13: a Q-learning backup

Background

Q-learning trains the current action value toward immediate reward plus the best estimated next action value.

Problem

The old value is $Q(s, a) = 1$. A transition gives reward -1. At the next state, the three action values are $[2, 5, 4]$. Let $\gamma = 0.9$ and learning rate $\alpha = 0.2$. Compute the target, TD error, and updated Q-value.

20.15 Worked problem 14: entropy changes the SAC objective

Background

SAC values expected reward plus α times policy entropy.

Problem

At one state, policy A has expected immediate score 5.0 and entropy 0.1. Policy B has score 4.8 and entropy 0.8. With $\alpha = 0.5$, which has the larger one-step soft objective?

20.16 Worked problem 15: behavior cloning averages modes

Background

Squared-error regression predicts the conditional mean when data contains uncertainty or several labels for the same input.

Problem

At an identical-looking observation, half the demonstrations steer left with action -1 and half steer right with action $+1$. What deterministic scalar action minimizes mean-squared error? Why may it be dangerous?

20.17 Worked problem 16: world-model error over a horizon

Background

Open-loop model predictions feed predicted state back into later predictions.

Problem

A simple position model has a consistent 5 mm forward error per predicted step. Ignoring all nonlinear amplification, what bias accumulates over 25 open-loop steps? Why is this a lower-complexity estimate rather than a guarantee?

20.18 Worked problem 17: mean return hides tail failure

Background

Expected return is not a per-episode safety guarantee.

Problem

Four hardware evaluations have returns [100, 100, 100, –100], where the last is a damaging fall. Compute the mean, median, and non-fall success rate. What must the report say?

20.19 Worked problem 18: stale asynchronous actions

Background

An asynchronous VLA can predict chunks while another loop executes prior actions. The relevant latency is observation age when an action is applied.

Problem

Image capture/preprocessing takes 20 ms, queue wait 15 ms, inference 70 ms, and transport 10 ms. How old is the observation when the first new action arrives? How many 50 Hz policy periods is that?

20.20 Worked problem 19: camera geometry and perception age

Background

An ideal depth camera back-projects horizontal pixel coordinate u using $X = Z(u - c_x)/f_x$. This gives a point in the camera frame; extrinsic calibration is still required for the robot frame. While the perception system computes, the robot continues moving, so geometric error and temporal age both consume clearance.

Problem

A camera has $f_x = 400$ pixels and $c_x = 320$ pixels. Pixel $u = 360$ has depth $Z = 1.5$ m. Compute camera-frame X. If the rover travels at 0.4 m/s and the measurement is 200 ms old at action time, how much unobserved forward travel must be added to the reasoning?

20.21 Worked problem 20: uncertainty-aware stopping speed

Background

Use conservative clearance $D_{safe} = \mu_D - k\sigma_D$ and stopping envelope

$$d_{stop} = v\tau + \frac{v^2}{2a_b} + m.$$

Solving $D_{safe} = d_{stop}$ yields a maximum speed under the model. It is a governor calculation, not a proof that Gaussian errors or constant braking hold.

Problem

Mean clearance is 0.70 m, standard deviation 0.05 m, and $k = 2$. Reaction age is 0.15 s, verified braking magnitude is 1.0 m/s², and residual margin is 0.10 m. Compute conservative clearance and maximum speed.

20.22 Worked problem 21: option return and elapsed-time discount

Background

A hierarchical option lasts several primitive steps. Its internal reward is discounted step by step, and the next high-level value is discounted by the entire duration. Forgetting duration makes long options appear too attractive.

Problem

An option lasts three steps with rewards $[2, -1, 3]$, discount $\gamma = 0.9$, and next-state best option value 5. Compute its internal return and full high-level target.

20.23 Worked problem 22: potential shaping telescopes

Background

Potential-based shaping adds $F(s_t, s_{t+1}) = \gamma\Phi(s_{t+1}) - \Phi(s_t)$. Discounted across a trajectory, intermediate potentials cancel. This is why progress pays while holding one state does not create an unlimited per-step jackpot under the theorem's setup.

Problem

For two transitions, let $\gamma = 0.9$ and potentials be $\Phi(s_0) = 0.2$, $\Phi(s_1) = 0.5$, and $\Phi(s_2) = 0.9$. Compute each shaping reward and their discounted sum. Verify it equals $-\Phi(s_0) + \gamma^2\Phi(s_2)$.

20.24 Worked problem 23: offline extrapolation error

Background

An offline critic sees only a fixed dataset. Function approximation can assign an unsupported action a high value even though no transition confirms it. A greedy actor then exploits the critic rather than the environment.

Problem

At one state, two dataset actions have estimated values 4 and 3. An unseen action has estimated value 9. What does naïve greedy policy improvement choose? Explain how Conservative Q-Learning and Implicit Q-Learning respond differently without claiming either knows the unseen action's true value.

20.25 Worked problem 24: action-chunk staleness

Background

An action chunk reduces model calls but actions later in the chunk are based on an older observation. Receding-horizon execution limits that open-loop age by executing only a prefix.

Problem

Inference finishes 100 ms after image capture. A policy predicts 10 actions for a 50 Hz controller. If all 10 execute, how old is the source image when the last action begins? If only the first 5 execute before replanning, what is the age of the fifth? Ignore other delays and count the first action at inference finish.

20.26 Worked problem 25: paired evaluation

Background

Using the same held-out scenarios for two policies can remove common scenario difficulty. Analyze per-scenario differences rather than comparing only two unpaired means.

Problem

Policy A scores [10, 12, 9, 13] and policy B scores [8, 11, 10, 9] on four matched scenarios. Compute paired differences and their mean. Why are four scenarios still insufficient for a broad generalization claim?

20.27 Worked problem 26: update-to-data ratio

Background

For off-policy learning, one convention defines update-to-data reuse as $U = GB/N_{new}$, where G is gradient batches, B is batch size, and N_{new} is newly collected transitions. Always state the convention because some tools count optimizer steps rather than sampled transition slots.

Problem

A collection cycle gathers 4096 × 24 transitions. The learner performs 96 gradient batches of size 1,024. Compute U. What would U be for 384 batches, and what risk grows as reuse increases?

20.28 Open exercises

1. Derive the maximum 20-second episode length in policy steps.
2. At iteration 1,000, what is the curriculum environment-step counter?
3. Draw the path from `head_pose` command to a reward and to the ONNX input.
4. Explain why body-pose user interface (UI) controls do not prove the velocity checkpoint learned body-pose control.
5. Design a five-case evaluation battery for exact-zero standing.
6. List three real measurements needed before transferring a policy to a new actuator.
7. Find one pure helper and its regression test in `tests/`; explain why the pure form is easier to test than a full simulator wrapper.

Return to the book index and repeat any lab whose terms or calculation are still unclear.

20.29 Folded solutions

Try each problem before opening its solution. The explanation—not only the final number—is the part to compare with your reasoning.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 1: observation dimensions — show solution

$$3 + 3 + 14 + 14 + 14 + 13 = 61$$

The first 48 values are proprioception/history-like action context, and the last 13 are intention. With 4,096 environments the actor input tensor is `(4096, 61)`.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 2: timestep and policy rate — show solution

$$\Delta t_{policy} = 0.005 \times 4 = 0.020 \text{ s}$$

Frequency is the inverse of period:

$$f = \frac{1}{0.020} = 50 \text{ Hz}$$

Five hundred policy steps last:

$$500 \times 0.020 = 10 \text{ s}$$

This is why a 500-step recorded rollout is a 10-second behavior sample.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 3: discounted return — show solution

$$G_0 = 1 + 0.99(1) + 0.99^2(1)$$

$$G_0 = 1 + 0.99 + 0.9801 = 2.9701$$

The third reward still matters strongly because it is only two steps away. This truncated example is for arithmetic; real GAE also uses a critic estimate beyond a rollout boundary when appropriate.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 4: PPO rollout size — show solution

$$64 \times 24 = 1,536$$

$$4096 \times 24 = 98,304$$

The five-iteration smoke train therefore performs real PPO updates but with a much smaller batch and duration than a full run. It validates plumbing, not locomotion quality.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 5: Gaussian tracking reward — show solution

At zero error:

$$e^{-(0/0.5)^2} = 1$$

At 0.1 rad:

$$e^{-(0.1/0.5)^2} = e^{-0.04} \approx 0.961$$

At 0.5 rad:

$$e^{-(0.5/0.5)^2} = e^{-1} \approx 0.368$$

This shows why a wide standard deviation gives useful gradient far away but makes small errors relatively cheap.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 6: PPO clipping — show solution

$$r = 0.13/0.10 = 1.3$$

Unclipped:

$$1.3 \times 2 = 2.6$$

Clipped ratio is 1.2:

$$1.2 \times 2 = 2.4$$

The objective uses the more conservative value 2.4, removing the incentive to increase this already-favored sample further during that update.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 7: penalty signs — show solution

For A, use a negative weight; for example:

$$-1.0 \times +0.4 = -0.4$$

For B, use a positive weight:

$$+1.0 \times -0.4 = -0.4$$

Using a negative weight for B gives `+0.4`, rewarding the violation. Confirm all logged penalty contributions remain nonpositive.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 8: why uniform sampling misses idle — show solution

The probability of any one exact real-number triple under continuous sampling is zero. “Very small” is also behaviorally different from an exact idle flag. The environment therefore needs an explicit zero-command bucket with nonzero probability.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 9: command versus action — show solution

```
desired forward speed            command (also included in observation)
measured head joint angle        proprioceptive observation
new left-knee position target    action after scale/offset mapping
desired head yaw delta           command (also included in observation)
previous actor output            observation/history context
```

Commands become part of the actor input; they are not outputs chosen by the motor policy.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 10: obstacle-avoidance architecture — show solution

It may learn post-contact reactions or a generally conservative gait. It cannot deliberately steer around an unseen box before contact because no pre-contact obstacle information reaches the actor. Add a local perception/planner that changes twist commands, or create a versioned exteroceptive policy input and matching runtime.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 11: actor versus critic information — show solution

The critic is discarded after training, so privileged velocity can improve its value/advantage estimates without becoming a runtime input. If the actor used perfect velocity but the real robot could not reproduce it with matching noise, delay, frame, and accuracy, deployment observations would come from a different distribution and behavior could fail.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 12: design one controlled experiment — show solution

```
hypothesis:
  turn-in-place examples are too rare

change:
  increase TURN_IN_PLACE_FRACTION from 0.15 to 0.25 only

preserve:
  task, robot, rewards, PPO config, training budget, evaluation commands

verification:
  config test -> CPU suite -> five-iteration smoke -> equal-budget train

metrics:
  turn success, yaw error, translation drift, fall rate, forward tracking

decision:
  keep only if turn improvement is repeatable and forward behavior remains
  inside its acceptance envelope
```

Changing entropy, yaw reward, command range, and sample fraction together would prevent a causal conclusion.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 13: a Q-learning backup — show solution

The greedy next value is 5:

$$y = -1 + 0.9(5) = 3.5.$$

The TD error is target minus old estimate:

$$\delta = 3.5 - 1 = 2.5.$$

Update only partway using α:

$$Q_{new} = 1 + 0.2(2.5) = 1.5.$$

The estimate rises because this transition was better than the old prediction.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 14: entropy changes the SAC objective — show solution

$$A : 5.0 + 0.5(0.1) = 5.05,$$

$$B : 4.8 + 0.5(0.8) = 5.20.$$

The soft objective prefers B despite slightly lower expected task score because it retains more action diversity. This does not mean deployment must sample unsafe random actions; the entropy term shapes training, and execution policy/ safety still must be specified.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 15: behavior cloning averages modes — show solution

For prediction x:

$$L(x) = \tfrac{1}{2}(x+1)^2 + \tfrac{1}{2}(x-1)^2 = x^2 + 1.$$

The derivative $2x$ is zero at $x = 0$, the mean. If left and right both avoid an obstacle, the average can drive straight into it. More context or a multimodal policy must distinguish/represent the alternatives.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 16: world-model error over a horizon — show solution

$$25 \times 5 \text{ mm} = 125 \text{ mm} = 0.125 \text{ m}.$$

Real error need not add linearly. A biased position changes future contact and policy inputs, which can amplify, cancel, or redirect error. Receding-horizon replanning replaces imagined state with observation before the full horizon.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 17: mean return hides tail failure — show solution

$$\text{mean} = \frac{100 + 100 + 100 - 100}{4} = 50.$$

The sorted middle pair is 100, 100, so median is 100. Success is 3/4 = 75%. The attractive median hides one severe failure; even the mean does not label its consequence. Report every trial, success/failure category, damage/safety intervention, and uncertainty. Four trials are too few for a reliable tail-risk estimate.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 18: stale asynchronous actions — show solution

$$20 + 15 + 70 + 10 = 115 \text{ ms}.$$

A 50 Hz period is 20 ms:

$$115/20 = 5.75 \text{ policy periods}.$$

The model acts from information nearly six local-control ticks old. A manipulator may tolerate that under slow receding-horizon execution; a disturbed balance loop likely cannot. Measure age/jitter and keep fast stabilization local.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 19: camera geometry and perception age — show solution

Back-project the horizontal coordinate:

$$X = Z\frac{u - c_x}{f_x} = 1.5\frac{360 - 320}{400} = 0.15 \text{ m}.$$

The point is 15 cm along the camera's declared positive horizontal axis. The robot travels during age:

$$d_{age} = v\tau = 0.4(0.200) = 0.080 \text{ m}.$$

Eight centimeters must be included before braking and residual margin. The two numbers describe different axes in this example; transform the point into the robot frame before using it as forward clearance.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 20: uncertainty-aware stopping speed — show solution

Conservative clearance is

$$D_{safe} = 0.70 - 2(0.05) = 0.60 \text{ m}.$$

Set the stopping envelope equal to 0.60 m:

$$0.60 = 0.15v + \frac{v^2}{2} + 0.10.$$

Equivalently, $v^2 + 0.30v - 1.0 = 0$. Take the nonnegative root:

$$v_{max} = \frac{-0.30 + \sqrt{0.30^2 + 4}}{2} \approx 0.861 \text{ m/s}.$$

Substitution gives roughly 0.60 m. This bound is only as defensible as the measured braking, age tail, calibration, uncertainty coverage, and margin.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 21: option return and elapsed-time discount — show solution

The internal option return is

$$R_o = 2 + 0.9(-1) + 0.9^2(3) = 3.53.$$

The next option is selected after three steps, so its value receives $\gamma^3 = 0.729$:

$$y = 3.53 + 0.729(5) = 7.175.$$

Using only one factor of γ would overvalue the future after this three-step option relative to a shorter option.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 22: potential shaping telescopes — show solution

For the first transition:

$$F_0 = 0.9(0.5) - 0.2 = 0.25.$$

For the second:

$$F_1 = 0.9(0.9) - 0.5 = 0.31.$$

Discount the second shaping reward because it arrives one step later:

$$F_0 + \gamma F_1 = 0.25 + 0.9(0.31) = 0.529.$$

The endpoints give the same result:

$$-\Phi(s_0) + \gamma^2 \Phi(s_2) = -0.2 + 0.9^2(0.9) = 0.529.$$

The intermediate $\Phi(s_1)$ cancels. Terminal handling and discount must match the theorem; adding an arbitrary "upright every step" bonus does not telescope.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 23: offline extrapolation error — show solution

Naïve greedy improvement chooses the unseen action with estimated value 9. The choice is based on critic extrapolation, not supporting evidence.

Conservative Q-Learning adds pressure lowering values of broadly sampled or policy-proposed actions relative to dataset actions, so the unsupported 9 should lose its easy advantage when the regularizer works. Implicit Q-Learning fits a state value from dataset action values using an upper expectile, then extracts a policy by giving higher imitation weight to above-value dataset actions; its central update does not maximize over the unseen action. Neither method observes the unseen action's true result. If dataset coverage omits every good action, conservatism cannot invent safe evidence.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 24: action-chunk staleness — show solution

A 50 Hz period is 20 ms. With the first action beginning at 100 ms, the tenth begins nine periods later:

$$A_{10} = 100 + 9(20) = 280 \text{ ms}.$$

The fifth begins four periods after the first:

$$A_5 = 100 + 4(20) = 180 \text{ ms}.$$

Replanning after five actions reduces the oldest executed source age by 100 ms in this simplified schedule. Real age also includes exposure, queues, transport, and scheduling jitter; disturbance can make even 180 ms unacceptable for balance.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 25: paired evaluation — show solution

Compute A minus B for each matched scenario:

$$d = [10 - 8,\ 12 - 11,\ 9 - 10,\ 13 - 9] = [2, 1, -1, 4].$$

The mean paired improvement is

$$\bar{d} = \frac{2 + 1 - 1 + 4}{4} = 1.5.$$

One scenario favors B, which a global mean could hide. Four selected scenarios give little information about training-seed variability, rare failures, or a larger target population. Preserve the pairs and add preregistered scenarios and independent training seeds.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Problem 26: update-to-data ratio — show solution

New transitions are

$$N_{new} = 4096(24) = 98{,}304.$$

For 96 batches:

$$U = \frac{96(1024)}{98{,}304} = 1.$$

For 384 batches, $U = 4$. Four times as many sampled replay slots are optimized per new transition. Greater reuse may improve data efficiency, but it increases compute and can overfit replay, amplify critic/extrapolation error, or let the actor exploit a lagging critic. It does not mean each transition is seen exactly four times because replay sampling is random.

REFERENCE ANSWER · CHECK AFTER ATTEMPTING

Open exercises 1–7 — show solutions and reference checks

1. At 50 Hz, a 20-second episode contains 20 s × 50 Hz = 1,000 policy steps.
2. Curriculum steps count environment steps per rollout fragment, so iteration 1,000 corresponds to 1,000 × 24 = 24,000 environment steps.
3. `CommandManager` samples `head_pose`; `generated_commands` appends its four values after the 48 proprioceptive values and 3D twist command, so they occupy actor indices 51–54. The `head_pose_tracking` reward reads the same command and the four physical head/neck joint positions. Export rebuilds this observation order, applies the saved normalizer, and embeds the actor in ONNX. This shared source and ordering are why command, reward, and deployment remain consistent.
4. Interface presence is not a learned objective. The six body-pose inputs and UI control can vary, but the main velocity recipe gives `body_pose_tracking` weight zero. A checkpoint might ignore those inputs or respond only through accidental correlations; testable command following requires nonzero task data, reward, and held-out evaluation.

5. A useful exact-zero battery includes: nominal quiet stand; randomized initial tilt; a bounded push followed by recovery; held-out friction and actuator-delay extremes; and a long-duration hold that exposes drift or heating. For every case report fall rate, tilt, position drift, action jitter, saturation, and recovery time across seeds—not just reward.
6. At minimum measure command-to-motion delay/bandwidth, torque or force versus command under load, and friction/deadband/backlash. Battery-voltage response, saturation, speed limits, thermal behavior, and sensor noise are also part of a defensible actuator model.
7. One example is the following pure helper and its focused test:

```
mdp.py::spin_wheel_differential_from_values
tests/test_spin.py::test_wheel_differential_from_values_is_pure
```

   The helper receives tensors and constants directly, so the test can cover its sign, scaling, and gating without constructing MuJoCo, managers, sensors, or GPU state. The simulator wrapper can then be tested separately for correct data extraction.

Primary Sources and Open-Source Study Index

Last reviewed: 2026-09-01.

This index supports the book's research chapters. It deliberately favors primary papers and official repositories. Inclusion means the source is useful for study and its identity/artifact was verified; it is not an endorsement of every claim or a declaration that the method is currently best for every task.

Inclusion and verification policy

A research entry should provide at least:

- a stable paper record (publisher, proceedings, OpenReview, or arXiv);
- identifiable authors and version/date;
- a clearly scoped empirical or theoretical claim; and
- preferably an official project page, source repository, data, or checkpoint.

For every source, readers should verify:

1. paper version and publication status;
2. task, embodiment, observation, action, and data setting;
3. training/evaluation compute and number of seeds/trials;
4. whether code/config/checkpoints/data reproduce the reported path; and
5. license and current dependency compatibility.

Links below point to papers or official author/project repositories, not search result pages.

Foundations and policy optimization

Reinforcement Learning: An Introduction, second edition (2018)

- MIT Press record
- Richard S. Sutton and Andrew G. Barto.
- Establishes: the standard progression from bandits and tabular prediction to function approximation and policy gradients.
- Study caution: it is a general RL foundation, not a current robot deployment manual.

Applied Dynamic Programming (1962)

- RAND publication record
- Richard E. Bellman and Stuart E. Dreyfus.
- Establishes: an early primary account of dynamic programming applied to sequential optimization, trajectories, feedback control, and related problems.
- Study caution: the historical formulation assumes models/representations unlike modern sampled deep robot learning.

Neuronlike Adaptive Elements That Can Solve Difficult Learning Control Problems (1983)

- Publisher record
- Andrew G. Barto, Richard S. Sutton, Charles W. Anderson.
- Establishes: an early actor-critic-style adaptive controller evaluated on a pole-balancing problem.
- Study caution: historical influence does not make its network or experiment a present-day robot baseline.

Q-learning (1992)

- Publisher record
- Christopher J. C. H. Watkins and Peter Dayan.
- Establishes: the tabular off-policy Q-learning update and convergence result under the paper's discrete, repeatedly sampled conditions.
- Does not establish: convergence with nonlinear neural approximation and arbitrary replay distributions.

REINFORCE (1992)

- Publisher record
- Ronald J. Williams.
- Establishes: stochastic connectionist update rules aligned with gradients of expected reinforcement in the paper's settings.
- Study caution: the basic estimator can have high variance; actor-critic, advantage, and trust-region machinery address later practical pressures.

Policy invariance under reward transformations (1999)

- Author-hosted paper
- Andrew Y. Ng, Daishi Harada, Stuart Russell.
- Establishes: conditions under which potential-based reward shaping preserves optimal policies.
- Study caution: terminal handling, discounting, and implementation must match the theorem; arbitrary dense bonuses are not covered.

A Natural Policy Gradient (2001)

- NeurIPS proceedings
- Sham Kakade.
- Establishes: policy-gradient steps adjusted for local policy-distribution geometry, a conceptual predecessor to trust-region methods.

Deterministic Policy Gradient and DDPG (2014/2015)

- Deterministic Policy Gradient paper
- DDPG paper
- David Silver et al.; Timothy Lillicrap et al.
- Establishes: deterministic actor gradients and an influential deep continuous-control actor-critic implementation with replay/targets.
- Study caution: TD3 was motivated by value-error and brittleness observed in this family.

Playing Atari with Deep Reinforcement Learning (2013)

- Paper
- Volodymyr Mnih et al.
- Establishes: an influential deep Q-learning result from pixel observations in discrete Atari action spaces.
- Does not establish: suitability of DQN for high-dimensional continuous robot commands.

Double DQN and Rainbow (2015/2017)

- Double DQN paper
- Rainbow paper
- Hado van Hasselt, Arthur Guez, David Silver; Matteo Hessel et al.
- Establishes: decoupled action selection/evaluation reduces DQN overestimation in the evaluated setting, and several value-learning extensions can be combined with measured interactions on Atari.
- Study caution: these are discrete-action results and not direct replacements for continuous robot actors.

High-Dimensional Continuous Control Using GAE (2015)

- Paper
- John Schulman et al.
- Establishes: the generalized advantage estimator and its bias/variance motivation in policy-gradient control.

Trust Region Policy Optimization (2015)

- Paper
- John Schulman et al.
- Establishes: the trust-region motivation that precedes PPO.

Proximal Policy Optimization Algorithms (2017)

- Paper
- John Schulman et al.
- Establishes: PPO surrogate objectives and benchmark evidence for repeated minibatch updates on fresh policy data.
- Study caution: clipping is a practical surrogate, not a universal monotonic improvement guarantee.

Addressing Function Approximation Error in Actor-Critic Methods (TD3, 2018)

- Paper
- Scott Fujimoto, Herke van Hoof, David Meger.
- Establishes: twin critics, delayed actor updates, and target smoothing as responses to continuous-control value error.

Soft Actor-Critic (2018)

- Paper
- Tuomas Haarnoja et al.
- Establishes: an off-policy maximum-entropy stochastic actor-critic method for continuous control.

Constrained Policy Optimization (2017)

- Paper
- Joshua Achiam et al.
- Establishes: policy optimization with expected constraints and stated theoretical properties.
- Does not replace: hard electrical, mechanical, or realtime safety.

Model-based and world-model learning

DreamerV3: Mastering Diverse Domains through World Models (2023)

- Paper
- Danijar Hafner et al.
- Establishes: latent world-model learning and imagined actor/critic training with one reported configuration across diverse evaluated domains.

DayDreamer: World Models for Physical Robot Learning (2022)

- Paper
- Danijar Hafner et al.
- Establishes: online world-model learning directly on four physical robots in the paper's tasks, including quadruped locomotion and manipulation.
- Study caution: real-world sample efficiency does not remove reset labor, hardware wear, latency, or safety constraints.

QT-Opt: Scalable Deep Reinforcement Learning for Vision-Based Robotic Manipulation (2018)

- Paper
- Dmitry Kalashnikov et al.
- Establishes: a distributed off-policy Q-learning system trained from a large mixture of autonomous and logged real-robot grasp attempts.
- Study caution: its data and infrastructure scale are part of the method's empirical setting and must accompany algorithm comparisons.

TD-MPC2 (2023)

- Paper
- Official code
- Nicklas Hansen, Hao Su, Xiaolong Wang.
- Establishes: scalable latent model-predictive control plus value learning across the paper's online continuous-control suite.

Imitation and offline learning

DAgger (2010/2011)

- Paper
- Stéphane Ross, Geoffrey Gordon, Drew Bagnell.
- Establishes: dataset aggregation to address sequential imitation's state distribution shift.

Generative Adversarial Imitation Learning (2016)

- Paper
- Jonathan Ho and Stefano Ermon.
- Establishes: matching an expert's occupancy distribution through an adversarial discriminator and a policy-optimization loop in the paper's control tasks.
- Study caution: the method needs interactive policy rollouts and can inherit discriminator instability; it is not ordinary supervised behavior cloning.

Decision Transformer (2021)

- Paper
- Official code
- Lili Chen et al.
- Establishes: casting offline control as return-conditioned sequence modeling on the paper's Atari, OpenAI Gym, and Key-to-Door settings.
- Does not establish: that asking for an arbitrarily high return creates behavior absent from the dataset, or that a return token supplies physical safety.

D4RL (2020)

- Paper
- Justin Fu et al.
- Establishes: offline-RL datasets/protocols designed to expose data-coverage challenges.

Conservative Q-Learning (2020)

- Paper
- Aviral Kumar et al.
- Establishes: conservative value regularization for fixed offline data.

Implicit Q-Learning (2021)

- Paper
- Ilya Kostrikov, Ashvin Nair, Sergey Levine.

- Establishes: offline improvement without evaluating arbitrary unseen policy actions during the primary value-learning step.

Cal-QL: Calibrated Offline RL Pre-Training (2023)

- NeurIPS paper
- Official code
- Mitsuhiko Nakamoto et al.
- Establishes: calibrating a conservative value initialization to reduce the early "unlearning" problem during offline-to-online fine-tuning on the paper's benchmark suite.
- Study caution: safe online improvement still requires a hardware envelope, replay/data-mixing choices, reset protocols, and measured real trials.

Efficient Online Reinforcement Learning with Offline Data / RLPD (2023)

- PMLR paper
- Official code
- Philip J. Ball et al.
- Establishes: a simple off-policy recipe that mixes offline and newly collected data, with ensemble and high update-to-data-ratio choices, in the paper's online fine-tuning experiments.
- Does not establish: that arbitrary logged robot data are safe, correctly labeled, or sufficiently covering for autonomous exploration.

ACT: Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware (2023)

- Paper
- Official code
- Tony Zhao et al.
- Establishes: action chunking and transformer-style imitation on the reported bimanual tasks.

Diffusion Policy (2023)

- Paper
- Official code
- Cheng Chi et al.
- Establishes: conditional diffusion for multimodal visual action sequences on the paper's manipulation benchmarks.

VQ-BeT: Behavior Generation with Latent Actions (2024)

- Paper
- Official code
- Seungjae Lee et al.
- Establishes: a residual vector-quantized action representation plus a transformer policy for multi-modal behavior generation in the evaluated imitation settings.
- Study caution: codebook collapse, action-unit conventions, and temporal chunk execution must be checked on a new embodiment.

3D Diffusion Policy / DP3 (2024)

- Paper
- Official code
- Yanjie Ze, Gu Zhang et al.
- Establishes: compact point-cloud conditioning for diffusion-policy learning on the paper's simulated and physical manipulation tasks.
- Study caution: depth quality, cropping, calibration, point sampling, and camera placement are part of the method rather than interchangeable inputs.

BAKU: An Efficient Transformer for Multi-Task Policy Learning (2024)

- Paper

- Official code
- Siddhant Haldar, Zhuoran Peng, Lerrel Pinto.
- Establishes: a modular transformer policy and action-head comparison across the paper's multi-task simulation and real-robot imitation settings.
- Study caution: compare data, observation encoders, action heads, and task sampling together; "transformer" alone is not the causal intervention.

DROID robot-manipulation dataset (2024)

- Paper
- Official dataset code
- DROID collaboration.
- Establishes: a large, distributed real-robot manipulation dataset and data collection/evaluation pipeline across the participating sites.
- Study caution: dataset size does not erase site, operator, camera, success label, embodiment, or action-schema heterogeneity.

robomimic

- Official code
- Study use: a modular imitation-learning framework with behavior-cloning, recurrent, transformer, and generative baselines plus robot datasets.
- Study caution: pin the exact dataset, observation modalities, low-dimensional keys, environment version, and evaluation protocol before comparing scores.

Sim-to-real and locomotion

Domain Randomization for Visual Transfer (2017)

- Paper
- Josh Tobin et al.
- Establishes: a foundational visual domain-randomization demonstration for sim-to-real object localization/control.

Dynamics Randomization for Robotic Control (2017)

- Paper
- Xue Bin Peng et al.
- Establishes: dynamics-randomized simulation policies transferred to a real object-pushing setup.

DeepMimic (2018)

- Paper
- Xue Bin Peng et al.
- Establishes: reinforcement learning of physics-based character skills from reference motions using phase-conditioned imitation and task rewards in the paper's simulated settings.
- Study caution: a human or character reference trajectory is not directly a feasible robot trajectory; morphology, contacts, torque limits, and retargeting must be modeled.

Learning Agile and Dynamic Motor Skills for Legged Robots (2019)

- Paper
- Jemin Hwangbo et al.
- Establishes: an influential actuator-aware simulated-training/real-ANYmal deployment pipeline.

Learning to Walk in Minutes (2021)

- Paper
- Official `legged_gym`
- Nikita Rudin et al.

- Establishes: massively parallel GPU locomotion training and terrain curriculum results in the reported setup.

Adversarial Motion Priors / AMP (2021)

- Paper
- Xue Bin Peng et al.
- Establishes: an adversarially learned motion-style reward that can be combined with task rewards without requiring phase alignment to a single reference trajectory in the evaluated simulated characters.
- Study caution: a discriminator rewards similarity to its training motion distribution, not robot feasibility or task completion by itself.

Rapid Motor Adaptation (2021)

- Paper
- Official project page
- Author-linked training code
- Ashish Kumar et al.
- Establishes: a base policy plus history-based adaptation module for the evaluated quadruped conditions.

Adversarial Skill Embeddings / ASE (2022)

- Paper
- Official code
- Xue Bin Peng et al.
- Establishes: a latent-conditioned low-level controller that learns a reusable motion-skill space from motion data and can be composed for downstream tasks in the paper's simulated humanoid setting.
- Study caution: latent diversity and imitation quality do not prove that a downstream task can safely or efficiently search the learned skill space.

Robust Perceptive Locomotion in the Wild (2022)

- Paper
- Takahiro Miki et al.
- Establishes: learned fusion of proprioception and unreliable exteroception for reported real quadruped deployments.

Extreme Parkour with Legged Robots (2023)

- Paper
- Cheng et al.
- Establishes: depth-conditioned parkour behaviors on the paper's low-cost quadruped and obstacle protocol.

Wheeled-Legged Navigation and Locomotion (2024)

- Paper
- Joonho Lee et al.
- Establishes: an integrated hierarchical learned locomotion/navigation system evaluated in the stated urban missions.

SoloParkour (2024)

- Paper
- Elliot Chane-Sane et al.
- Establishes: constrained visual locomotion initialized from privileged experience on Solo-12.

MuJoCo Playground (2025)

- Paper
- Official code
- Establishes: an open GPU-accelerated MuJoCo suite with reported robot-learning and sim-to-real examples.

MaskedMimic and ProtoMotions (2024)

- Paper
- Official code
- Establishes: a masked, partially specified motion-imitation controller and an open training framework for the reported physics-based humanoid tasks.
- Study caution: broad motion in a simulated humanoid does not imply actuator, contact, timing, or safety compatibility with a physical robot.

ASAP: Aligning Simulation and Real-World Physics for Humanoid Robot (2025)

- Paper
- Official code
- Establishes: a two-stage motion imitation and real-world-dynamics alignment approach evaluated on the paper's humanoid motion-tracking tasks.
- Study caution: the learned residual captures discrepancies inside the measured operating region; it is not a certificate outside that support.

BeyondMimic (2025 preprint)

- Paper
- Official code
- Establishes: the paper's whole-body humanoid tracking recipe, including motion retargeting and deployable policy components, on its stated robots and motion suite.
- Study caution: this is a recent preprint; verify revisions, released assets, and the exact real-robot trial protocol before treating comparisons as final.

Learning Whole-Body Humanoid Locomotion via Motion Generation and Motion Tracking (2026 preprint)

- Paper
- Establishes: a recent reported approach to terrain-aware whole-body humanoid motion generation in the paper's specified data and evaluation setting.
- Study caution: this is a 2026 preprint. Recheck its current version, publication status, code availability, and independent reproduction before making a design commitment.

FastTD3: Simple, Fast, and Capable Reinforcement Learning for Humanoid Control (2025)

- Paper
- Official code
- Establishes: a tuned, massively parallel Twin Delayed Deep Deterministic Policy Gradient recipe with rapid learning on the paper's humanoid-control benchmarks.
- Study caution: the result is a coupled algorithm, implementation, batching, replay, update-ratio, simulator, and hardware recipe—not evidence that the TD3 name alone causes the speed.

Learning Sim-to-Real Humanoid Locomotion in 15 Minutes (2025 preprint)

- Paper
- Official code
- Younggyo Seo et al.
- Establishes: reported FastSAC/FastTD3 recipes at massive simulation scale on the stated GPUs, humanoids, and protocols.
- Study caution: a training-time headline is hardware/configuration dependent.

FastDSAC: Constrained Exploration for Scalable Humanoid Locomotion (2026 preprint)

- Paper
- Official code
- Establishes: a recent distributional Soft Actor-Critic variant and training recipe evaluated on the paper's humanoid-control suite.
- Study caution: this is a new 2026 preprint. Treat the result as provisional until revisions, official artifacts, exact budgets, and independent evidence have been checked.

PACE sim-to-real (active project; 2025 paper record in project)

- Official code and documentation
- Establishes: an open, measurement-driven workflow for identifying actuator and joint dynamics on supported legged systems.
- Study caution: active APIs and platform-specific models must be versioned.

Generalist robot policies and data

RT-1: Robotics Transformer for Real-World Control at Scale (2022)

- Paper
- Anthony Brohan et al.
- Establishes: a transformer policy trained on the paper's multi-task real robot dataset, with image and language inputs and discretized action tokens.
- Does not establish: that language alone supplies geometric safety or that the reported policy transfers unchanged to arbitrary embodiments.

RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control (2023)

- Paper
- Anthony Brohan et al.
- Establishes: co-fine-tuning vision-language models on robot trajectories and web-scale visual-language data in the reported manipulation evaluations.
- Study caution: inspect which models, weights, robot data, and evaluation interfaces are available before calling a result reproducible.

Open X-Embodiment and RT-X (2023)

- Paper
- Official dataset organization
- Open X-Embodiment Collaboration et al.
- Establishes: a large cross-institution collection of robot datasets and cross-embodiment policy experiments under the paper's unified format.
- Study caution: common serialization does not make action semantics, camera geometry, control rates, or data quality identical.

Octo (2024)

- Paper
- Official code
- Establishes: an open generalist policy pretrained on a large multi-robot demonstration collection and fine-tuned in the evaluated settings.

OpenVLA (2024)

- Paper
- Official code

- Establishes: an open 7B vision-language-action model, training recipe, and evaluated fine-tuning/serving results.

FAST action tokenization (2025)

- Paper
- Official implementation in `openpi`
- Establishes: Frequency-space Action Sequence Tokenization (FAST), which uses a discrete cosine transform and byte-pair encoding to compress continuous robot action chunks for autoregressive vision-language-action training in the paper's datasets and evaluations.
- Study caution: token compression changes sequence length and modeling convenience; it does not by itself establish closed-loop stability or safe low-level control.

OpenVLA-OFT (2025)

- Paper
- Official code
- Establishes: an optimized fine-tuning recipe for OpenVLA, including parallel decoding and continuous action representations, on the reported manipulation benchmarks.
- Study caution: compare against the exact OpenVLA checkpoint, data split, action normalization, image preprocessing, and control-frequency contract.

π_0 (2024)

- Paper
- Kevin Black et al.
- Establishes: a VLM plus flow-matching action model evaluated on the stated multi-platform manipulation data and tasks.
- Availability caution: verify which weights, data, and code required for a claimed result are public before planning a reproduction.

$\pi_{0.5}$: A Vision-Language-Action Model with Open-World Generalization (2025)

- Paper
- Official `openpi` code
- Karl Pertsch et al.
- Establishes: the reported co-training and hierarchical action-generation approach for long-horizon manipulation evaluations.
- Study caution: distinguish the paper's full internal data recipe from the checkpoints and fine-tuning paths actually released in `openpi`.

GR00T N1 (2025)

- Paper
- Official code
- NVIDIA et al.
- Establishes: a dual-system vision-language/action architecture and released tooling for the humanoid manipulation settings described by the project.
- Study caution: check robot support, dataset license, inference hardware, and checkpoint scope at the pinned revision.

Gemini Robotics (2025)

- Technical report
- Google DeepMind et al.
- Establishes: reported embodied-reasoning and vision-language-action results for the models, robots, data, and evaluation suites described in the report.
- Availability caution: a technical report is not an open reproduction bundle. Verify public weights, training data, evaluator code, and access terms before selecting it for a reproducible student capstone.

SmolVLA (2025)

- Paper
- Official LeRobot code
- Hugging Face et al.
- Establishes: a compact open VLA recipe evaluated with community robot data and asynchronous inference in the stated tasks.
- Study caution: small parameter count is not a timing or safety guarantee on a particular robot computer.

LeRobot

- Official code
- Establishes: an active open ecosystem for robot datasets, models, training, and hardware integrations.
- Study caution: the repository evolves quickly; pin a release/commit and read each policy's learning setting instead of calling all of it RL.

Green-VLA (2026 preprint)

- Paper
- Establishes: a staged curriculum from foundational vision-language models through multimodal grounding, multi-embodiment pretraining, embodiment-specific adaptation, and reinforcement-learning alignment for the paper's Green humanoid and benchmark settings.
- Study caution: this is a 2026 preprint. Recheck version, official code and weights, data/preprocessing, baselines, real-trial protocol, and independent evidence. Here "Green" names the robot/platform; it is not an energy claim.

Language-grounded planning and skill composition

Do As I Can, Not As I Say / SayCan (2022)

- Paper
- Michael Ahn et al.
- Establishes: combining language-model skill relevance with learned affordance/value estimates for the reported mobile-manipulation tasks.
- Does not establish: that unconstrained model text should become motor commands; the candidate skill set and local feasibility model are essential.

Code as Policies (2022)

- Paper
- Jacky Liang et al.
- Establishes: generating compositional robot-policy programs from language in the demonstrated tabletop and mobile manipulation settings.
- Study caution: generated code still needs constrained APIs, validation, timeouts, and a trusted execution boundary.

VoxPoser (2023)

- Paper
- Wenlong Huang et al.
- Establishes: composing language-conditioned 3D value maps with a motion planner in the reported manipulation experiments.
- Study caution: the perception and geometric-planning assumptions are part of the result and can fail independently of the language model.

Evaluation and reproducibility

Deep Reinforcement Learning That Matters (2017)

- Paper
- Peter Henderson et al.
- Establishes: empirical evidence that implementation and experimental choices materially affect deep-RL comparisons.

Deep RL at the Edge of the Statistical Precipice / RLiable (2021)

- Paper
- Official code
- Rishabh Agarwal et al.
- Establishes: uncertainty-aware aggregate evaluation methods for the few-run deep-RL regime.
- Maintenance caution: the official repository was archived in October 2025. Pin a commit and environment, and treat the equations/paper as the authority if a future dependency conflict changes executable behavior.

Machine Learning Reproducibility Challenge program report (2021)

- JMLR paper
- Establishes: lessons from a structured, multi-year effort to independently reproduce machine-learning papers, including recurring reporting and artifact problems.
- Study caution: a reproduction can test a bounded claim and implementation; it does not prove that all results or future versions are correct.

ACM artifact review and badging policy

- Official ACM policy
- Study use: precise vocabulary for artifacts that are available, functional, reusable, or whose results were reproduced/replicated under a review process.
- Study caution: read the badge definition and review scope. A badge is not a blanket endorsement of safety, scientific validity, or every paper claim.

Open robot-learning ecosystems and benchmarks

These projects are useful substrates, not a leaderboard declaring one universal winner. Their rapidly changing interfaces make a pinned revision part of every experiment.

Gymnasium

- Official code
- Study use: a standard environment/application programming interface and reference environments for reinforcement-learning experiments.
- Study caution: distinguish `terminated` from `truncated`; incorrect bootstrap handling changes the target even when the program still runs.

Stable-Baselines3 and CleanRL

- Stable-Baselines3 official code
- CleanRL official code
- Study use: tested library-style baselines and compact single-file implementations, respectively, for checking algorithm mechanics.
- Study caution: neither substitutes for matching environment wrappers, budgets, network sizes, evaluation, or robot-specific safety contracts.

robosuite, ManiSkill, RLBench, and LIBERO

- robosuite official code
- ManiSkill official code
- RLBench official code
- LIBERO official code

- Study use: complementary simulation and benchmark ecosystems for robot manipulation, demonstrations, multi-task learning, and lifelong learning.
- Study caution: scores across suites are not directly comparable. Record the task revision, assets, control mode, observations, demonstrations, reset and success logic, and evaluation seeds.

Case-study projects

Microduck RL

- Official training repository
- Microduck runtime repository
- Study use: complete small-biped PPO task definitions, BAM actuator modeling, domain randomization, ONNX export, and sim-to-real runtime contracts.

RSL-RL

- Official code
- Study use: compact robot-focused runners, PPO, actor/critic models, storage, and student-teacher mechanisms.

Isaac Lab

- Official code
- Study use: multi-modal simulator workflows, actuator/sensor models, domain randomization, RL and imitation integrations.

Genesis

- Official code
- Study use: a modern general robotics physics platform and an alternative vectorized simulation stack.
- Availability caution: distinguish released open simulator features from project-roadmap/generative-system descriptions.

Maintaining this index

When adding a source:

1. link the exact paper/version and official repository;
2. summarize the narrow evidence in original words;
3. add one “does not establish” or study-caution boundary;
4. record the review date if a project is fast-moving; and
5. never update a benchmark superlative without rechecking the protocol and newer primary results.